

普中 51 单片机开发攻略
www.prechin.cn

普中 51 单片机开发攻略



开源共享

携手共进

深圳普中科技有限公司

官方网站: www.prechin.cn

技术论坛: www.prechin.net

咨询电话: 0755-36564227

目录

普中 51 单片机开发攻略.....	1
第 1 章 如何使用本攻略.....	1
第 2 章 开发板功能及使用介绍.....	3
2.1 开发板功能介绍.....	4
2.1.1 普中 51-单核-A2 开发板.....	4
2.2 开发板使用方法.....	6
2.2.1 CH340 驱动安装.....	6
2.2.2 程序的烧录（下载）.....	8
2.2.3 51 仿真器调试.....	13
2.2.4 接线方法及实验现象.....	13
第 3 章 51 单片机介绍.....	16
3.1 51 单片机介绍.....	17
3.1.1 单片机简介.....	17
3.1.2 51 单片机简介.....	17
3.2 STC89Cxx 芯片介绍.....	19
3.2.1 STC89C52 芯片简介.....	20
3.2.2 STC89C52 芯片内部结构图.....	21
3.3 51 单片机能做什么.....	22
3.4 怎样学好 51 单片机.....	25
课后作业.....	26
第 4 章 C 语言基础.....	27
4.1 C 语言介绍.....	28
4.1.1 C51 和 C 的区别.....	29
4.2 C 语言基础知识.....	30
4.2.1 数据类型.....	30
4.2.2 C51 运算量.....	32
4.2.3 C51 运算符.....	37
4.2.4 C51 表达式及符合语句.....	41
4.2.5 C51 基本结构和相关语句.....	42
4.2.6 函数.....	49
4.2.7 C51 构造数据类型.....	51
4.3 一个简单的单片机 C 程序.....	53
课后作业.....	53
第 5 章 单片机最小系统.....	54
5.1 51 单片机最小系统构成.....	55
5.1.1 晶振电路.....	56
5.1.2 复位电路.....	56
5.1.3 电源电路.....	57
5.1.4 下载电路.....	58
课后作业.....	59

第 6 章 KEILC51 安装与工程建立.....	60
6.1 KEIL C51 软件获取.....	61
6.2 KEIL C51 软件安装.....	62
6.3 KEIL C51 软件破解.....	64
6.4 创建 51 单片机工程.....	67
课后作业.....	76
第 7 章 点亮第一个 LED 灯.....	77
7.1 51 单片机 GPIO 介绍.....	78
7.1.1 GPIO 概念.....	78
7.1.2 GPIO 结构.....	79
7.2 LED 简介.....	81
7.3 硬件设计.....	81
7.4 软件设计.....	82
7.4.1 LED 闪烁实验.....	84
7.4.2 LED 流水灯实验.....	85
7.5 实验现象.....	89
课后作业.....	90
第 8 章 蜂鸣器实验.....	90
8.1 蜂鸣器介绍.....	91
8.2 硬件设计.....	92
8.3 软件设计.....	93
8.4 实验现象.....	95
第 9 章 动态数码管实验.....	96
9.1 数码管介绍.....	97
9.1.1 多位数码管简介.....	97
9.1.2 数码管动态显示原理.....	97
9.2 74HC245 和 74HC138 芯片介绍.....	98
9.2.1 74HC245 芯片简介.....	98
9.2.2 74HC138 芯片简介.....	100
9.3 硬件设计.....	101
9.4 软件设计.....	102
9.5 实验现象.....	105
课后作业.....	105
第 10 章 独立按键实验.....	106
10.1 按键介绍.....	107
10.1.1 独立按键检测原理.....	108
10.2 硬件设计.....	108
10.3 软件设计.....	109
10.4 实验现象.....	111
第 11 章 矩阵按键实验.....	112
11.1 矩阵按键介绍.....	113
11.2 硬件设计.....	114
11.3 软件设计.....	115
11.4 实验现象.....	118

课后作业.....	119
第 12 章 单片机 IO 扩展（串转并）--74HC595.....	120
12.1 74HC595 芯片介绍.....	121
12.2 硬件设计.....	122
12.3 软件设计.....	122
12.4 实验现象.....	125
课后作业.....	126
第 13 章 LED 点阵实验.....	126
13.1 LED 点阵介绍.....	127
13.2 硬件设计.....	128
13.3 软件设计.....	129
13.3.1 LED 点阵（点亮一个点）.....	129
13.3.2 LED 点阵（显示数字）.....	132
13.3.3 LED 点阵（显示图像）.....	138
13.4 实验现象.....	141
课后作业.....	142
第 14 章 直流电机实验.....	143
14.1 直流电机介绍.....	144
14.2 ULN2003 芯片介绍.....	145
14.3 硬件设计.....	146
14.4 软件设计.....	147
14.5 实验现象.....	149
课后作业.....	149
第 15 章 中断系统.....	150
15.1 中断介绍.....	151
15.1.1 中断概念.....	151
课后作业.....	157
第 16 章 外部中断实验.....	158
16.1 外部中断介绍.....	159
16.2 外部中断配置.....	161
16.3 硬件设计.....	162
16.4 软件设计.....	163
16.5 实验现象.....	165
课后作业.....	166
第 17 章 定时器中断实验.....	166
17.1 定时器介绍.....	167
17.1.1 51 单片机定时器原理.....	168
17.2 定时器配置.....	172
17.3 硬件设计.....	173
17.4 软件设计.....	174
17.5 实验现象.....	176
课后作业.....	177
第 18 章 EEPROM-IIC 实验.....	178
18.1 I2C 介绍.....	179

18.1.1 I2C 物理层.....	179
18.1.2 I2C 协议层.....	180
18.2 AT24C02 芯片介绍.....	183
18.3 硬件设计.....	185
18.4 软件设计.....	186
18.5 实验现象.....	196
课后作业.....	197
第 19 章 DS18B20 温度传感器实验.....	198
19.1 DS18B20 介绍.....	199
19.2 硬件设计.....	204
19.3 软件设计.....	205
19.4 实验现象.....	213
课后作业.....	214
第 20 章 DS1302 时钟实验.....	215
20.1 DS1302 时钟芯片介绍.....	216
20.1.1 DS1302 简介.....	216
20.1.2 DS1302 使用.....	217
20.2 硬件设计.....	221
20.3 软件设计.....	222
20.4 实验现象.....	229
课后作业.....	230
第 21 章 红外通信实验.....	231
21.1 红外遥控介绍.....	232
21.1.1 红外线简介.....	232
21.1.2 红外遥控的原理.....	232
21.2 硬件设计.....	235
21.3 软件设计.....	235
21.4 实验现象.....	241
课后作业.....	242
第 22 章 ADC 模数转换实验.....	243
22.1 ADC 介绍.....	244
22.1.1 ADC 简介.....	244
22.1.2 ADC 转换原理.....	245
22.2 XPT2046 芯片介绍.....	247
22.3 硬件设计.....	251
22.4 软件设计.....	251
22.5 实验现象.....	257
课后作业.....	258
第 23 章 DAC 数模转换实验.....	259
23.1 DAC 介绍.....	260
23.1.1 DAC 简介.....	260
23.1.2 DAC 工作原理.....	261
23.2 PWM 介绍.....	262
23.3 硬件设计.....	263

23.4 软件设计.....	264
23.5 实验现象.....	267
课后作业.....	268
第 24 章 LCD1602 液晶显示实验.....	269
24.1 LCD1602 介绍.....	270
24.1.1 LCD1602 简介.....	270
24.1.2 LCD1602 常用指令.....	272
24.1.3 LCD1602 使用.....	273
24.2 硬件设计.....	275
24.3 软件设计.....	275
24.4 实验现象.....	282
课后作业.....	283
第 25 章 LCD12864 液晶显示实验.....	284
25.1 LCD12864 介绍.....	285
25.1.1 LCD12864 简介.....	285
25.1.2 MiniLCD12864 介绍.....	286
25.2 硬件设计.....	286
25.3 软件设计.....	288
25.3.1 显示文字.....	289
25.3.2 显示图片.....	303
25.4 实验现象.....	306
课后作业.....	307

第 1 章 如何使用本攻略

学习本开发攻略主要参考的文档有《STC89Cxx 中文参考手册》，这是 STC 官方手册，里面包含了 STC89Cxx 单片机内部所有资源介绍，非常详细。大家在学习 51 单片机的时候可以参考下这个文档，特别是涉及到外设寄存器部分。该文档在光盘的“\6--开发板芯片资料\STC89Cxx 中文参考手册.pdf”。在后面具体的章节中也会提到所要参考的文档等提示信息。

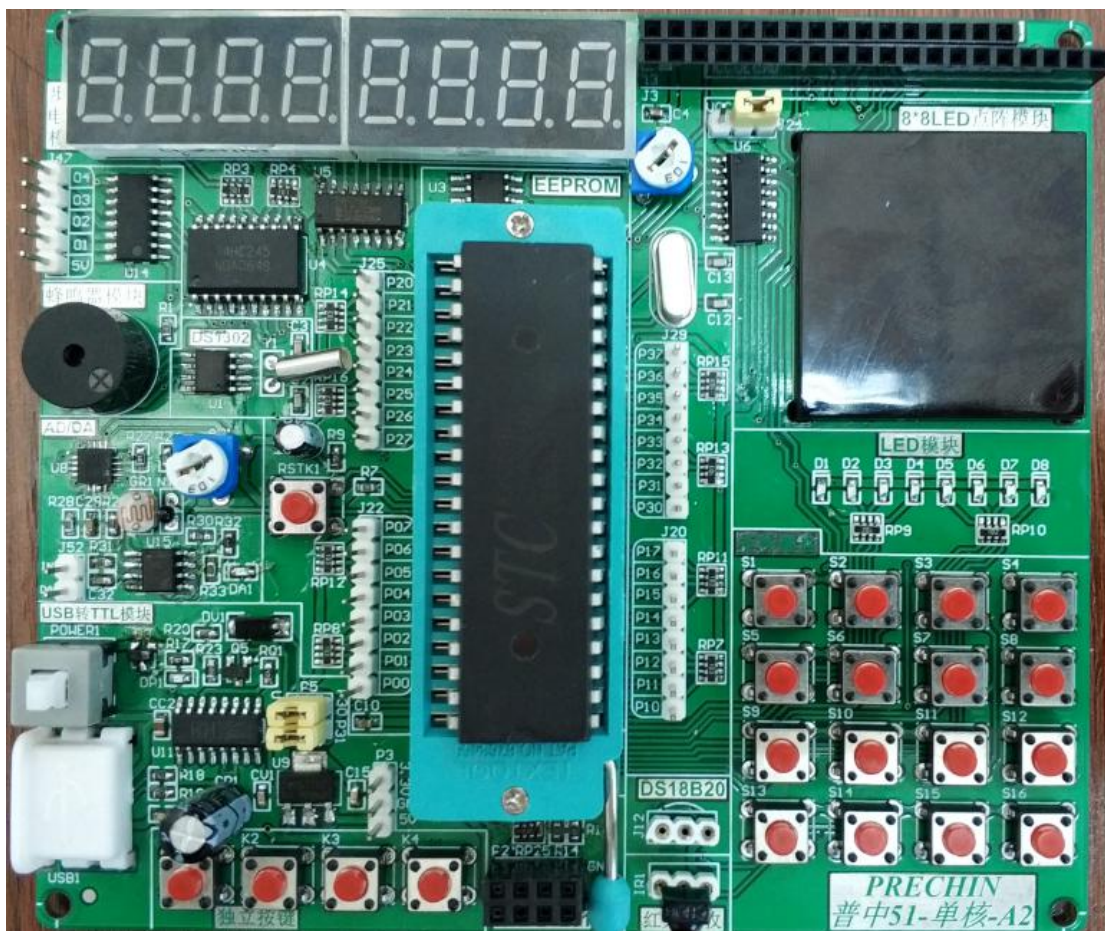
本攻略编写风格是：

- (1) 外设介绍，包括外设内部结构框图等
- (2) 外部芯片使用方法讲解
- (3) 硬件讲解
- (4) 软件分析
- (5) 实验现象

通过上述几大块的介绍让您彻底掌握 51 单片机开发。

本开发攻略配套的实验平台为：**普中 51-单核-A2 开发板**，这一款开发板采用的是单 CPU 设计，用的是 STC 公司生产的经典 51 内核芯片 STC89C52，这是一款拥有 64KB FLASH 超大存储器的 51 单片机，可容纳更大更复杂的程序，而且本款开发板部分 I/O 口与部分模块相互关联，省去了接线的麻烦，并且我们配备了强大的外围设备，如：LCD1602、LCD12864、DS18B20 等，还配备了学习光盘资料让 51 单片机的初学者可以更快的上手学习。STC89C52 不仅适合初学者对 51 单片机的入门学习，也适合学习者深入学习 51 单片机，该芯片通常用来开发生活中实际产品，具有非常强的实用、实战价值，而且只要您学会了 STC89CXX 的开发，将来在通往 STM32 嵌入式或嵌入式单片机的学习路上会更加的得心应手。

普中 51-单核-A2 开发板外观图如下：（此图没有采用包装拍摄，实际产品比此图更好看）



在学习的过程中，如果遇到什么问题，可以到我们公司论坛：
www.prechin.net 发帖交流，也可联系我们技术电话：0755-36564227，我们共同进步。

鉴于作者水平有限，难免会有纰漏，还请热心的读者指正并发到论坛或者邮箱 1819370695@qq.com，好让我们改进，祝您生活学习愉快。在 51 单片机的学习过程中，普中与您同行！

深圳普中科技有限公司

官方网站：www.prechin.cn
技术论坛：www.prechin.net
咨询电话：0755-36564227

第 2 章 开发板功能及使用介绍

本章将向大家介绍普中 51-单核-A2 开发板和与之教程兼容的其他几款开发板（这里以普中 51-双核-A6 开发板介绍）的功能及使用方法，通过本章的学习，让大家能快速上手开发板的学习。本章分为如下几部分内容：

2.1 开发板功能介绍

2.2 开发板使用方法

普中 51 单片机开发板

2.1 开发板功能介绍

本套教程可兼容普中 51-单核-A2、51-双核-A6、普中 51-双核-A7、HC6800-EM3 V3.0、HC6800-EM3 V2.2、HC6800-EM3 V2.2C 开发板，普中 51-双核-A6 开发板与普中 51-双核-A7、HC6800-EM3 V3.0、HC6800-EM3 V2.2、HC6800-EM3 V2.2C 开发板内的模块设计绝大多数一样，只是部分模块设计不同而已，比如普中 51-双核-A6，A2 开发板采用的是 STC89C52 和 STC8A8K64S4A12 双 51 内核 CPU 设计，对于普中 51-单核-A2 等，默认使用的是 STC89C52 单 CPU 设计，其内部模块与普中 51-双核-A6 开发板几乎一样，只是一个单 CPU 设计，一个是双 CPU 设计，而且普中 51-单核-A2 开发板部分 I/O 口与部分模块相互关联，而其它部分开发板 I/O 口连接完全开放，所以这部分开发板可以按照普中 51-单核-A2、HC6800-EM3 V3.0、HC6800-EM3 V2.2、HC6800-EM3 V2.2C 开发板的实验接线方式操作，因此教程也就通用。

2.1.1 普中 51-单核-A2 开发板

本款开发板采用的是单 CPU 设计，采用的是 STC 推出的新一代超强抗干扰/高速/低功耗的 51 单片机 STC89C52，它是采用 8051 核的 ISP (In System Programming) 在系统可编程芯片，指令代码完全兼容传统的 8051 单片机，最高工作时钟频率为 80MHz，片内含 4K Bytes 的可反复擦写 1000 次的 Flash 只读程序存储器，器件兼容标准 MCS-51 指令系统及 80C51 引脚结构，芯片内集成了通用 8 位中央处理器和 ISP Flash 存储单元，具有在系统可编程 (ISP) 特性，配合 PC 端的控制程序即可将用户的程序代码下载进单片机内部，省去了购买通用编程器，而且速度更快。STC89C52 系列单片机是单时钟/机器周期 (1T) 的兼容 8051 内核单片机，是高速/低功耗的新一代 8051 单片机，全新的流水线/精简指令集结构，内部集成 MAX810 专用复位电路，不管是 51 单片机的初学者还是想深入学习 51 单片机的研究者，STC89C516 都能满足。普中 51-双核-A6 开发板各功能模块如下：



按照模块序号，我们依次进行介绍：

序号	模块	功能描述
1	五线四相步进电机驱动模块	使用 ULN2003 芯片, 可驱动直流电机、五线四相步进电机等
2	动态数码管模块	使用 74HC245 芯片和和 74HC138 译码器驱动 2 个四位一体共阴数码管
3	LCD1602/LCD12864 液晶接口	连接 LCD1602 液晶屏或连接 LCD12864 液晶屏
4	EEPROM 模块	使用 AT24C02 芯片, 存储容量为 256 字节, 可实现 IIC-EEPROM 功能, 存储的数据掉电不丢失
5	8*8 单色 LED 点阵模块	可独立控制 LED 显示数字、字符、简单汉字图形等
6	蜂鸣器模块	使用无源蜂鸣器, 可实现报警提示等功能

7	DS1302 时钟模块	使用 DS1302 时钟芯片，可实现数字时钟功能
8	STC89C52 单片机接口座和 I/O 管脚	可固定单片机，并将单片机 I/O 口全部引出，方便用户二次开发
9	LED 流水灯模块	使用 8 个小灯，可实现 LED 流水灯控制
10	ADC 模块	使用 XPT2046 芯片，可实现模拟信号采集转换，可设计简易电压表等
11	DAC (PWM) 模块	使用 LM358 芯片，可实现模拟信号输出、PWM 控制
12	USB 转 TTL 串口模块	使用 CH340 芯片，可实现 USB 转 TTL 串口功能，既可下载程序，又可实现串口通信
13	矩阵键盘模块	使用 4*4 矩阵键盘，可实现键盘输入控制
14	独立按键模块	使用 4 个按键，可实现按键控制
15	NRF24L01 模块接口	支持 NRF24L01 模块，可实现 2.4G 无线
16	DS18B20 接口	可接入 DS18B20 温度传感器进行温度检测
17	红外接收模块	使用一体化红外接收头，可实现红外遥控通信
18	复位按键	系统复位按钮

2.2 开发板使用方法

2.2.1 CH340 驱动安装

上一节，我们已经介绍了开发板各个模块的功能，下面我们看看如何使用这款开发板。

首先，拿到开发板，我们要安装下 USB 转串口 CH340 驱动，对于大多数电脑系统，将 USB 线连接电脑和开发板的 USB 接口后会自动检测安装 CH340 驱动，如果您的电脑没有自动安装 CH340 驱动，没关系，我们可以手动安装，打开光盘目录“\3--开发工具\2. 开发板驱动\开发板 USB 转串口 CH340 驱动”，如下：



双击 CH341SER 应用程序，出现如下界面，点击安装即可。

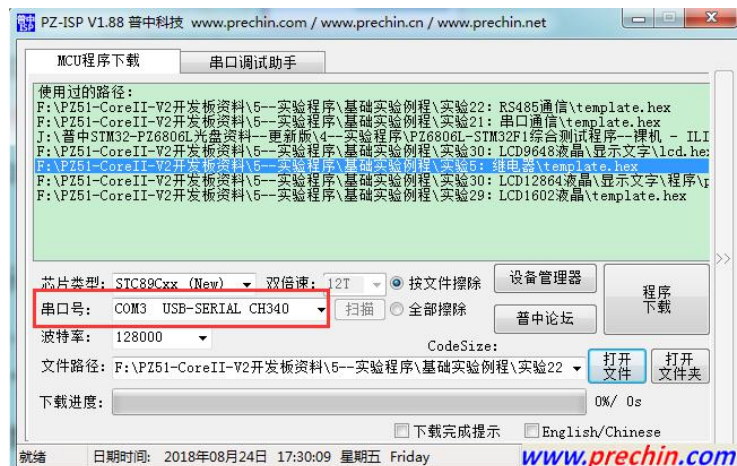


一段时间后，如果安装成功会显示如下界面：（前提：必须使用 USB 线将电脑 USB 口和开发板 USB 接口连接）



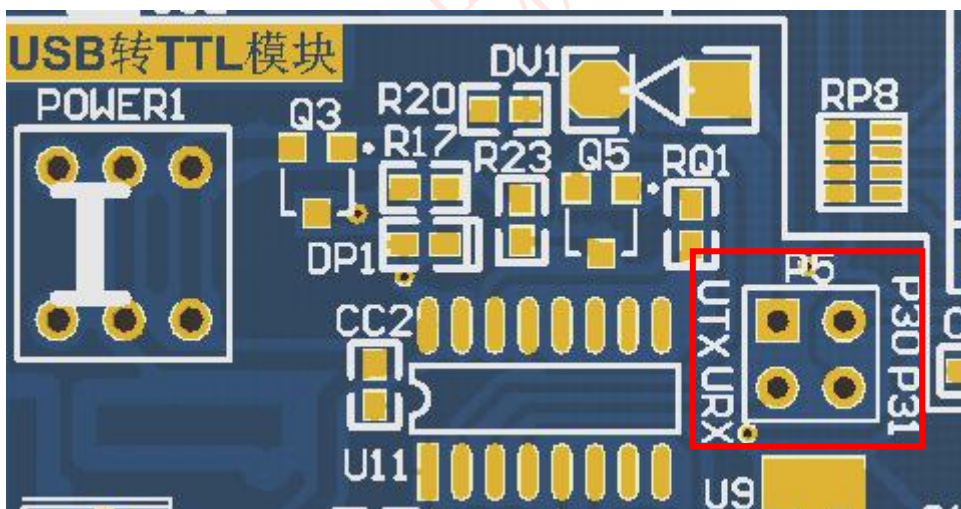
如果显示“驱动预安装成功”或者“驱动安装失败”等提示信息，表明驱动安装不成功。这时可以打开光盘目录“\3--开发工具\2. 开发板驱动\驱动安装失败解决方法”，安装对应的驱动。如果还是安装失败，可以重新换条 USB 线（支持安卓手机数据线）再次安装测试，如果还是安装失败，还可以下载一个驱动精灵，让其自动检测硬件驱动，一般通过这几个步骤都是可以解决驱动安装失败等情况，假如还是安装失败，请联系我们技术电话：0755-36564227，或者到我们公司论坛：www.prechin.net 发帖咨询，我们技术看到后会第一时间给您处理。

驱动安装成功后可以打开打开光盘目录“\3--开发工具\3. 程序烧入软件\普中科技烧写软件（推荐使用）\PZ-ISP 自动下载软件.exe”软件，查看串口号是否显示有“CH340”字样的串口，如果有则证明驱动安装成功，否则失败。如下：



2.2.2 程序的烧录（下载）

其次，安装好 CH340 驱动后，我们就可以下载程序了，在下载程序前先确认下开发板上的 USB 转 TTL 串口模块上的 P5 短接片是否短接（P31T 与 URXD 连接，P30R 与 UTXD 连接），出厂的时候该短接片默认已经短接好，如下所示：

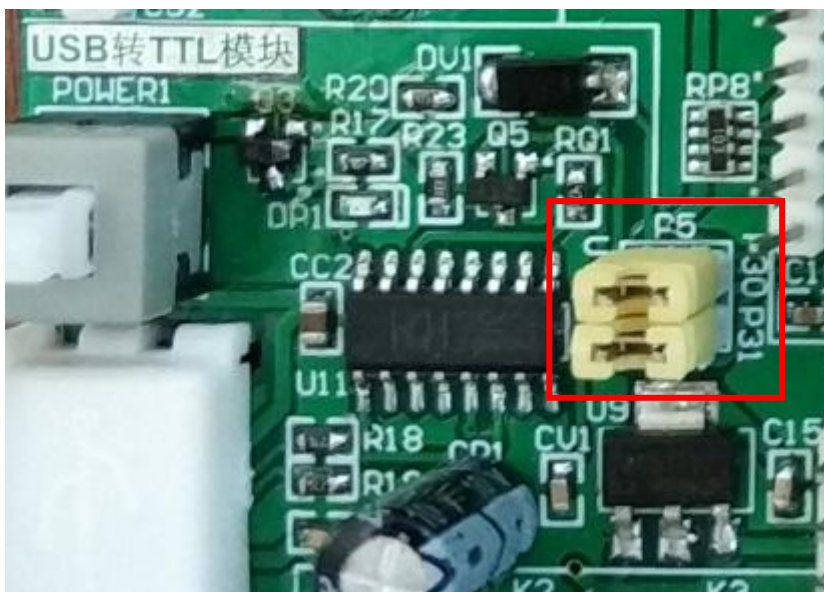


可以使用本公司开发的自动下载软件，软件在光盘目录“\3--开发工具\3.程序烧入软件”内，可以看到里面有两个烧写软件，一个是“STC-ISP 下载”软件，另一个是我们公司自动下载软件。STC-ISP 下载软件是需要冷启动，即先点击下载然后开启电源，操作较为复杂，所以不推荐使用该软件。而本公司自主研发的自动下载软件，只需一键即可下载，操作非常简单，推荐大家使用该软件下载程序。

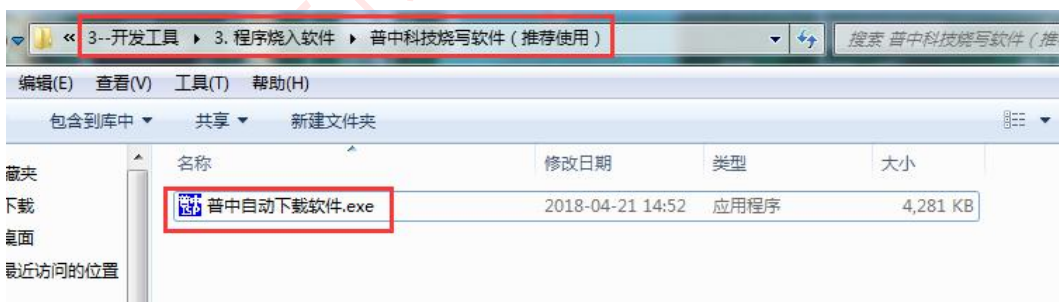
注意：要下载程序，前提条件是你必须使用 USB 线将开发板和电脑连接好，

且安装好 CH340 驱动，有关驱动的安装可参考“\1--新手必看\普中 51-双核-A6 开发板入门教程.pdf”。下面我们介绍如何给普中 51-单核-A2 开发板下载程序。

1、首先我们需要将开发板上的 USB 转 TTL 模块的 P5 端子使用短接片短接好，出厂的时候开发板默认已经短接好（URXD 与 P31T，UTXD 与 P30R），用户无需改动，如下图：



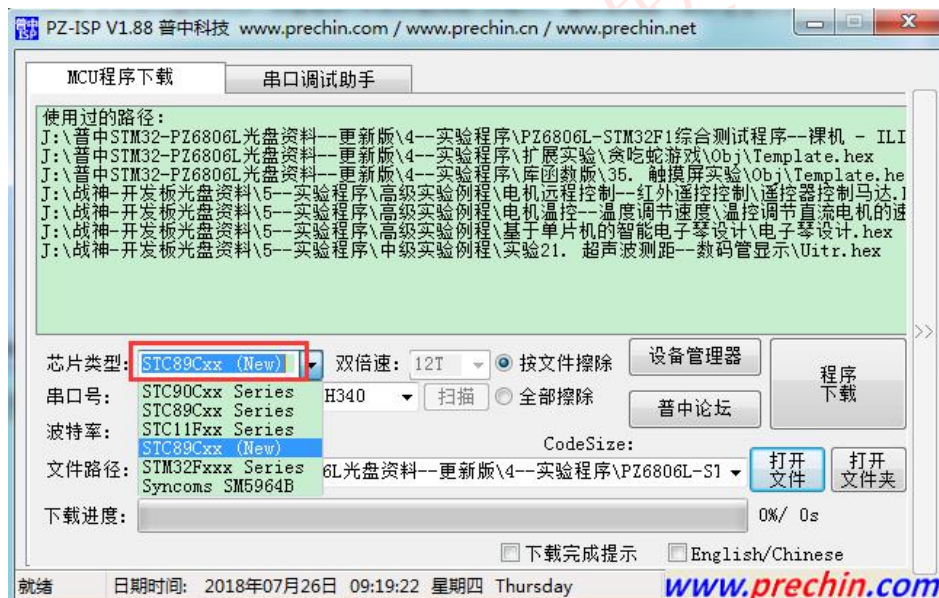
2、然后打开光盘“\3--开发工具\3. 程序烧入软件\普中科技烧写软件（推荐使用）\普中自动下载软件.exe”，如下图：



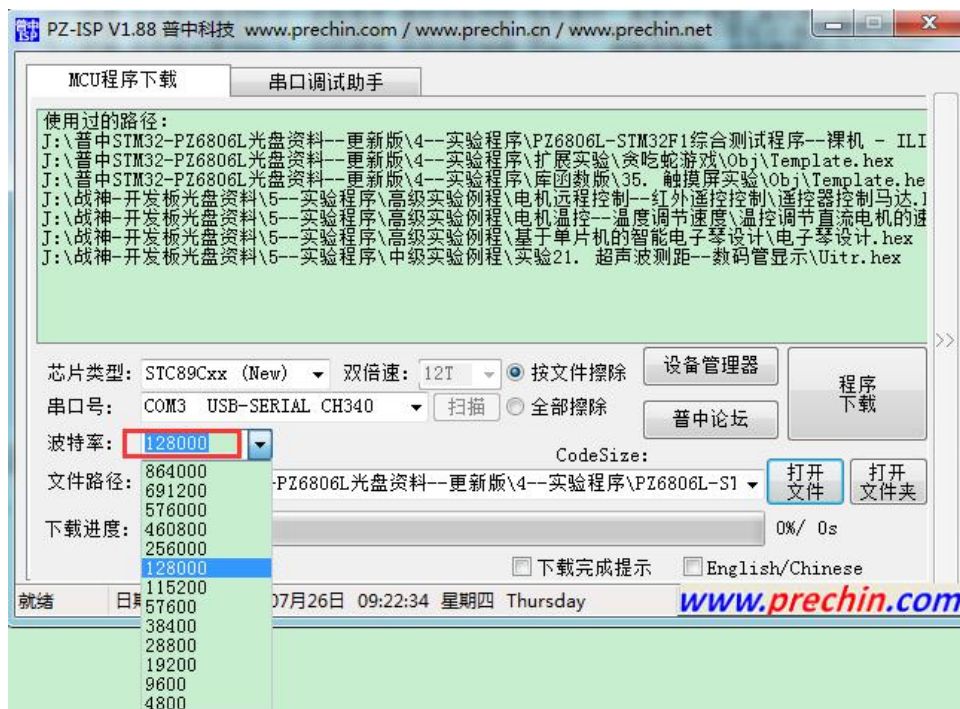
鼠标双击该软件，弹出如下界面：（注意软件的版本号）（注意：此时默认你已经安装好了 CH340 驱动，此时可以看到对应的串口号，我们这里显示的是“COM3 USB-SERIAL CH340”，你们电脑不一定是这个串口）



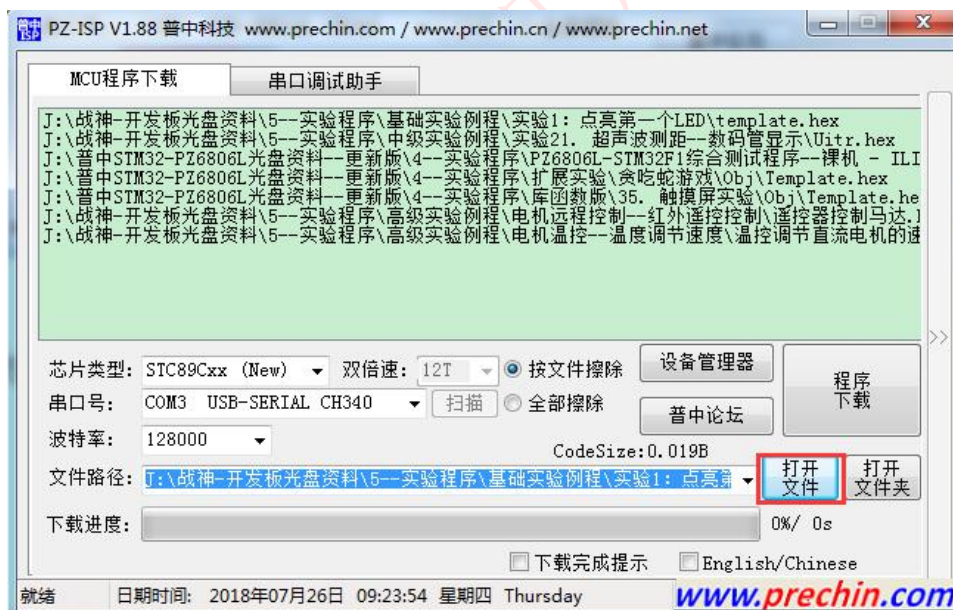
3、选择芯片类型为“STC89Cxx (New)”，如下图所示：



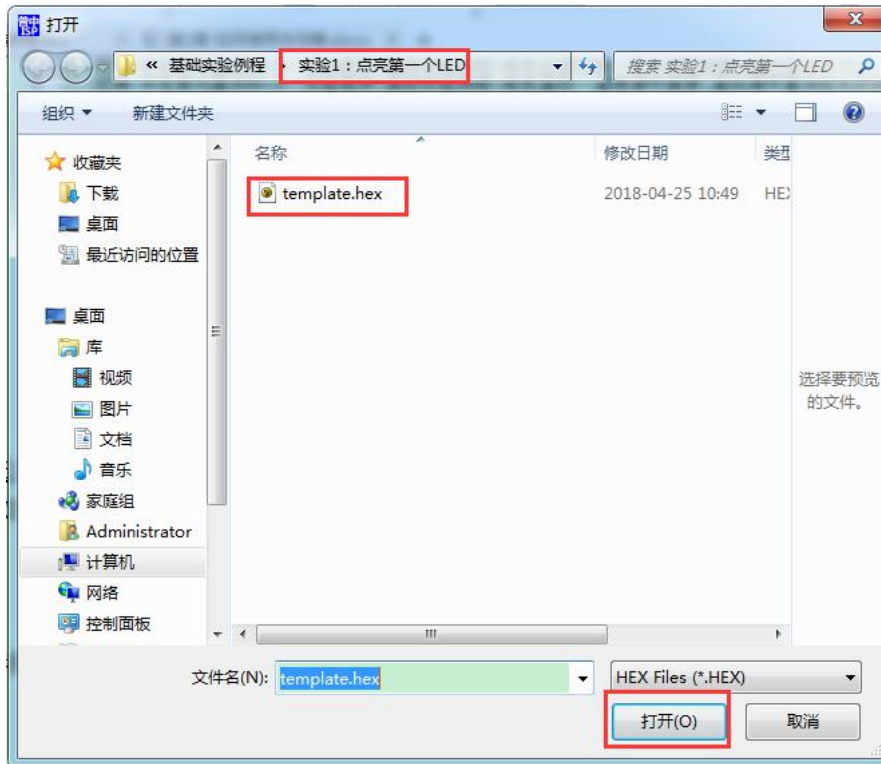
4、将波特率设置为“12800”（如果发现此波特率下载速度比较慢，可以提高波特率，如果下载失败，可以把波特率降低，总之选择一个能下载的波特率），如下图所示：



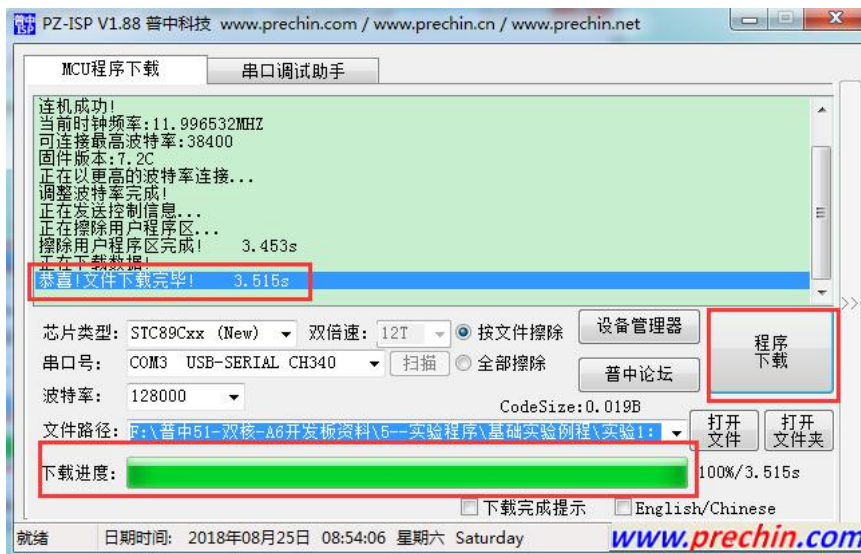
其他的选项我们保持默认设置。下面点击“打开文件”，操作如下：



选择我们实验程序的 HEX 文件，点击“打开”即可选择好需要下载的程序，操作如下：



点击“程序下载”按钮即可完成程序下载。当程序下载完成会提示程序下载成功，操作如下：



2.2.3 51 仿真器调试

开发板还支持普中 51 仿真器在线仿真调试（前提需购买 51 仿真器），但是需要在 KEIL 软件内设置，在光盘的“\3--开发工具\5. 51 仿真器驱动的安装”内也有相应的教程。

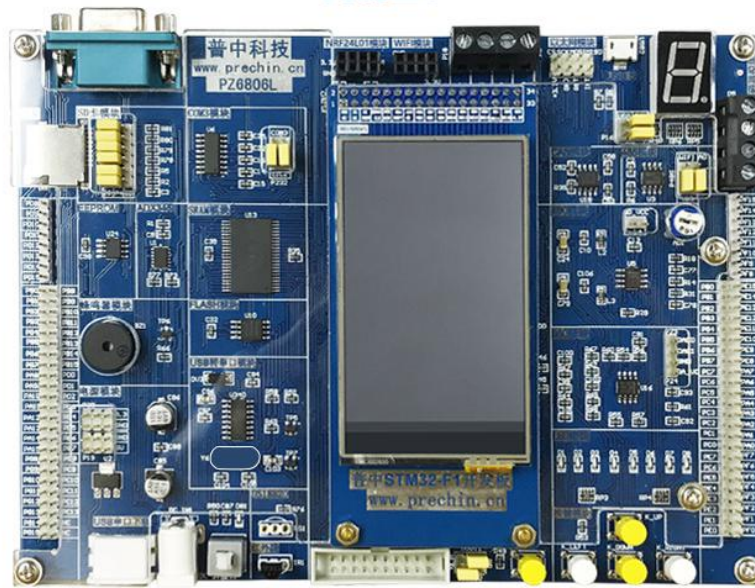
2.2.4 接线方法及实验现象

最后，程序下载成功后即可观察到实验现象。在后面章节的实验中，本教程都有一节“实验现象”介绍，大家在做实验时可以打开看下。

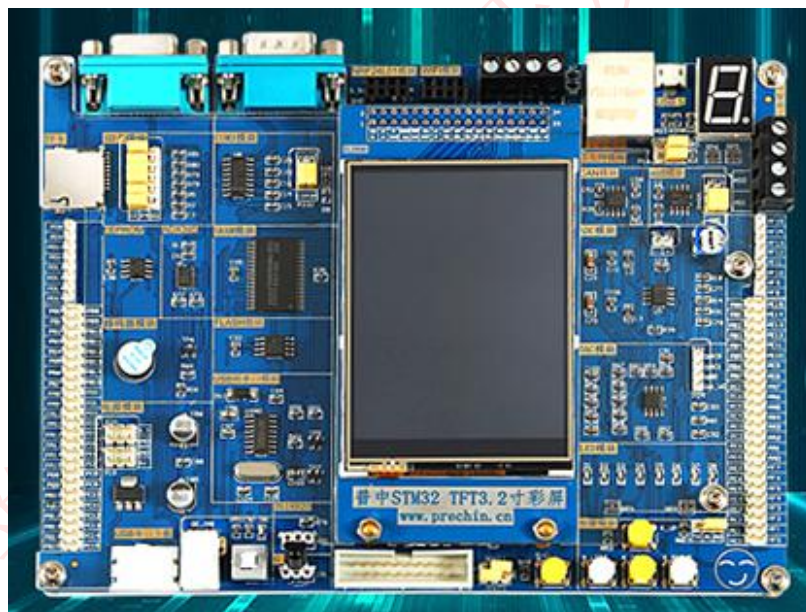
本套教程针对的是 51 单片机开发讲解，对于 STM32F1 或 STM32F4 开发，大家可以参考普中 STM32-PZ6806L/STM32-PZ6806D 和 PZ6808L-F4 开发板教程，STM32-PZ6806L/STM32-PZ6806D 是针对 STM32F1 的开发，而 PZ6808L-F4 是针对 STM32F4 的开发，这几款 STM32 开发板都是非常具有针对性的，对于学完 51 或 STM8 单片机后想学习 STM32 是非常不错的选择，特别是 PZ6808L-F4 开发板，它采用双 CPU 设计，使用了 STM32F407ZGT6 和 STM32F103C8T6 芯片。即可学习 STM32F1 又可学习 STM32F4。

它们的外观图如下：

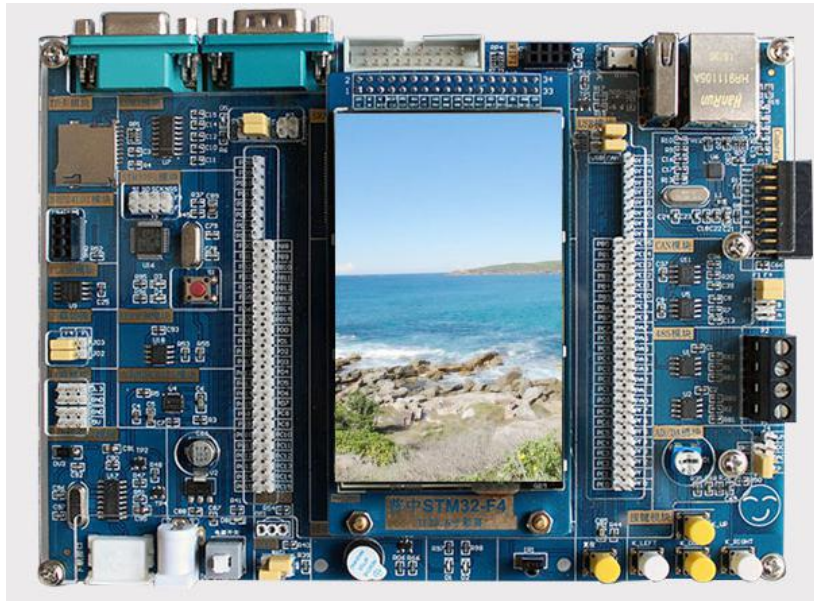
①STM32-PZ6806L 开发板



②STM32-PZ6806D 开发板



③STM32-PZ6808L-F4 开发板



第 3 章 51 单片机介绍

到目前为止，在电子界，如果想进入微控制器领域的学习，相信绝大多数的人会认为要从 51 单片机开始。不错，在众多微控制器中，51 单片机确实是非常适合初学者入门学习的，因此很有必要先认识和了解下 51 单片机。本章将向大家介绍 51 单片机，通过本章的学习，让大家对 51 单片机有一个深入的了解，为后面 51 单片机的学习做好铺垫。本章分为如下几部分内容：

- 3.1 51 单片机介绍
- 3.2 STC89Cxx 芯片介绍
- 3.3 51 单片机能做什么
- 3.4 怎样学好 51 单片机

3.1 51 单片机介绍

3.1.1 单片机简介

一台能够工作的计算机要有这样几个部份构成：CPU（进行运算、控制）、RAM（数据存储）、ROM（程序存储）、输入/输出设备（例如：串行口、并行输出口等）。在个人计算机上这些部份被分成若干块芯片，安装一个称之为主板的印刷线路板上。而在单片机中，这些部份全部被做到一块集成电路芯片中了，所以就称为单片机（也称微控制器 MCU），而且有一些单片机中除了上述部份外，还集成了其它部份如 A/D，D/A 等。体积不大，一般用 40 脚封装，当然功能多一些单片机也有引脚比较多的，如 68 引脚，功能少的只有 10 多个或 20 多个引脚，有的甚至只 8 只引脚。

3.1.2 51 单片机简介

51 单片机是对所有兼容 Intel 8031 指令系统的单片机的统称。该系列单片机的始祖是 Intel 的 8004 单片机，后来随着 Flash rom 技术的发展，8004 单片机取得了长足的进展，成为应用最广泛的 8 位单片机之一，其代表型号是 ATME1 公司的 AT89 系列，它广泛应用于工业测控系统之中。很多公司都有 51 系列的兼容机型推出，今后很长的一段时间内将占有大量市场。51 单片机是基础入门的一个单片机，还是应用最广泛的一种。需要注意的是 51 系列的单片机一般不具备自编程能力。

80C51 是 MCS-51 系列中的一个典型品种；其它厂商以 8051 为基核开发出的 CMOS 工艺单片机产品统称为 80C51 系列。当前常用的 80C51 系列单片机主要产品有：

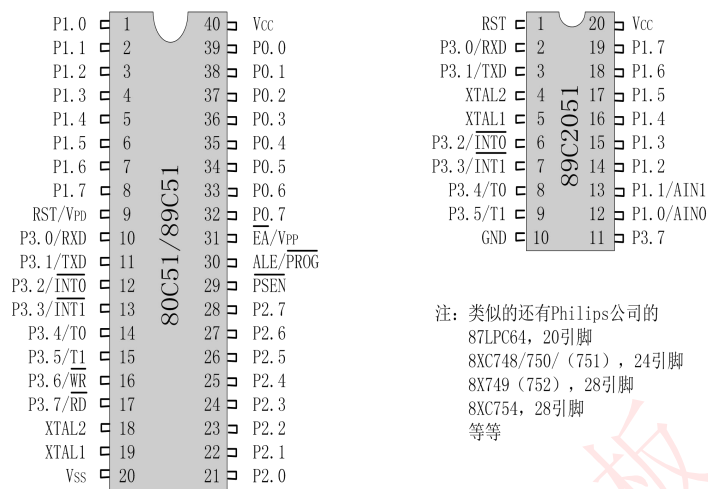
Intel(英特尔)的：80C31、80C51、87C51，80C32、80C52、87C52 等；

ATMEL(艾德梅尔)的：89C51、89C52、89C2051，89S51（RC），89S52（RC）等；

Philips(飞利浦)、华邦、Dallas(达拉斯)、Siemens(西门子)等公司的许多产品；

STC(国产宏晶)单片机：89c51、89c52、89c516、90c516 等众多品牌。

80C51 芯片管脚图如下所示：



上图中左图是 51 单片机非常经典的管脚图，通常采用的是 DIP-40 封装。

其内部资源如下：

- 8 位 CPU
- 4kbytes 程序存储器 (ROM) (52 为 8K)
- 128bytes 的数据存储器 (RAM) (52 有 256bytes 的 RAM)
- 32 条 I/O 口线
- 111 条指令，大部分为单字节指令
- 21 个专用寄存器
- 2 个可编程定时/计数器
- 5 个中断源，2 个优先级 (52 有 6 个)
- 一个全双工串行通信口
- 外部数据存储器寻址空间为 64kB
- 外部程序存储器寻址空间为 64kB
- 逻辑操作位寻址功能
- 双列直插 40PinDIP 封装
- 单一+5V 电源供电

CPU：由运算和控制逻辑组成，同时还包括中断系统和部分外部特殊功能寄存器；

RAM：用以存放可以读写的数据，如运算的中间结果、最终结果以及欲显示

的数据；

ROM：用以存放程序、一些原始数据和表格；

I/O 口：四个 8 位并行 I/O 口，既可用作输入，也可用作输出

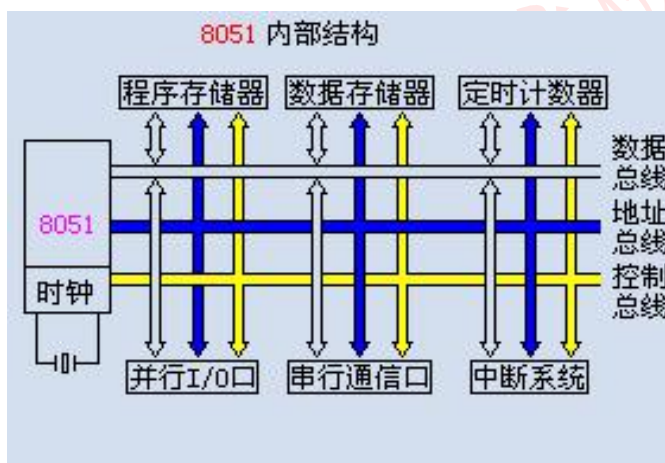
T/C：两个定时/计数器，既可以工作在定时模式，也可以工作在计数模式；

五个中断源的中断控制系统；

一个全双工 UART（通用异步接收发送器）的串行 I/O 口，用于实现单片机之间或单片机与微机之间的串行通信；

片内振荡器和时钟产生电路，石英晶体和微调电容需要外接。最佳振荡频率为 6M—12M。

其内部结构图如下所示：



总线（BUS）是单片机各部件之间传送信息的公共通道。单片机中有内部总线和外部总线两类，内部总线是 CPU 内部之间的连线，外部总线是指 CPU 与其它部件之间的连线；外部总线有三种：数据总线 DB（Data Bus），地址总线 AB（Address Bus）和控制总线 CB（Control Bus）。

更多关于 51 单片机的相关知识，大家可以到百度上搜索，51 单片机的资料是非常多的，可以说在 MCU 中没有比它更多更详细的了。

3.2 STC89Cxx 芯片介绍

前面介绍了很多款基于 MCS-51 内核设计的 51 单片机，这么多款 51 单片机是不是我们都要去学习呢？当然不是，既然他们的内核指令都是一样的，那我们只需要学会一款典型并具代表性的 51 单片机即可，其他的都是可以兼容的。

早期很长一段时间，51 单片机大部分使用的是 AT89C51 或者 AT89C52。随着我国芯片技术的快速发展，宏晶公司推出的增强型 51 单片机 STC89Cxx/STC90Cxx 等系列更受大众喜爱，除内部资源及功能大大增强外，还有一个非常重要的是它支持 ISP（在系统可编程）/IAP（在应用可编程），无需专用编程器或专用仿真器。宏晶公司推出的 STC51 芯片种类非常多，我们只需选择一款经典的学习即可。我们开发板上使用的是 STC89C52 或者 STC89C516，这两款芯片以及 STC89Cxx 或者 STC90Cxx 除 Flash 和 RAM 容量差异外，内部资源功能几乎一样，这里我们就以 STC89C516 芯片来给大家介绍。

3.2.1 STC89C52 芯片简介

STC89C52 是 STC 推出的新一代超强抗干扰/高速/低功耗的 51 单片机，它是采用 8051 核的 ISP（In System Programming）在系统可编程芯片，指令代码完全兼容传统的 8051 单片机，最高工作时钟频率为 80MHz，片内含 4K Bytes 的可反复擦写 1000 次的 Flash 只读程序存储器，器件兼容标准 MCS-51 指令系统及 80C51 引脚结构，芯片内集成了通用 8 位中央处理器和 ISP Flash 存储单元，具有在系统可编程（ISP）特性，配合 PC 端的控制程序即可将用户的程序代码下载进单片机内部，省去了购买通用编程器，而且速度更快。STC89C516 系列单片机是单时钟/机器周期(1T)的兼容 8051 内核单片机，是高速/低功耗的新一代 8051 单片机，全新的流水线/精简指令集结构，内部集成 MAX810 专用复位电路。其特性功能主要如下：

- 1：增强型 8051 单片机，6 时钟/机器周期和 12 时钟/机器周期可任意选择，指令代码完全兼容传统 8051。
- 2：工作电压： 5.5V - 3.3V（5V 单片机） / 3.6V - 2.0V（3V 单片机）
- 3：工作频率范围： 0~40MHz，相当于普通 8051 的 0~80MHz，实际工作频率可达 48MHz
- 4：用户应用程序空间： 4K / 8K / 13K / 16K / 32K / 64K 字节（STC89C516）
- 5：片上集成 1280 字节或 512 字节或 256 字节 RAM
- 6：通用 I/O 口(35/39 个)， 复位后为：P1/P2/P3/P4 是准双向口/弱上拉（普通 8051 传统 I/O 口）； P0 口是开漏输出，作为总线扩展用时，不用加上拉电阻，

作为 I/O 口用时，需加上拉电阻。

7: ISP（在系统可编程） / IAP（在应用可编程），无需专用编程器，无需专用仿真器，可通过串口（RxD/P3.0, TxD/P3.1）直接下载用户程序，数秒即可完成一片。

8: 有 EEPROM 功能

9: 看门狗

10: 内部集成 MAX810 专用复位电路（HD 版本和 90C 版本才有），外部晶体 20M 以下时，可省外部复位电路, 复位脚可直接接地。

11: 有 3 个 16 位定时器/计数器，其中定时器 0 还可以当成 2 个 8 位定时器使用。

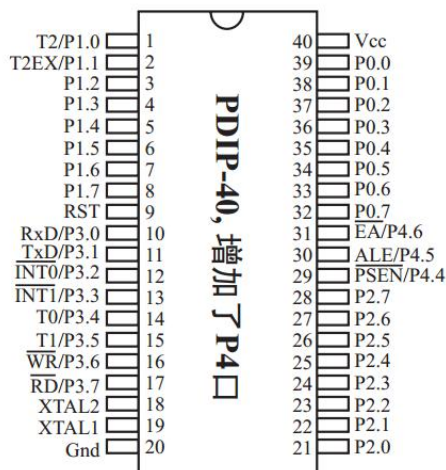
12: 外部中断 4 路, 下降沿中断或低电平触发中断, Power Down 模式可由外部中断低电平触发中断方式唤醒

13: 通用异步串行口 (UART)，还可用定时器软件实现多个 UART

14: 工作温度范围：-40 ~ +85℃（工业级） / 0 ~ 75℃（商业级）

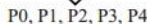
15: 封装：LQFP-44, PDIP-40, PLCC-44, PQFP-44。

该芯片管脚图如下图所示：



3.2.2 STC89C52 芯片内部结构图

STC89C516 系列单片机的内部结构框图如下图所示：



想要了解更多 STC89C5xx 系列芯片介绍的，可以百度搜索对应数据手册。在我们光盘内也提供了该芯片的数据手册，可作为 STC89C5XX 单片机学习参考。

单片机说白了，就是一种微型的电脑。只要我们稍加编程，再加上一系列的外围电子设备，就可以发挥强大的功能，接下来给大家简单介绍一下，51 单片机能干什么？

(1) 自动化技术。51 单片机属于控制类数字芯片，工业上，农业上，我们的自动化技术都可以用 51 单片机实现。



(2) 智能测量仪器。比如一些智能化的测量设备，比如我们经常使用的万用表。



(3) 家用、商用电子产品。比如我们平时用的微波炉，公司的门禁系统等等。



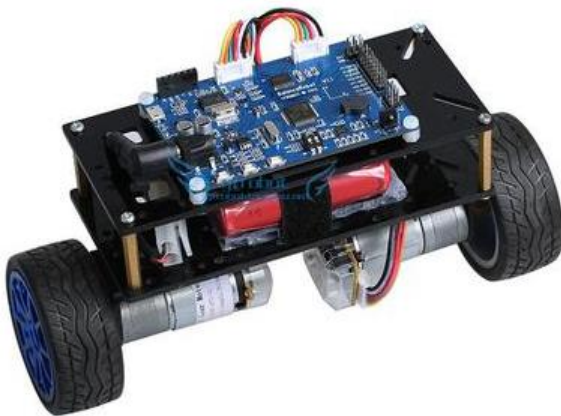
(4) 通信。比如我们上网用的“猫”，还有我们的手机，这些都可以用单片机实现。



(5) 军事领域。导弹、飞机、坦克和潜艇等都可以利用单片机来增强它们的威力。

日常生活中可见的电子产品：

智能手环，微型四轴飞行器，平衡车、扫地机、移动 POST 机，智能电饭锅，3D 打印机、机器人等。



总 之 学 好 51 单 片 机 用 处 是 大 大 的 ， 而 且 学 好 51 单 片 机 对 后 期 学

习 ARM 或者其他高级单片机帮助是非常大的。在众多 51 芯片中，STC89CXX 或者 STC90CXX 系列芯片是首选。大家只要认真学习本套教程，相信学好 51 并不是什么难事！

3.4 怎样学好 51 单片机

(1) 基本外设：

GPIO 输入输出，外部中断，定时器，串口。理解了这四个外设，基本就入门了一款 MCU。

(3) 掌握基础的数字电路和模拟电路知识

51 单片机开发中，涉及的电路并不是很复杂，对于初学者来说只需要一些基础的数电和模电知识。比如：二极管导通特性、三极管工作原理等。至于一些数字芯片，只需要学会看对应的数据手册即可。这些在我们后面章节内容也都会一一介绍。

(2) 深入学习部分：

对 51 单片机来说，定时器、中断方面的使用是难点，所以该部分内容一定要多学习几遍，有不明白的就要学会查询资料，可以百度也可以咨询我们技术。当把难点部分研究透了，相信 51 单片机的相关开发都会游刃有余。

(3) C 语言能力要加强

C 语言是嵌入式开发的基础中的基础。如果 C 语言不过关，大大限制嵌入式学习进度，以及嵌入式学习深度。嵌入式高手都是 C 高手。选择谭浩强的《C 程序设计 第四版》入门及巩固 C 语言基础是不错的，对于 51 单片机的开发来说，基本上掌握 C 语言基础知识部分即可，对 C 语言功底要求不是太高。但如果想要使用高端单片机开发项目，仅靠这本书远远不够，可以参考《C 与指针》、《C 指针编程之道》等。我们已经在光盘“\7--单片机相关资料\C 语言加油站”内提供给大家。

(4) 多动手编程

请一定要记住：编程能力是练出来的，不是看出来的。很多初学者在看视频时，程序的编写都能看懂，等到脱离视频自己动手编写时发现不知道怎么去编写，而且编写的一些语句有错误，比如字母大小写问题，关键字写错等问题。

(5) 遇到问题多百度多浏览论坛

在学习过程中不可能是一帆风顺的，会经常遇到各种各样的问题，有问题了先要学会自己去思考解决，可以多百度或者去一些比较大的技术论坛交流。不要一遇到问题就直接联系技术，即使技术帮你解决了，过段时间后可能你又忘了，所以对你的技术成长没有任何帮助。这里推荐几个论坛，大家可以在里面相互交流。

普中科技论坛：www.prechin.net

51 单片机论坛：<http://bbs.2lic.com/icfilter-typeid-11-211.html>

课后作业

- (1) 搜索 51 单片机资料，了解 STC89Cxx 管脚介绍，为后面章节做好铺垫。

第 4 章 C 语言基础

在前面章节，我们已经介绍过 51 单片机的学习方法，其中非常重要的一点是要具备一定的 C 语言基础知识。考虑到部分初学者的专业问题，可能未学习过 C 语言，因此很有必要为大家介绍下 C 语言基础知识和 C51 知识，由于 C51 和 C 几乎类似，因此不去划分 C51 和 C，而统一为 C。如果有 C 语言基础的朋友可以绕过本章内容，当然还是推荐大家看下，就当做是复习。通过本章的学习，让大家能掌握 C 语言基础知识，为后面 51 单片机的编程学习做好铺垫。本章分为如下几部分内容：

- 4.1 C 语言介绍
- 4.2 C 语言基础知识
- 4.3 一个简单的单片机 C 程序

4.1 C 语言介绍

C 语言是在 70 年代初问世的。一九七八年由美国电话电报公司(AT&T) 贝尔实验室正式发表了 C 语言。同时由 B. W. Kernighan 和 D. M. Ritchie 合著了著名的“THE C PROGRAMMING LANGUAGE”一书。通常简称为《K&R》，也有人称之为《K&R》标准。但是，在《K&R》中并没有定义一个完整的标准 C 语言，后来由美国国家标准协会(American National Standards Institute) 在此基础上制定了一个 C 语言标准，于一九八三年发表。通常称之为 ANSI C。

早期的 C 语言主要是用于 UNIX 系统。由于 C 语言的强大功能和各方面的优点逐渐为人们认识，到了八十年代，C 开始进入其它操作系统，并很快在各类大、中、小和微型计算机上得到了广泛的使用，成为当代最优秀的程序设计语言之一。

在实际工程应用中，51 单片机的程序设计一般都是采用 C 语言编写，通过相应的编译器，得到可执行代码，以提高程序开发效率。但由于单片机的内部资源有限，并结合单片机的特点（如位操作），与标准 C 语言相比有所不同，称为 C51 程序。

C 语言的特点

- 语言简洁、紧凑，使用方便、灵活。
- 运算符丰富。
- 数据结构丰富。具有现代化语言的各种数据结构。
- 可进行结构化程序设计。
- 可以直接对计算机硬件进行操作。
- 生成的目标代码质量高，程序执行效率高。
- 可移植性好。

C 语言程序采用函数结构，每个 C 语言程序由一个或多个函数组成，在这些函数中至少应包含一个主函数 main()，也可以包含一个 main() 函数和若干个其它的功能函数。不管 main() 函数放于何处，程序总是从 main() 函数开始执行，执行到 main() 函数结束则结束。在 main() 函数中调用其它函数，其它函数也可以相互调用，但 main() 函数只能调用其它的功能函数，而不能被其它的函数所调用。功能函数可以是 C 语言编译器提供的库函数，也可以是由用户定义的自定义函数。在编制 C 程序时，程序的开始部分一般是预处理命令、函数说明和变量

定义等。

用 C 语言编写 51 单片机程序与用汇编语言编写 51 单片机程序不同，汇编语言必须要考虑其存储器结构，尤其必须考虑其片内数据存储器与特殊功能寄存器的使用以及按实际地址处理端口数据。

用 C 语言编写的 51 单片机应用程序，则不用像汇编语言那样须具体组织、分配存储器资源和处理端口数据，但在 C 语言编程中，对数据类型与变量的定义，必须要与单片机的存储结构相关联，否则编译器不能正确地映射定位。

用 C 语言编写单片机应用程序与标准的 C 语言程序也有相应的区别：C 语言编写单片机应用程序时，需根据单片机存储结构及内部资源定义相应的数据类型和变量，而标准的 C 语言程序不需要考虑这些问题。

C51 包含的数据类型、变量存储模式、输入输出处理、函数等方面与标准的 C 语言有一定的区别。其它的语法规则、程序结构及程序设计方法等与标准的 C 语言程序设计相同。

4.1.1 C51 和 C 的区别

C51 的语法规则、程序结构及程序设计方法都与标准的 C 语言程序设计相同，但 C51 程序与标准的 C 程序在以下几个方面不一样：

(1) C51 中定义的库函数和标准 C 语言定义的库函数不同。标准的 C 语言定义的库函数是按通用微型计算机来定义的，而 C51 中的库函数是按 51 单片机相应情况来定义的；

(2) C51 中的数据类型与标准 C 的数据类型也有一定的区别，在 C51 中还增加了几种针对 51 单片机特有的数据类型；

(3) C51 变量的存储模式与标准 C 中变量的存储模式不一样，C51 中变量的存储模式是与 51 单片机的存储器紧密相关；

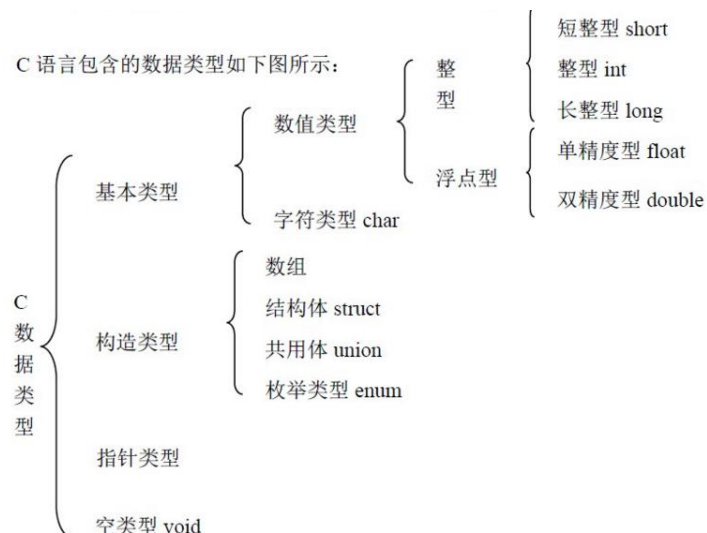
(4) C51 与标准 C 的输入输出处理不一样，C51 中的输入输出是通过 51 串行口来完成的，输入输出指令执行前必须要对串行口进行初始化；

(5) C51 与标准 C 在函数使用方面也有一定的区别，C51 中有专门的中断函数。

4.2 C 语言基础知识

4.2.1 数据类型

C 语言包含的数据类型如下图所示：



C51 的数据类型分为基本数据类型和组合数据类型，情况与标准 C 中的数据类型基本相同，但其中 char 型与 short 型相同，float 型与 double 型相同，另外，C51 中还有专门针对于 51 单片机的特殊功能寄存器型和位类型。

(1) C51 基本数据类型如下图所示：

类型	符号	关键字	所占位数	数的表示范围
整型	有	(signed) int	16	-32768~32767
		(signed) short	16	-32768~32767
		(signed) long	32	-2147483648~2147483647
	无	unsigned int	16	0~65535
		unsigned short int	16	0~65535
		unsigned long int	32	0~4294967295
实型	有	float	32	3.4e-38~3.4e38
	有	double	64	1.7e-308~1.7e308
字符型	有	char	8	-128~127
	无	unsigned char	8	0~255

(2) *指针型：

指针型本身就是一个变量，在这个变量中存放的指向另一个数据的地址。这个指针变量要占用一定的内存单元，对不同的处理器其长度不一样，在 C51 中它

的长度一般为 1~3 个字节。

（3）特殊功能寄存器型

这是 C51 扩充的数据类型，用于访问 51 单片机中的特殊功能寄存器数据，它分 sfr 和 sfr16 两种类型。其中：sfr 为字节型特殊功能寄存器类型，占一个内存单元，利用它可以访问 51 内部的所有特殊功能寄存器；sfr16 为双字节型特殊功能寄存器类型，占用两个字节单元，利用它可以访问 51 内部的所有两个字节的特殊功能寄存器。在 C51 中对特殊功能寄存器的访问必须先用 sfr 或 sfr16 进行声明。

（4）位类型

这也是 C51 中扩充的数据类型，用于访问 51 单片机中的可寻址的位单元。在 C51 中，支持两种位类型：bit 型和 sbit 型。它们在内存中都只占一个二进制位，其值可以是“1”或“0”。其中：用 bit 定义的位变量在 C51 编译器编译时，在不同的时候位地址是可以变化的，而用 sbit 定义的位变量必须与 51 单片机的一个可以寻址位单元或可位寻址的字节单元中的某一位联系在一起，在 C51 编译器编译时，其对应的位地址是不可变化的。

KEIL C51 编译器能够识别的基本数据类型：

基本数据类型	长度	取值范围
unsigned char	1 字节	0~255
signed char	1 字节	-128~+127
unsigned int	2 字节	0~65535
signed int	2 字节	-32768~+32767
unsigned long	4 字节	0~4294967295
signed long	4 字节	-2147483648~+2147483647
float	4 字节	$\pm 1.175494\text{E}-38 \sim \pm 3.402823\text{E}+38$
bit	1 位	0 或 1
sbit	1 位	0 或 1
sfr	1 字节	0~255
sfr16	2 字节	0~65535

在 C51 语言程序中，有可能会出现在运算中数据类型不一致的情况。C51 允

许任何标准数据类型的隐式转换，隐式转换的优先级顺序如下：

Bit→char→int→long→float→signed→unsigned

也就是说，当 char 型与 int 型进行运算时，先自动对 char 型扩展为 int 型，然后与 int 型进行运算，运算结果为 int 型。C51 除了支持隐式类型转换外，还可以通过强制类型转换符“()”对数据类型进行人为的强制转换。

C51 编译器除了能支持以上这些基本数据类型之外，还能支持一些复杂的组合型数据类型，如数组类型、指针类型、结构类型、联合类型等这些复杂的数据类型。对于初学者我们要求先掌握 C 语言基础知识，把基础的掌握了在学习复杂的。

4.2.2 C51 运算量

(1) 常量

常量是指在程序执行过程中其值不能改变的量。在 C51 中支持整型常量、浮点型常量、字符型常量和字符串型常量。

1， 整型常量

整型常量也就是整型常数，根据其值范围在计算机中分配不同的字节数来存放。在 C51 中它可以表示成以下几种形式：

十进制整数。如 234、-56、0 等。

十六进制整数。以 0x 开头表示，如 0x12 表示十六进制数 12H。

长整数。在 C51 中当一个整数的值达到长整型的范围，则该数按长整型存放，在存储器中占四个字节，另外，如一个整数后面加一个字母 L，这个数在存储器中也按长整型存放。如 123L 在存储器中占四个字节。

2， 浮点型常量

浮点型常量也就是实型常数。有十进制表示形式和指数表示形式。十进制表示形式又称定点表示形式，由数字和小数点组成。如 0.123、34.645 等都是十进制数表示形式的浮点型常量。指数表示形式为：[] 数字 [. 数字] e [] 数字

例如：123.456e-3、-3.123e2 等都是指数形式的浮点型常量。

3， 字符型常量

字符型常量是用单引号引起的字符，如 ‘a’、‘1’、‘F’ 等。可以是可

显示的 ASCII 字符，也可以是不可显示的控制字符。对不可显示的控制字符须在前面加上反斜杠“\”组成转义字符。利用它可以完成一些特殊功能和输出时的格式控制。常用的转义字符如下表所示。

转义字符	含 义	ASCII 码（十六进制数）
\0	空字符（null）	00H
\n	换行符（LF）	0AH
\r	回车符（CR）	0DH
\t	水平制表符（HT）	09H
\b	退格符（BS）	08H
\f	换页符（FF）	0CH
\'	单引号	27H
\"	双引号	22H
\\	反斜杠	5CH

4. 字符串型常量

字符串型常量由双引号“”括起的字符组成。如“D”、“1234”、“ABCD”等。注意字符串常量与字符常量是不一样的，一个字符常量在计算机内只用一个字节存放，而一个字符串常量在内存中存放时不仅双引号内的字符一个占一个字节，而且系统会自动的在后面加一个转义字符“\0”作为字符串结束符。因此不要将字符常量和字符串常量混淆，如字符常量‘A’和字符串常量“A”是不一样的。

（2）变量

变量是在程序运行过程中其值可以改变的量。一个变量由两部分组成：变量名和变量值。在 C51 中，变量在使用前必须对变量进行定义，指出变量的数据类型和存储模式。以便编译系统为它分配相应的存储单元。定义的格式如下：

[存储种类] 数据类型说明符 [存储器类型] 变量名 1[=初值]，变量名 2[初值]…；

1. 数据类型说明符

在定义变量时，必须通过数据类型说明符指明变量的数据类型，指明变量在存储器中占用的字节数。可以是基本数据类型说明符，也可以是组合数据类型说明符，还可以是用 typedef 定义的类型别名。

在 C51 中，为了增加程序的可读性，允许用户为系统固有的数据类型说明符用 typedef 起别名，格式如下：

typedef c51 固有的数据类型说明符 别名；

定义别名后，就可以用别名代替数据类型说明符对变量进行定义。别名可以用大写，也可以用小写，为了区别一般用大写字母表示。

【例】 typedef 的使用。

```
typedef unsigned int u8;
typedef unsigned char u16;
u8 a1=0x12;
u16 a2=0x1234;
```

2，变量名

变量名是 C51 区分不同变量，为不同变量取的名称。在 C51 中规定变量名可以由字母、数字和下划线三种字符组成，且第一个字母必须为字母或下划线。变量名有两种：普通变量名和指针变量名。它们的区别是指针变量名前面要带“*”号。

3，存储种类

存储种类是指变量在程序执行过程中的作用范围。C51 变量的存储种类有四种，分别是自动(auto)、外部(extern)、静态(static)和寄存器(register)。

a. auto:

使用 auto 定义的变量称为自动变量，其作用范围在定义它的函数体或复合语句内部，当定义它的函数体或复合语句执行时，C51 才为该变量分配内存空间，结束时占用的内存空间释放。自动变量一般分配在内存的堆栈空间中。定义变量时，如果省略存储种类，则该变量默认为自动(auto)变量。

b. extern:

使用 extern 定义的变量称为外部变量。在一个函数体内，要使用一个已在该函数体外或别的程序中定义过的外部变量时，该变量在该函数体内要用

extern 说明。外部变量被定义后分配固定的内存空间，在程序整个执行时间内都有效，直到程序结束才释放。

c. static:

使用 static 定义的变量称为静态变量。它又分为内部静态变量和外部静态变量。在函数体内部定义的静态变量为内部静态变量，它在对应的函数体内有效，一直存在，但在函数体外不可见，这样不仅使变量在定义它的函数体外被保护，还可以实现当离开函数时值不被改变。外部静态变量是在函数外部定义的静态变量。它在程序中一直存在，但在定义的范围之外是不可见的。如在多文件或多模块处理中，外部静态变量只在文件内部或模块内部有效。

d. register:

使用 register 定义的变量称为寄存器变量。它定义的变量存放在 CPU 内部的寄存器中，处理速度快，但数目少。C51 编译器编译时能自动识别程序中使用频率最高的变量，并自动将其作为寄存器变量，用户可以无需专门声明。

4. 存储器类型

存储器类型是用于指明变量所处的单片机的存储器区域情况。存储器类型与存储种类完全不同。C51 编译器能识别的存储器类型有以下几种，见表所示。

存储器类型	描 述
data	直接寻址的片内 RAM 低 128B，访问速度快
bdata	片内 RAM 的可位寻址区（20H~2FH），允许字节和位混合访问
idata	间接寻址访问的片内 RAM，允许访问全部片内 RAM
pdata	用 Ri 间接访问的片外 RAM 的低 256B
xdata	用 DPTR 间接访问的片外 RAM，允许访问全部 64k 片外 RAM
code	程序存储器 ROM 64k 空间

定义变量时也可以省“存储器类型”，省时 C51 编译器将按编译模式默认存储器类型。

5. 特殊功能寄存器变量

51 系列单片机片内有许多特殊功能寄存器，通过这些特殊功能寄存器可以控

制 51 系列单片机的定时器、计数器、串口、I/O 及其它功能部件，每一个特殊功能寄存器在片内 RAM 中都对应于一个字节单元或两个字节单元。

在 C51 中，允许用户对这些特殊功能寄存器进行访问，访问时须通过 sfr 或 sfr16 类型说明符进行定义，定义时须指明它们所对应的片内 RAM 单元的地址。格式如下：

sfr 或 sfr16 特殊功能寄存器名=地址；

sfr 用于对 51 单片机中单字节的特殊功能寄存器进行定义，sfr16 用于对双字节特殊功能寄存器进行定义。特殊功能寄存器名一般用大写字母表示。地址一般用直接地址形式。

【例】特殊功能寄存器的定义。

```
sfr PSW=0xd0;
```

```
sfr SCON=0x98;
```

```
sfr TMOD=0x89;
```

```
sfr P1=0x90;
```

```
sfr16 DPTR=0x82;
```

```
sfr16 T1=0x8A;
```

6. 位变量

在 C51 中，允许用户通过位类型符定义位变量。位类型符有两个：bit 和 sbit。可以定义两种位变量。

bit 位类型符用于定义一般的可位处理位变量。它的格式如下：

bit 位变量名；

在格式中可以加上各种修饰，但注意存储器类型只能是 bdata、data、idata。只能是片内 RAM 的可位寻址区，严格来说只能是 bdata。

sbit 位类型符用于定义在可位寻址字节或特殊功能寄存器中的位，定义时须指明其位地址，可以是位直接地址，可以是可位寻址变量带位号，也可以是特殊功能寄存器名带位号。格式如下：

sbit 位变量名=位地址；

如位地址为位直接地址，其取值范围为 0x00~0xff；如位地址是可位寻址变量带位号或特殊功能寄存器名带位号，则在它前面须对可位寻址变量或特殊功能

寄存器进行定义。字节地址与位号之间、特殊功能寄存器与位号之间一般用“^”作间隔。如定义 51 单片机管脚：

```
sbit LED=P1^0;
```

在 C51 中，为了用户处理方便，C51 编译器把 51 单片机的常用的特殊功能寄存器和特殊位进行了定义，放在一个“reg51.h”或“reg52.h”的头文件中，当用户要使用时，只须要在使用之前用一条预处理命令#include <reg52.h>把这个头文件包含到程序中，然后就可使用特殊功能寄存器名和特殊位名称。

4.2.3 C51 运算符

(1) 赋值运算符

赋值运算符“=”，在 C51 中，它的功能是将一个数据的值赋给一个变量，如 x=10。利用赋值运算符将一个变量与一个表达式连接起来的式子称为赋值表达式，在赋值表达式的后面加一个分号“；”就构成了赋值语句，一个赋值语句的格式如下：

变量=表达式；

执行时先计算出右边表达式的值，然后赋给左边的变量。例如：

```
x=8+9;    /*将 8+9 的值赋给变量 x*/
```

```
x=y=5;    /*将常数 5 同时赋给变量 x 和 y*/
```

在 C51 中，允许在一个语句中同时给多个变量赋值，赋值顺序自右向左。

(2) 算术运算符

C51 中支持的算术运算符有：

+ 加或取正值运算符

- 减或取负值运算符

* 乘运算符

/ 除运算符

% 取余运算符

加、减、乘运算相对比较简单，而对于除运算，如相除的两个数为浮点数，则运算的结果也为浮点数，如相除的两个数为整数，则运算的结果也为整数，即为整除。如 25.0/20.0 结果为 1.25，而 25/20 结果为 1。

对于取余运算，则要求参加运算的两个数必须为整数，运算结果为它们的余数。例如： $x=5\%3$ ，结果 x 的值为 2。

(3) 关系运算符

C51 中有 6 种关系运算符：

> 大于
< 小于
>= 大于等于
<= 小于等于
== 等于
!= 不等于

关系运算用于比较两个数的大小，用关系运算符将两个表达式连接起来形成的式子称为关系表达式。关系表达式通常用来作为判别条件构造分支或循环程序。关系表达式的一般形式如下：

表达式 1 关系运算符 表达式 2

关系运算的结果为逻辑量，成立为真（1），不成立为假（0）。其结果可以作为一个逻辑量参与逻辑运算。例如： $5>3$ ，结果为真（1），而 $10==100$ ，结果为假（0）。

注意：关系运算符等于“==”是由两个“=”组成。

(4) 逻辑运算符

C51 有 3 种逻辑运算符：

|| 逻辑或
&& 逻辑与
! 逻辑非

关系运算符用于反映两个表达式之间的大小关系，逻辑运算符则用于求条件式的逻辑值，用逻辑运算符将关系表达式或逻辑量连接起来的式子就是逻辑表达式。

逻辑与，格式：

条件式 1 && 条件式 2

当条件式 1 与条件式 2 都为真时结果为真（非 0 值），否则为假（0 值）。

逻辑或，格式：

条件式 1 || 条件式 2

当条件式 1 与条件式 2 都为假时结果为假（0 值），否则为真（非 0 值）。

逻辑非，格式：

! 条件式

当条件式原来为真（非 0 值），逻辑非后结果为假（0 值）。当条件式原来为假（0 值），逻辑非后结果为真（非 0 值）。

例如：若 $a=8$, $b=3$, $c=0$ ，则 $!a$ 为假， $a \&\& b$ 为真， $b \&\& c$ 为假。

(5) 位运算符

C51 语言能对运算对象按位进行操作，它与汇编语言使用一样方便。位运算是按位对变量进行运算，但并不改变参与运算的变量的值。如果要求按位改变变量的值，则要利用相应的赋值运算。C51 中位运算符只能对整数进行操作，不能对浮点数进行操作。C51 中的位运算符有：

& 按位与
| 按位或
^ 按位异或
~ 按位取反
<< 左移
>> 右移

【例】 设 $a=0x45=01010100B$ ， $b=0x3b=00111011B$ ，则 $a\&b$ 、 $a|b$ 、 $a\wedge b$ 、 $\sim a$ 、 $a\<<2$ 、 $b\>>2$ 分别为多少？

$a\&b=00010000b=0x10$ 。

$a|b=01111111B=0x7f$ 。

$a\wedge b=01101111B=0x6f$ 。

$\sim a=10101011B=0xab$ 。

$a\<<2=01010000B=0x50$ 。

$b\>>2=00001110B=0x0e$ 。

(6) 复合赋值运算符

C51 语言中支持在赋值运算符“=”的前面加上其它运算符，组成复合赋值运算符。下面是 C51 中支持的复合赋值运算符

+=	加法赋值	?+	减法赋值
*=	乘法赋值	/=	除法赋值
%=	取模赋值	&=	逻辑与赋值
=	逻辑或赋值	^=	逻辑异或赋值
~=	逻辑非赋值	>>=	右移位赋值
<<=	左移位赋值		

复合赋值运算的一般格式如下：

变量 复合运算赋值符 表达式

它的处理过程：先把变量与后面的表达式进行某种运算，然后将运算的结果赋给前面的变量。其实这是 C51 语言中简化程序的一种方法，大多数二目运算都可以用复合赋值运算符简化表示。例如：a+=6 相当于 a=a+6；a*=5 相当于 a=a*5；b&=0x55 相当于 b=b&0x55；x>>=2 相当于 x=x>>2。

(7) 逗号运算符

在 C51 语言中，逗号“，”是一个特殊的运算符，可以用它将两个或两个以上的表达式连接起来，称为逗号表达式。逗号表达式的一般格式为：

表达式 1，表达式 2，……，表达式 n

程序执行时对逗号表达式的处理：按从左至右的顺序依次计算出各个表达式的值，而整个逗号表达式的值是最右边的表达式（表达式 n）的值。例如：x=(a=3, 6*3) 结果 x 的值为 18。

(8) 条件运算符

条件运算符“?:”是 C51 语言中唯一的一个三目运算符，它要求有三个运算对象，用它可以三个表达式连接在一起构成一个条件表达式。条件表达式的一般格式为：

逻辑表达式? 表达式 1: 表达式 2

其功能是先计算逻辑表达式的值，当逻辑表达式的值为真（非 0 值）时，将计算的表达式 1 的值作为整个条件表达式的值；当逻辑表达式的值为假（0 值）

时，将计算的表达式 2 的值作为整个条件表达式的值。例如：条件表达式 $\text{max}=(a>b)?a:b$ 的执行结果是将 a 和 b 中较大的数赋值给变量 max。

(9) 指针与地址运算符

指针是 C51 语言中的一个十分重要的概念，在 C51 中的数据类型中专门有一种指针类型。指针为变量的访问提供了另一种方式，变量的指针就是该变量的地址，还可以定义一个专门指向某个变量的地址的指针变量。

为了表示指针变量和它所指向的变量地址之间的关系，C51 中提供了两个专门的运算符：

* 指针运算符

& 取地址运算符

指针运算符“*”放在指针变量前面，通过它实现访问以指针变量的内容为地址所指向的存储单元。例如：指针变量 p 中的地址为 2000H，则 *p 所访问的是地址为 2000H 的存储单元， $x=*p$ ，实现把地址为 2000H 的存储单元的内容送给变量 x。

取地址运算符“&”放在变量的前面，通过它取得变量的地址，变量的地址通常送给指针变量。例如：设变量 x 的内容为 12H，地址为 2000H，则 &x 的值为 2000H，如有一指针变量 p，则通常用 $p=\&x$ ，实现将 x 变量的地址送给指针变量 p，指针变量 p 指向变量 x，以后可以通过 *p 访问变量 x。

4.2.4 C51 表达式及符合语句

(1) 表达式语句

在表达式的后边加一个分号“;”就构成了表达式语句，如：

$a=++b*9;$

$x=8; y=7;$

$++k;$

可以一行放一个表达式形成表达式语句，也可以一行放多个表达式形成表达式语句，这时每个表达式后面都必须带“;”号，另外，还可以仅由一个分号“;”占一行形成一个表达式语句，这种语句称为空语句。

空语句在程序设计中通常用于两种情况：

①在程序中为有关语句提供标号，用以标记程序执行的位置。例如采用下面的语句可以构成一个循环。

```
repeat: ;  
:  
goto repeat;
```

②在用 while 语句构成的循环语句后面加一个分号，形成一个不执行其它操作的空循环体。这种结构通常用于对某位进行判断，当不满足条件则等待，满足条件则执行。

(2) 复合语句

复合语句是由若干条语句组合而成的一种语句，在 C51 中，用一个大括号“{ }”将若干条语句括在一起就形成了一个复合语句，复合语句最后不需要以分号“;”结束，但它内部的各条语句仍需以分号“;”结束。复合语句的一般形式为：

```
{  
    局部变量定义;  
    语句 1;  
    语句 2;  
}
```

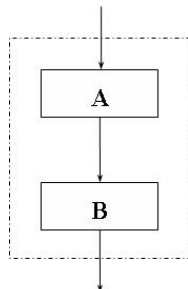
复合语句在执行时，其中的各条单语句按顺序依次执行，整个复合语句在语法上等价于一条单语句，因此在 C51 中可以将复合语句视为一条单语句。通常复合语句出现在函数中，实际上，函数的执行部分（即函数体）就是一个复合语句；复合语句中的单语句一般是可执行语句，此外还可以是变量的定义语句（说明变量的数据类型）。在复合语句内部语句所定义的变量，称为该复合语句中的局部变量，它仅在当前这个复合语句中有效。利用复合语句将多条单语句组合在一起，以及在复合语句中进行局部变量定义是 C51 语言的一个重要特征。

4.2.5 C51 基本结构和相关语句

(1) C51 的基本结构

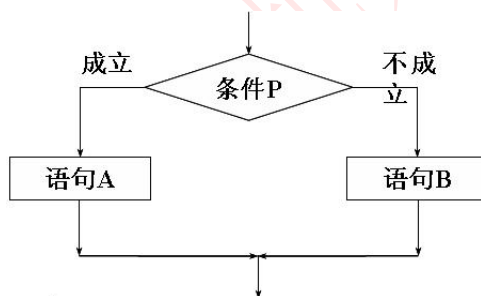
1，顺序结构

顺序结构是最基本、最简单的结构，在这种结构中，程序由低地址到高地址依次执行，如图给出顺序结构流程图，程序先执行 A 操作，然后再执行 B 操作。



2，选择结构

选择结构可使程序根据不同的情况，选择执行不同的分支，在选择结构中，程序先都对一个条件进行判断。当条件成立，即条件语句为“真”时，执行一个分支，当条件不成立时，即条件语句为“假”时，执行另一个分支。如图：当条件 S 成立时，执行分支 A，当条件 P 不成立时，执行分支 B。



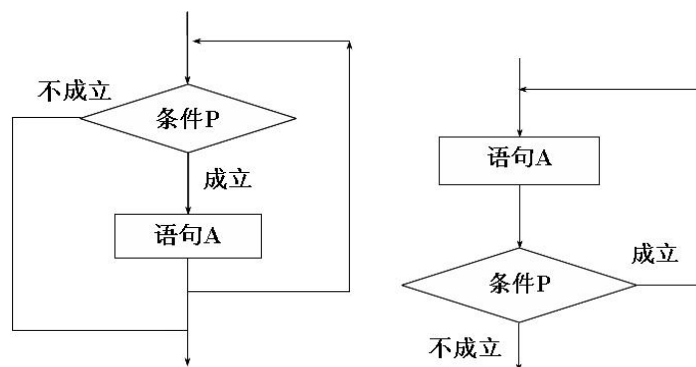
在 C51 中，实现选择结构的语句为 if/else, if/else if 语句。另外在 C51 中还支持多分支结构，多分支结构既可以通过 if 和 else if 语句嵌套实现，可用 switch/case 语句实现。

3，循环结构

在程序处理过程中，有时需要某一段程序重复执行多次，这时就需要循环结构来实现，循环结构就是能够使程序段重复执行的结构。循环结构又分为两种：当（while）型循环结构和直到（do...while）型循环结构。

①当型循环结构

当型循环结构如图：当条件 P 成立（为“真”）时，重复执行语句 A，当条件不成立（为“假”）时才停止重复，执行后面的程序。



②直到型循环结构

直到型循环结构，先执行语句 A，再判断条件 P，当条件成立（为“真”）时，再重复执行语句 A，直到条件不成立（为“假”）时才停止重复，执行后面的程序。构成循环结构的语句主要有：while、do while、for、goto。

(2) C51 的相关语句

1, if 语句

if 语句是 C51 中的一个基本条件选择语句，它通常有三种格式：

- ①if （表达式） {语句； }
- ②if （表达式） {语句 1； } else {语句 2； }
- ③if （表达式 1） {语句 1； }
 else if （表达式 2） （语句 2； ）
 else if （表达式 3） （语句 3； ）

 else if （表达式 n-1） （语句 n-1； ）
 else {语句 n}

【例】 if 语句的用法。

(1) if (x!=y) printf(“x=%d,y=%d\n”,x,y);

执行上面语句时，如果 x 不等于 y，则输出 x 的值和 y 的值。

(2) if (x>y) max=x;

 else max=y;

执行上面语句时，如 x 大于 y 成立，则把 x 送给最大值变量 max，如 x 大于 y 不成立，则把 y 送给最大值变量 max。使 max 变量得到 x、y 中的大数。

(3) if (score>=90) printf(“Your result is an A\n”);

```
else if (score>=80) printf( "Your result is an B\n" );  
else if (score>=70) printf( "Your result is an C\n" );  
else if (score>=60) printf( "Your result is an D\n" );  
else printf( "Your result is an E\n" );
```

执行上面语句后，能够根据分数 score 分别打出 A、B、C、D、E 五个等级。

2, switch/case 语句

if 语句通过嵌套可以实现多分支结构，但结构复杂。switch 是 C51 中提供的专门处理多分支结构的多分支选择语句。它的格式如下：

```
switch (表达式)  
{  
case 常量表达式 1: {语句 1; }break;  
case 常量表达式 2: {语句 2; }break;  
.....  
case 常量表达式 n: {语句 n; }break;  
default: {语句 n+1; }
```

说明如下：

- (1) switch 后面括号内的表达式，可以是整型或字符型表达式。
- (2) 当该表达式的值与某一“case”后面的常量表达式的值相等时，就执行该“case”后面的语句，然后遇到 break 语句退出 switch 语句。若表达式的值与所有 case 后的常量表达式的值都不相同，则执行 default 后面的语句，然后退出 switch 结构。
- (3) 每一个 case 常量表达式的值必须不同否则会出现自相矛盾的现象。
- (4) case 语句和 default 语句的出现次序对执行过程没有影响。
- (5) 每个 case 语句后面可以有“break”，也可以没有。有 break 语句，执行到 break 则退出 switch 结构，若没有，则会顺次执行后面的语句，直到遇到 break 或结束。
- (6) 每一个 case 语句后面可以带一个语句，也可以带多个语句，还可以不带。语句可以用花括号括起，也可以不括。
- (7) 多个 case 可以共用一组执行语句。

【例】 switch/case 语句的用法。

对学生成绩划分为 A~D，对应不同的百分制分数，要求根据不同的等级打印出它的对应百分数。可以通过下面的 switch/case 语句实现。

.....

```
switch (grade)
{
case 'A' ; printf (" 90~100\n" ) ; break;
case 'B' ; printf (" 80~90\n" ) ; break;
case 'C' ; printf (" 70~80\n" ) ; break;
case 'D' ; printf (" 60~70\n" ) ; break;
case 'E' ; printf (" <60\n" ) ; break;
default; printf (" error" \n)
}
```

3, while 语句

while 语句在 C51 中用于实现当型循环结构，它的格式如下：

```
while (表达式)
{语句; } /*循环体*/
```

while 语句后面的表达式是能否循环的条件，后面的语句是循环体。当表达式为非 0（真）时，就重复执行循环体内的语句；当表达式为 0（假），则中止 while 循环，程序将执行循环结构之外的下一条语句。它的特点是：先判断条件，后执行循环体。在循环体中对条件进行改变，然后再判断条件，如条件成立，则再执行循环体，如条件不成立，则退出循环。如条件第一次就不成立，则循环体一次也不执行。

4, do...while 语句

Do...while 语句在 C51 中用于实现直到型循环结构，它的格式如下：

```
do
{语句; } /*循环体*/
while (表达式);
```

它的特点是：先执行循环体中的语句，后判断表达式。如表达式成立（真），则再执行循环体，然后又判断，直到有表达式不成立（假）时，退出循环，执行

do while 结构的下一条语句。do while 语句在执行时，循环体内的语句至少会被执行一次。

5, for 语句

for (表达式 1; 表达式 2; 表达式 3)

{语句; } /*循环体*/

for 语句后面带三个表达式，它的执行过程如下：

(1) 先求解表达式 1 的值。

(2) 求解表达式 2 的值，如表达式 2 的值为真，则执行循环体中的语句，然后执行下一步 (3) 的操作，如表达式 2 的值为假，则结束 for 循环，转到最后一步。

(3) 若表达式 2 的值为真，则执行完循环体中的语句后，求解表达式 3，然后转到第四步。

(4) 转到 (2) 继续执行。

(5) 退出 for 循环，执行下面的一条语句。

在 for 循环中，一般表达式 1 为初值表达式，用于给循环变量赋初值；表达式 2 为条件表达式，对循环变量进行判断；表达式 3 为循环变量更新表达式，用于对循环变量的值进行更新，使循环变量能不满足条件而退出循环。

6, 循环的嵌套

在一个循环的循环体中允许又包含一个完整的循环结构，这种结构称为循环的嵌套。外面的循环称为外循环，里面的循环称为内循环，如果在内循环的循环体内又包含循环结构，就构成了多重循环。在 C51 中，允许三种循环结构相互嵌套。

【例】用嵌套结构构造一个延时程序。

```
void delay(unsigned int x)
{
    unsigned char j;
    while(x--)
    {for (j=0;j<125;j++);}
}
```


这里，用内循环构造一个基准的延时，调用时通过参数设置外循环的次数，这样就可以形成各种延时关系。

7, break 和 continue 语句

break 和 continue 语句通常用于循环结构中，用来跳出循环结构。但是二者又有所不同，下面分别介绍。

(1) break 语句

前面已介绍过用 break 语句可以跳出 switch 结构，使程序继续执行 switch 结构后面的一个语句。使用 break 语句还可以从循环体中跳出循环，提前结束循环而接着执行循环结构下面的语句。它不能用在除了循环语句和 switch 语句之外的任何其它语句中。

【例】下面一段程序用于计算圆的面积，当计算到面积大于 100 时，由 break 语句跳出循环。

```
for (r=1; r<=10; r++)
{
    area=pi*r*r;
    if (area>100) break;
    printf( "%f\n" , area);
}
```

(2) continue 语句

continue 语句用在循环结构中，用于结束本次循环，跳过循环体中 continue 下面尚未执行的语句，直接进行下一次是否执行循环的判定。

continue 语句和 break 语句的区别在于：continue 语句只是结束本次循环而不是终止整个循环；break 语句则是结束循环，不再进行条件判断。

【例】 输出 100~200 间不能被 3 整除的数。

```
for (i=100; i<=200; i++)
{
    if (i%3==0) continue;
    printf( "%d  ", i);
}
```

在程序中，当 i 能被 3 整除时，执行 continue 语句，结束本次循环，跳过 printf 函数，只有能被 3 整除时才执行 printf 函数。

8, return 语句

return 语句一般放在函数的最后位置，用于终止函数的执行，并控制程序返回调用该函数时所处的位置。返回时还可以通过 return 语句带回返回值。return 语句格式有两种：

(1) return;

(2) return (表达式);

如果 return 语句后面带有表达式，则要计算表达式的值，并将表达式的值作为函数的返回值。若不带表达式，则函数返回时将返回一个不确定的值。通常我们用 return 语句把调用函数取得的值返回给主调用函数。

4.2.6 函数

(1) 函数的定义

函数定义的一般格式如下：

函数类型 函数名(形式参数表) [reentrant][interrupt m][using n]

形式参数说明

```
{  
    局部变量定义  
    函数体  
}
```

前面部件称为函数的首部，后面称为函数的尾部，格式说明：

1. 函数类型

函数类型说明了函数返回值的类型。

2. 函数名

函数名是用户为自定义函数取的名字以便调用函数时使用。

3. 形式参数表

形式参数表用于列录在主调函数与被调用函数之间进行数据传递的形式参数。

【例】定义一个返回两个整数的最大值的函数 max()。

```
int max(int x,int y)
{
    int z;
    z=x>y?x: y;
    return (z) ;
}
```

4. reentrant 修饰符

这个修饰符用于把函数定义为可重入函数。所谓可重入函数就是允许被递归调用的函数。函数的递归调用是指当一个函数正被调用尚未返回时，又直接或间接调用函数本身。一般的函数不能做到这样，只有重入函数才允许递归调用。

关于重入函数，注意以下几点：

(1) 用 reentrant 修饰的重入函数被调用时，实参表内不允许使用 bit 类型的参数。函数体内也不允许存在任何关于位变量的操作，更不能返回 bit 类型的值。

(2) 编译时，系统为重入函数在内部或外部存储器中建立一个模拟堆栈区，称为重入栈。重入函数的局部变量及参数被放在重入栈中，使重入函数可以实现递归调用。

(3) 在参数的传递上，实际参数可以传递给间接调用的重入函数。无重入属性的间接调用函数不能包含调用参数，但是可以使用定义的全局变量来进行参数传递。

5. interrupt m 修饰符

interrupt m 是 C51 函数中非常重要的一个修饰符，这是因为中断函数必须通过它进行修饰。在 C51 程序设计中，当函数定义时用了 interrupt m 修饰符，系统编译时把对应函数转化为中断函数，自动加上程序头段和尾段，并按 51 系统中断的处理方式自动把它安排在程序存储器中的相应位置。

在该修饰符中，m 的取值为 0~31，对应的中断情况如下：

0——外部中断 0

1——定时/计数器 T0

- 2——外部中断 1
- 3——定时/计数器 T1
- 4——串行口中断
- 5——定时/计数器 T2
- 其它值预留。

(2) 函数的调用与声明

1, 函数的调用

函数调用的一般形式如下：

函数名（实参列表）；

对于有参数的函数调用，若实参列表包含多个实参，则各个实参之间用逗号隔开。

按照函数调用在主调函数中出现的位置，函数调用方式有以下三种：

- (1) 函数语句。把被调用函数作为主调用函数的一个语句。
- (2) 函数表达式。函数被放在一个表达式中，以一个运算对象的方式出现。这时的被调用函数要求带有返回语句，以返回一个明确的数值参加表达式的运算。
- (3) 函数参数。被调用函数作为另一个函数的参数。

2, 自定义函数的声明

在 C51 中，函数原型一般形式如下：

[extern] 函数类型 函数名（形式参数表）；

函数的声明是把函数的名字、函数类型以及形参的类型、个数和顺序通知编译系统，以便调用函数时系统进行对照检查。函数的声明后面要加分号。

如果声明的函数在文件内部，则声明时不用 extern，如果声明的函数不在文件内部，而在另一个文件中，声明时须带 extern，指明使用的函数在另一个文件中。

4.2.7 C51 构造数据类型

(1) 数组

1, 一维数组

一维数组只有一个下标，定义的形式如下：

数据类型说明符 数组名[常量表达式][={初值，初值……}]

各部分说明如下：

(1) “数据类型说明符”说明了数组中各个元素存储的数据的类型。

(2) “数组名”是整个数组的标识符，它的取名方法与变量的取名方法相同。

(3) “常量表达式”，常量表达式要求取值要为整型常量，必须用方括号“[]”括起来。用于说明该数组的长度，即该数组元素的个数。

(4) “初值部分”用于给数组元素赋初值，这部分在数组定义时属于可选项。对数组元素赋值，可以在定义时赋值，也可以定义之后赋值。在定义时赋值，后面须带等号，初值须用花括号括起来，括号内的初值两两之间用逗号间隔，可以对数组的全部元素赋值，也可以只对部分元素赋值。初值为 0 的元素可以只用逗号占位而不写初值 0。

例如：下面是定义数组的两个例子。

```
unsigned char x[5];  
unsigned int y[3]={1,2,3};
```

第一句定义了一个无符号字符数组，数组名为 x，数组中的元素个数为 5。

第二句定义了一个无符号整型数组，数组名为 y，数组中元素个数为 3，定义的同时给数组中的三个元素赋初值，赋初值分别为 1、2、3。

需要注意的是，C51 语言中数组的下标是从 0 开始的，因此上面第一句定义的 5 个元素分别是：x[0]、x[1]、x[2]、x[3]、x[4]。第二句定义的 3 个元素分别是：y[0]、y[1]、y[2]。赋值情况为：y[0]=1；y[1]=2；y[2]=3。

C51 规定在引用数组时，只能逐个引用数组中的各个元素，而不能一次引用整个数组。但如果是字符数组则可以一次引用整个数组。

2，字符数组

用来存放字符数据的数组称为字符数组，它是 C 语言中常用的一种数组。字符数组中的每一个元素都用来存放一个字符，也可用字符数组来存放字符串。字符数组的定义下一般数组相同，只是在定义时把数据类型定义为 char 型。

例如：char string1[10];

```
char string2[20];
```

上面定义了两个字符数组，分别定义了 10 个元素和 20 个元素。

在 C51 语言中，字符数组用于存放一组字符或字符串，字符串以“\0”作为结束符，只存放一般字符的字符数组的赋值与使用和一般的数组完全相同。对于存放字符串的字符数组。既可以对字符数组的元素逐个进行访问，也可以对整个数组按字符串的方式进行处理。

到这里我们就把 51 单片机涉及到的基础 C 语言知识简单介绍了下，大家还可以打开我们的光盘资料“\7--单片机相关资料\C 语言加油站\[我的第①本 c 语言编程书：C 语言从入门到精通]. 国家 863 中部软件孵化器. 扫描版. pdf”来详细的学习 C 语言基础知识。

4.3 一个简单的单片机 C 程序

根据前面小节介绍，我们知道一个 C 程序必须有且只有一个 main 函数，程序的执行都是从该函数开始。我们使用 C 语言来开发 51 单片机程序就会涉及到单片机内部寄存器的操作，这些在 KEIL 开发环境中都已经做好了，我们只需要将“reg51.h”或“reg52.h”头文件包括进来即可。下面我们就列举一个简单的单片机 C 程序，如下：

```
#include "reg52.h"

void main()
{
}

}
```

课后作业

(1) 花一点时间学习下光盘资料“\7--单片机相关资料\C 语言加油站\[我的第①本 c 语言编程书：C 语言从入门到精通]. 国家 863 中部软件孵化器. 扫描版. pdf”文档，查找变量命名的规则、if/switch 语句、for 语句、while 语句、一维数组的使用方法。（温馨提示：这些知识也是在 51 单片机开发过程中应用最多的）

第 5 章 单片机最小系统

在前面章节，我们已经介绍了 51 单片机的 C 语言基础知识，该部分可以说是软件部分，至于硬件部分本章就来简单介绍下。相信很多学完 51 单片机的朋友在后期可能会做一些具体的综合项目，在项目中涉及到的硬件电路各种各样，但唯一不变的是 51 单片机最小系统，因此有些朋友就会去自己制作最小系统。本章就来介绍 51 单片机的最小系统构成，通过本章的学习，让大家能够掌握 51 单片机最小系统的构成，为后期自己搭建最小系统提供理论基础。本章分为如下几部分内容：

5.1 51 单片机最小系统构成

5.1 51 单片机最小系统构成

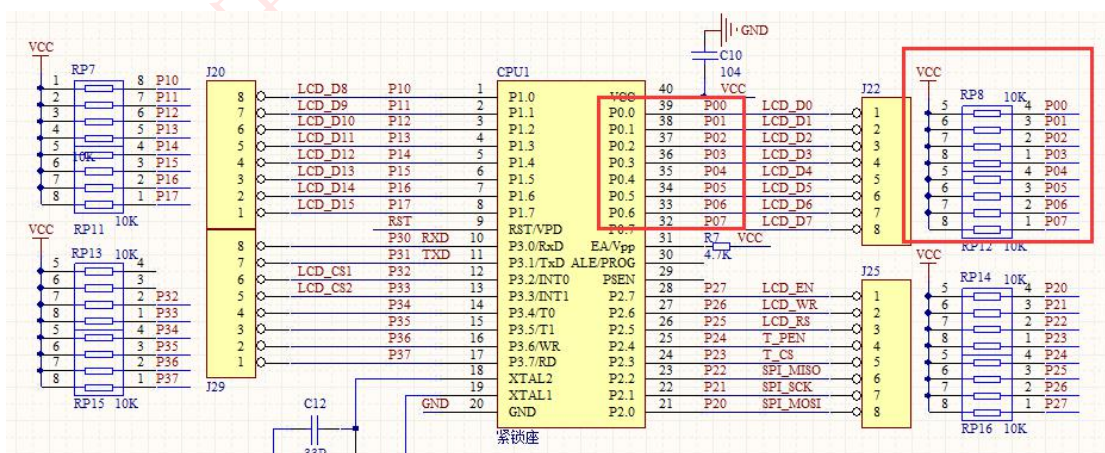
若要使系统正常运行，必须确保单片机的最小系统稳定工作。51 单片机的最小系统由以下几部分组成：

- (1) 晶振电路
- (2) 复位电路
- (3) 电源电路
- (4) 程序下载电路

第(4)是我们添加进来的,实际上最小系统只由前面3个部分组成。为什么要加入第(4)部分呢?仅靠前面3个部分电路只能使单片机正常运行,但如果我们要给系统更新程序即烧入程序时就没有办法了,所以我们将第(4)部分电路也加入到最小系统当中,这时候我们就可以给系统任意烧入程序了。

晶振电路提供时钟给单片机工作，犹如人的心脏。复位电路提供系统复位操作，当系统出现运行不正常或者死机等情况时，可以通过复位按键重新启动系统。电源电路也是非常关键的一个部分，因为单片机对供电电压是有要求的，如果电压过大将烧坏芯片，如果电压过小系统将运行不了。所以如果一个合适稳定的电源电路是非常关键的。

还有一点非常重要，因为 51 单片机的 P0 口是漏极开路的，要让它输出高电平就必须外接上拉电阻，原理图中如下：

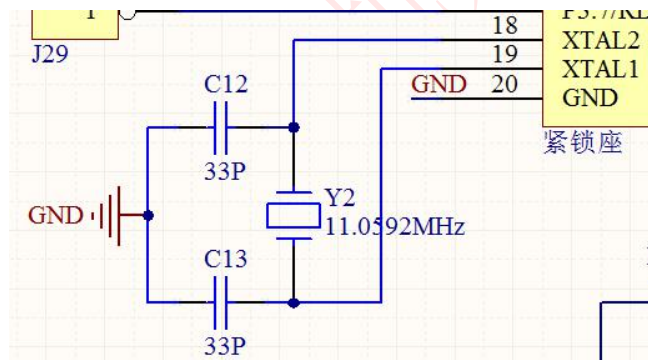


相同网络标号的表示其线路是连接在一起的，这个在后面章节会介绍。为了增强其他 I/O 口的驱动能力，开发板上也都为其外接了上拉电阻（除了串口），通常上拉电阻选择 10K 左右。

下面就来介绍下这 4 部分的电路。

5.1.1 晶振电路

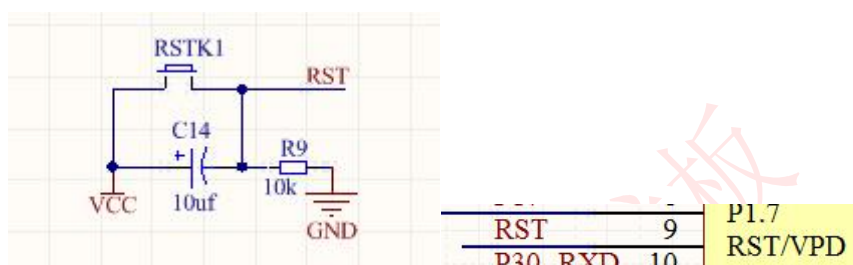
由于单片机正常工作需要一个时钟，因此就需要在单片机晶振引脚上外接一个晶振（我们使用的 STC89CXX 单片机晶振引脚是 18 和 19 脚），至于需要多大晶振这就取决于你所使用的单片机，由于我们使用的是 51 单片机，其时钟频率可在 0-40MHZ 上运行，一般情况下我们建议选择 12M（适合计算延时时间）或者是 11.0592M（适合串口通信）。若直接将此晶振接入单片机晶振引脚，会发现系统工作不稳定，这是因为晶振起振的一瞬间会产生一些电感，为了消除这个电感所带来的干扰，可以在此晶振两端分别加上一个电容，电容的选取需要无极性的，另一端需要共地。根据选取的晶振大小决定电容值，通常电容可在 10-33PF 值范围内选取。我们使用的是 33PF 电容。这样一来就构成了晶振电路。只有保证晶振电路稳定，单片机才能继续工作。其电路如下所示：



5.1.2 复位电路

前面我们已经将晶振电路搭建完成，我们知道晶振犹如人的心脏，需要无时无刻给单片机提供运行周期。但即使时钟周期在不停的运行，系统也有可能会出现崩溃或者瘫痪状态。这就好比人会生病一样，人生病就得看医生，服用医生开的药后重新获得正常状态。那么单片机呢，它是如何获取重生的？这就需要设计一个复位电路来实现此功能。我们知道单片机引脚当中就有一个 RST 复位引脚，而 STC89CXX 单片机又是高电平复位，所以只需要让这个引脚保持一段时间高电平就可以。要实现此功能通常有两种方式，一种是通过按键进行手动复位，

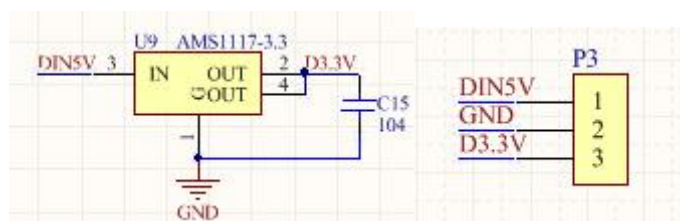
还有一种是上电复位，即电源开启后自动复位。手动复位是通过一个按键及电容电阻所组成，利用按键的开关功能实现复位，按键按下后 VCC 直接进入单片机 RST 引脚，松开后 VCC 断开，RST 被电阻拉为低电平。这一合一开就实现了手动复位。而自动复位主要是利用 RC 充放电功能，电源已开启，由于电容隔直，VCC 直接进入 RST，然后电容开始慢慢充电，直到充电完成，此时 RST 被电阻拉低。这样就起到上电复位的效果。这里我们采用手动复位。不到系统崩溃，我们几乎不会操作复位。复位电路如下图所示：



当按下按键 RSTK1，VCC 直接连接到 RST 复位脚，VCC 是高电平所以系统复位。本来复位电路只需上图左半部分，即按键 RSTK1 和 C14、R9 组成的复位电路，但考虑到开发板兼容 AVR、ARM 等单片机，所以使用一个三极管 Q2 将复位信号翻转，即变为低电平复位，此时要记得将 J31 端子的 2 和 3 脚短接。

5.1.3 电源电路

任何电子器件都需要有一个合适的电源进行供电，这就好比人要吃饭一样，没有电源，系统是不会工作的。STC89CXX 单片机的工作电压是 3.3~5.5V 范围，通常我们使用 5V 直流。将电源接入到各芯片电源引脚即可。开发板电源电路如下图所示：



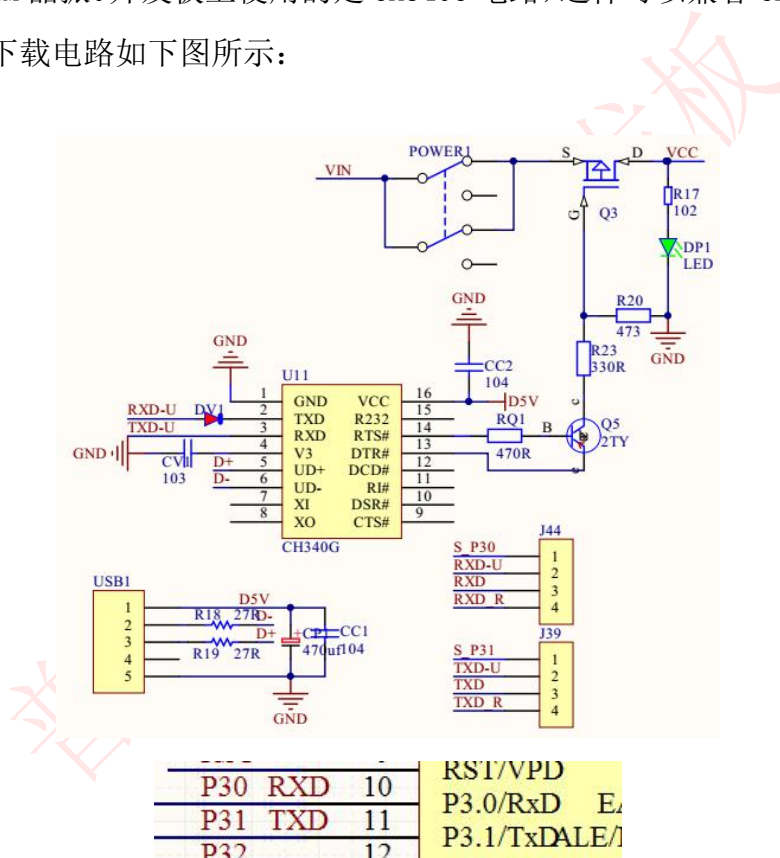
开发板上还预留了 P3（5V/3V）端子，这些端子可以很方便的给外部模块连接电源。

5.1.4 下载电路

在前面小节中已经介绍了为什么要加下载电路到最小系统中，这里就不再重复。程序要烧写到单片机内是通过上位机（PC 机）及对应的软件将编译器生成的 xxx.HEX 文件通过单片机串口写入进去。

我们知道现在的笔记本电脑没有 RS232 接口，所以要使用 USB 转 TTL 串口电平芯片来建立 PC 机和单片机数据传输通路。通常使用 CH340G 或者 CH340C 芯片来完成电平转换。CH340G 需外接 12M 晶振，而 CH340C 内部自带晶振，所以可以不接外部 12M 晶振。开发板上使用的是 CH340G 电路，这样可以兼容 CH340C 电路。

开发板下载电路如下图所示:



本电路是我们自主研发的一键自动下载电路，无需冷启动。

从上图可以知道，USB1 接口即为程序下载接口，D-和 D+连接到 CH340 芯片的 D-和 D+，然后 CH340 芯片的串口 TXD 和 RXD 管脚就连接到单片机的串口(P3. 0、P3. 1)上，这里不是直接连接到单片机串口，而是通过 J44 和 J39 端子进行转接。这样做不仅可以让开发板 USB 接口下载程序，还可以使用 RS232 接口下载程序，这部分内容在后面章节会介绍。而且还有一点比较重要的是可以让单片机串口不受 CH340 或者 RS232 干扰，这样开发板也可以当成 USB 转 TTL 模块使用，该设计

也是考虑 WIFI、蓝牙等模块与串口通信不受干扰问题。

USB1 接口不仅可以作为程序下载口，还可以作为串口通信口，因为它本身就是实现串口下载。同时还可以作为电源供电口，可以看到 USB1 的管脚 1 就是 5V 电源脚，所以开发板的可以直接使用 USB 线来供电，如果提供的 USB 线有问题，可以使用安卓手机数据线，接口都是兼容的。当电源开关打开后，电源指示灯 DP1 即会点亮，表明系统电源正常。

至此，我们就把 51 单片机的最小系统介绍完，大家在制作最小系统时，可以参考我们的电路，这样成功率会更高。

课后作业

第 6 章 KEILC51 安装与工程建立

本章将向大家介绍 KEIL C51 软件的安装，通过本章的学习，我们可以在电脑上安装一个 KEIL C51 软件，为后面学习程序的开发做好铺垫。本章分为如下几部分内容：

- 6.1 KEIL C51 软件获取
- 6.2 KEIL C51 软件安装
- 6.3 KEIL C51 软件破解
- 6.4 创建 51 单片机工程

6.1 KEIL C51 软件获取

要在电脑上成功安装 KEIL C51，首先必须要有安装包，我们可以通过万能的百度，搜索关键字“KEIL C51 下载”，上面会有很多下载渠道，也可以在 KEIL 的官网上下载：<https://www.keil.com/download/product/>，打开界面如下图所示。



不过我们光盘内已经给大家提供了下载链接，在光盘“5--开发工具\1. KEIL 编程软件”内，大家直接下载即可，省去了查找下载的时间。我们使用 KEIL C51 是 9.01 版本，如果后面出了更高的版本选择性升级即可，不过也没有必要什么都追求最新的，用习惯了一个软件就行。

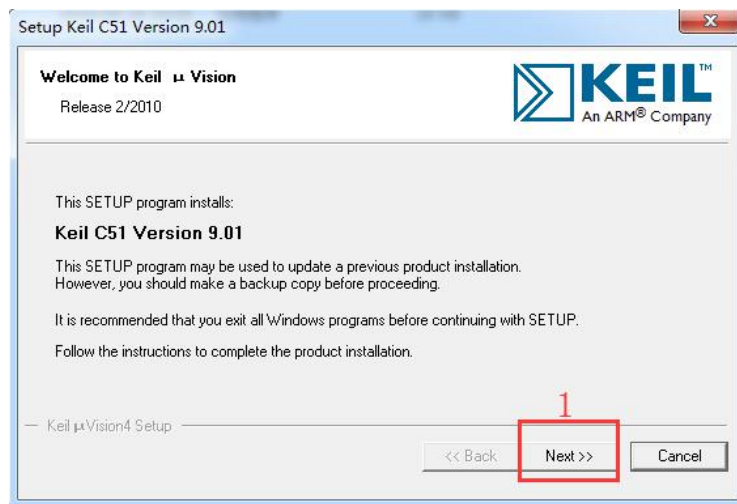
使用我们提供的下载链接下载的软件包，其内部含有如下图所示文件。



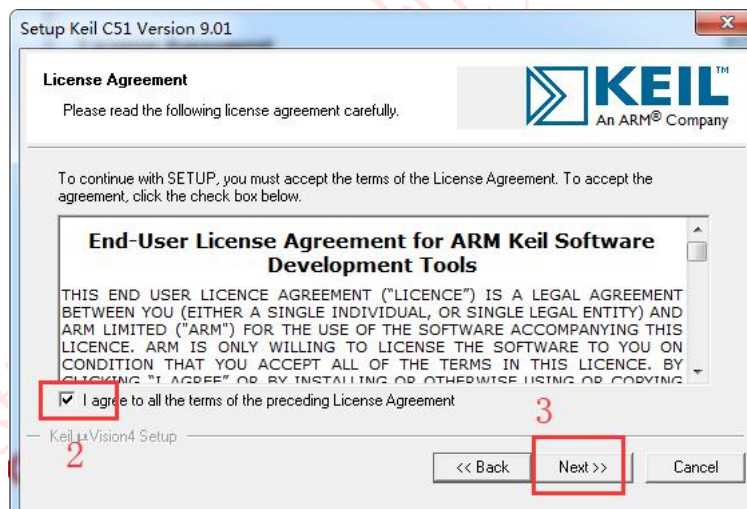
C51V901.exe 是 KEIL C51 软件的安装文件，KEIL_Lic.exe 是用来破解 KEIL C51 软件的工具，使其能够编译大小超过 2K 的程序文件。

6.2 KEIL C51 软件安装

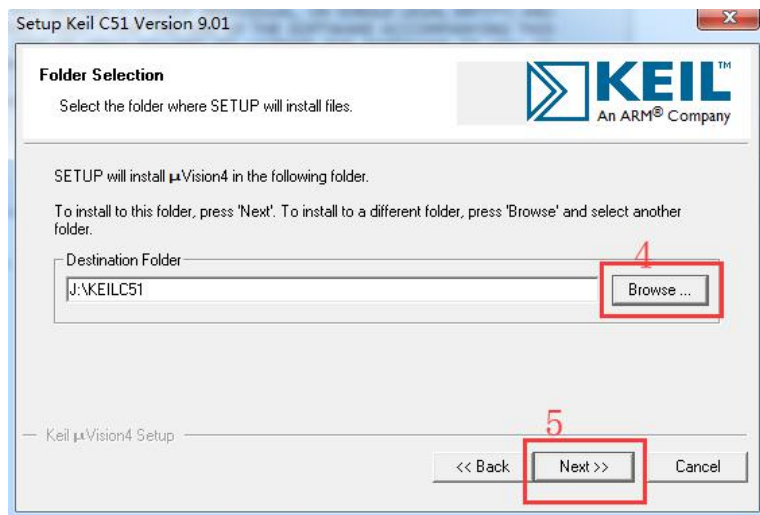
软件包下载完成之后，我们双击 C51V901.exe 这个应用程序，弹出如下所示对话框。



点击 Next 按钮。弹出如下对话框。



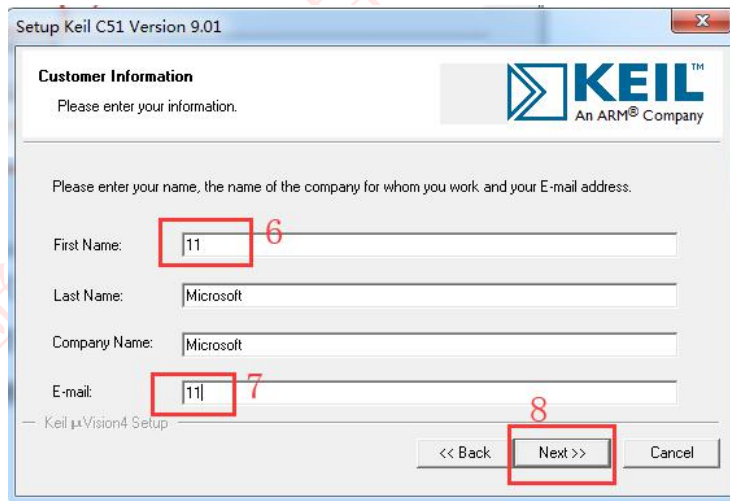
在红框 2 中勾选上，点击 Next 按钮。弹出如下对话框。



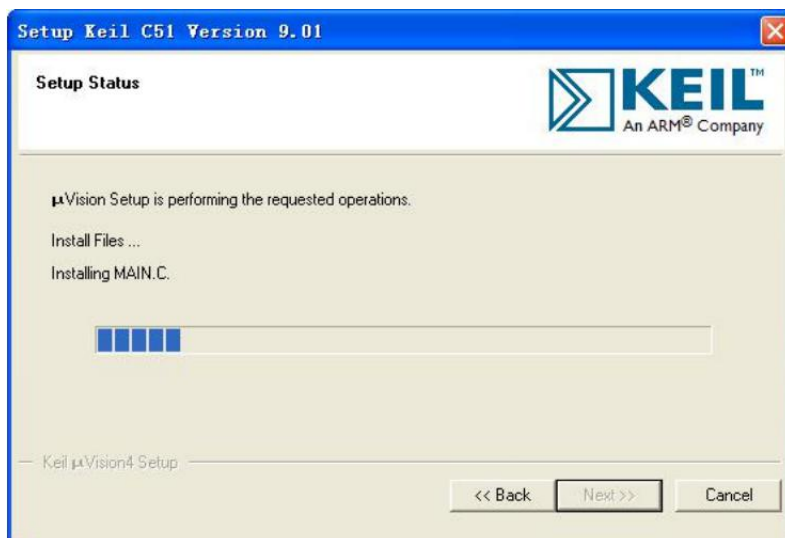
点击红框 4 选择软件安装的路径，我们这里在 J 盘内新建一个文件夹，命名为 KEILC51，将其安装到该文件夹内，方便软件的管理。特别要注意：

(1) 软件安装保存路径不能出现中文或者特殊字符，否则会出现很多奇怪的错误，到时候很难找问题。

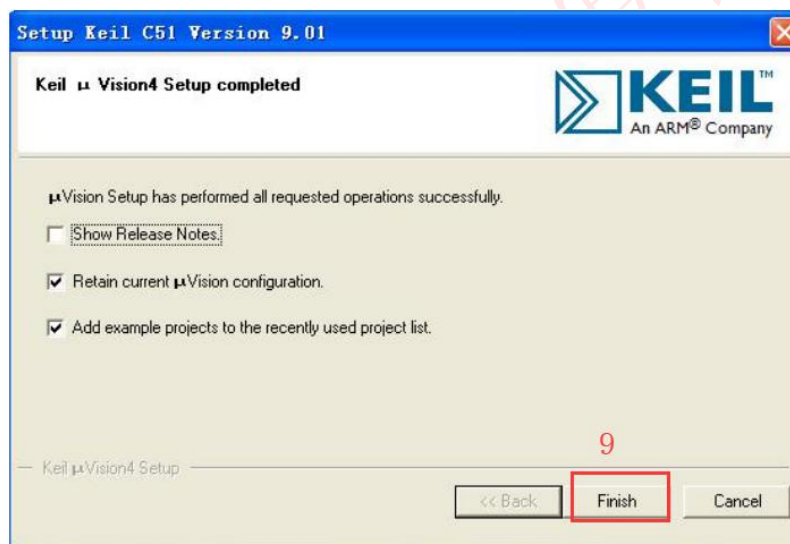
(2) 不要将 KEIL5 软件和 KEIL4 或者 51 的 KEIL 安装在一个文件夹内。然后点击 Next。弹出如下对话框。



在红色框 6 和 7 中我们随便输入一些东西，我们这里输入数字 11，当然也可以输入空格，但是一定要输入，否则红色框 8 就一直是灰色状态，输入完成后点击 Next，弹出如下对话框。



说明 KEIL C51 软件正在安装，只要等一段时间即可，安装完成以后会出现如下界面。



点击 Finish 按钮完成 KEIL C51 软件的安装过程。KEIL C51 安装完成后在电脑桌面上一般会有一个快捷方式，如下所示：

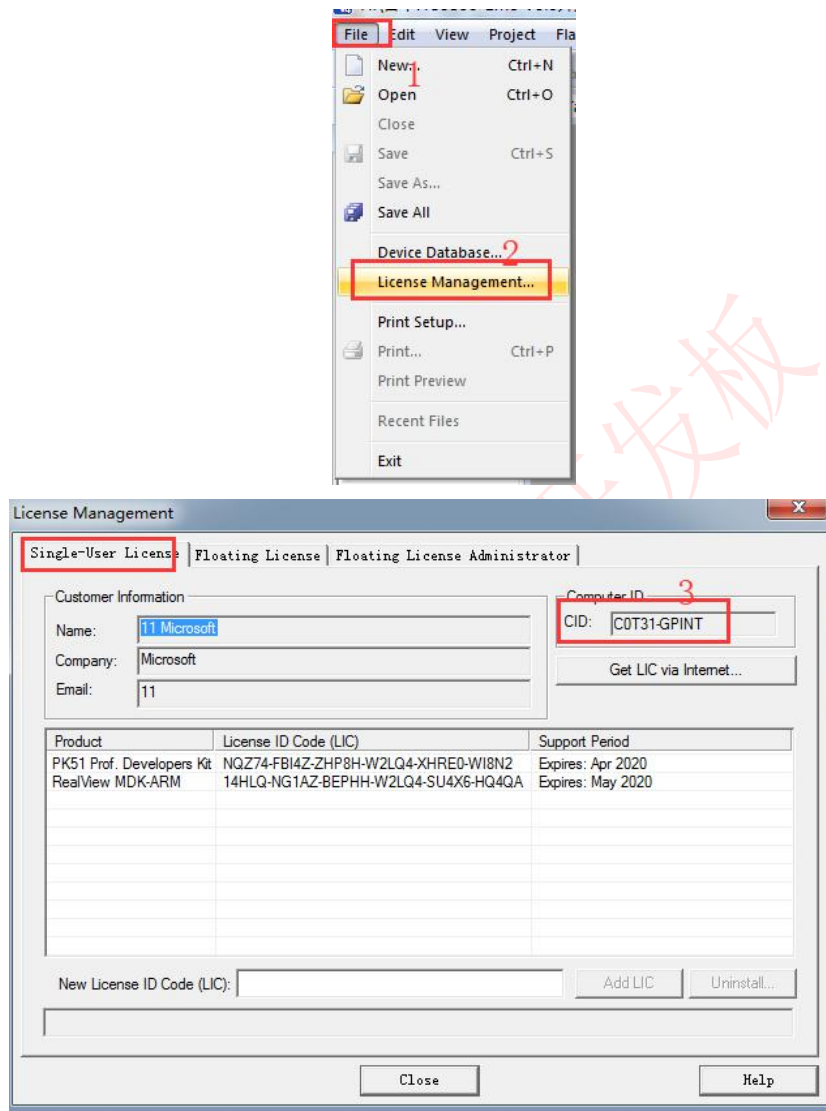


6.3 KEIL C51 软件破解

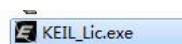
上一节我们已经安装好了 KEIL C51 编程软件，可以使用它来编译一些 51 单片机代码量小于 2K 的小工程。但是有时候我们开发的项目可能比较大，程序代

码超过了 2K，这时候就需要来破解 KEIL C51，因此 KEIL C51 软件的破解非常必要。下面就来介绍下如何破解。

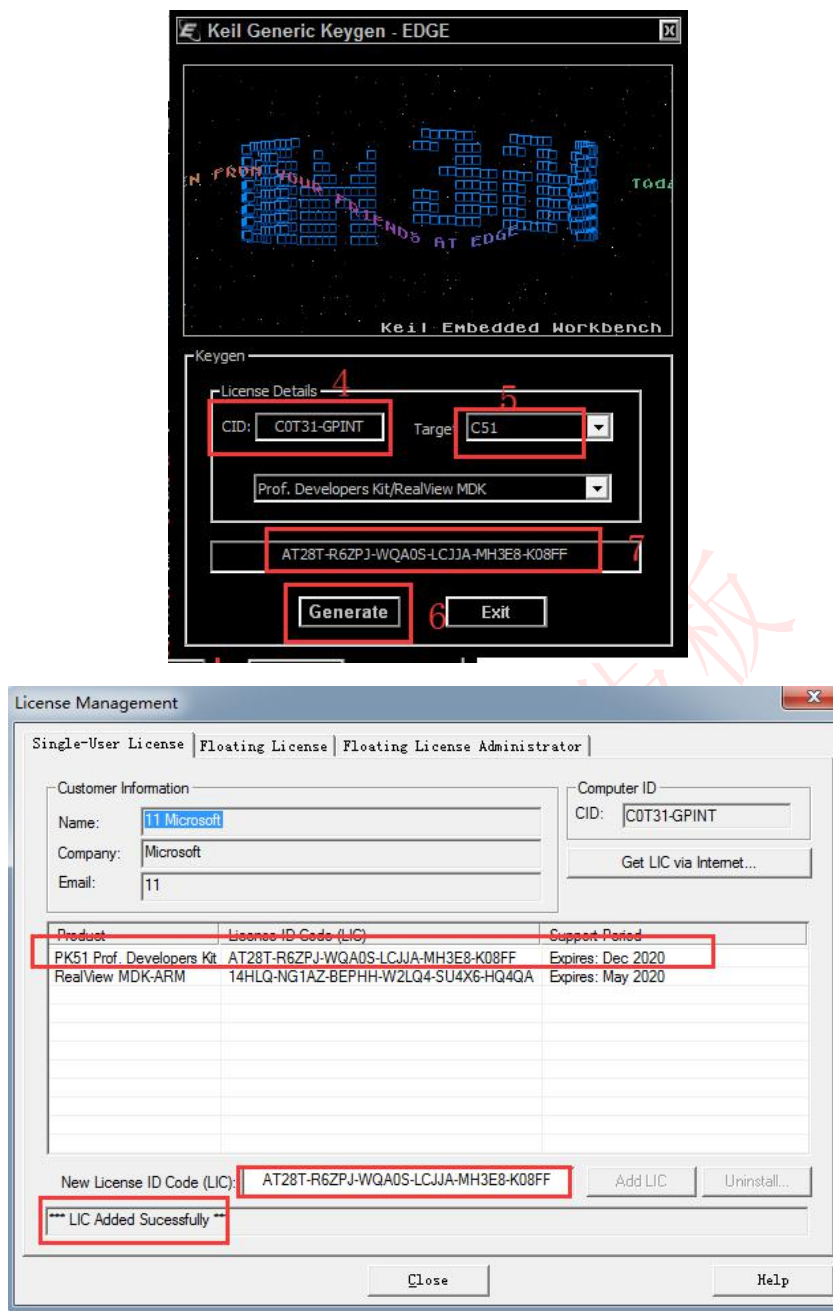
(1) 首先打开 KEIL C51 软件，点击 File/License Management.., 复制红色框 3 中的 CID。 操作步骤如下：



然后我们打开之前下载的 KEIL C51 软件包， 里面还有一个 KEIL_Lic.exe 文件， 如下所示：



此文件用于破解，直接双击它，将刚才从 KEIL C51 中复制的 CID 码粘贴到下面红色框 4 中，并且在红色框 5 选择 C51， 然后点击红色框 6， 生成破解码， 将此破解码即红色框 7 内容复制到 KEIL C51 软件内 LIC 框中， 点击 Add LIC 即可破解， 如果破解成功则会显示。 操作步骤如下：



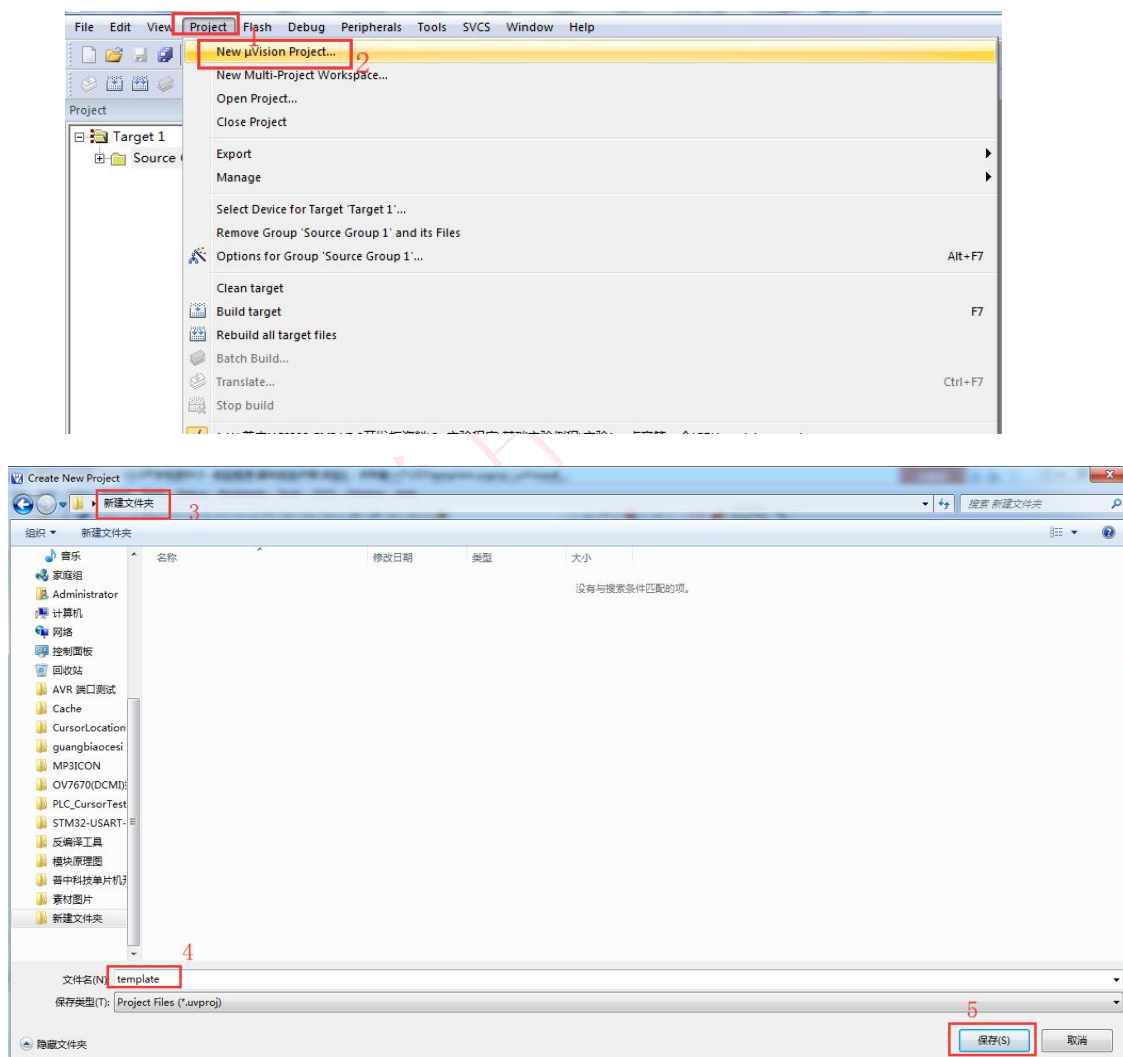
如果破解失败，在打开 KEIL C51 和注册机的时候，点击快捷方式图标，选择右键： Run AsAdministrator ,中文系统应该是“以管理员权限运行”，然后就可以正常加 licence 了。建议最好所有允许的用户，都以管理员身份运行，以避免重复破解。

至此，我们就将 KEIL C51 软件安装和破解成功。后面就可以使用该软件来编写我们的 51 单片机程序。

6.4 创建 51 单片机工程

前面几节，我们介绍了如何安装和破解 KEIL C51 软件，这一节我们来介绍下如何使用该软件创建一个 51 单片机工程模板，为后面章节程序的编写带来方便。

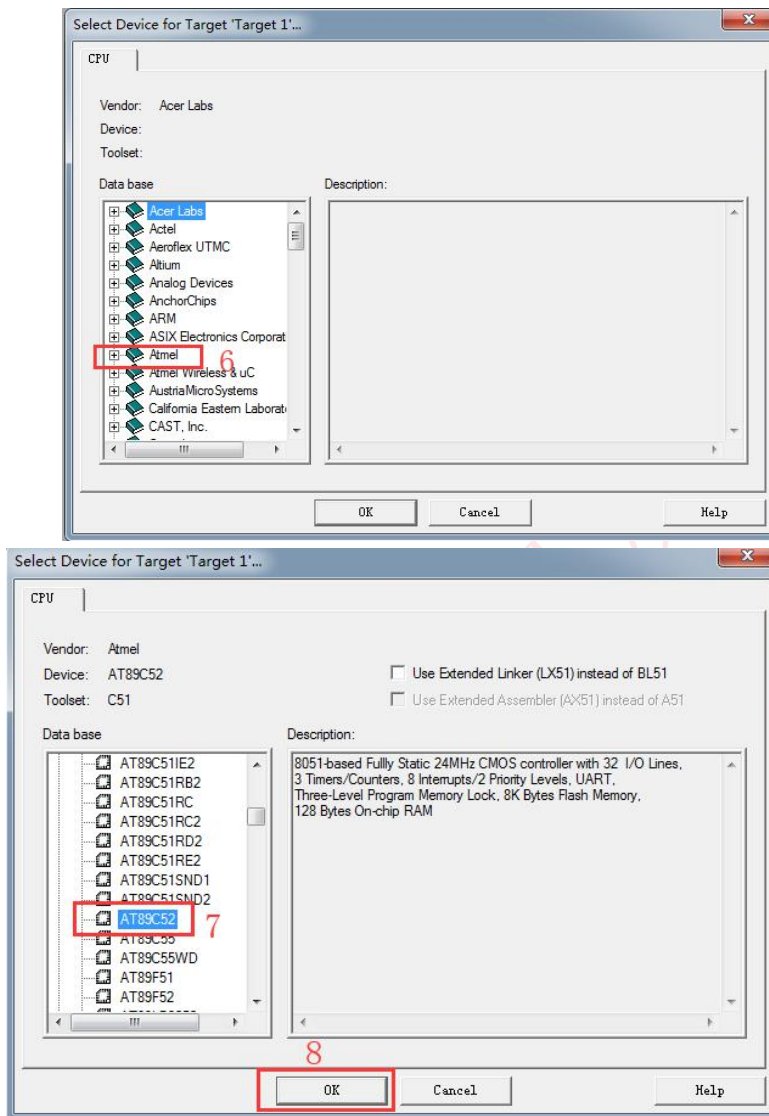
(1) 打开 KEIL C51 软件，新建一个工程，工程名根据喜好命名，但是要注意使用英文，如果使用中文名可能会出现一些奇怪的错误，这里我们命名为 template，直接保存在你所要保存的文件夹下。具体步骤如下：



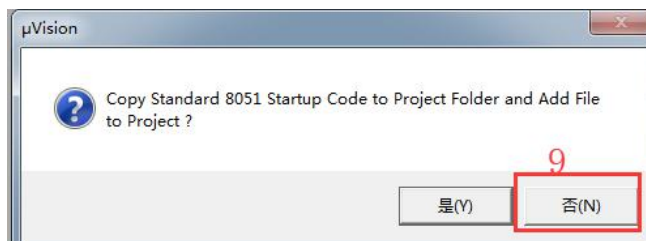
(2) 选择单片机型号。

这个根据开发板使用的 CPU 具体的型号来选择，我们普中 51-双核-A6/A7、HC6800-EM3 V2.2、HC6800-EM3 V2.2C、HC6800-EM3 V3.0 开发板采用的是 STC89C516 芯片，开拓者、HC6800-ES V2.0、普中 51-单核-A2 使用的是 STC89C52

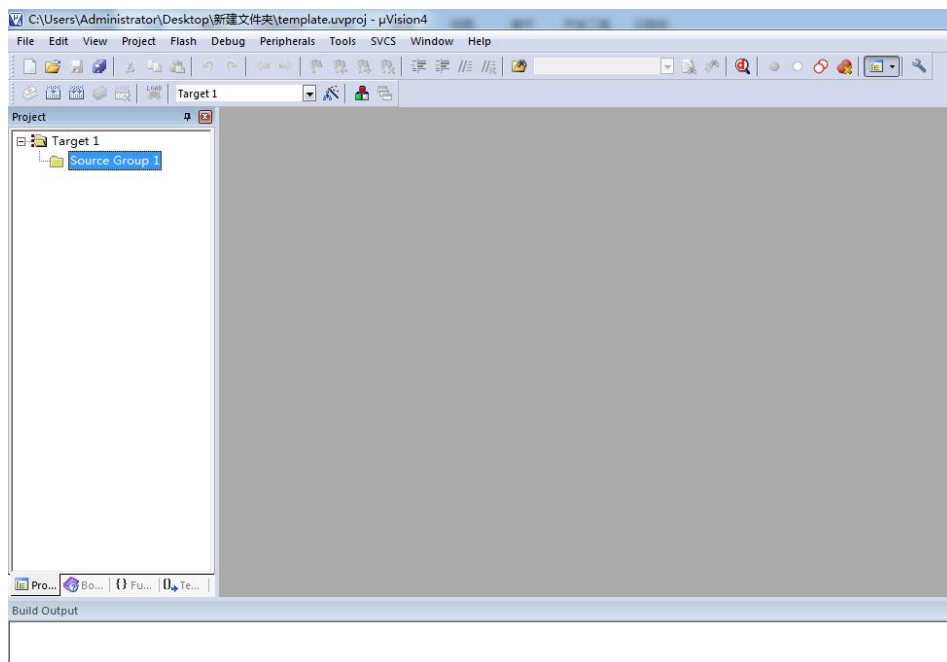
芯片。但实际上在 KEIL C51 软件内找不到这些芯片类型，但我们可以选择 AT89C51 或者 AT89C52，它们都是 51 内核，软件代码兼容。这里选择 AT89C52，具体操作如下：



点击 OK 键后，弹出如下对话框：

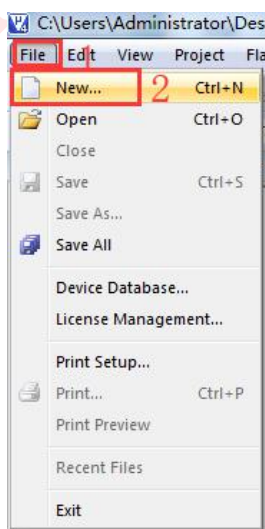


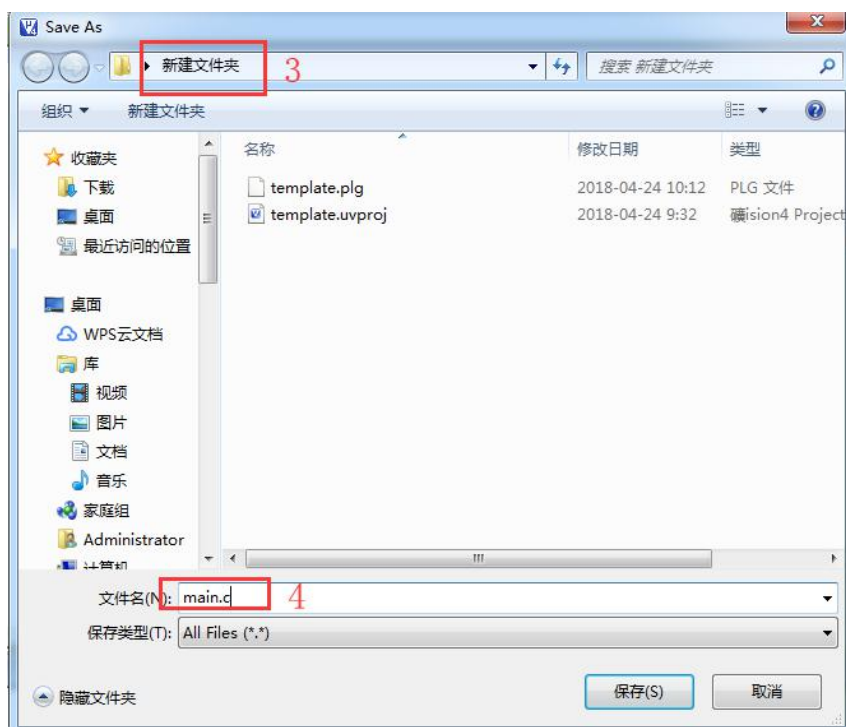
该对话框内容是提示我们是否要将 8051 启动文件添加到工程中，这里我们选择“否”，原因是 KEIL C51 内已经帮我们完成了启动，所以只需要编写应用程序即可。选择“否”后界面如下：



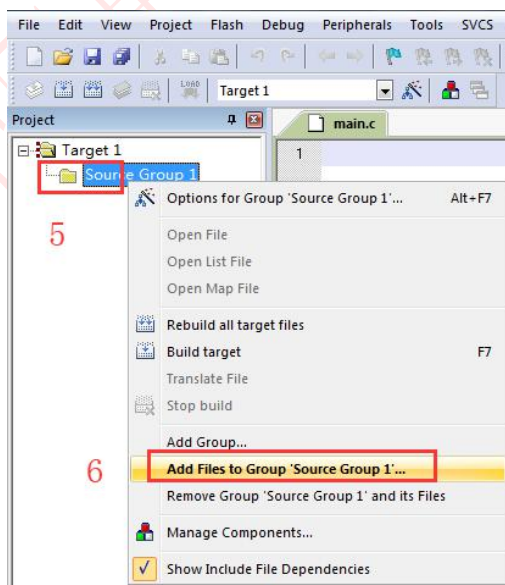
(3) 给工程添加文件。

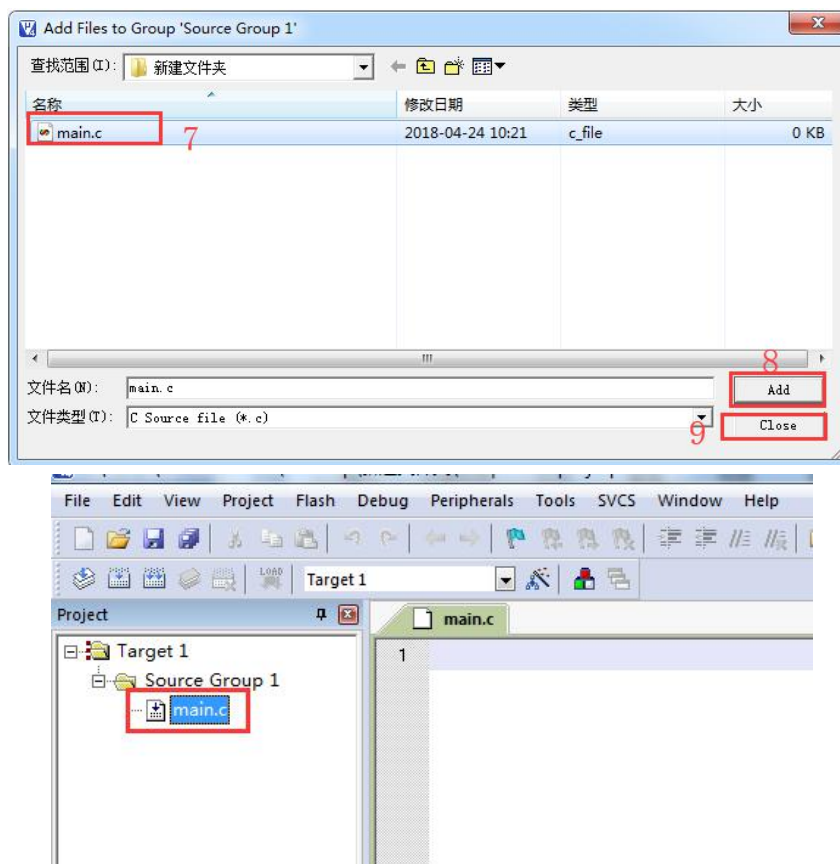
选择 File/New... 或者使用工具栏的图标  来新建一个文件，后点击保存，系统会自动定位到我们工程目录，只需要在文件名栏输入新建的文件名即可，一个 51 单片机工程必须含有且仅有一个 main 函数，所以新建的文件命名为 main.c，点击保存。具体操作步骤如下：





然后将新建的 main.c 文件添加到工程中，选择工程组“Source Group 1”，鼠标右键选择“Add Files to Group ‘Source Group 1’ ...”，然后选择对应的 xxx.c 文件，点击 Add 键后在点击 Close 关闭。这时工程中就显示已加入的文件。具体操作如下：

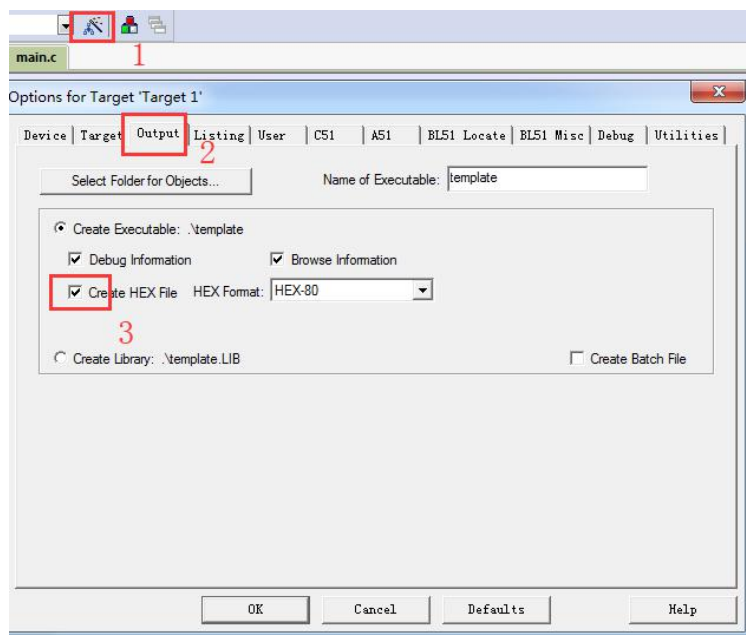




(4) 配置魔术棒选项卡

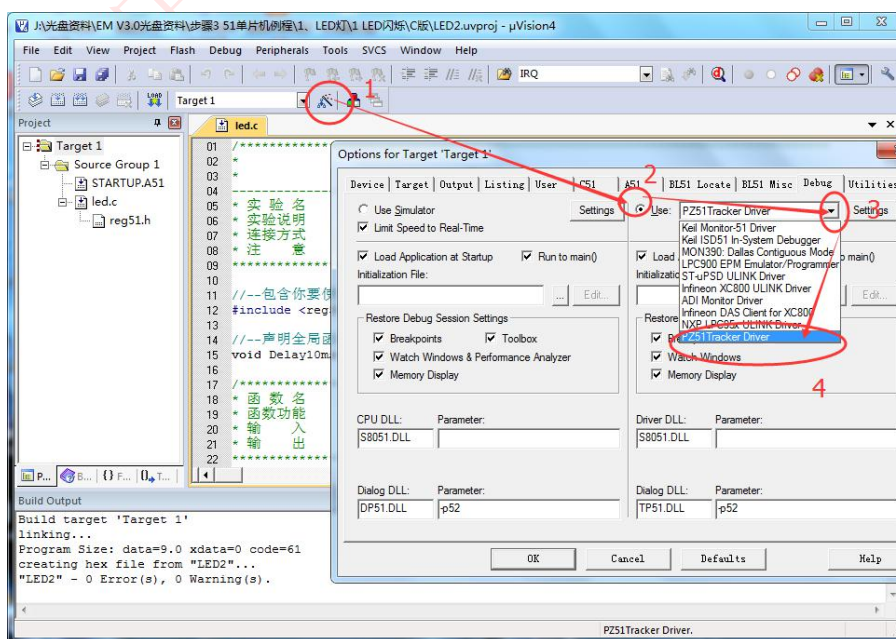
这一步的配置工作非常重要，很多人自己编写程序编译后发现找不到 .HEX 文件，或者使用我们 51 仿真器的时候发下仿真失败，这些问题都是在这个地方没有配置好导致的。

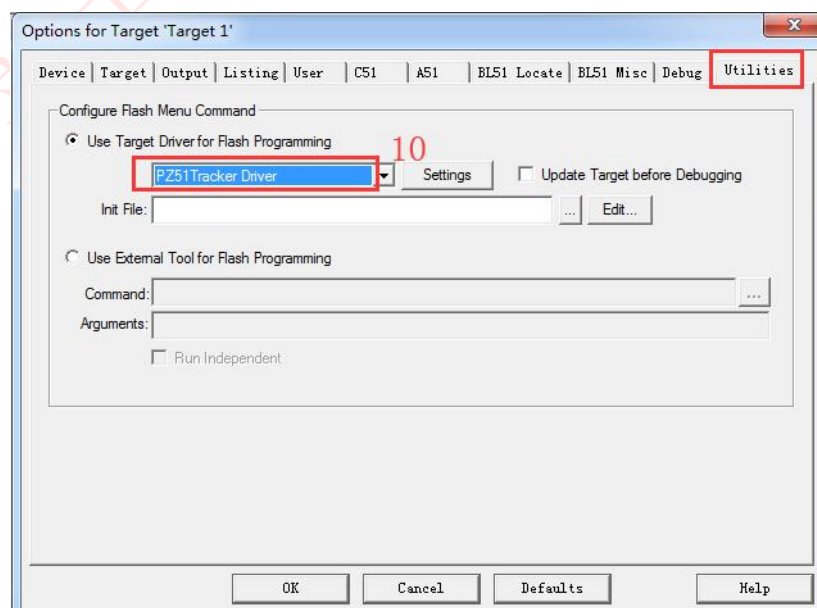
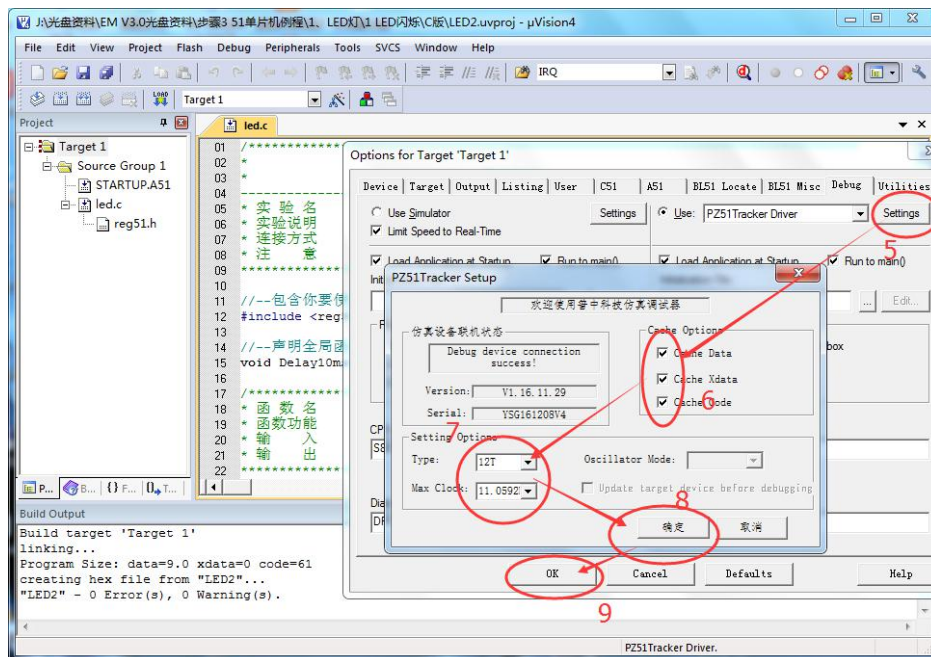
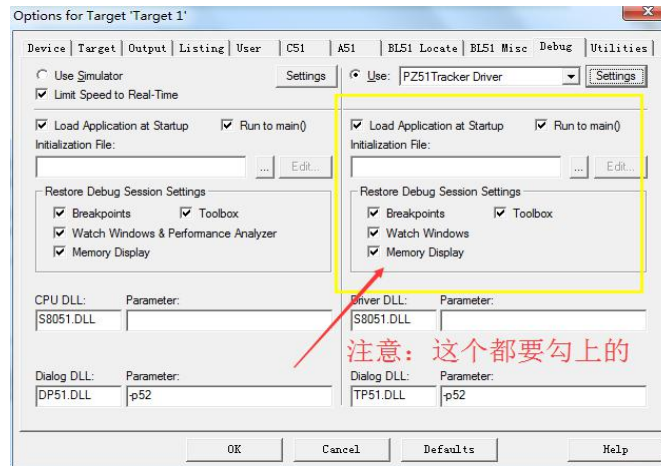
1，选择魔术棒工具的 Output 选项卡，勾选红色框 3，即程序编译成功后会产生 HEX 文件。具体操作如下：



2，51 仿真器配置

当你购买了 51 仿真器且你的仿真器安装好驱动并且连接了电脑和板子，那么就可以开始配置。如果使用我们公司的 51 仿真器的朋友，相关驱动的安装请查看“\3--开发工具\5. 51 仿真器驱动的安装”教程。将 51 仿真器直接插在单片机卡座上，仿真器采用防呆设计，方向错了没法卡紧。仿真器连接电脑，同时仿真器也给开发板供电，所以使用仿真器可不接电源线。要使用我们 51 仿真器在线调试程序，需要在 KEIL 软件内进行相应的配置。具体的配置如下（我们已经将 51 仿真器连接开发板）：





根据上图从 1 到 10 步骤设置完成之后就可以进行在线仿真调试了。

最后点击 OK 按钮即可。然后双击工程组中的 main.c 文件，我们输入如下内容：

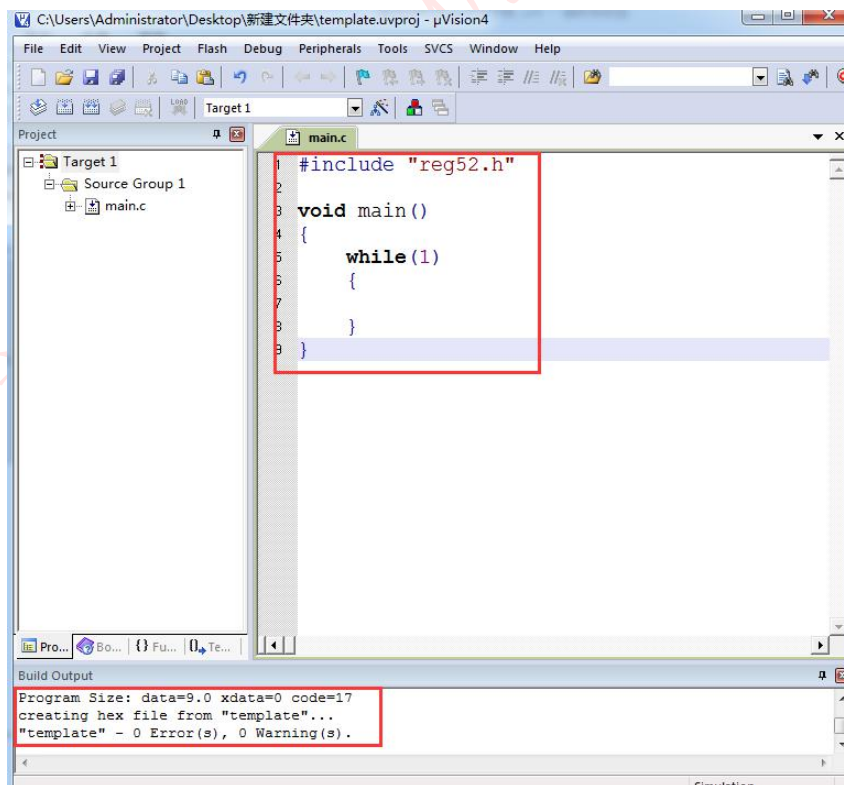
```
#include "reg52.h"

void main()
{
    while(1)
    {

    }
}
```

编译后结果 0 错误 0 警告，表明我们创建的 51 单片机工程完全正确。

如下：

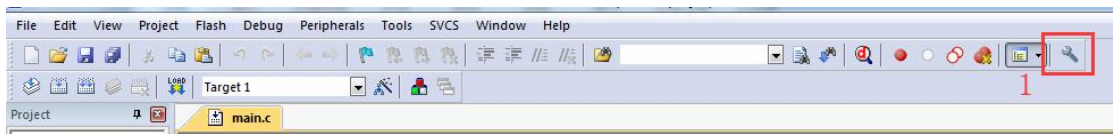


(5) 修改编程区内字体大小

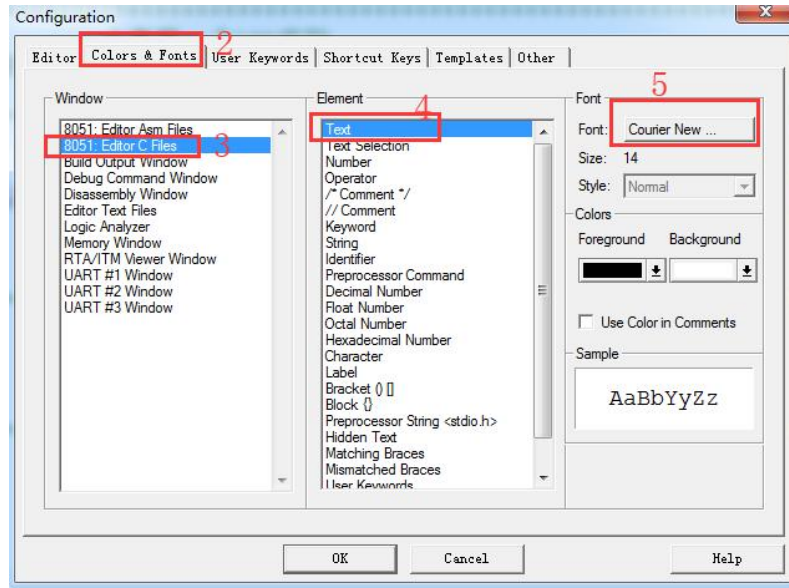
很多朋友可能会觉得，KEIL C51 默认提供的字体比较小，看起来非常不舒服。

下面我们就来介绍如何在 KEIL C51 软件内修改编程区字体大小问题。

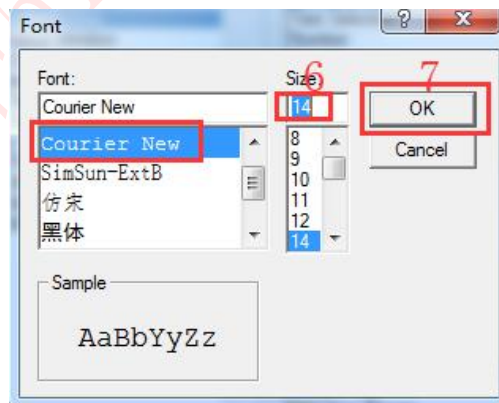
1，首先在 KEIL C51 软件的工具栏中内找到一个“小扳手”，如下所示：



2，然后点击它，弹出如下设置界面，按照设置界面的红色标号顺序进入。



3，点击标号 5 按钮时用于选择设置当前字体大小等信息的，选择适合自己的字体大小即可，然后点击 OK，设置完成。如下所示：



最后设置完成后，编程区内的字体就会发生改变。

更多有关 KEIL C51 软件的使用小技巧大家可以通过百度了解学习。

课后作业

- (1) 按照上述方法创建一个自己的 51 单片机工程模板。

普中51单片机开发板

第 7 章 点亮第一个 LED 灯

不论学习什么单片机，最简单的外设莫过于 IO 口的高低电平控制，本章将向大家介绍如何在创建好的工程模板上，通过控制 51 单片机的 GPIO 使开发板上的 LED 灯点亮。通过本章的学习，让大家学会如何在程序中操作 51 单片机的 GPIO 口输出高低电平。本章分为如下几部分内容：

7.1 51 单片机 GPIO 介绍

7.2 LED 简介

7.3 硬件设计

7.4 软件设计

7.5 实验现象

7.1 51 单片机 GPIO 介绍

7.1.1 GPIO 概念

GPIO (general purpose input output) 是通用输入输出端口的简称，可以通过软件来控制其输入和输出。51 单片机芯片的 GPIO 引脚与外部设备连接起来，从而实现与外部通讯、控制以及数据采集的功能。不过 GPIO 最简单的应用还属点亮 LED 灯了，只需通过软件控制 GPIO 输出高低电平即可。当然 GPIO 还可以作为输入控制，比如在引脚上接入一个按键，通过电平的高低判断按键是否按下。

我们开发板上使用的 51 单片机型号是 STC89C52 或 STC89C516，此芯片共有 40 引脚，芯片引脚图如下图所示：

1	P1.0	VCC	40
2	P1.1	P0.0	39
3	P1.2	P0.1	38
4	P1.3	P0.2	37
5	P1.4	P0.3	36
6	P1.5	P0.4	35
7	P1.6	P0.5	34
8	P1.7	P0.6	33
9	RST/VPD	P0.7	32
10	P3.0/RxD	EA/Vpp	31
11	P3.1/TxD	ALE/PROG	30
12	P3.2/INT0	PSEN	29
13	P3.3/INT1	P2.7	28
14	P3.4/T0	P2.6	27
15	P3.5/T1	P2.5	26
16	P3.6/WR	P2.4	25
17	P3.7/RD	P2.3	24
18	XTAL2	P2.2	23
19	XTAL1	P2.1	22
20	GND	P2.0	21

那么是不是所有引脚都是 GPIO 呢？当然不是，51 单片机引脚可以分为这么几大类：

- (1) 电源引脚：引脚图中的 VCC、GND 都属于电源引脚。
- (2) 晶振引脚：引脚图中的 XTAL1、XTAL2 都属于晶振引脚。
- (3) 复位引脚：引脚图中的 RST/VPD 属于复位引脚，不做其他功能使用。
- (4) 下载引脚：51 单片机的串口功能引脚 (TXD、RXD) 可以作为下载引脚使用。
- (5) GPIO 引脚：引脚图中带有 Px.x 等字样的均属于 GPIO 引脚。从引脚图可以看出，GPIO 占用了芯片大部分的引脚，共达 32 个，分为了 4 组，P0、P1、P2、P3，每组为 8 个 IO，而且在 P3 组中每个 IO 都具备额外功能，只要通过相

应的寄存器设置即可配置对应的附加功能，同一时刻，每个引脚只能使用该引脚的一个功能。

对于这么多 GPIO 管脚，我们怎么知道具体某个引脚有什么功能呢？很简单，可以查阅 STC89CXX 芯片数据手册获取信息，数据手册在我们光盘“\6--芯片资料”内，里面有一个 STC89CXX 数据手册.pdf。里面的第 23 页中就有介绍，我们截取了一部分内容如下图所示：

管脚	管脚编号			说明
	LQFP44	PDIP40	PLCC44	
P0.0 ~ P0.7	37-30	39-32	43~36	P0:P0口既可作为输入/输出口，也可作为地址/数据复用总线使用。当P0口作为输入/输出口时，P0是一个8位准双向口，上电复位后处于开漏模式。P0口内部无上拉电阻，所以作I/O口必须外接10K~4.7K的上拉电阻。当P0作为地址/数据复用总线使用时，是低8位地址线[A0~A7]，数据线的[D0~D7]，此时无需外接上拉电阻。
P1.0/T2	40	1	2	P1.0 标准I/O口 PORT1[0]
				T2 定时器/计数器2的外部输入
P1.1/T2EX	41	2	3	P1.1 标准I/O口 PORT1[1]
				T2EX 定时器/计数器2捕捉/重装方式的触发控制
P1.2	42	3	4	标准I/O口 PORT1[2]
P1.3	43	4	5	标准I/O口 PORT1[3]
P1.4	44	5	6	标准I/O口 PORT1[4]
P1.5	1	6	7	标准I/O口 PORT1[5]
P1.6	2	7	8	标准I/O口 PORT1[6]

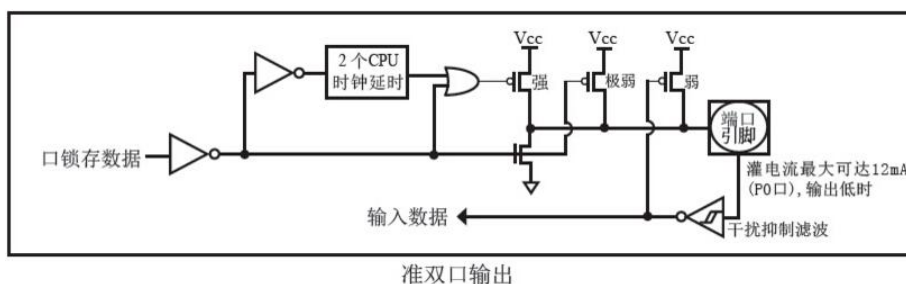
从上图中我们可以获取引脚的名字和引脚功能等信息。这个我们开发板芯片原理图内已经将引脚所有功能都标进去了，所以后面也不需要查找具体引脚有什么功能，直接看原理图即可。

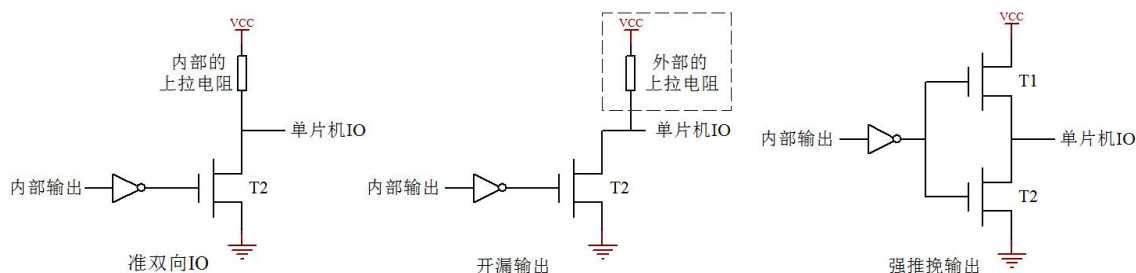
7.1.2 GPIO 结构

前面我们讲解了 51 单片机 GPIO 的基本概念及引脚分类。现在我们看下 GPIO 内部的结构是怎样的。

51 单片机的 I/O 口都可以同时作为输入和输出之用。除了 P0 口以外，其他的 I/O 口都有内部上拉电阻。3 个内部上拉电阻都有不同的功用，要理解这个电路需要一定的模拟电路的知识。

内部上拉电阻示意图如下所示：





由于相关的硬件配置，在编程时有以下需要注意的地方：

- ①当 I/O 口被用作输入用途时，读取输入前要保证锁存器状态为 1。
- ②当将 I/O 口由低拉高时，指令执行后需要 1~2 个机器周期才能使实际的输出变成高电平。

在外围电路上需要注意的地方：

- ①P0 口做普通 I/O 使用时，要加上拉电路，因为其没有内部上拉。
- ②为防止电流过大，必要时外加限流电阻。

上下拉电阻的作用：

①上拉电阻就是将不确定的信号通过一个电阻拉到高电平，同时此电阻起到一个限流的作用，下拉就是下拉到低电平。

②OC 门要输出高电平，外部必须加上拉电阻。

③加大普通 IO 口的驱动能力。

④起到限流的作用。

⑤抵抗电磁干扰。

上下拉电阻的选取原则：

- 1、从降低功耗方面考虑应该足够大，因为电阻越大，电流越小。
- 2、从确保足够的引脚驱动能力考虑应该足够小，电阻越小，电流才能越大。
- 3、开漏输出时，过大的上拉电阻会导致信号上升沿变缓，如下所示。



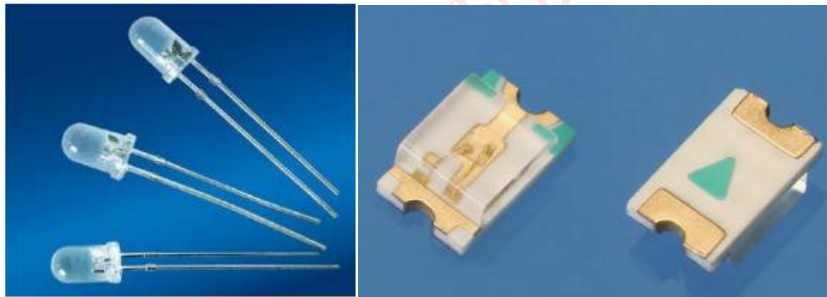
如果不了解 51 单片机 IO 口内部结构也没有关系，大家只要记住两点：

- (1) 51 单片机所有 IO 口都是双向的，即可以作为输入也可以作为输出使用。
- (2) 由于 P0 口是漏极开路的，所以要操作 P0 口必须外接上拉电阻，其他

P1、P2、P3 口都内部自带上拉电阻，可以不加，如果要增强 I/O 口驱动能力，可以外接上拉电阻。

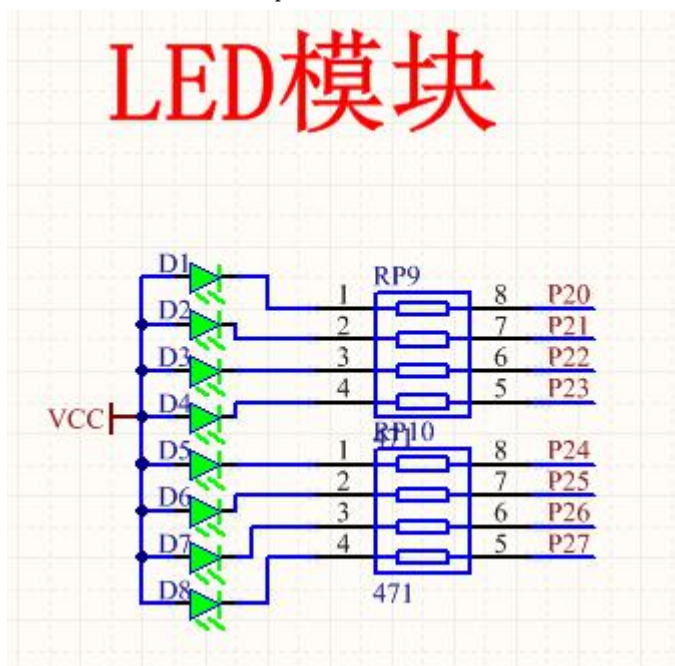
7.2 LED 简介

LED 即发光二极管。它具有单向导电性，通过 5mA 左右电流即可发光，电流越大，其亮度越强，但若电流过大，会烧毁二极管，一般我们控制在 3 mA-20mA 之间，通常我们会在 LED 管脚上串联一个电阻，目的就是限制通过发光二极管的电流不要太大，因此这些电阻又可以称为“限流电阻”。当发光二极管发光时，测量它两端电压约为 1.7V，这个电压又叫做发光二极管的“导通压降”。下图左右分别为直插式发光二极管和贴片式发光二极管实物图。发光二极管正极又称阳极，负极又称阴极，电流只能从阳极流向阴极。直插式发光二极管长脚为阳极，短脚为阴极。仔细观察贴片式发光二极管正面的一端有彩色标记，通常有标记的一端为阴极。



7.3 硬件设计

在我们开发板上有 8 个 LED 连接单片机 P2 I/O 口上，具体电路如下图所示：



该电路即可实现 LED 实验，还可以实现交通灯实验（只需按照交通灯颜色摆放即可），所以我们也称该电路为 LED/交通灯电路。

从上图中可以看出，该电路是集成的，LED 控制管脚直接连接到 51 单片机的 I0 上。LED 采用共阳接法，即所有 LED 阳极管脚接电源 VCC，阴极管脚通过一个 470 欧的限流电阻接到 P2 口上。根据前面 LED 的介绍我们知道，要让 LED 发光即对应的阴极管脚就应该为低电平，若为高电平则熄灭。

如果要想 51 单片机控制 LED，就必须通过单片机管脚在 P2 I0 口上输出低电平。本章所要实现的功能是：点亮 D1 指示灯。由于 LED 模块与 I0 已经连接，所以使用单片机的 P2 I0 口即可。

7.4 软件设计

本章所要实现的功能是：点亮 D1 发光二极管，即让 P0.0 管脚输出一个低电平。完成后可再控制 D1 指示灯闪烁，即间隔一定时间点亮和熄灭 D1 指示灯。

我们直接复制上一章创建的工程，在此模板基础上进行程序开发。为了能够与开发攻略教程对应，将复制过来的模板文件夹重新命名为“实验 1：点亮第一个 LED”。打开工程直接在 main.c 源文件内进行编程，main.c 内代码如下：

```
/*  
*****  
*****  
*                               点亮第一个 LED 实验
```

```

*
实现现象：下载程序后 D1 指示灯点亮
注意事项：无

*****
*****/

#include "reg52.h"    //此文件中定义了单片机的一些特殊功能寄存器

sbit led=P2^0;        //将单片机的 P2.0 端口定义为 led

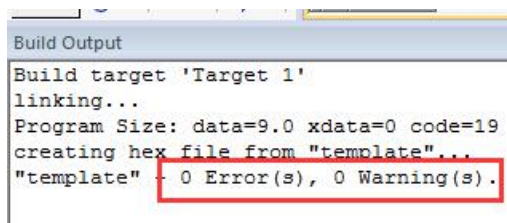
/*****
*****
* 函 数 名      : main
* 函数功能      : 主函数
* 输    入      : 无
* 输    出      : 无
*****
*****/
void main()
{
    led=0; //P0.0 端口设置为低电平
    while(1)
    {

    }
}

```

main.c 文件内代码非常少也很简单，首先将 51 单片机的头文件包含进来，然后使用 sbit 关键字来定义 P0.0 管脚，定义好后即可使用 led 来替代 P0.0 口的操作。主函数功能非常简单，首先让 led 为低电平，即 P0.0 口为低电平，D1 指示灯即点亮，然后进入 while 循环，单片机此时一直在 while 内循环操作。当然该语句也可以放在 while 循环语句内，同样会点亮 D1 指示灯。在 51 单片机程序开发中，main 函数内通常都会有一个 while 循环，在循环体内不断执行我们所要实现的功能。对于 LED 这种一次性操作的器件来说，可以把控制语句放在 while 之前来执行。

到这里整个程序就编写完成，我们编译一下，如下图所示：



可以看到没有错误，也没有警告。从编译信息可以看出，我们的代码占用 FLASH 大小为：19 字节，所用的 SRAM 大小为：9 个字节（9+0）。这里我们解释一下，编译结果里面的几个数据的意义：

Code：表示程序所占用 FLASH 的大小。

data：数据储存器内部 RAM 占用大小。

xdata：数据储存器外部 RAM 占用大小。

有了这个就可以知道你当前使用的 flash 和 sram 大小了。一定要注意的是程序的大小不是 .hex 文件的大小，而是编译后的 Code 和 data 之和。

7.4.1 LED 闪烁实验

如果要实现 LED 闪烁，很简单，只需循环让 D1 指示灯先亮一会后熄灭。这里就有一个延时问题，如何来产生延时呢？在前面介绍 C 语言时讲解过循环语句，只需要编写一个循环函数即让 CPU 不断循环执行即可实现延时功能。依据人的肉眼余晖效应，延时时间肯定不能太短，否则就没法观察。在我们的“\5--实验程序\基础实验例程\实验 2：LED 闪烁”例程内延时函数如下：

```
/******  
*****  
* 函数名      : delay  
* 函数功能    : 延时函数，i=1 时，大约延时 10us  
*****  
*****/  
void delay(u16 i)  
{  
    while(i--);  
}
```

因此 LED 闪烁程序就很容易编写出来，具体的大家可以打开相应工程查看，这里就不多说了。

7.4.2 LED 流水灯实验

如果要想实现 LED 流水灯，很简单，只需循环让 D1-D8 指示灯逐个点亮。同样本实验也需要延时，这个在前面已介绍，这里就不多说。要实现循环点亮，可以使用最笨的方法，实现方法：点亮 D1 且把 D2-D8 熄灭，延时一段时间后再点亮 D2 且把 D1、D3-D8 熄灭，延时一段时间后再点亮 D3 且把 D1-D2、D4-D8 熄灭，如此循环，这样就可以很简单的实现 LED 流水灯实验，当然我们不推荐此种方法。在前面我们学习了 C 语言相关的基础知识，里面有移位以及循环语句操作，根据流水灯实现原理，即 IO 口由低往高或者由高往低逐个输出低电平特点，那么我们可以将移位操作以及循环结合进来。实现代码如下：

```
#include "reg52.h"           //此文件中定义了单片机的一些特殊功能寄存器

#include<intrins.h>          //因为要用到左右移函数，所以加入这个头文件

typedef unsigned int u16;     //对数据类型进行声明定义
typedef unsigned char u8;

#define led P2               //将 P2 口定义为 led 后面就可以使用 led 代替 P2 口

/*****
*****
* 函 数 名      : delay
* 函数功能      : 延时函数，i=1 时，大约延时 10us
*****
*****/

void delay(u16 i)
{
    while(i--);
}
```

```
/******  
*****  
* 函 数 名      : main  
* 函数功能      : 主函数  
* 输    入      : 无  
* 输    出      : 无  
*****  
*****/  
  
void main()  
{  
    u8 i;  
    led=~0x01;//0x01 取反即为 0xFE  
    delay(50000); //大约延时 450ms  
    while(1)  
    {  
        for(i=0;i<8;i++)  
        {  
            P2=~(0x01<<i);    //将 1 右移 i 位, 然后将结果赋值到 P0 口  
            delay(50000); //大约延时 450ms  
        }  
    }  
}
```

进入 main 函数后首先 led=~0x01, 因为 LED 是低电平点亮, 所以 0x01 取反后的结果是 0xFE, 对应二进制数为 1111 1110, 即最低位为 0, 因此最开始的 D1 指示灯会点亮, 然后延时一段时间进入 while 循环, 由于要实现 8 个 LED 从 D1→D8 循环点亮, 因此可以使用 for 循环语句循环 8 次, 每循环一次, 点亮的小灯向右移动一个即 P2 口输出的低电平要左移一位, 因此可以使用 P2=~(0x01<<i); 语句实现。0x01<<i 表示每次 i 增加 1 次, 0x01 中的 1 就移动多少位, 因为 1 (高电

平) 不会让 LED 点亮, 需要取反后变为低电平 0 才能点亮, 所以最后的结果需要取反后给 P0 口, 并且每次循环都要延时一段时间, 这样我们肉眼才能分辨出来。

(1) 左移_crol_、右移_cror_函数的使用

除了使用 for 循环语句实现移位, KEIL C51 软件内还有对应的移位库函数, 左移函数是_crol_(), 右移函数是_cror_(), 要使用这两个函数在我们的程序中必须要包含 intrins.h 头文件。这两个移位函数大家可以百度了解下, 其内部实现过程我们是看不到的, 该移位函数实现的移位功能就相当于一个队列内循环移动, 如果是左移, 那么最高位就被移到最低位了, 次高位变为最高位, 依次类推。使用左移、右移函数实现的流水灯操作代码如下:

```
#include "reg52.h"           //此文件中定义了单片机的一些特殊功能寄存器

#include<intrins.h>          //因为要用到左右移函数, 所以加入这个头文件

typedef unsigned int u16;     //对数据类型进行声明定义
typedef unsigned char u8;

#define led P2               //将 P2 口定义为 led 后面就可以使用 led 代替 P2 口

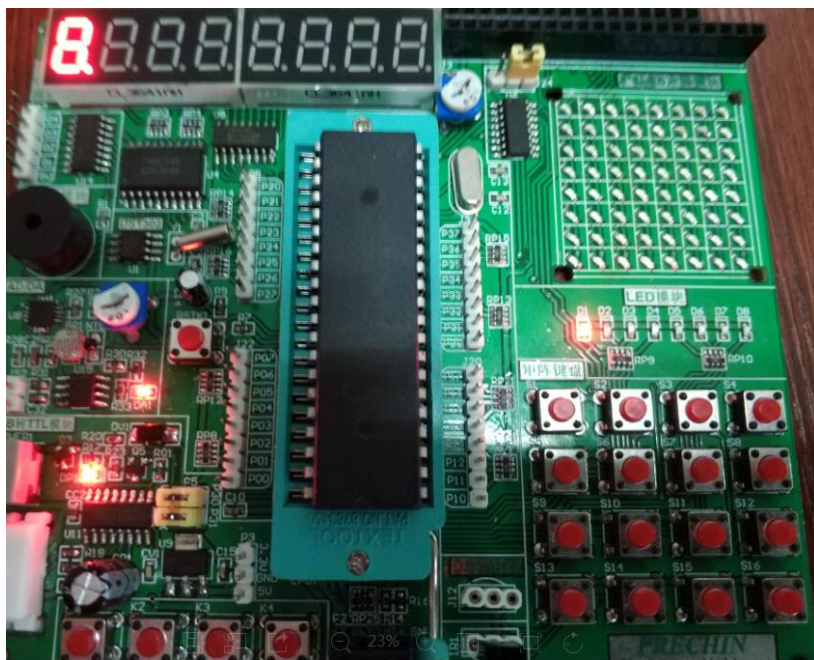
/*****
*****
* 函 数 名          : delay
* 函数功能          : 延时函数, i=1 时, 大约延时 10us
*****
*****/

void delay(u16 i)
{
    while(i--);
}
```

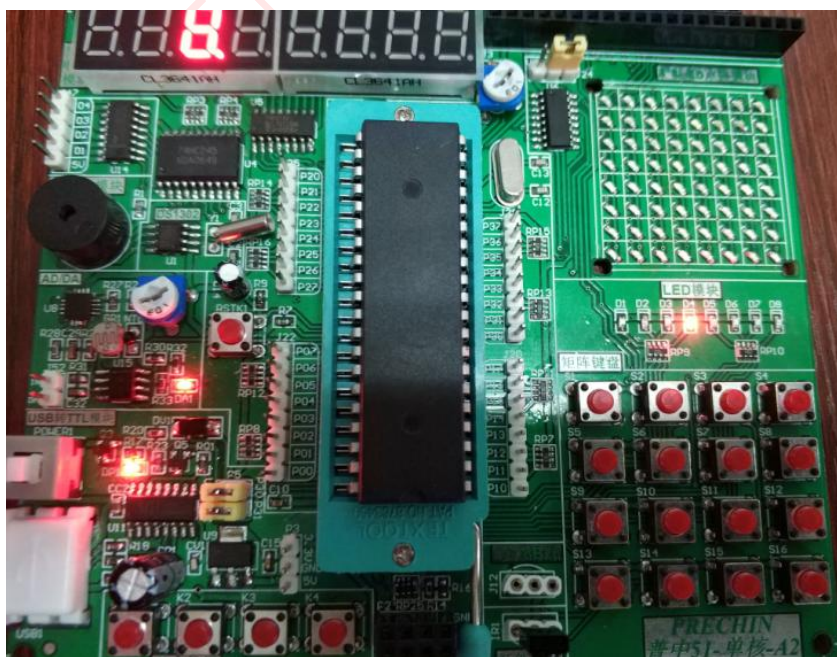
```
/******  
*****  
* 函 数 名      : main  
* 函数功能      : 主函数  
* 输    入      : 无  
* 输    出      : 无  
*****  
*****/  
void main()  
{  
    u8 i;  
    led=~0x01;  
    delay(50000); //大约延时 450ms  
    while(1)  
    {  
        for(i=0;i<7;i++) //将 led 左移一位  
        {  
            led=_crol_(led, 1);  
            delay(50000); //大约延时 450ms  
        }  
        for(i=0;i<7;i++) //将 led 右移一位  
        {  
            led=_cror_(led, 1);  
            delay(50000); //大约延时 450ms  
        }  
    }  
}
```

7.5 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 驱动串口），把编写好的程序编译后将编译产生的 .hex 文件烧入到芯片内（程序的烧入具体操作步骤可参考“第 2 章 开发板功能及使用介绍”的程序烧入（下载）章节），按照如下图所示可以看到开发板上 LED 模块的 D1 指示灯点亮。



(1) “实验 3：LED 流水灯”实验现象如下：



课后作业

(1) 按照上述方法使 D1 指示灯闪烁。

(1) 实现 LED 流水灯效果

(温馨提示：注意控制好延时时间，时间太短，肉眼看不出现象。在编写 LED 流水灯程序时，可以使用如下库函数：

循环左移函数 `_crol_(a,b)` ;a 是左移的值，b 是左移的位数。包含在 `instrins.h` 库函数里面。

循环右移函数， `_cror_(a,b)` ;a 是右移的值，b 是右移的位数。包含在 `instrins.h` 库函数里面)

第 8 章 蜂鸣器实验

在前面章节中我们已经介绍了如何控制 51 单片机的 I/O 口输出高低电平，本章我们通过另外一个实验来讲述 51 单片机 I/O 口的输出。通过 STC89C5XX 芯片的一个 I/O 口控制板载有源/无源蜂鸣器（战神使用的是有源蜂鸣器，V3.0/V2.2/V2.2C 使用的是无源）实现蜂鸣器控制。学习本章可以参考前面 LED 实验章节内容。若结合视频学习效果更佳。本章分为如下几部分内容：

8.1 蜂鸣器介绍

8.2 硬件设计

8.3 软件设计

8.4 实验现象

8.1 蜂鸣器介绍

蜂鸣器是一种一体化结构的电子讯响器，采用直流电压供电，广泛应用于计算机、打印机、复印机、报警器、电子玩具、汽车电子设备、电话机、定时器等电子产品中作发声器件。蜂鸣器主要分为压电式蜂鸣器和电磁式蜂鸣器两种类型。

压电式蜂鸣器主要由多谐振荡器、压电蜂鸣片、阻抗匹配器及共鸣箱、外壳等组成。多谐振荡器由晶体管或集成电路构成，当接通电源后（1.5~15V 直流工作电压），多谐振荡器起振，输出 1.5~5kHz 的音频信号，阻抗匹配器推动压电蜂鸣片发声。

电磁式蜂鸣器由振荡器、电磁线圈、磁铁、振动膜片及外壳等组成。接通电源后，振荡器产生的音频信号电流通过电磁线圈，使电磁线圈产生磁场，振动膜片在电磁线圈和磁铁的相互作用下，周期性地振动发声。

其实一句话就可概括它们之间的区别，想要压电式蜂鸣器发声，需提供一定频率的脉冲信号；想要电磁式蜂鸣器发声，只需提供电源即可。

我们战神-开发板上使用的蜂鸣器是有源蜂鸣器，属于电磁式蜂鸣器类型。这里说的有源，并不是指电源的意思，而是指蜂鸣器内部是否含有振荡电路，有源蜂鸣器内部自带振荡电路，只需提供电源即可发声，而无源蜂鸣器则需提供一定频率的脉冲信号才能发声，频率大小通常在 1.5-5kHz 之间。有源蜂鸣器实物图如下图所示：



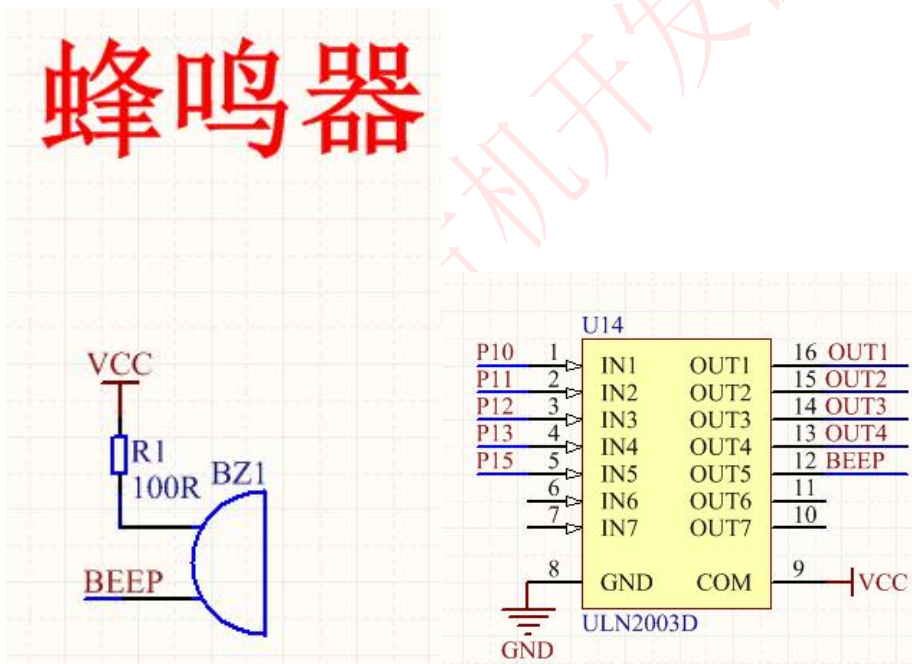
如果给有源蜂鸣器加一个 1.5-5kHz 的脉冲信号，同样也会发声，而且改变这个频率，就可以调节蜂鸣器音调，产生各种不同音色、音调的声音。如果改变输出电平的高低电平占空比，则可以改变蜂鸣器的声音大小。

有关蜂鸣器的详细介绍，大家可以在百度上查找了解。

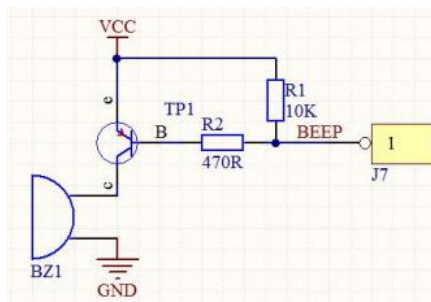
8.2 硬件设计

在前面章节中我们已经对 51 单片机的 GPIO 做了简单介绍，并且还使用了其中 I/O 口直接控制开发板上的 LED。对于本章要实现蜂鸣器的控制，我们能否直接使用 STC89CXX 的 I/O 口驱动呢？答案是否定的，因为 51 单片机 I/O 口的驱动能力较弱（即使外接上拉电阻），而蜂鸣器驱动需要约 30mA，所以非常困难，即使可以驱动，那对于整个芯片的其 I/O 剩下驱动能力就更加弱甚至无法工作。所以我们不会直接使用 I/O 口驱动蜂鸣器，而是通过三极管把电流放大后再驱动蜂鸣器，这样 51 单片机的 I/O 口只需要提供不到 1mA 的电流就可控制蜂鸣器。所以我们也经常说到 51 单片机芯片是用来做控制的，而不是驱动。

我们普中 51-单核-A2 开发板上的蜂鸣器模块电路如下图所示：



从上图中可以看出，该电路是集成的，蜂鸣器控制管脚直接连接到 51 单片机的 P1~5 I/O 口上。开发板上使用的是无源蜂鸣器，它需要一定频率的脉冲（高低电平）才会发声，图中 TP1 PNP 三极管起到一个电流放大作用，从而可以驱动蜂鸣器。当 J7 端子有一个高电平进来时，PNP 三极管 TP1 截止，蜂鸣器不得电，当 J7 端子有一个低电平进来时，PNP 三极管 TP1 导通，蜂鸣器得电，如果 J7 端子有一个一定频率的脉冲信号（高低电平不断翻转）时，无源蜂鸣器发出声音。而有源蜂鸣器则只需电源即可发声，如果您使用的是有源蜂鸣器就需要考虑外界对 J7 引脚电平的干扰问题。大家使用有源蜂鸣器时可参考如下电路设计：



通过电阻 R2 和 PNP 三极管 TP1 进行电流放大，从而驱动蜂鸣器。电阻 R1 是一个上拉电阻，用来防止蜂鸣器误发声。当 BEEP 引脚输出低电平时，PNP 三极管导通，蜂鸣器发声；当 BEEP 引脚输出高电平时，PNP 三极管截止，蜂鸣器停止发声。

如果要想 51 单片机控制蜂鸣器，就必须将单片机管脚通过导线连接到 J7 端子上。本章所要实现的功能是：让蜂鸣器发出声音。由于蜂模块电路是集成的，所以直接使用单片机管脚即可。

8.3 软件设计

本章所要实现的功能是：让蜂鸣器发出声音，即让 P1.5 管脚输出一个低电平，完成后可再让 P1.5 输出一定频率的脉冲控制有源蜂鸣器。

我们直接复制上一章创建的工程，在此模板基础上进行程序开发。为了能够与开发攻略教程对应，将复制过来的模板文件夹重新命名为“实验 4：蜂鸣器”。打开工程直接在 main.c 源文件内进行编程，main.c 内代码如下：

```
/*  
*****  
  
*  
实验现象：下载程序后“蜂鸣器模块”的蜂鸣器发声  
接线说明：单片机-->蜂鸣器模块(具体接线图可见开发攻略对应实验的“实验现象”章节)  
  
注意事项：  
  
*****  
*****  
*/
```

```

*/

#include "reg52.h"           //此文件中定义了单片机的一些特殊功能
寄存器
#include<intrins.h>         //因为要用到左右移函数,所以加入这个头文件

typedef unsigned int u16;    //对数据类型进行声明定义
typedef unsigned char u8;

sbit beep=P1^5;

/*****
*****
* 函 数 名          : delay
* 函数功能          : 延时函数, i=1 时, 大约延时 10us
*****
*****/
void delay(u16 i)
{
    while(i--);
}

/*****
*****
* 函 数 名          : main
* 函数功能          : 主函数
* 输    入          : 无
* 输    出          : 无
*****
*****/
void main()
{
    while(1)
    {
        beep=~beep;
        delay(100); //延时大约 1ms    通过修改此延时时间达到不同的发
声效果
    }
}

```

main.c 文件内代码非常少也很简单, 首先将 51 单片机的头文件包含进来, 然后使用 sbit 关键字来定义 P1.5 管脚, 定义好后即可使用 beep 来替代 P1.5 口的操作。主函数功能非常简单, 直接进入 while 循环, 在循环内不断对 beep

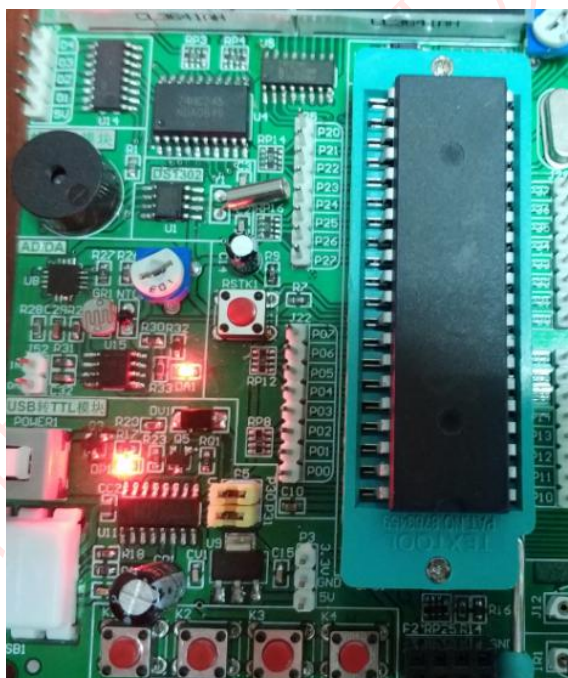
取反，然后延时一定时间，即 P1.5 间隔一定时间输出高低电平，这样就会产生脉冲信号，延时函数在前面章节已介绍过，这里不再叙述。将程序编译无误后下载到开发板上，使用导线连接好即可听到蜂鸣器发出声音了。

大家也可以打开蜂鸣器工程程序查看，这里就不多说。

要想改变音调可以修改延时时间，但要注意频率不能太大或者太小，具体大家可以试着调试。本实验程序同样也可以控制有源蜂鸣器发声。

8.4 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 驱动串口），把编写好的程序编译后将编译产生的 .hex 文件烧入到芯片内，按照如下图所示的接线方式可以听到开发板上蜂鸣器发出美妙的声音。



第 9 章 动态数码管实验

在前面章节中我们已经介绍了如何使用 51 单片机控制静态数码管显示，如果在设计系统时需要显示多位数值，这个时候采用静态数码管显示就非常不好，因此就需要采用另外一种显示方式，即数码管动态显示。开发板上板载 2 个四位一体的共阴数码管，本章我们就来介绍下如何使用 51 单片机控制动态数码管显示。本章所要实现的功能是：控制动态数码管从左至右显示数字 0-7。学习本章可以参考前面的实验章节内容。若结合视频学习效果更佳。本章分为如下几部分内容：

9.1 数码管介绍

9.2 74HC245 和 74HC138 芯片介绍

9.3 硬件设计

9.4 软件设计

9.5 实验现象

9.1 数码管介绍

上一章我们主要是介绍一位数码管的内部结构及控制原理。下面我们再来介绍下多位数码管及动态显示原理的相关知识。

9.1.1 多位数码管简介

多位数码管，即是两个或两个以上单个数码管并列集中在一起形成一体的数码管。当多位一体时，它们内部的公共端是独立的，而负责显示什么数字的段线全部是连接在一起的，独立的公共端可以控制多位一体中的哪一位数码管点亮，而连接在一起的段线可以控制这个能点亮数码管亮什么数字，通常我们把公共端叫做“位选线”，连接在一起的段线叫做“段选线”，有了这两个线后，通过单片机及外部驱动电路就可以控制任意的数码管显示任意的数字了。

一般一位数码管有 10 个引脚，二位数码管也是 10 个引脚，四位数码管是 12 个引脚，关于具体的引脚及段、位标号大家可以查询相关资料，最简单的办法就是用数字万用表测量，若没有数字万用表也可用 5V 直流电源串接 1k 电阻后测量，将测量结果记录，通过统计便可绘制出引脚标号。多位数码管有许多是按一定要求设计的，引脚不完全按照一般规则设定，所以需要在使用时查找手册，最直接的办法就是按照数码管上的标示向生产商要求。

我们开发板上使用了 2 个四位一体的共阴数码管，这样可在上面同时显示 8 个数值。

9.1.2 数码管动态显示原理

多位数码管依然可以静态显示，但是显示时要么只显示一位数码管，否则一体的多位同时显示必须时显示相同内容。当多位数码管应用于某一系统时，它们的“位选”是可独立控制的，而“段选”是连接在一起的，我们可以通过位选信号控制哪几个数码管亮，而在同一时刻，位选选通的所有数码管上显示的数字始终都是一样的，因为它们的段选是连接在一起的，所以送入所有数码管的段选

信号都是相同的，那么它们显示的数字必定一样，数码管的这种显示方法叫做静态显示。

而动态显示，就是利用减少段选线，分开位选线，利用位选线不同时选择通断，改变段选数据来实现的。比如在第一次选中第一位数码管时，给段选数据 0，下一次位选中第二位数码管时显示 1。为了在显示 1 的时候，0 不会消失（当然实际上是消失了），必须在人肉眼观察不到的时间里再次点亮第一次点亮的 0。而这时就需要记住，人的肉眼正常情况下只能分辨变化超过 24ms 间隔的运动。也就是说，在下次点亮 0 这个数字的时间差不得大于 24ms。这时就会发现，数码管点亮是在向右或者向左一位一位点亮，形成了动态效果。如果把间隔时间改长就能直接展现这现象。

数码管动态显示的应用非常多，所以大家一定要认真学好数码管的动态显示方法。

9.2 74HC245 和 74HC138 芯片介绍

通过前面内容的介绍我们知道，要使单片机能控制开发板上 2 位一体的共阴数码管显示，仅靠单片机 I/O 口来驱动是不行的，这里就需要增加外部驱动芯片，开发板上使用的是 74HC245 芯片。2 个 4 位一体的共阴数码管的位选线有 8 根，直接让单片机 I/O 口控制是没有任何问题的，但考虑到 51 单片机 I/O 口资源的限制，通常会使用一种 I/O 扩展芯片，比如 74HC138、74HC164、74HC595 芯片等，只需要很少的单片机 I/O 口就可以扩展出 8 个控制口，通过级联方式甚至可扩展出更多的控制口（这个在后面 LED 点阵实验章节中会有详细介绍）。我们开发板上使用的是 74HC138 译码器芯片，只需单片机 3 个 I/O 口就可以实现 8 个位选管脚的控制，大大节省了芯片的 I/O 资源。

下面就来看看这两个芯片功能及使用方法。

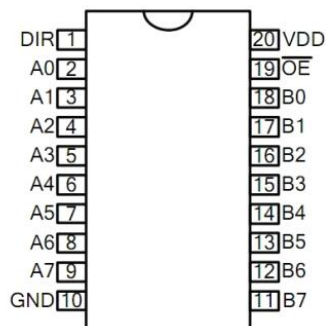
9.2.1 74HC245 芯片简介

74HC245 是一种三态输出、八路信号收发器，主要应用于大屏显示，以及其它的消费类电子产品中增加驱动。

（1）主要特性

- ①采用 CMOS 工艺
- ②宽电压工作范围：3.0V-5.0V
- ③双向三态输出
- ④八线双向收发器
- ⑤封装形式：SOP20、SOP20-2、TSSOP20、DIP20

(2) 管脚功能定义



管脚定义说明

符号	管脚名称	管脚号	说明
A0—A7	数据输入/输出	2—9	
B0—B7	数据输入/输出	18—11	
$\overline{\text{OE}}$	输出使能	19	
DIR	方向控制	1	DIR=1,A→B; DIR=0,B→A
GND	逻辑地	20	逻辑地
VDD	逻辑电源	10	电源端

功能真值表

输出使能	输出控制	工作状态
$\overline{\text{OE}}$	DIR	
L	L	Bn 输入 An 输出
L	H	An 输入 Bn 输出
H	X	高阻态

从上面的管脚功能定义说明及真值表可以知道该芯片使用方法非常简单，给 OE 使能管脚低电平，DIR 管脚为高电平传输方向是 A→B 输出，为低电平传输方向是 B→A，至于输出高电平还是输出低电平取决于输入端的状态，如果输入为低电平，输出即为低；输入为高电平，输出即为高。如果 OE 使能管脚为高电平，不论 DIR 管脚是高还是低，输出是高阻态。

通常我们使用 74HC245 芯片用作驱动只会让其在在一个方向输出，即 DIR 管脚为高电平，传输方向是 A→B。

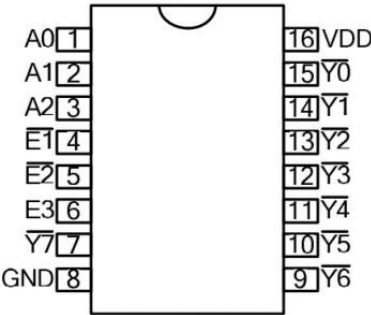
9.2.2 74HC138 芯片简介

74HC138D 是一种三通道输入、八通道输出译码器，主要应用于消费类电子产品。

(1) 主要特性

- ①采用 CMOS 工艺
- ②低功耗
- ③工作电压：3.0V-5.0V
- ④封装形式：SOP16

(2) 管脚功能定义



管脚说明

名称	功能说明	管脚号
$\overline{Y0} \sim \overline{Y6}, \overline{Y7}$	数据输出	15—9,7
A0—A2	数据输入	1—3
$\overline{E1}, \overline{E2}, \overline{E3}$	使能控制	4—6
VDD	逻辑电源	16
GND	逻辑地	8

真值表：

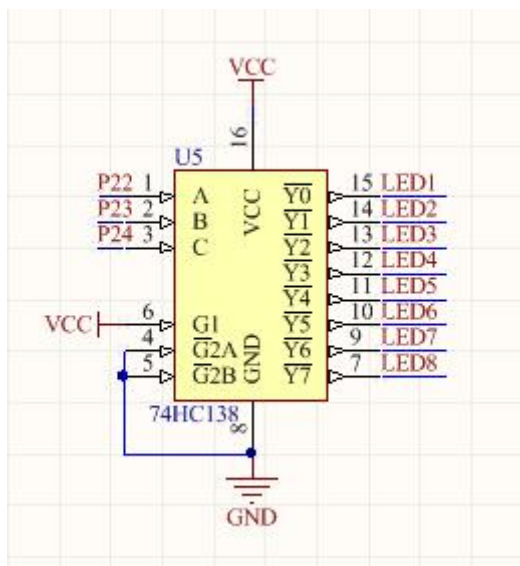
输入						输出							
$\overline{E1}$	$\overline{E2}$	E3	A0	A1	A2	$\overline{Y0}$	$\overline{Y1}$	$\overline{Y2}$	$\overline{Y3}$	$\overline{Y4}$	$\overline{Y5}$	$\overline{Y6}$	$\overline{Y7}$
H	X	X	X	X	X	H	H	H	H	H	H	H	H
X	H	X	X	X	X	H	H	H	H	H	H	H	H
X	X	L	X	X	X	H	H	H	H	H	H	H	H
L	L	H	L	L	L	L	H	H	H	H	H	H	H
L	L	H	H	L	L	H	L	H	H	H	H	H	H
L	L	H	L	H	L	H	H	L	H	H	H	H	H
L	L	H	H	H	L	H	H	H	L	H	H	H	H
L	L	H	L	L	H	H	H	H	H	L	H	H	H
L	L	H	H	L	H	H	H	H	H	H	L	H	H
L	L	H	L	H	H	H	H	H	H	H	H	L	H
L	L	H	H	H	H	H	H	H	H	H	H	H	L

这里给大家总结一个方法：A0、A1、A2 输入就相当于 3 位 2 进制数，A0 是低位，A1 是次高位，A2 是高位。而 Y0-Y7 具体哪一个输出有效电平，就看输入二进制对应的十进制数值。比如输入是 101（A2，A1，A0），其对应的十进制数是 5，所以 Y5 输出有效电平（低电平）。

9.3 硬件设计

我们开发板上的动态数码管模块电路如下图所示:





从上图中可以看出，该电路是集成的，动态数码管的控制管脚直接连接到 51 单片机的 I/O 上，段选端连接到 P0 上，位选端连接到 LED P2 口上。段选端通过 74HC245 芯片驱动这两个共阴数码管的 a-dp 段。

如果要想 51 单片机控制动态数码管，就必须通过单片机管脚输出低电平再有 138 译码器控制位选点亮数码管。由于动态数码管模块电路是集成的，所以使用单片机管脚即可。

9.4 软件设计

本章所要实现的功能是：控制动态数码管从左至右显示数字 0~7。

我们直接复制上一章创建的工程，在此模板基础上进行程序开发。为了能够与开发攻略教程对应，将复制过来的模板文件夹重新命名为“实验 7：动态数码管显示”。打开工程直接在 `main.c` 源文件内进行编程，`main.c` 内代码如下：

```
/*****
```

实验现象：下载程序后“动态数码管模块”从左至右显示 0-7

接线说明： 单片机-->动态数码管模块(具体接线图可见开发攻略对应实验的“实验现象”章节)

注意事项:

***** /

```
#include "reg52.h"
```

//此文件中定义了单片机的一些特殊功能

寄存器

```

typedef unsigned int u16;      //对数据类型进行声明定义
typedef unsigned char u8;

sbit LSA=P2^2;
sbit LSB=P2^3;
sbit LSC=P2^4;

u8 code smgduan[17]={0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07,
                    0x7f, 0x6f, 0x77, 0x7c, 0x39, 0x5e, 0x79, 0x71}; // 显示
0~F 的值

/*****
*****
* 函 数 名          : delay
* 函数功能          : 延时函数, i=1 时, 大约延时 10us
*****
*****/
void delay(u16 i)
{
    while(i--);
}

/*****
*****
* 函 数 名          : DigDisplay
* 函数功能          : 数码管动态扫描函数, 循环扫描 8 个数码管显示
*****
*****/
void DigDisplay()
{
    u8 i;
    for(i=0; i<8; i++)
    {
        switch(i) //位选, 选择点亮的数码管,
        {
            case(0):
                LSA=1;LSB=1;LSC=1; break;//显示第 0 位
            case(1):
                LSA=0;LSB=1;LSC=1; break;//显示第 1 位
            case(2):
                LSA=1;LSB=0;LSC=1; break;//显示第 2 位
            case(3):

```

```

        LSA=0;LSB=0;LSC=1; break;//显示第 3 位
    case(4):
        LSA=1;LSB=1;LSC=0; break;//显示第 4 位
    case(5):
        LSA=0;LSB=1;LSC=0; break;//显示第 5 位
    case(6):
        LSA=1;LSB=0;LSC=0; break;//显示第 6 位
    case(7):
        LSA=0;LSB=0;LSC=0; break;//显示第 7 位
    }
    P0=smgduan[i]; //发送段码
    delay(100); //间隔一段时间扫描
    P0=0x00; //消隐
}
}

/*****
*****
* 函 数 名      : main
* 函数功能      : 主函数
* 输    入      : 无
* 输    出      : 无
*****
*****/
void main()
{
    while(1)
    {
        DigDisplay(); //数码管显示函数
    }
}

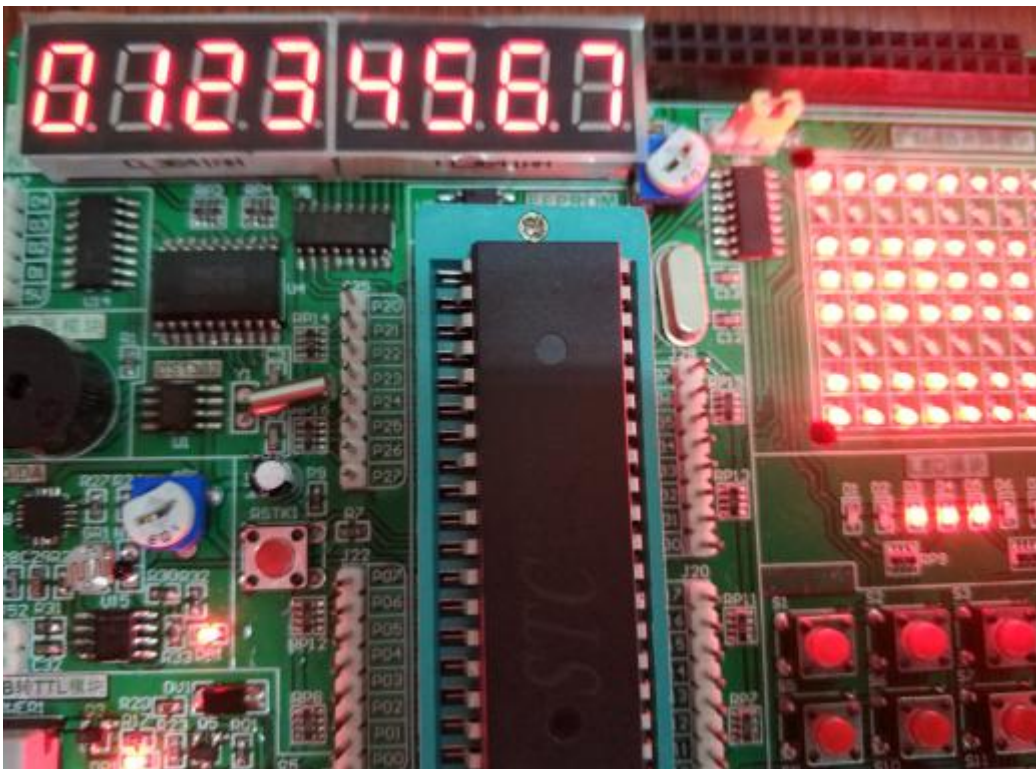
```

main.c 文件内代码非常少也很简单，首先将 51 单片机的头文件包含进来，然后定义单片机控制 38 译码器的制引脚，并将共阴数码管 0-F 段码数据使用数组定义好。主函数功能非常简单，直接进入 while 循环，在循环体内执行 DigDisplay() 数码管动态显示函数。该函数是根据前面介绍动态数码管工作原理所编写，即选中所要显示的那位数码管，然后发送在该位数码管上所要显示的段码数据，延时一定时间后在将段选口清零即消隐（消除之前的显示），如此循环 8 次即可实现 8 位数码管显示。在动态显示中，要注意延时时间，只要保证在人肉眼所能感觉时间之内即可让我们看到所要显示的内容就像是同时显示一样。如果时间过长或者过短都可能会影响数码管的显示效果，大家可以在例程基础上试

着修改延时观察效果。

9.5 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 驱动串口），把编写好的程序编译后将编译产生的 .hex 文件烧入到芯片内，按照如下图所示的接线方式可以看到开发板上动态数码管模块中的 8 个数码管从左至右同时显示数字 0-7。



课后作业

- (1) 在 8 位数码管上从左至右显示“1314-520”。
- (2) 使用单片机 P1 口直接控制数码管位选，而不经 38 译码器。（温馨提示：可将选通每位数码管位选值赋值给 P1 口即可）

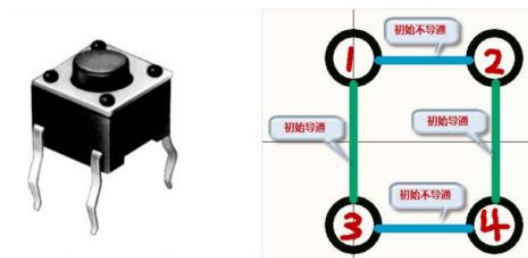
第 10 章 独立按键实验

前几章我们介绍的都是 I/O 口输出的使用，这一章我们通过独立按键实验来介绍下 I/O 口作为输入的使用。开发板上板载 8 个独立按键。通过本章的介绍让大家学会如何使用 51 单片机来检测独立按键的控制。本章所要实现的功能是：通过开发板上的独立按键控制 D1 指示灯亮灭。学习本章可以参考前面的实验章节内容。若结合视频学习效果更佳。本章分为如下几部分内容：

- 10.1 按键介绍
- 10.2 硬件设计
- 10.3 软件设计
- 10.4 实验现象

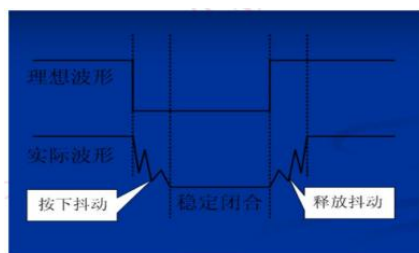
10.1 按键介绍

按键是一种电子开关，使用时轻轻按开关按钮就可使开关接通，当松开手时，开关断开。我们开发板上使用的按键及内部简易图如下图所示：



按键管脚两端距离长的表示默认是导通状态，距离短的默认是断开状态，如果按键按下，初始导通状态变为断开，初始断开状态变为导通。

通常的按键所用开关为机械弹性开关，当机械触点断开、闭合时，电压信号如下图所示：



由于机械点的弹性作用，按键开关在闭合时不会马上稳定的接通，在断开时也不会一下子断开，因而在闭合和断开的瞬间均伴随着一连串的抖动。抖动时间的长短由按键的机械特性决定的，一般为 5ms 到 10ms。按键稳定闭合时间的长短则由操作人员的按键动作决定的，一般为零点几秒至数秒。按键抖动会引起按键被误读多次。为了确保 CPU 对按键的一次闭合仅作一次处理，必须进行消抖。

按键消抖有两种方式，一种是硬件消抖，另一种是软件消抖。为了使电路更

加简单，通常采用软件消抖。我们开发板也是采用软件消抖，一般来说一个简单的按键消抖就是先读取按键的状态，如果得到按键按下之后，延时 10ms，再次读取按键的状态，如果按键还是按下状态，那么说明按键已经按下。其中延时 10ms 就是软件消抖处理，至于硬件消抖，大家可以百度了解下，网上都有非常详细的介绍。这里给大家列出单片机常用的软件去抖动方法：

- 1，先设置 I/O 口为高电平（由于开发板 I/O 都有上拉电阻，所以默认 I/O 为高电平）。
- 2，读取 I/O 口电平确认是否有按键按下。
- 3，如有 I/O 电平为低电平后，延时几个毫秒。
- 4，再读取该 I/O 电平，如果任然为低电平，说明对应按键按下。
- 5，执行相应按键的程序。

键盘分为编码键盘和非编码键盘。键盘上闭合键的识别由专用的硬件编码器实现，并产生键编码号或键值的称为编码键盘，如计算机键盘。而靠软件编程来识别的键盘称为非编码键盘，在单片机组成的各种系统中，用的较多的是非编码键盘。非编码键盘又分为独立键盘和行列式键盘（常说的矩阵键盘）。本章介绍的就是独立式键盘。

10.1.1 独立按键检测原理

独立按键电路构成是由各个按键的一个管脚连接在一起接地，按键其他引脚分别接到单片机 I/O 口。

我们知道单片机的 I/O 口既可作为输出也可作为输入使用，当检测按键时用的是它的输入功能，独立按键的一端接地，另一端与单片机的某个 I/O 口相连，开始时先给该 I/O 口赋一高电平，然后让单片机不断地检测该 I/O 口是否变为低电平，当按键闭合时，即相当于该 I/O 口通过按键与地相连，变成低电平，程序一旦检测到 I/O 口变为低电平则说明按键被按下，然后执行相应的指令。

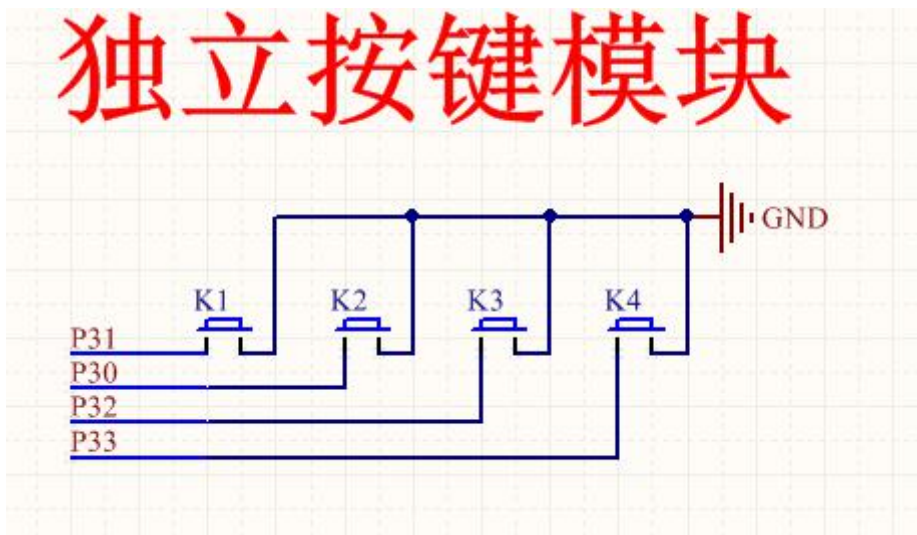
10.2 硬件设计

本实验使用到硬件资源如下：

- （1）LED 模块中的 D1 指示灯

(2) K1 按键

LED 模块电路在前面章节都介绍过，这里就不多说，我们开发板上的独立按键模块电路如下图所示：



从上图中可以看出，该电路是独立的，4 个独立按键的控制管脚直接连接到 51 单片机的 P3⁰~P3³ I/O 上。

如果要想 51 单片机能够检测按键是否按下，就必须通过单片机管脚来控制独立按键。由于独立按键模块电路是集成的，所以使用单片机 P3⁰~P3³ 管脚即可。

10.3 软件设计

本章所要实现的功能是：通过开发板上的独立按键控制 D1 指示灯亮灭。

我们直接复制上一章创建的工程，在此模板基础上进行程序开发。为了能够与开发攻略教程对应，将复制过来的模板文件夹重新命名为“实验 8：独立按键”。打开工程直接在 main.c 源文件内进行编程，main.c 内代码如下：

```
/*  
*****  
*****  
  
*  
实验现象：下载程序后按下 K1 按键可以对 D1 小灯状态取反  
接线说明：（具体接线图可见开发攻略对应实验的“实验现象”章节）  
*/
```

注意事项:

```

*****
*****

*/

#include "reg52.h"          //此文件中定义了单片机的一些特殊功能
寄存器

typedef unsigned int u16;    //对数据类型进行声明定义
typedef unsigned char u8;

sbit k1=P3^1;  //定义 P31 口是 k1
sbit led=P2^0;  //定义 P20 口是 led

/*****
*****
* 函 数 名          : delay
* 函数功能          : 延时函数, i=1 时, 大约延时 10us
*****
*****/
void delay(u16 i)
{
    while(i--);
}

/*****
*****
* 函 数 名          : keypros
* 函数功能          : 按键处理函数, 判断按键 K1 是否按下
*****
*****/
void keypros()
{
    if(k1==0)          //检测按键 K1 是否按下
    {
        delay(1000);  //消除抖动 一般大约 10ms
        if(k1==0)    //再次判断按键是否按下
        {
            led=~led;  //led 状态取反
        }
        while(!k1);   //检测按键是否松开
    }
}

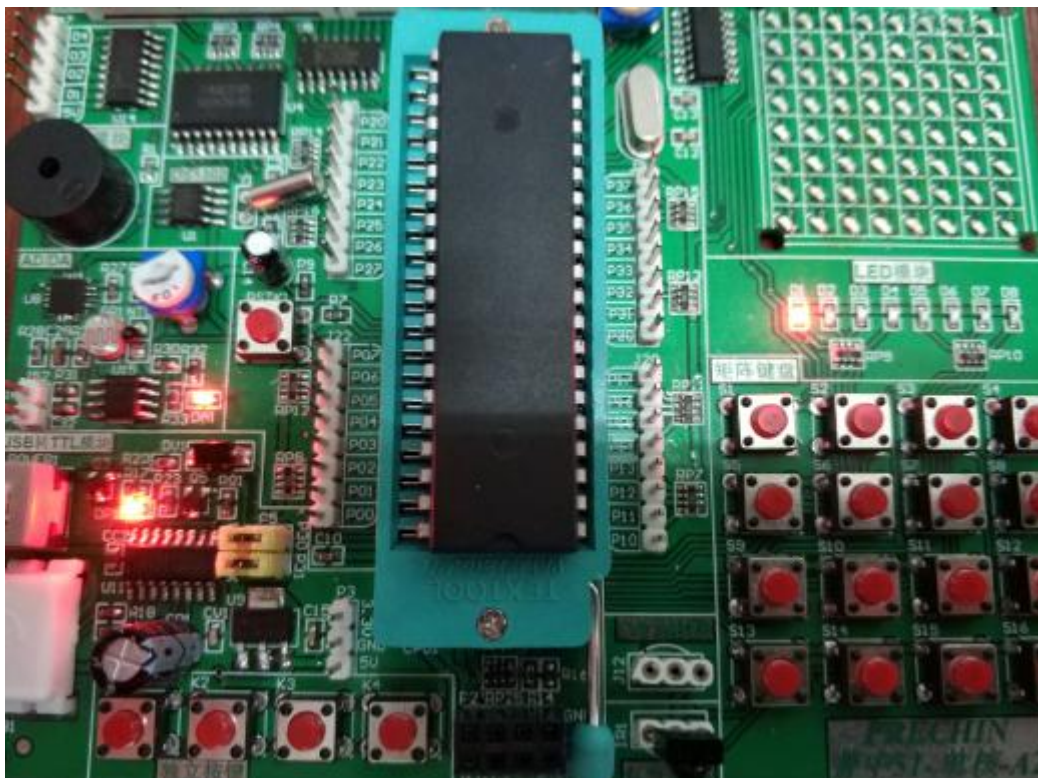
```

```
    }  
}  
  
/*****  
*****  
* 函 数 名      : main  
* 函数功能      : 主函数  
* 输    入      : 无  
* 输    出      : 无  
*****  
*****/  
void main()  
{  
    led=1;  
    while(1)  
    {  
        keypros(); //按键处理函数  
    }  
}
```

main.c 文件内代码非常少也很简单，首先将 51 单片机的头文件包含进来，然后定义按键 K1 和 LED1 的控制脚。主函数功能非常简单，首先让 led 设置为高电平，即熄灭 D1 指示灯。然后进入 while 循环，在循环体内执行 keypros() 按键处理函数。该函数是根据前面介绍独立按键工作原理所编写，即判断按键 K1 是否按下，如果按下则延时 10ms 进行消抖，再次判断按键 K1 是否按下，如果 K1 确实按下，则执行 LED 状态翻转控制，最后判断按键是否松开。

10.4 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 驱动串口），把编写好的程序编译后将编译产生的 .hex 文件烧入到芯片内，按照如下图所示的接线方式可以看到当按键 K1 按下后，指示灯 D1 点亮，再次按下则熄灭，如此循环。



第 11 章 矩阵按键实验

上一章我们已经介绍了独立按键的构成以及使用方法，本章我们介绍按键电路的另一种模型：矩阵按键。开发板上板载了一个 4*4 矩阵键盘。通过本章的介绍让大家学会如何使用 51 单片机来检测矩阵按键的控制。本章所要实现的功能是：通过开发板上的矩阵键盘控制静态数码管显示对应的键值 0-F。学习本章可以参考前面的实验章节内容。若结合视频学习效果更佳。本章分为如下几部分内

容:

11.1 矩阵按键介绍

11.2 硬件设计

11.3 软件设计

11.4 实验现象

11.1 矩阵按键介绍

独立键盘与单片机连接时，每一个按键都需要单片机的一个 I/O 口，若某单片机系统需较多按键，如果用独立按键便会占用过多的 I/O 口资源。单片机系统中 I/O 口资源往往比较宝贵，当用到多个按键时为了减少 I/O 口引脚，引入了矩阵键盘。

本章以 4*4 矩阵键盘为例讲解其工作原理和检测方法。开发板上将 16 个按

键排成 4 行 4 列，第一行将每个按键的一端连接在一起构成行线，第一列将每个按键的另一端连接在一起构成列线，这样便一共有 4 行 4 列共 8 根线，我们将这 8 根线连接到单片机的 8 个 I/O 口上，通过程序扫描键盘就可检测 16 个键。用这种方法我们也可实现 3 行 3 列 9 个键、5 行 5 列 25 个键、6 行 6 列 36 个键甚至更多。

无论是独立键盘还是矩阵键盘，单片机检测其是否被按下的依据都是一样的，也就是检测与该键对应的 I/O 口是否为低电平。独立键盘有一端固定为低电平，单片机写程序检测时比较方便。而矩阵键盘两端都与单片机 I/O 口相连，因此在检测时需编程通过单片机 I/O 口送出低电平。检测方法有多种，最常用的是行列扫描和线翻转法。

行列扫描法检测时，先送一列为低电平，其余几列全为高电平(此时我们确定了列数)，然后立即轮流检测一次各行是否有低电平，若检测到某一行低电平(这时我们又确定了行数)，则我们便可确认当前被按下的键是哪一行哪一列的，用同样方法轮流送各列一次低电平，再轮流检测一次各行是否变为低电平，这样即可检测完所有的按键，当有键被按下时便可判断出按下的键是哪一个键。当然我们也可以将行线置低电平，扫描列是否有低电平。从而达到整个键盘的检测。

线翻转法，就是使所有行线为低电平时，检测所有列线是否有低电平，如果有，就记录列线值；然后再翻转，使所有列线都为低电平，检测所有行线的值，由于有按键按下，行线的值也会有变化，记录行线的值。从而就可以检测到全部按键。

矩阵键盘也少不了按键消抖的环节，本章实验中采用的是行列扫描法来检测哪个按键按下。

11.2 硬件设计

本实验使用到硬件资源如下：

- (1) 动态数码管
- (2) 4*4 矩阵按键

静态数码管模块电路在前面章节都介绍过，这里就不多说，我们开发板上的

矩阵按键模块

The diagram illustrates a 4x4 matrix keypad module. The keypad is composed of 16 buttons, labeled S1 through S16, arranged in a 4x4 grid. The columns are connected to microcontroller pins P10, P11, P12, and P13. The rows are connected to microcontroller pins P14, P15, P16, and P17. Each button is connected to a row and a column line, with pull-up resistors on the column lines.

如果要想 51 单片机能够检测按键是否按下，就必须通过单片机管脚来控制矩阵按键。由于矩阵按键模块电路是集成的，所以使用单片机 P1 管脚即可。

本章所要实现的功能是：通过开发板上的矩阵键盘控制动态数码管显示对应的键值 0-F。

```

/*****
****

```

```

*
实验现象：下载程序后数码管显示 0，按下矩阵按键上的按键显示对应的数字

注意事项：

*****
*****

*/
#include "reg52.h"          //此文件中定义了单片机的一些特殊功能寄存器

typedef unsigned int u16;    //对数据类型进行声明定义
typedef unsigned char u8;

#define GPIO_DIG P0
#define GPIO_KEY P1

u8 KeyValue; //用来存放读取到的键值

u8 code smgduan[17]={0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8,
                    0x80, 0x90, 0x88, 0x83, 0xc6, 0xa1, 0x86, 0x8e}; // 显示 0~F 的值

/*****
*****
* 函 数 名      : delay
* 函数功能      : 延时函数，i=1 时，大约延时 10us
*****
*****/
void delay(u16 i)
{
    while(i--);
}

/*****
*****
* 函 数 名      : KeyDown
* 函数功能      : 检测有按键按下并读取键值
* 输    入      : 无

```

```

* 输 出      : 无
*****
*****/
void KeyDown(void)
{
    char a=0;
    GPIO_KEY=0x0f;
    if(GPIO_KEY!=0x0f)//读取按键是否按下
    {
        delay(1000); //延时 10ms 进行消抖
        if(GPIO_KEY!=0x0f)//再次检测键盘是否按下
        {
            //测试列
            GPIO_KEY=0X0F;
            switch(GPIO_KEY)
            {
                case(0X07):  KeyValue=0;break;
                case(0X0b):  KeyValue=1;break;
                case(0X0d):  KeyValue=2;break;
                case(0X0e):  KeyValue=3;break;
            }
            //测试行
            GPIO_KEY=0XF0;
            switch(GPIO_KEY)
            {
                case(0X70):  KeyValue=KeyValue;break;
                case(0Xb0):  KeyValue=KeyValue+4;break;
                case(0Xd0):  KeyValue=KeyValue+8;break;
                case(0Xe0):  KeyValue=KeyValue+12;break;
            }
        }
    }
    while((a<50)&&(GPIO_KEY!=0xf0)) //检测按键松手检测
    {
        delay(100);
        a++;
    }
}

/*****
*****
* 函 数 名      : main

```

```

* 函数功能      : 主函数
* 输    入      : 无
* 输    出      : 无
*****
*****/
void main()
{

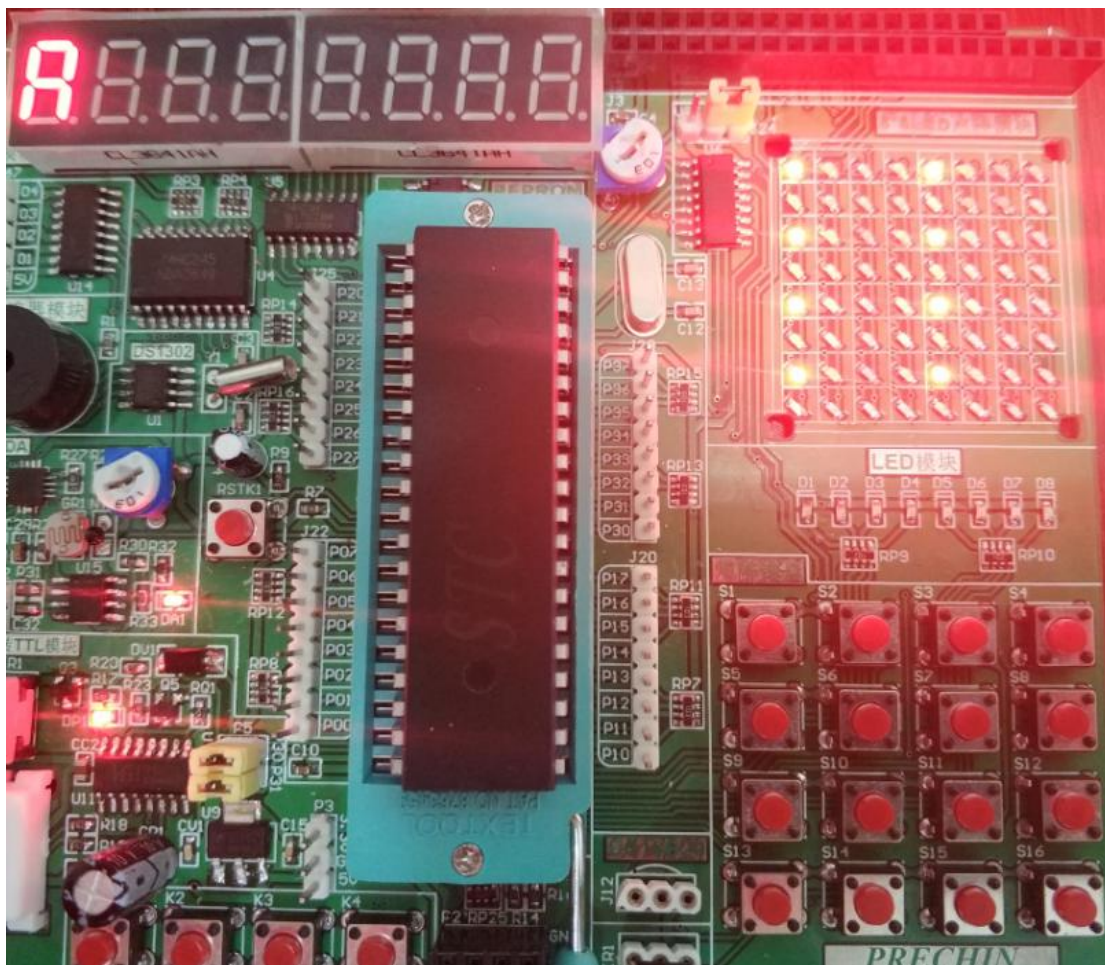
    while(1)
    {
        KeyDown();      //按键判断函数
        GPIO_DIG=~smgduan[KeyValue];    //
    }
}

```

main.c 文件内代码非常少也很简单，首先将 51 单片机的头文件包含进来，使用 define 宏定义静态数码管和矩阵按键控制端口，这样一方面是方便程序的编写，另一方面方便程序修改和移植。然后定义程序中所要使用到的变量，用于存储相应的值。主函数功能非常简单，直接进入 while 循环，在循环体内执行 KeyDown() 矩阵按键扫描检测函数。该函数是根据前面介绍矩阵按键行列扫描原理所编写，具体大家看代码。通过该函数可获取相应的键值存储在全局变量 KeyValue 中，然后将该键值作为数组下标调用对应的段码在静态数码管上显示。

11.4 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 驱动串口），把编写好的程序编译后将编译产生的 .hex 文件烧入到芯片内，按照如下图所示的接线方式可以看到按下矩阵键盘键对应显示如下：



课后作业

- (1) 使用矩阵按键设计一个简易计算器。
- (2) 使用矩阵按键设计一个简易电子琴。

(温馨提示：可参考前面实验章节将内容进行整合，可以在百度上搜索相关资料，网上有很多)

第 12 章 单片机 IO 扩展(串转并)--74HC595

在上一章节，我们介绍了如何使用 74HC165 来扩展 IO，它是一种并口转串口的方式。本章就来介绍另外一种 IO 口扩展方式-串转并，使用的芯片是 74HC595。开发板板载 1 个 74HC595 芯片，通过芯片级联扩展了 8 个 IO。通过本章的学习，让大家学会使用 74HC595 芯片来扩展 IO 口。本章所要实现的功能是：使用开发板上的 74HC595 模块控制 LED。学习本章可以参考前面的实验章节内容。若结合视频学习效果更佳。本章分为如下几部分内容：

12.1 74HC595 芯片介绍

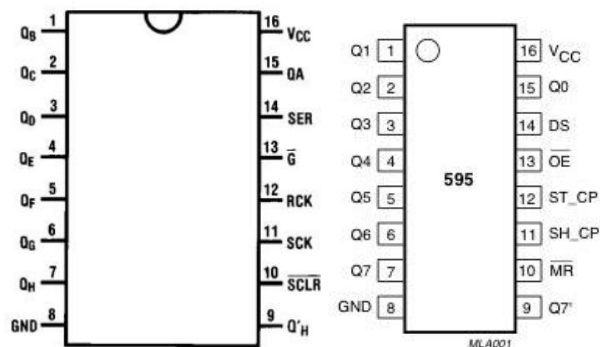
12.2 硬件设计

12.3 软件设计

12.4 实验现象

12.1 74HC595 芯片介绍

74HC595 是一个 8 位串行输入、并行输出的位移寄存器，其中并行输出为三态输出（即高电平、低电平和高阻抗）。芯片管脚及功能说明如下：



上面两张都是 74HC595 芯片管脚图，细心的朋友就会发现左侧的 1 脚是 QB，而右侧芯片的 1 脚是 Q1，左侧芯片的 11 脚是 SCK，而右侧芯片的 11 脚是 SH_CP，还有很多其他管脚不一样，其实这个都没有什么，不同人在绘制芯片管脚图时命名可能不一样而已，看一个芯片重点是管脚功能。

15 和 1 到 7 脚 QA—QH：并行数据输出

9 脚 QH 非：串行数据输出

10 脚 SCLK 非（MR）：低电平复位引脚

11 脚 SCK（SHCP）：移位寄存器时钟输入

12 脚 RCK（STCP）：存储寄存器时钟输入

13 脚 G 非（OE）：输出有效

14 脚 SER（DS）：串行数据输入

74HC595 是具有 8 位移位寄存器和一个存储器，三态输出功能。移位寄存器和存储器是分别的时钟。数据在 SCK 的上升沿输入，在 RCK 的上升沿进入到存储器中。如果两个时钟连在一起，则移位寄存器总是比存储器早一个脉冲。移位寄存器有一个串行输入（DS），和一个串行输出（QH 非），和一个异步的低电平复位，存储器有一个并行 8 位的，具有三态的总线输出，当 MR 为高电平，OE 为低电平时，数据在 SHCP 上升沿进入移位寄存器，在 STCP 上升沿输出到并行端口。

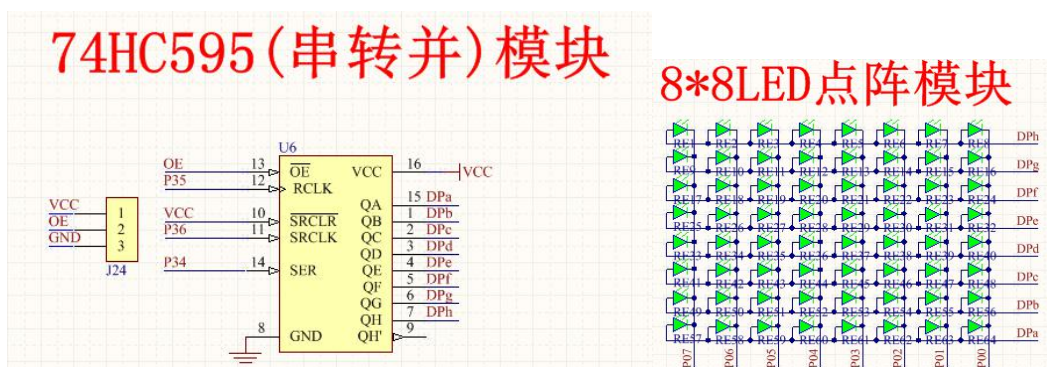
有关 74HC595 芯片的更多详细介绍，大家可以在光盘“\6—开发板芯片资料”内查看。

12.2 硬件设计

本实验使用到硬件资源如下：

- (1) 8X8LED 点阵模块
- (2) 74HC595 模块

LED 模块电路在前面章节都介绍过，这里就不多说，我们开发板上的 74HC595 模块电路如下图所示：



从上图中可以看出，该电路是集成的，74HC595 需要用到的控制管脚 RCLK、SRCLK、SER 直接连接到 51 单片机的 P3⁴~P3⁶ IO 口上，输出端则是直接连接到 LED 点阵模块的行端口上。

如果要想 51 单片机能够控制 74HC595 输出数据，就必须通过单片机管脚直接控制 74HC595。由于 74HC595 模块电路是集成的，所以使用单片机 P3⁴~P3⁶ 管脚即可。

12.3 软件设计

本章所要实现的功能是：使用开发板上的 74HC595 模块控制 LED。

我们直接复制上一章创建的工程，在此模板基础上进行程序开发。为了能够与开发攻略教程对应，将复制过来的模板文件夹重新命名为“实验 11：单片机 IO 扩展--74HC595”。打开工程直接在 main.c 源文件内进行编程，main.c 内代码如下：

```
/*
*****
*****

*
实验现象：下载程序后，LED 灯从左往右点亮，类似流水灯效果
*/
```

接线说明：（具体接线图可见开发攻略对应实验的“实验现象”章节）

注意事项：

```

*****
*****

*/

#include "reg51.h"          //此文件中定义了单片机的一些特殊功能
寄存器
#include "intrins.h"

typedef unsigned int u16;    //对数据类型进行声明定义
typedef unsigned char u8;
u8 ledNum;

//--定义使用的 I/O 口--//
sbit SRCLK=P3^6;
sbit RCLK=P3^5;
sbit SER=P3^4;
sbit LED=P0^7;

/*****
*****
* 函 数 名          : delay
* 函数功能          : 延时函数, i=1 时, 大约延时 10us
*****
*****/
void delay(u16 i)
{
    while(i--);
}

/*****
*****
* 函 数 名          : Hc595SendByte(u8 dat)
* 函数功能          : 向 74H595 发送一个字节的数
* 输    入          : 无
* 输    出          : 无
*****

```

```

*****/
void Hc595SendByte(u8 dat)
{
    u8 a;

    SRCLK = 1;
    RCLK = 1;

    for(a=0;a<8;a++)        //发送 8 位数
    {
        SER = dat >> 7;      //从最高位开始发送
        dat <<= 1;

        SRCLK = 0;          //发送时序
        _nop_();
        _nop_();
        SRCLK = 1;
    }
    RCLK = 0;
    _nop_();
    _nop_();
    RCLK = 1;
}

/*****
*****
* 函 数 名      : main
* 函数功能      : 主函数
* 输    入      : 无
* 输    出      : 无
*****
*****/
void main()
{
    LED=0;

    ledNum = ~0x01;

    while(1)
    {
        Hc595SendByte(ledNum);
        ledNum = _crol_(ledNum, 1);
    }
}

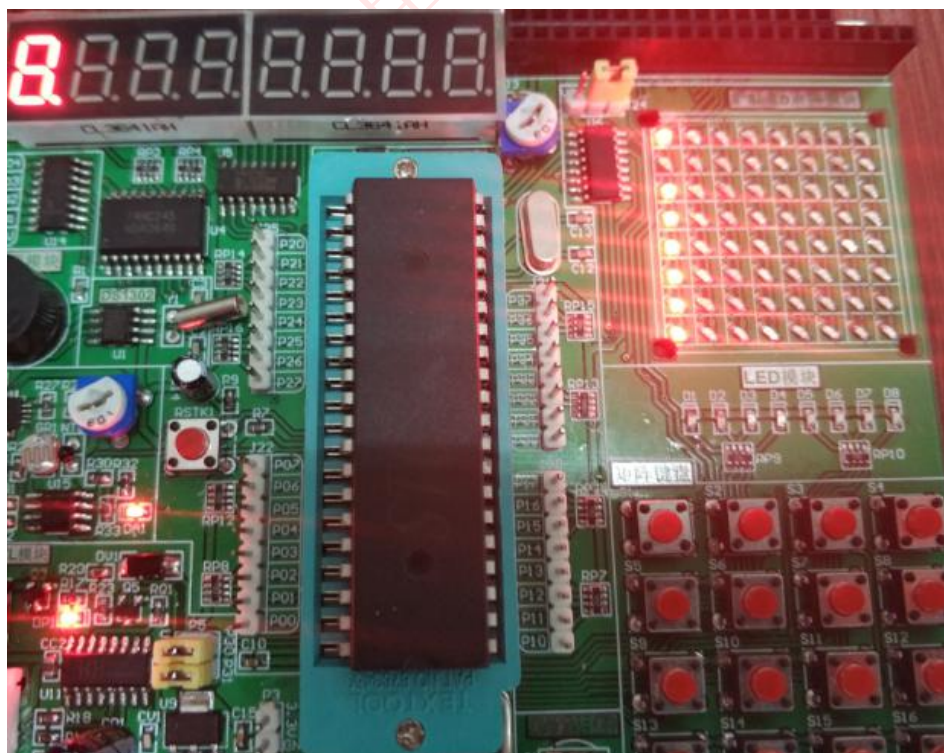
```

```
    delay(50000);  
  }  
}
```

main.c 文件内代码非常少也很简单，首先将 51 单片机的头文件包含进来，然后定义 74HC595 控制端口。主函数功能非常简单，首先定义一个变量 ledNum，用于存储所要发送的数据，初始值为 $\sim 0x01$ ，即取反为 0xFE。然后进入 while 循环，在循环体内执行 Hc595SendByte() 数据发送函数，该函数带一个形参，我们把 ledNum 值传递进去，函数将接收的参数从高位依次发送，如何获取高位，很简单，只需将数据右移 7 位，即 $dat \gg 7$ 。函数内的实现方式与 74HC165 类似，只不过这里是发送。然后调用 _crol_ 左移函数将 ledNum 中的 0 进行移位，该函数在 LED 实验章节就已介绍，这里不多说，为了保证我们能看出 LED 灯状态变化，这里加入了一个延时。这样 ledNum 值的变化就相当于 LED 灯状态的变化。

12.4 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 驱动串口），把编写好的程序编译后将编译产生的 .hex 文件烧入到芯片内，可以看到 8X8LED 点阵模块的第一列指示灯像流水灯一样滚动显示。



课后作业

- (1) 使用 74HC595 (A) 控制静态数码管显示 0-F。
 - (2) 使用 74HC595 (A) 的并行输出口连接动态数码管的位选，实现在动态数码管上显示 0-7。
- (温馨提示：将前面的几个实验进行整合)

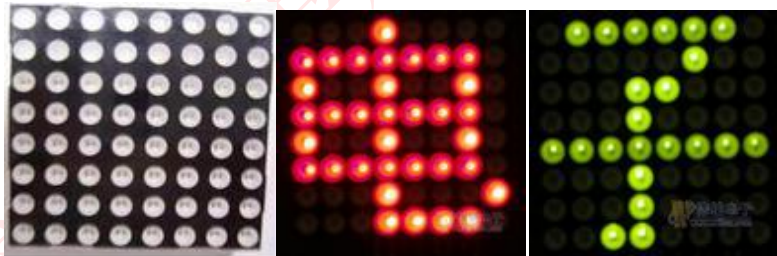
第 13 章 LED 点阵实验

在前面章节，我们介绍过静态数码管和动态数码管，其中动态数码管是一种应用非常多的显示设备，除此之外还有很多应用广泛的显示装置，比如 LED 点阵屏、LCD 液晶显示器等。我们开发板上使用了 64 个红色 LED 组成的 8*8LED 点阵。本章就来介绍下 LED 点阵的使用，通过本章的学习，让大家能够在 8*8LED 点阵屏上显示信息。本章所要实现的功能是：在 8*8LED 点阵屏上点亮一个点，上述功能实现后再显示数字字符等。学习本章可以参考前面的实验章节内容。若结合视频学习效果更佳。本章分为如下几部分内容：

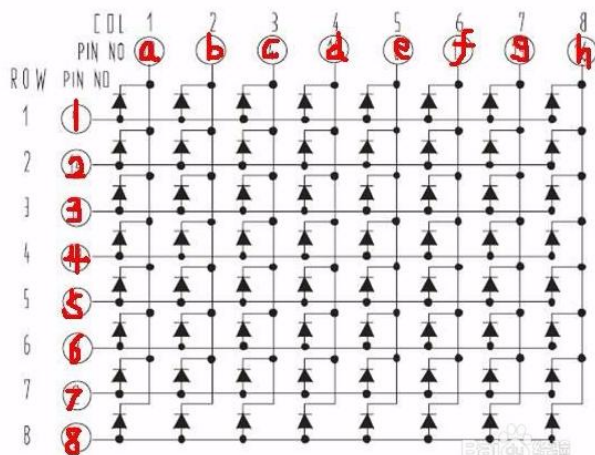
- 13.1 LED 点阵介绍
- 13.2 硬件设计
- 13.3 软件设计
- 13.4 实验现象

13.1 LED 点阵介绍

LED 点阵是由发光二极管排列组成的显示器件, 在我们日常生活的电器中随处可见, 被广泛应用于汽车报站器, 广告屏等。如下所示:



通常应用较多的是 8*8 点阵, 然后使用多个 8*8 点阵可组成不同分辨率的 LED 点阵显示屏, 比如 16*16 点阵可以使用 4 个 8*8 点阵构成。因此理解了 8*8LED 点阵的工作原理, 其他分辨率的 LED 点阵显示屏都是一样的。这里以 8*8LED 点阵来做介绍。其内部结构图如下所示:



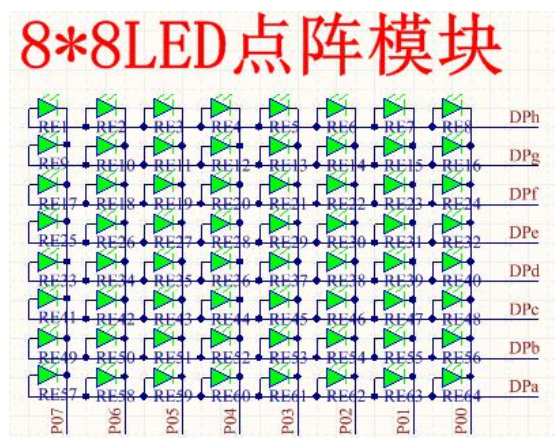
8*8 点阵共由 64 个发光二极管组成, 且每个发光二极管是放置在行线和列线的交叉点上, 当对应的某一行置 1 电平, 某一列置 0 电平, 则相应的二极管就亮; 如要将第一个点点亮, 则 1 脚接高电平 a 脚接低电平, 则第一个点就亮了; 如果要将第一行点亮, 则第 1 脚要接高电平, 而 (a、b、c、d、e、f、g、h) 这些引脚接低电平, 那么第一行就会点亮; 如要将第一列点亮, 则第 a 脚接低电平, 而 (1、2、3、4、5、6、7、8) 接高电平, 那么第一列就会点亮。由此可见, LED 点阵的使用也是非常简单的。

13.2 硬件设计

本实验使用到硬件资源如下:

- (1) 8*8LED 点阵模块
- (2) 74HC595 模块

74HC595 模块电路在前面章节都介绍过, 这里就不多说, 我们开发板上的 8*8 双色点阵模块电路如下图所示:



从上图中可以看出，该电路是集成的，电路中的 RE57~RE64 是 LED 点阵的第 1-8 列，通过 P0 I/O 口控制。RE8~RE64 是 LED 点阵的第 1-8 行，通过 74HC595 进行控制。DPa-DPh 是 LED 点阵的 8 行控制管脚，通过动态数码管模块中的 74HC245 进行驱动，这些行的引出端子都是 LED 的阴极。

根据前面介绍我们知道，8*8LED 点阵共有 64 个 LED，如果仅使用单片机 I/O 口来驱动可能效果不是很好，为了能够有效驱动 8*8LED 点阵且最大减少 I/O 口的占用，我们使用前面介绍的 74HC595 模块。74HC595 模块电路在前面已做介绍，通过较少的 I/O 口即可控制较为复杂的电路。

由于 74HC595 模块电路是集成的，所以使用单片机 P3⁴~P3⁶ 管脚即可，8X8LED 点阵模块电路也是集成的，所以列控制使用单片机 P0 管脚控制即可。

13.3 软件设计

本章所要实现的功能是：在点阵屏上点亮一个点，上述功能实现后再显示数字字符。

13.3.1 LED 点阵（点亮一个点）

我们直接复制上一章创建的工程，在此模板基础上进行程序开发。为了能够与开发攻略教程对应，将复制过来的模板文件夹重新命名为“实验 12：LED 点阵（点亮一个点）”。打开工程直接在 main.c 源文件内进行编程，main.c 内代码如下：

```
/*  
*****  
实验现象： 下载程序后，LED 点阵左上角第一个点的 LED 被点亮果  
  
接线说明： （具体接线图可见开发攻略对应实验的“实验现象”章节）  
  
注意事项：  
  
*****  
*****/
```

```

#include "reg51.h"           //此文件中定义了单片机的一些特殊功能寄存器
#include "intrins.h"

typedef unsigned int u16;     //对数据类型进行声明定义
typedef unsigned char u8;

//--定义使用的 I/O 口--//
sbit SRCLK=P3^6;
sbit RCLK=P3^5;
sbit SER=P3^4;
sbit LED=P0^7;

/*****
*****
* 函 数 名      : Hc595SendByte(u8 dat1,u8 dat2)
* 函数功能    : 通过 595 发送 2 个字节的数据
* 输    入    : dat1: 第 2 个 595 输出数值
*              * dat2: 第 1 个 595 输出数值
* 输    出    : 无
*****
*****/
void Hc595SendByte(u8 dat1,u8 dat2)
{
    u8 a;

    SRCLK = 1;
    RCLK = 1;

    for(a=0;a<8;a++)        //发送 8 位数
    {
        SER = dat1 >> 7;    //从最高位开始发送
        dat1 <<= 1;

        SRCLK = 0;          //发送时序
        _nop_();
        _nop_();
        SRCLK = 1;
    }

    for(a=0;a<8;a++)        //发送 8 位数
    {
        SER = dat2 >> 7;    //从最高位开始发送
        dat2 <<= 1;
    }
}

```

```
    SRCLK = 0;          //发送时序
    _nop_();
    _nop_();
    SRCLK = 1;
}

RCLK = 0;
_nop_();
_nop_();
RCLK = 1;
}

/*****
*****
* 函数名      : main
* 函数功能    : 主函数
* 输入       : 无
* 输出       : 无
*****
*****/
void main()
{
    LED=0; //使第一列为低电平。

    while(1)
    {
        Hc595SendByte(0xfe, 0x01);
    }
}
```

main.c 文件内代码非常少也很简单，首先将 51 单片机的头文件包含进来，然后定义 74HC595 控制端口。主函数功能非常简单，直接进入 while 循环调用 Hc595SendByte(0xfe, 0x01) 函数 2 次将点阵的行列控制数据输入到 74HC595 中：

第 1 个输出的是 0xfe，也就是八列（RE1-RE8），列是低电平有效，所以这八列有七列是无效的，点不亮，只有第一列是低电平，是有效的，第一列上的灯可以点亮。注意：这里是“可以”点亮，还必须要行为高电平才一定点亮。

第 2 个输出的是 0x01，也就是八行（A-Dp），行是高电平有效，所以这八行有七行是无效的，点不亮，只有第一行是高电平，是有效的，第一行上的灯可以点亮。注意：这里是“可以”点亮，还必须要列为低电平才一定点亮。

该函数内部实现过程与前面介绍的 Hc595SendByte() 是一样的，只不过这里进行了 2 次数据的传输。

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 驱动串口），把编写好的程序编译后将编译产生的 .hex 文件烧入到芯片内，按照如下图所示的接线方式可以看到 LED 点阵的左上角第一个灯点亮。

13.3.2 LED 点阵（显示数字）

点亮一个 LED 灯很简单，可是如何点亮多个 LED 灯呢？如果需要一次显示多个怎么办？从原理图上可以看到每一行上都连接着多个 LED 灯，每一列上也都连接着多个 LED 灯，如果要点亮一个，按照上面原理可以，但是要同时点亮多个怎么办？

那么就需要用到动态数码管的动态扫描原理。首先如何点亮一行上面多个灯或者一列上面多个灯？明显需要某行或某列有效，同时使多列或多行有效。比如在第一行有效的情况下，有效列与这一行交点上的 LED 灯就会被点亮。那么实现一行或一列点亮会比较容易。如何实现不同行不同列上的灯被多个点亮呢？是否是行有效，列有效就可以？并不是！

要实现行列不同位置亮灯，需要使用动态显示的方法，也要结合扫描的方法。在第一行亮灯一段时间以后灭掉，点亮第二行一段时间以后灭掉，点亮第三行一段时间以后灭掉，如此点亮，直到十六行全部点亮一次，在第一行点亮到最后一行灭掉的总时间不能超过人肉眼可识别的时间，即 24 毫秒。在每一行点亮的时候，给列一个新的数据，此时对应列而在该行上要点亮的灯的数据。这样就想数码管的动态显示一样，只不过数码管的 LED 灯是段值。这里使用 LED 点阵显示数字，也是多个 LED 同时点亮。

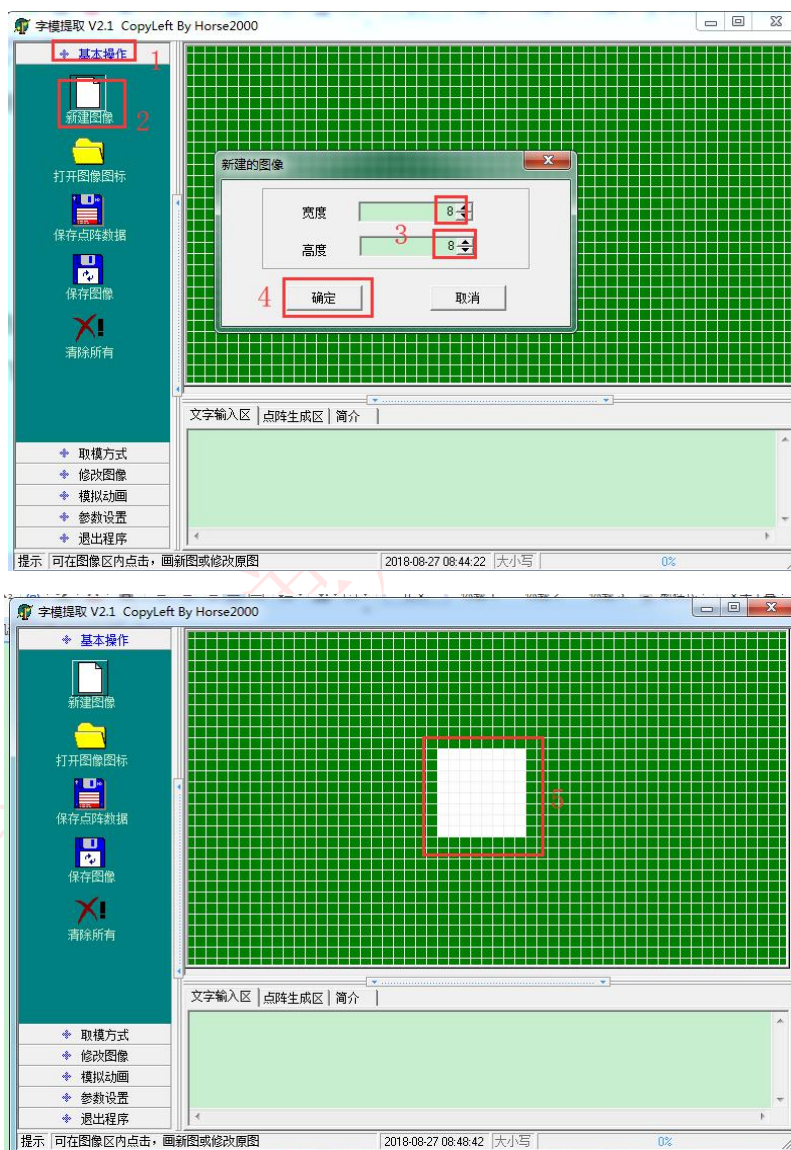
要想在点阵上显示数字等字符，首先要获取在 LED 点阵上显示数字字符所需的数据，即一个数字字符在 LED 点阵上显示，对应的每行每列都会有一些灯点亮或者熄灭，这样就会构成一组数据，也就是数字字符的显示数据，我们只要将这些数据通过 74HC595 发送到点阵对应的行或列就能显示数字字符。

数字字符数据如何获取呢？这里给大家介绍一个非常好用的工具-取字模软件。该软件在“\3--开发工具\4. 常用辅助开发软件\文字取模软件”内，如下

所示：



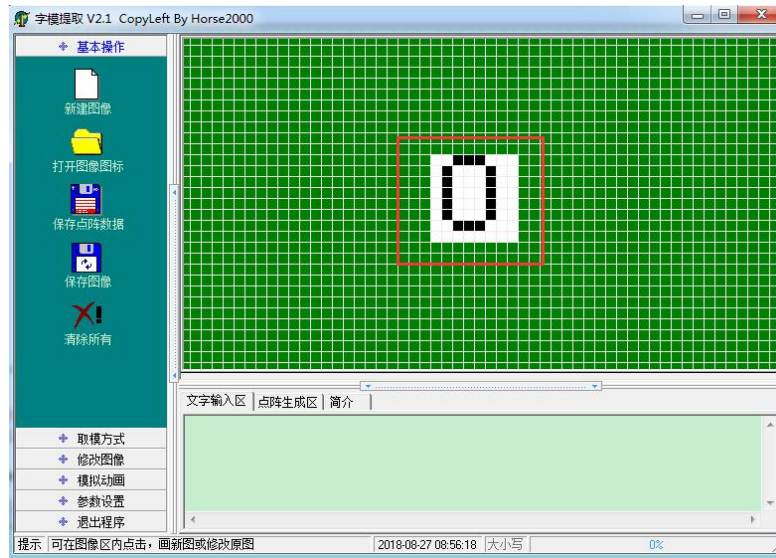
双击打开该软件，首先选择“基本操作->新建图像”，设置图像的宽度和高度为8，点击确定后将在显示窗口出现一个 8*8 的白色格子，这个就类似于我们的 8*8LED 点阵了，具体操作如下：



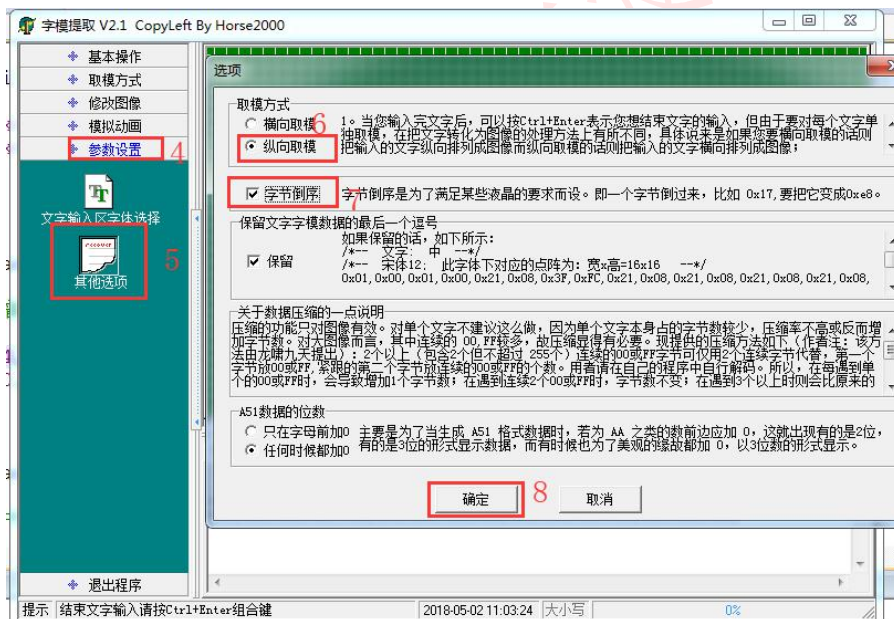
然后可以在这个 8*8 白色格子里面点击，点击后即会在对应位置出现一个黑点，表示在 LED 点阵对应位置显示，未点击位置（白色）表示 LED 点阵对应位置不显示。

比如我们想在 8*8LED 点阵上显示数字 0，那么可以在图中 8*8 白色框内

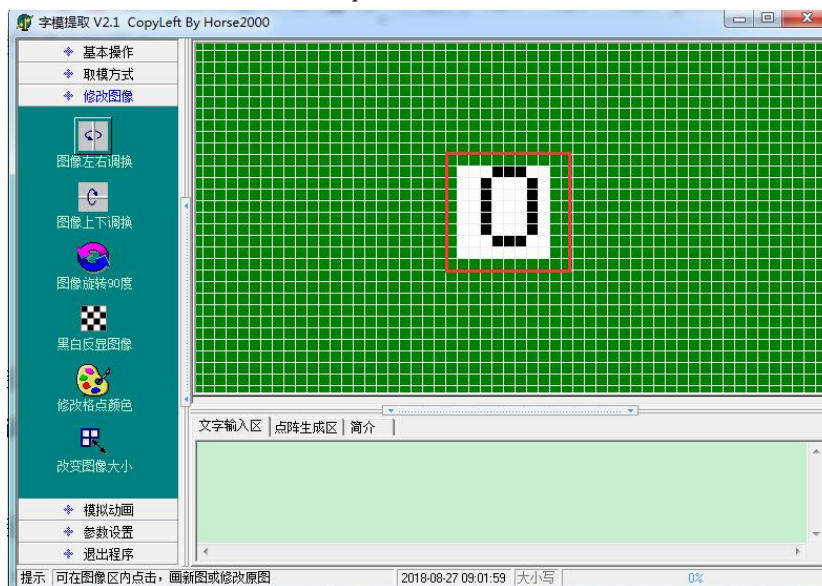
通过点击对应位置描述出一个数字 0 的外形，如下所示：



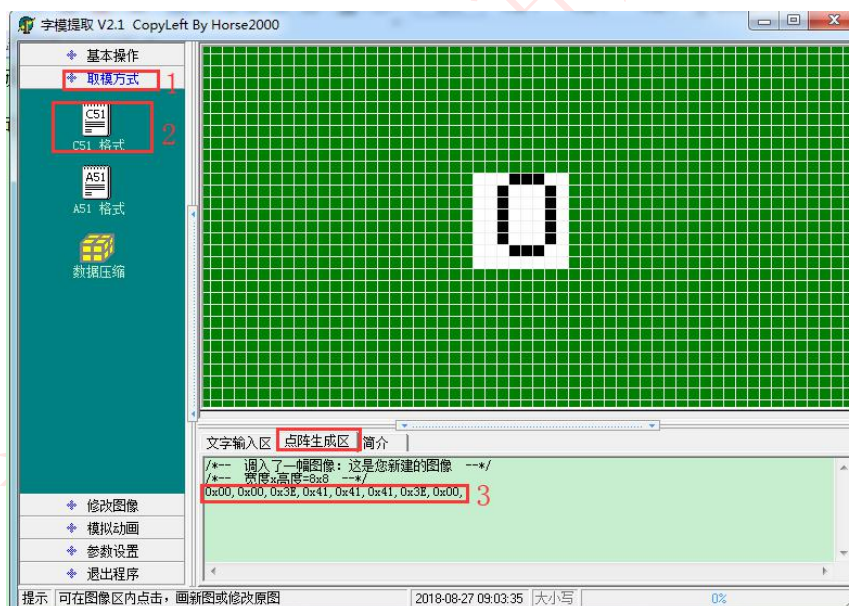
然后设置取模数据的取模方式等内容，具体操作如下：



然后点击“修改图像→图像左右调换”，此时在显示区，数字 0 的显示就变成左右调换后的结果，具体操作如下：



然后点击“取模方式”，选择 C51 格式选项，然后在点阵生成区会自动会生成数字字符对应的数据（如果你是使用汇编开发，那么汇编对应的汉字数据可选择 A51 格式）。如下所示：



到这里我们就将数字 0 的数据生成了，然后复制所生成的数据到我们程序内定义的数组中，如下所示：

```
u8 ledduan[]={0x00, 0x00, 0x3E, 0x41, 0x41, 0x41, 0x3E, 0x00}; // 类似于  
动态数码管的段选
```

至于其他数字或者字符及简单图形的显示取模方法与上述类似。

既然是动态扫描，那么就需要不断的扫描每列，因此可以把 LED 点阵的列控制也用数组存储起来，为后面循环调用提供方便。如下：

```
u8 ledwei[]={0x7f,0xbf,0xdf,0xef,0xf7,0xfb,0xfd,0xfe}; // 类似于
动态数码管的位选
```

对于这些数据的由来，大家可以按照前面介绍的点阵显示原理使用代入法将数据代入到 74HC595 发送数据函数就可以很容易理解。

我们直接复制上一章创建的工程，在此模板基础上进行程序开发。为了能够与开发攻略教程对应，将复制过来的模板文件夹重新命名为“实验 13: LED 点阵（显示数字）”。打开工程直接在 main.c 源文件内进行编程，main.c 内代码如下：

```

/*****
*****
实验现象：下载程序后，LED 点阵显示数字 0

接线说明：（具体接线图可见开发攻略对应实验的“实验现象”章节）

注意事项：

*****
*****/

#include "reg51.h"          //此文件中定义了单片机的一些特殊功能
寄存器
#include "intrins.h"

typedef unsigned int u16;    //对数据类型进行声明定义
typedef unsigned char u8;

//--定义使用的 I/O 口--//
sbit SRCLK=P3^6;
sbit RCLK=P3^5;
sbit SER=P3^4;

u8 ledduan[]={0x00,0x00,0x3E,0x41,0x41,0x41,0x3E,0x00}; // 类似于
动态数码管的段选
u8 ledwei[]={0x7f,0xbf,0xdf,0xef,0xf7,0xfb,0xfd,0xfe}; // 类似于
动态数码管的位选

```

```

/*****
*****
* 函 数 名      : delay
* 函数功能      : 延时函数, i=1 时, 大约延时 10us
*****
*****/
void delay(u16 i)
{
    while(i--);
}

void LED1(u8 dat1)
{
    P0=dat1;
}

/*****
*****
* 函 数 名      : Hc595SendByte(u8 dat1,u8 dat2)
* 函数功能      : 通过 595 发送 2 个字节的数据
* 输    入      : dat1: 第 2 个 595 输出数值
*                * dat2: 第 1 个 595 输出数值
* 输    出      : 无
*****
*****/
void Hc595SendByte(u8 dat)
{
    u8 a;

    SRCLK = 1;
    RCLK = 1;

    for(a=0;a<8;a++)    //发送 8 位数
    {
        SER = dat >> 7;    //从最高位开始发送
        dat <<= 1;

        SRCLK = 0;    //发送时序
        _nop_();
        _nop_();
        SRCLK = 1;
    }
    RCLK = 0;
}

```



```
_nop_();
_nop_();
RCLK = 1;
}

/*****
*****
* 函 数 名      : main
* 函数功能      : 主函数
* 输    入      : 无
* 输    出      : 无
*****
*****/
void main()
{
    u8 i=0;

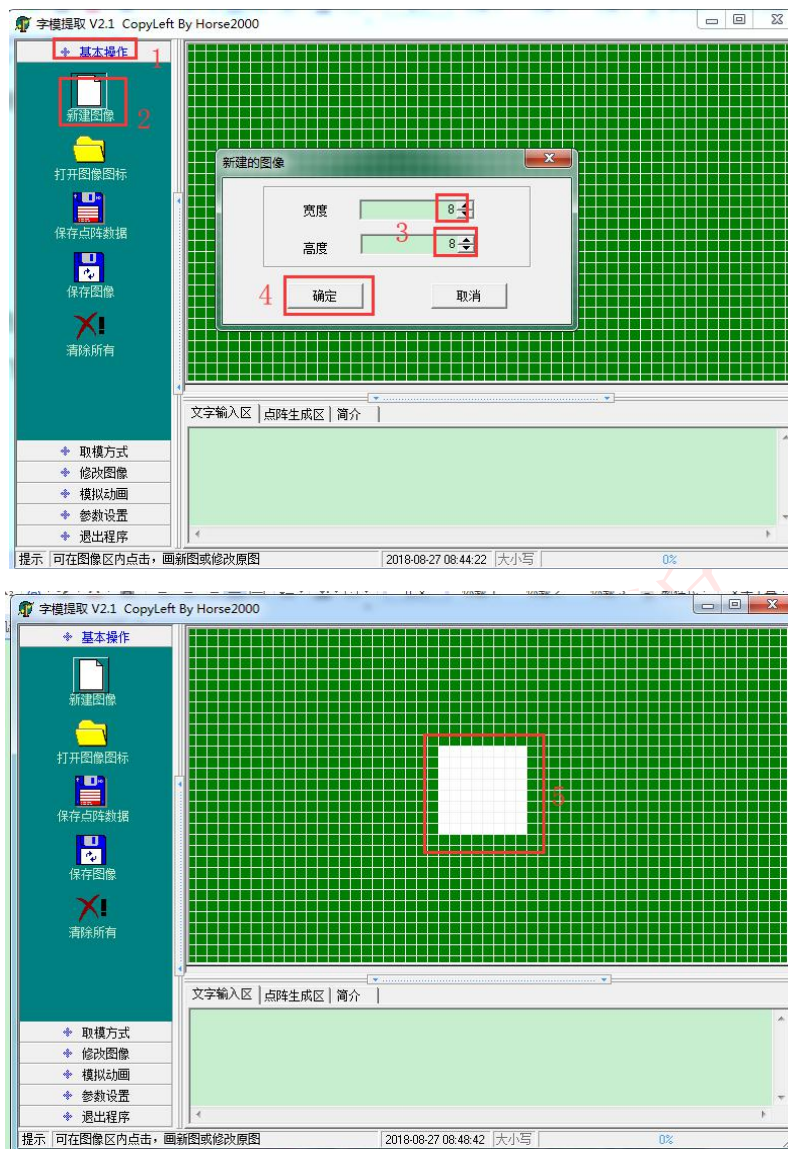
    while(1)
    {
        for(i=0;i<8;i++)
        {
            Hc595SendByte(ledwei[i]);
            LED1(ledwei[i]);
            delay(10);
        }
    }
}
```

该实验程序与前面介绍点亮一个点程序几乎一样，只不过，这里是显示数字字符，增加了动态扫描部分和数字数据部分。这里就不在重复介绍，不理解的可以回过头重新查看。大家可以通过增大 delay 函数时间就可以观察到扫描过程。

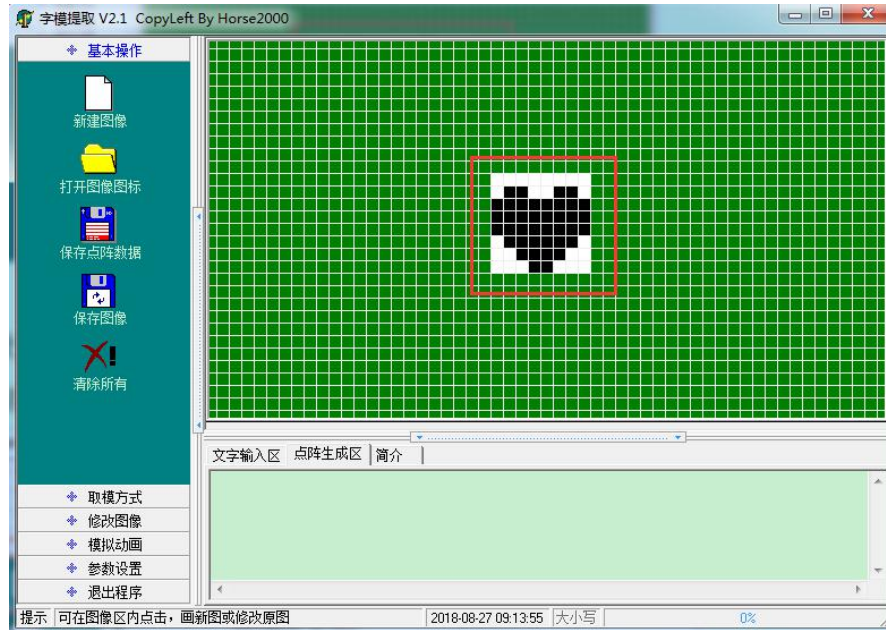
13.3.3 LED 点阵（显示图像）

该取模软件不仅可以用来取汉字、数字等字符数据，还可以用来取图像数据，这里以“心型”图像为例，教大家如何使用该软件来获取图像数据。

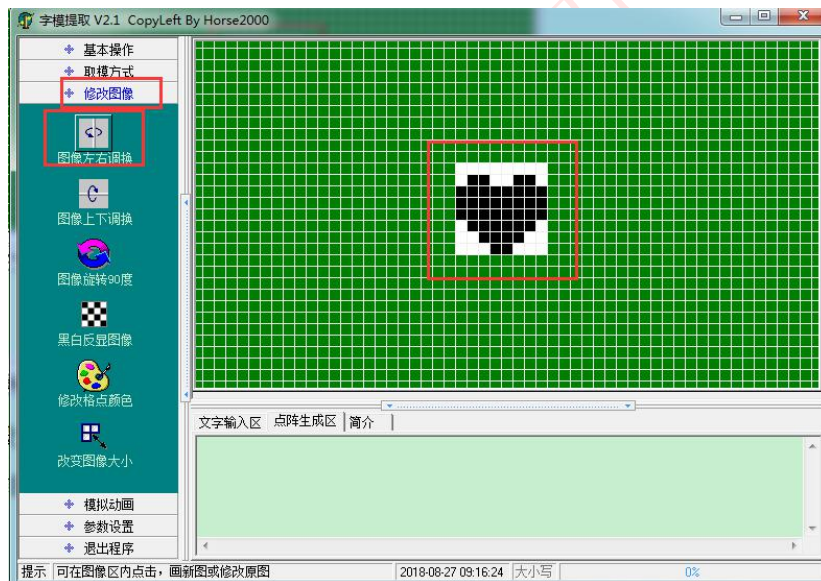
1、新建图像。弹出选择框。开发板 LED 点阵是 8X8 的，就建一个高 8 宽 8 的。



2、选中图像需用格点。这里每一个格点代表一个 LED 灯，点击可选中，再次点击也可取消。在界面上点出爱心。如下图：



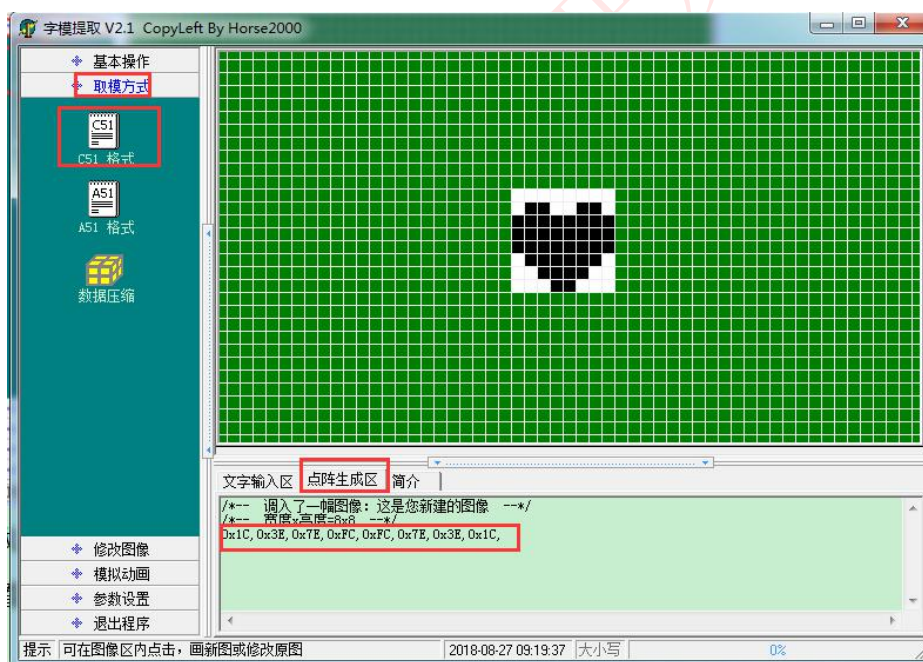
3、然后点击“修改图像->图像左右调换”，此时在显示区的显示就变成左右调换后的结果，如下图：



4、参数设置。“参数设置”选项中有许多可选项，可以自己进行设置，得到自己想要的结果。可参考前面取模里面的设置。如下所示：



5、生成数据。设计好图案和参数以后，就需要生成可用数据，对应自己的开发芯片进行生成格式选择。当点击格式以后会出现相应数据。如下图所示：

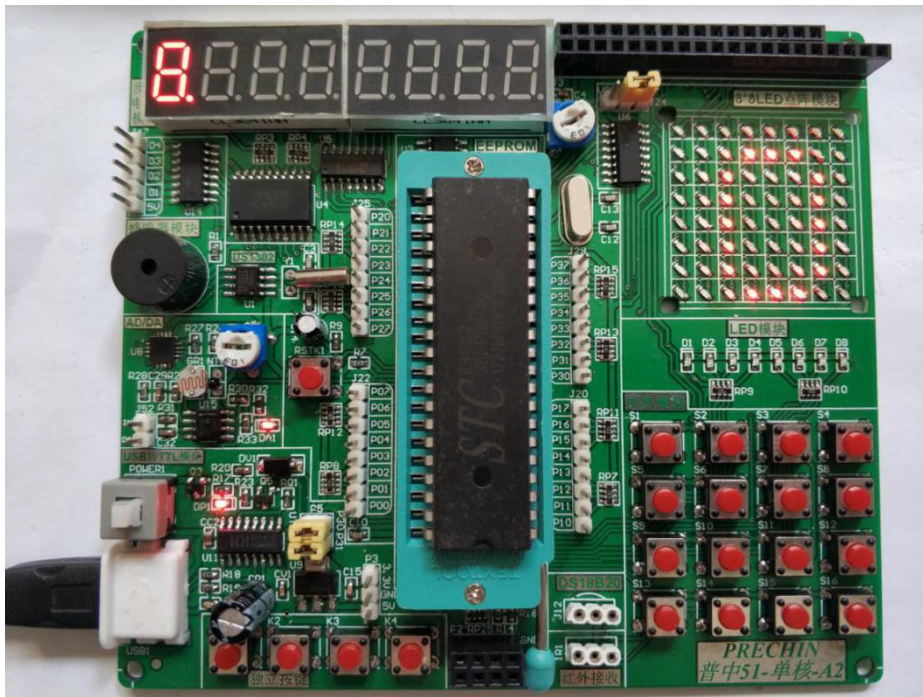


这里将生成的图形数据复制出来粘贴到例程中，替换数字字符 0 的数据。编译下载，就会看到 LED 点阵上显示这个心型图像。

13.4 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 驱动串口），把编写好的程序编译后将编译产生的 .hex 文件烧入到芯片内，按照如下

图所示的接线方式可以看到 LED 点阵模块上显示数字 0。



课后作业

- (1) 使用绿色来显示数字例程。
 - (2) 在 LED 点阵上显示心型图像，且使用双色来显示。
- (温馨提示：参考本章方法，双色即同时控制红色和绿色)

第 14 章 直流电机实验

在 51 单片机的应用中，电机控制方面的应用也是非常多的。所以很有必要先了解下电机相关的知识以及如何使用 51 单片机控制电机，在未学习 PWM 之前，我们先简单使用 GPIO 来控制电机的正反转和停止。不能直接使用 GPIO 来驱动电机，而需要相应的驱动芯片来实现，我们开发板上集成了两种直流电机驱动电路，一种使用的驱动芯片是 TC1508S，该芯片是一款专用电机驱动芯片，不仅可以用来驱动我们的直流电机，还可用来驱动开发板配置的四线双极性步进电机，这个在后面章节会介绍。另一种使用的驱动芯片是 ULN2003，该芯片是一个单片高电压、高电流的达林顿晶体管阵列集成电路。不仅可以用来驱动我们的直流电机，还可用来驱动五线四相步进电机，比如 28BYJ-48 步进电机。本章使用的是 ULN2003 芯片来驱动，本章所要实现的功能是：直流电机工作约 5S 后停止。学习本章可以参考前面的实验章节内容。若结合视频学习效果更佳。本章分为如下几部分内容：

14.1 直流电机介绍

14.2 ULN2003 芯片介绍

14.3 硬件设计

14.4 软件设计

14.5 实验现象

14.1 直流电机介绍

直流电机是指能将直流电能转换成机械能（直流电动机）或将机械能转换成直流电能（直流发电机）的旋转电机。它是能实现直流电能和机械能互相转换的电机。当它作电动机运行时是直流电动机，将电能转换为机械能；作发电机运行时是直流发电机，将机械能转换为电能。

直流电机的结构应由定子和转子两大部分组成。直流电机运行时静止不动的部分称为定子，定子的主要作用是产生磁场，由机座、主磁极、换向极、端盖、轴承和电刷装置等组成。运行时转动的部分称为转子，其主要作用是产生电磁转矩和感应电动势，是直流电机进行能量转换的枢纽，所以通常又称为电枢，由转轴、电枢铁心、电枢绕组、换向器和风扇等组成。

直流电机没有正负之分，在两端加上直流电就能工作。需要知道直流电机的额定电压和额定功率，不能使之长时间超负荷运作。在交换接线后，可以形成正反转。

开发板配置的直流电机为 5V 直流电机，其主要参数如下：

轴长：8mm

轴径：2mm

电压：1-6v

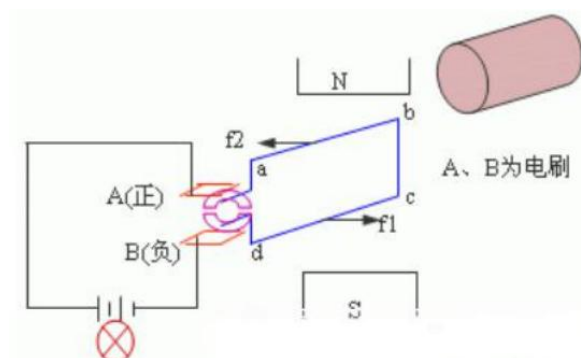
参考电流：0.35-0.4A

3v 转速：17000-18000 转每分钟

外观实物图如下：



其内部结构如下图所示：



14.2 ULN2003 芯片介绍

前面我们说过，51 单片机主要是用来控制而非驱动，如果直接使用芯片的 GPIO 管脚去驱动大功率器件，要么将芯片烧坏，要么就是驱动不起来。所以要驱动大功率器件，比如电机。就必须搭建外部驱动电路，我们开发板上集成了两种直流电机驱动电路，一种使用的驱动芯片是 L6219，该芯片是一个双极型单片集成 circuits，旨在控制和驱动一个双极步进电机或双向控制直流电机。它不仅可以用来驱动我们的直流电机，还可用来驱动开发板配置的四线双极性步进电机，这个在后面章节会介绍。另一种使用的驱动芯片是 ULN2003，该芯片是一个单片高电压、高电流的达林顿晶体管阵列集成电路。不仅可以用来驱动我们的直流电机，还可用来驱动五线四相步进电机，比如 28BYJ-48 步进电机。本章我们使用 ULN2003 来驱动直流电机，至于 L6219 芯片在后面章节会介绍。下面来具体了解下 ULN2003 芯片的使用。

ULN2003 是一个单片高电压、高电流的达林顿晶体管阵列集成电路。它是由 7 对 NPN 达林顿管组成的，它的高电压输出特性和阴极箝位二极管可以转换感应负载。单个达林顿对的集电极电流是 500mA。达林顿管并联可以承受更大的电流。此电路主要应用于继电器驱动器，字锤驱动器，灯驱动器，显示驱动器（LED 气体放电），线路驱动器和逻辑缓冲器。ULN2003 的每对达林顿管都有一个 2.7k 串联电阻，可以直接和 TTL 或 5V CMOS 装置。

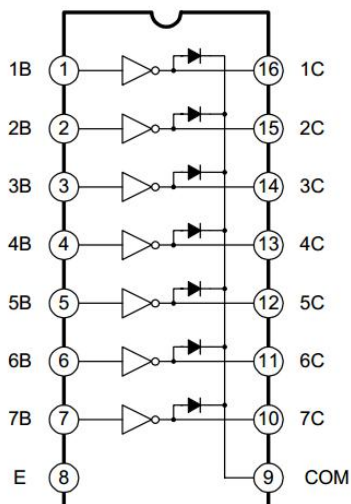
（1）主要特点

- ①500mA 额定集电极电流（单个输出）
- ②高电压输出：50V
- ③输入和各种逻辑类型兼容

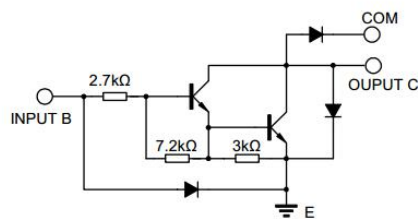
④继电器驱动器

(2) 逻辑框图

逻辑框图



示意图(每对达林顿管)



从上图可以很容易理解该芯片的使用方法，其内部实际上就相当于非门电路，即输入高输出为低，输入低输出高。

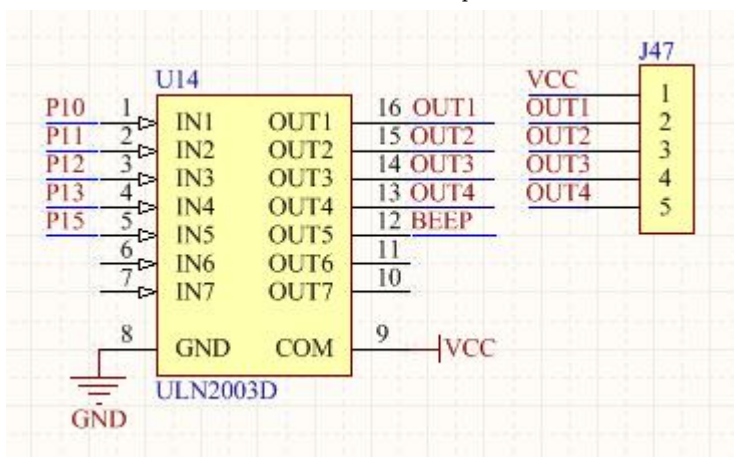
若使用该芯片驱动直流电机，只可实现单方向控制，电机一端接电源正极，另一端接芯片的输出口。若想控制五线四相步进电机，则可将四路输出接到步进电机的四相上，电机另一条线接电源正。芯片的详细介绍可参考光盘资料“\6—开发板芯片资料\ULN2003 中文手册.pdf”。

14.3 硬件设计

本实验使用到硬件资源如下：

(1) 五线四项步进电机驱动模块

我们开发板上的五线四项步进电机驱动模块电路如下图所示：



从上图中可以看出，该电路是集成的，芯片的输入通过单片机 P1⁰~P1⁵ IO 口提供，芯片的输出由 J47 端子引出。

由于该模块电路是集成的，所以使用单片机 P1⁰~P1⁵ 管脚即可。

注意：直流电机无方向，线接反只是转动方向不同而已。

14.4 软件设计

本章所要实现的功能是：直流电机工作约 5S 后停止。

我们直接复制上一章创建的工程，在此模板基础上进行程序开发。为了能够与开发攻略教程对应，将复制过来的模板文件夹重新命名为“实验 14：直流电机”。打开工程直接在 main.c 源文件内进行编程，main.c 内代码如下：

```
/*  
*****  
*                               直流电机实验  
*  
*  
实现现象：下载程序后，按照光盘内操作视频接线，直流电机旋转大约 5S，  
然后停止  
注意事项：无  
  
*****  
*****/  
  
#include "reg52.h"           //此文件中定义了单片机的一些特殊功能  
寄存器  
#include<intrins.h>         //因为要用到左右移函数，所以加入这个头文件  
  
typedef unsigned int u16;     //对数据类型进行声明定义
```

```

typedef unsigned char u8;

sbit moto=P1^0;

/*****
*****
* 函 数 名      : delay
* 函数功能      : 延时函数, i=1 时, 大约延时 10us
*****
*****/
void delay(u16 i)
{
    while(i--);
}

/*****
*****
* 函 数 名      : main
* 函数功能      : 主函数
* 输    入      : 无
* 输    出      : 无
*****
*****/
void main()
{
    u8 i;
    moto=0;          //关闭电机
    for(i=0;i<100;i++) //循环 100 次, 也就是大约 5S
    {
        moto=1;      //开启电机
        delay(5000); //大约延时 50ms
    }
    moto=0;          //关闭电机

    while(1)
    {

    }
}

```

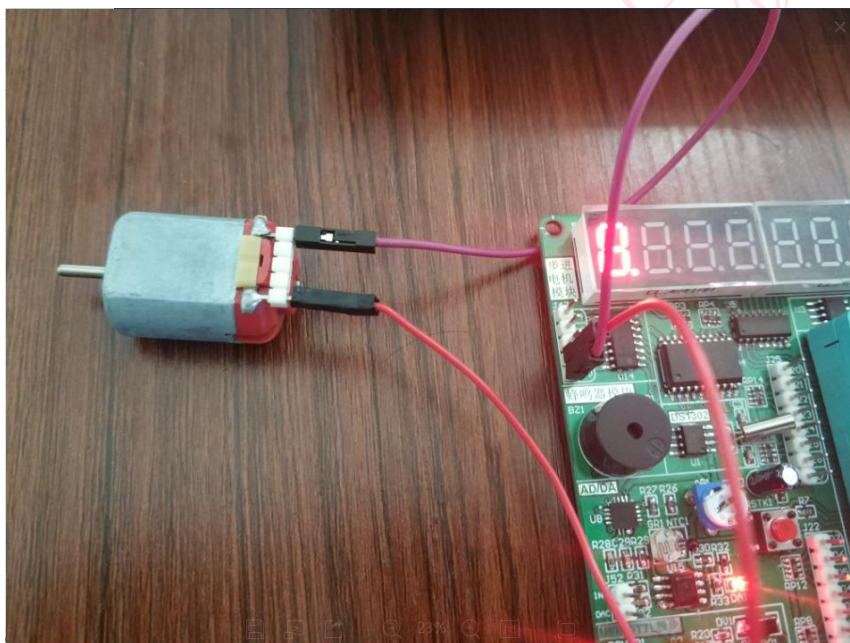
main.c 文件内代码非常少也很简单, 首先将 51 单片机的头文件包含进来, 然后定义直流电机控制端口。主函数功能非常简单, 首先定义变量 i 用于循环次数累计, 初始时关闭电机即使 moto 为 0, 对应输出就为高电平, 电机不工作。

然后使用 for 循环 100 次，开启电机，每次约延时 50ms，for 循环结束后关闭电机。最后进入 while 循环。

延时时间长短可通过修改 i 值或者 delay 实现，但是要注意 i 值的变量类型，不能超过 255，如果超过可使用 u16 类型来定义。

14.5 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 驱动串口），把编写好的程序编译后将编译产生的 .hex 文件烧入到芯片内，按照如下图所示的接线方式可以看到直流电机旋转约 5 秒后停止。



课后作业

- (1) 使用独立按键控制直流电机的转动和停止。
 - (2) 使用 ULN2003 芯片的其他通道来控制直流电机
- （温馨提示：当选择某个输入，与之对应的输出就必须连接电机）

第 15 章 中断系统

在前面章节，我们通过介绍 51 单片机 I/O 口学习了很多相关的实验。从最简单的 LED 控制到数码管动态扫描再到 LED 点阵控制以及电机控制，无不体现学习 51 单片机 I/O 口操作的重要性。在前面章节，我们还提到过学习 51 单片机的重点及难点主要有中断、定时器、串口等内容，对这部分内容大家一定要好好掌握，如果这部分没有学好就不能说学会了 51 单片机。我们专门使用一章来介绍 51 单片机的中断系统，为后面学习外部中断、定时器中断、串口中断做好铺垫。本章分为如下几部分内容：

15.1 中断介绍

15.2 中断结构及相关寄存器

15.1 中断介绍

15.1.1 中断概念

中断是为使单片机具有对外部或内部随机发生的事件实时处理而设置的，中断功能的存在，很大程度上提高了单片机处理外部或内部事件的能力。它也是单片机最重要的功能之一，是我们学习单片机必须要掌握的。很多初学者被困在中断中，学了很久仍然不知道中断究竟是如何运作的。千万不要认为它有多难，其实只要掌握正确的学习方法，没有哪个知识点是学不会的。

为了能让大家更容易理解中断概念，我们先来举一个生活事例：你打开火，烧上一壶水。然后去洗衣服，在洗衣服的过程中，突然听到水壶发出水开的报警声，这时，你停止洗衣服动作，立即去关掉火，然后将开水灌入暖水瓶中，灌完开水后，你又回去继续洗衣服。这个过程中实际上就发生了一次中断。

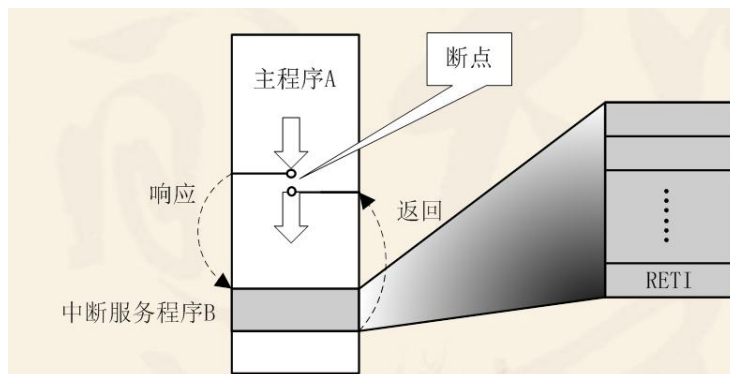
对于单片机来讲，中断是指 CPU 在处理某一事件 A 时，发生了另一事件 B，请求 CPU 迅速去处理(中断发生)；CPU 暂时停止当前的工作(中断响应)，转去处理事件 B(中断服务)；待 CPU 将事件 B 处理完毕后，再回到原来事件 A 被中断的地方继续处理事件 A(中断返回)，这一过程称为中断。



再回来看前面讲的生活事例，与单片机中断结合分析，你的主任务是洗衣服，水开报警这是一个中断请求，这一时刻相当于断点处，你响应中断去关火，然后将开水灌入暖水瓶中，这一动作实际上就是处理中断程序，灌完开水后再回去继续洗衣服，相当于处理完中断程序后再返回主程序继续执行主程序。这里需要注意的是，水开是随时都有可能的，但是无论什么时候开，只要一开你将立即去处理它，处理完后再回来继续接着洗刚才那件衣服。单片机在执行程序时，中断也随时有可能发生，但无论何时发生，只要一旦发生，单片机将立即暂停当前程序，

赶去处理中断程序，处理完中断程序后再返回刚才暂停处接着执行原来的程序。

单片机在执行程序时其程序流程图如下：



引起 CPU 中断的根源称为中断源。中断源向 CPU 提出中断请求，CPU 暂时中断原来的事务 A，转去处理事件 B，对事件 B 处理完毕后，再回到原来被中断的地方(即断点)，称为中断返回。实现上述中断功能的部件称为中断系统(中断机构)。

当中央处理机 CPU 正在处理某件事的时候外界发生了紧急事件请求，要求 CPU 暂停当前的工作，转而去处理这个紧急事件，处理完以后，再回到原来被中断的地方，继续原来的工作，这样的过程称为中断。实现这种功能的部件称为中断系统，请示 CPU 中断的请求源称为中断源。微型机的中断系统一般允许多个中断源，当几个中断源同时向 CPU 请求中断，要求为它服务的时候，这就存在 CPU 优先响应哪一个中断源请求的问题。通常根据中断源的轻重缓急排队，优先处理最紧急事件的中断请求源，即规定每一个中断源有一个优先级别。CPU 总是先响应优先级别最高的中断请求。

当 CPU 正在处理一个中断源请求的时候(执行相应的中断服务程序)，发生了另外一个优先级比它还高的中断源请求。如果 CPU 能够暂停对原来中断源的服务程序，转而去处理优先级更高的中断请求源，处理完以后，再回到原低级中断服务程序，这样的过程称为中断嵌套。这样的中断系统称为多级中断系统，没有中断嵌套功能的中断系统称为单级中断系统。

中断的开启与关闭、设置启用哪一个中断等都是由单片机内部的一些特殊功能寄存器来决定的，在前面章节的学习中我们仅对单片机 I/O 口操作过（实际上操作 I/O 口即操作 I/O 口寄存器，只不过编译器已经帮我们把 I/O 口寄存器封装好直接操作 I/O 即可，这些可在 51 单片机头文件内查看），从本章开始就会介绍单

片机内部更多的特殊功能寄存器以及如何配置它实现相应的功能。

随着计算机技术的应用，人们发现中断技术不仅解决了快速主机与慢速 I/O 设备的数据传送问题，而且还具有如下优点：

- ①分时操作。CPU 可以分时为多个 I/O 设备服务，提高了计算机的利用率；
- ②实时响应。CPU 能够及时处理应用系统的随机事件，系统的实时性大大增强；
- ③可靠性高。CPU 具有处理设备故障及掉电等突发性事件能力，从而使系统可靠性提高。

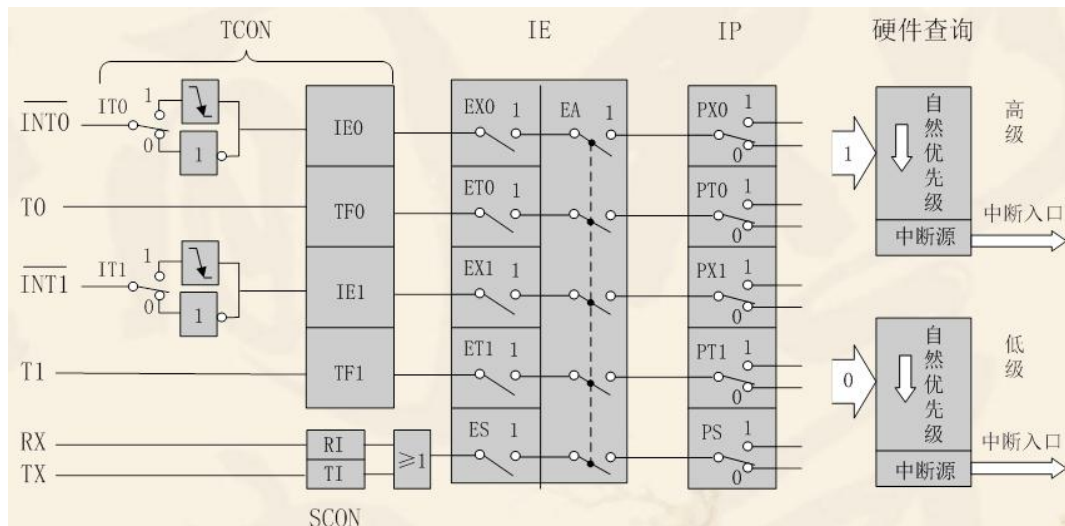
15.2 中断结构及相关寄存器

STC89C5X 系列单片机提供了 8 个中断请求源，它们分别是：外部中断 0 (INT0)、外部中断 1 (INT1)、外部中断 2 (INT2)、外部中断 3 (INT3)、定时器 0 中断、定时器 1 中断、定时器 2 中断、串口 (UART) 中断。（注意：51 系列单片机一定有基本的 5 个中断，但不全有 8 个中断，需要查看芯片手册，通常我们使用的都是基本的 5 个中断：INT0、INT1、定时器 0/1，串口中断）。所有的中断都具有四个中断优先级（基本型只有两个）。用户可以用关总中断允许位 (EA/IE. 7) 或相应中断的允许位来屏蔽所有的中断请求，也可以用打开相应的中断允许位来使 CPU 响应相应的中断申请。其中有些中断源可以用软件独立地控制为开中断或关中断状态。每一个中断的优先级别均可用软件设置。高优先级的中断请求可以打断低优先级的中断，反之，低优先级的中断请求不可以打断高优先级及同优先级的中断。当两个相同优先级的中断同时产生时，将由查询次序来决定系统先响应哪个中断。STC89C5X 系列单片机的各个中断查询次序表如下图所示：

中断源	中断向量地址	相同优先级内的查询次序	中断优先级设置 (IPH, IP)	优先级0 (最低)	优先级1	优先级2	优先级3 (最高)	中断请求标志位	中断允许控制位
$\overline{\text{INT0}}$ (外部中断 0)	0003H	0 (highest)	PX0H, PX0	0, 0	0, 1	1, 0	1, 1	IE0	EX0/EA
Timer 0	000BH	1	PT0H, PT0	0, 0	0, 1	1, 0	1, 1	TF0	ET0/EA
$\overline{\text{INT1}}$ (外部中断 1)	0013H	2	PX1H, PX1	0, 0	0, 1	1, 0	1, 1	IE1	EX1/EA
Timer 1	001BH	3	PT1H, PT1	0, 0	0, 1	1, 0	1, 1	TF1	ET1/EA
UART	0023H	4	PSH, PS	0, 0	0, 1	1, 0	1, 1	RI+TI	
Timer 2	002BH	5	PT2H, PT2	0, 0	0, 1	1, 0	1, 1	TF2 + EXF2	(ET2)/EA
$\overline{\text{INT2}}$ (外部中断 2)	0033H	6	PX2H, PX2	0, 0	0, 1	1, 0	1, 1	IE2	EX2/EA
$\overline{\text{INT3}}$ (外部中断 3)	003BH	7 (lowest)	PX3H, PX3	0, 0	0, 1	1, 0	1, 1	IE3	EX3/EA

通过设置新增加的特殊功能寄存器 IPH 中的相应位，可将中断优先级设为四级，如果只设置 IP 或 XICON，那么中断优先级就只有两级，与传统 8051 单片机两级中断优先级完全兼容。上图中的中断查询次序即为中断号，这个中断号在编程时非常重要，当中断来临时，只有中断号正确才能进入中断。

下面我们以 51 单片机一定有的 5 个中断来介绍，其内部结构框图如下所示：



①INT0 对应的是 P3.2 口的附加功能，可由 IT0 (TCON.0) 选择其为低电平有效还是下降沿有效。当 CPU 检测到 P3.2 引脚上出现有效的中断信号时，中断标志 IE0 (TCON.1) 置 1，向 CPU 申请中断。

②INT1 对应的是 P3.3 口的附加功能，可由 IT1 (TCON.2) 选择其为低电平有效还是下降沿有效。当 CPU 检测到 P3.3 引脚上出现有效的中断信号时，中断标志 IE1 (TCON.3) 置 1，向 CPU 申请中断。

③T0 对应的是 P3.4 口的附加功能，TF0 (TCON.5)，片内定时/计数器 T0 溢出中断请求标志。当定时/计数器 T0 发生溢出时，置位 TF0，并向 CPU 申请中断。

④T1 对应的是 P3.5 口的附加功能，TF1 (TCON.7)，片内定时/计数器 T1 溢出中断请求标志。当定时/计数器 T1 发生溢出时，置位 TF1，并向 CPU 申请中断。

⑤RXD 和 TXD 对应对应的是 P3.0 和 P3.1 口的附加功能，RI (SCON.0) 或 TI (SCON.1)，串行口中断请求标志。当串行口接收完一帧串行数据时置位 RI 或当串行口发送完一帧串行数据时置位 TI，向 CPU 申请中断。

(1) 中断允许控制

CPU 对中断系统所有中断以及某个中断源的开放和屏蔽是由中断允许寄存器

IE 控制的。

位	7	6	5	4	3	2	1	0	
字节地址: A8H	EA			ES	ET1	EX1	ET0	EX0	IE

EX0 (IE. 0)，外部中断 0 允许位；

ET0 (IE. 1)，定时/计数器 T0 中断允许位；

EX1 (IE. 2)，外部中断 1 允许位；

ET1 (IE. 3)，定时/计数器 T1 中断允许位；

ES (IE. 4)，串行口中断允许位；

EA (IE. 7)，CPU 中断允许（总允许）位。

(2) 中断请求标志 TCON

I	位	7	6	5	4	3	2	1	0	
字节地址: 88H	TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0		TCON

IT0 (TCON. 0)，外部中断 0 触发方式控制位。

当 IT0=0 时，为电平触发方式。

当 IT0=1 时，为边沿触发方式（下降沿有效）。

IE0 (TCON. 1)，外部中断 0 中断请求标志位。

IT1 (TCON. 2)，外部中断 1 触发方式控制位。

IE1 (TCON. 3)，外部中断 1 中断请求标志位。

TF0 (TCON. 5)，定时/计数器 T0 溢出中断请求标志位。

TF1 (TCON. 7)，定时/计数器 T1 溢出中断请求标志位。

(3) 中断优先级

同一优先级中的中断申请不止一个时，则有中断优先权排队问题。同一优先级的中断优先权排队，由中断系统硬件确定的自然优先级形成，其排列如所示：

各中断源响应优先级及中断服务程序入口表

中断源	中断标志	中断服务程序入口	优先级顺序
外部中断 0 ($\overline{\text{INT0}}$)	IE0	0003H	高
定时/计数器 0 (T0)	TF0	000BH	↓
外部中断 1 ($\overline{\text{INT1}}$)	IE1	0013H	↓
定时/计数器 1 (T1)	TF1	001BH	↓
串行口	RI 或 TI	0023H	低

(4) 中断号

中断源符号	名称	中断引起原因	中断号
/INT0	外部中断 0	P3.2 引脚低电平或下降沿信号	0
T0	定时器 0 中断	定时/计数器 0 计数回 0 溢出	1
/INT1	外部中断 1	P3.3 引脚低电平或下降沿信号	2
T1	定时器 1 中断	定时/计数器 1 计数回 0 溢出	3
TI/RI	串行口中断	串行通信完成一帧数据发送或接收引起中断	4

(5) 中断响应条件

- ①中断源有中断请求；
- ②此中断源的中断允许位为 1；
- ③CPU 开中断（即 EA=1）。

以上三条同时满足时，CPU 才有可能响应中断。在使用中断时我们需要做什么呢？

- ①你想使用的中断是哪个？选择相应的中断号；
- ②你所希望的触发条件是什么？
- ③你希望在中断之后干什么？

我们以外部中断 0 为例，如下：

主程序中需要有以下代码：

EA=1; //打开总中断开关

EX0=1; //开外部中断 0

IT0=0/1; //设置外部中断的触发方式

中断服务函数：

```
void int0() interrupt 0 using 1
{
    //编写用户所需的功能代码
}
```

在中断函数中 int0 是函数名，可自定义，但必须符合 C 语言标识符定义规则，interrupt 是一个关键字，表示 51 单片机中断。后面的“0”是中断号，外部中断 0 中断号为 0，这个可参考前面的内容。后面的 using 1 可省略不写。

至此我们就把 51 单片机的中断简单介绍了下，如果大家还有不明白的可先暂时放下，可先通过实验掌握 51 单片机中断的使用方法，对于理论方面的知识后续可以深入研究。

课后作业

(1) 百度查询 51 单片机中断相关内容，巩固中断相关知识。

(温馨提示：51 单片机中断（外部中断、定时器中断、串口中断）)

第 16 章 外部中断实验

上一章我们介绍了 51 单片机的中断，这一章我们就来学习下 51 单片机的外部中断，通过上一章的介绍我们知道，51 单片机外部中断有 2 个，外部中断 0 和外部中断 1，它们的使用方法是一样的，所以只要学会一个即可掌握所有外部中断。本章所要实现的功能是：使用独立按键 K3 控制 LED 亮灭，K3 连接外部中断 0（P3.2）管脚。学习本章可以参考前面的实验章节内容。若结合视频学习效果更佳。本章分为如下几部分内容：

16.1 外部中断介绍

16.2 外部中断配置

16.3 硬件设计

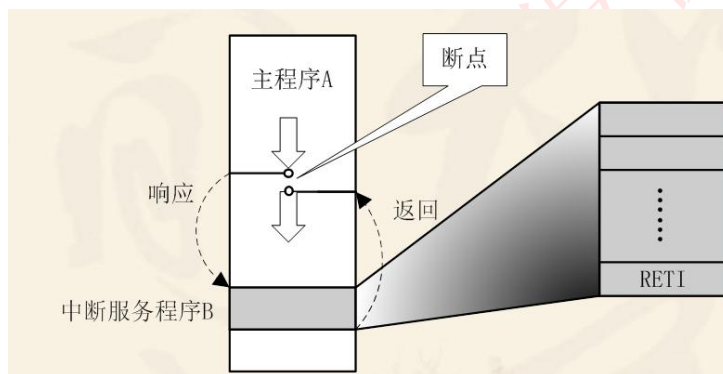
16.4 软件设计

16.5 实验现象

16.1 外部中断介绍

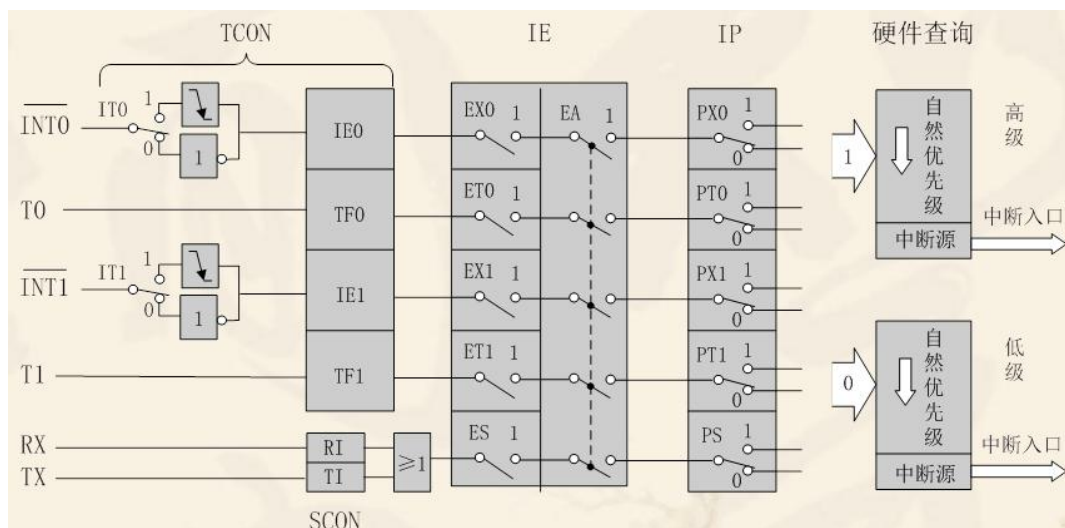
上一章我们介绍了 51 单片机的中断，我们在回顾一下 51 单片机的中断。

当中央处理机 CPU 正在处理某件事的时候外界发生了紧急事件请求，要求 CPU 暂停当前的工作，转而去处理这个紧急事件，处理完以后，再回到原来被中断的地方，继续原来的工作，这样的过程称为中断。实现这种功能的部件称为中断系统，请示 CPU 中断的请求源称为中断源。微型机的中断系统一般允许多个中断源，当几个中断源同时向 CPU 请求中断，要求为它服务的时候，这就存在 CPU 优先响应哪一个中断源请求的问题。通常根据中断源的轻重缓急排队，优先处理最紧急事件的中断请求源，即规定每一个中断源有一个优先级别。CPU 总是先响应优先级别最高的中断请求。中断示意图如下所示：



STC89C5X 系列单片机提供了 4 个外部中断：外部中断 0 (INT0)、外部中断 1 (INT1)、外部中断 2 (INT2)、外部中断 3 (INT3)。（注意：51 系列单片机一定有基本的 2 个外部中断，但不全有 4 个中断，需要查看芯片手册，通常我们都是使用基本的 2 个外部中断：INT0 和 INT1）。

下面我们来看下外外部中断结构图，如下所示：



图中 INT0 和 INT1 即为外部中断 0 和外部中断 1 输入口。

①INT0 对应的是 P3.2 口的附加功能，可由 IT0 (TCON.0) 选择其为低电平有效还是下降沿有效。当 CPU 检测到 P3.2 引脚上出现有效的中断信号时，中断标志 IE0 (TCON.1) 置 1，向 CPU 申请中断。

②INT1 对应的是 P3.3 口的附加功能，可由 IT1 (TCON.2) 选择其为低电平有效还是下降沿有效。当 CPU 检测到 P3.3 引脚上出现有效的中断信号时，中断标志 IE1 (TCON.3) 置 1，向 CPU 申请中断。

(1) 中断允许控制

CPU 对中断系统所有中断以及某个中断源的开放和屏蔽是由中断允许寄存器 IE 控制的。

位	7	6	5	4	3	2	1	0	
字节地址: A8H	EA			ES	ET1	EX1	ET0	EX0	IE

EX0 (IE.0)，外部中断 0 允许位；

ET0 (IE.1)，定时/计数器 T0 中断允许位；

EX1 (IE.2)，外部中断 1 允许位；

ET1 (IE.3)，定时/计数器 T1 中断允许位；

ES (IE.4)，串行口中断允许位；

EA (IE.7)，CPU 中断允许（总允许）位。

(2) 中断请求标志 TCON

I	位	7	6	5	4	3	2	1	0	
字节地址: 88H		TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0	TCON

IT0 (TCON.0)，外部中断 0 触发方式控制位。

当 IT0=0 时，为电平触发方式。

当 IT0=1 时，为边沿触发方式（下降沿有效）。

IE0 (TCON.1)，外部中断 0 中断请求标志位。

IT1 (TCON.2)，外部中断 1 触发方式控制位。

IE1 (TCON.3)，外部中断 1 中断请求标志位。

TF0 (TCON.5)，定时/计数器 T0 溢出中断请求标志位。

TF1 (TCON.7)，定时/计数器 T1 溢出中断请求标志位。

16.2 外部中断配置

我们知道要让 51 单片机发生中断必须要满足以下 3 个条件，这 3 个条件的顺序可以任意：

- ①中断源有中断请求；
- ②此中断源的中断允许位为 1；
- ③CPU 开中断（即 EA=1）。

比如我们配置外部中断 0，对应的配置程序如下：

EA=1; //打开总中断开关

EX0=1; //开外部中断 0

IT0=0/1; //设置外部中断的触发方式

如果要配置的是外部中断 1，只需将 EX0 改为 EX1，IT0 改为 IT1。

因为独立按键一端是共地的，当按下后对应单片机 IO 口被拉低，而默认单片机 IO 口是高电平，这样就有一个下降沿过程，所以通常使用外部中断都是配置为下降沿触发，即 IT0=1；

在编写程序时通常我们会将外部中断的配置放到一个自定义函数内便于管理维护。如下所示：

```
void Int0Init()
```



```
{  
    //设置 INT0  
    IT0=1;//跳变沿出发方式（下降沿）  
    EX0=1;//打开 INT0 的中断允许。  
    EA=1;//打开总中断  
}
```

当触发中断后即会进入中断服务函数，外部中断 0 中断服务函数如下：

```
void Int0() interrupt 0    //外部中断 0 的中断函数  
{  
    //执行所需的功能  
}
```

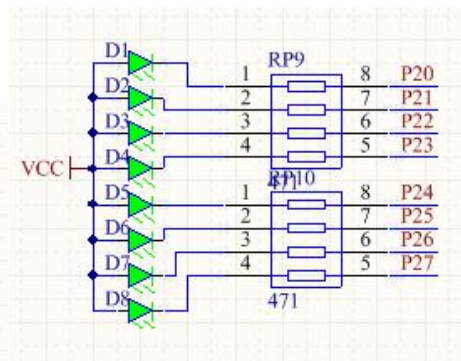
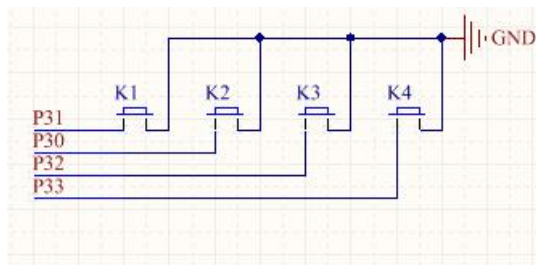
在中断函数中 int0 是函数名，可自定义，但必须符合 C 语言标识符定义规则，interrupt 是一个关键字，表示 51 单片机中断。后面的“0”是中断号，外部中断 0 中断号为 0，如果是外部中断 1，则中断号为 2，这个可参考中断章节的内容。

16.3 硬件设计

本实验使用到硬件资源如下：

- （1）独立按键模块（K3）
- （2）LED 模块（D1）

LED 模块和独立按键模块电路在前面章节都介绍过，这里就不多说。如果要想使用外部中断 0 控制 LED，就必须通过单片机外部中断管脚与按键 K3 连接。由于开发板上部分模块电路是集成的，所以直接使用单片机 P3² 和 P2⁰ 管脚即可。



16.4 软件设计

本章所要实现的功能是：使用独立按键 K3 控制 LED 亮灭，K3 连接外部中断 0（P3.2）管脚。

我们直接复制上一章创建的工程，在此模板基础上进行程序开发。为了能够与开发攻略教程对应，将复制过来的模板文件夹重新命名为“实验 16：外部中断 0”。打开工程直接在 main.c 源文件内进行编程，main.c 内代码如下：

```
/*
*****
*****
实验现象：下载程序后，操作 K3 按键使 D1 状态取反

接线说明：（具体接线图可见开发攻略对应实验的“实验现象”章节）

注意事项：

*****
*****/

#include "reg52.h"          //此文件中定义了单片机的一些特殊功能寄存器

typedef unsigned int u16;    //对数据类型进行声明定义
typedef unsigned char u8;

sbit k3=P3^2; //定义按键 K3
sbit led=P2^0; //定义 P20 口是 led

/*
*****
*****
*/
```

```

*****
* 函 数 名      : delay
* 函数功能      : 延时函数, i=1 时, 大约延时 10us
*****
*****/
void delay(u16 i)
{
    while(i--);
}

/*****
*****
* 函 数 名      : Int1Init()
* 函数功能      : 设置外部中断 1
* 输    入      : 无
* 输    出      : 无
*****
*****/
void Int0Init()
{
    //设置 INT0
    IT0=1; //跳变沿出发方式(下降沿)
    EX0=1; //打开 INT0 的中断允许。
    EA=1; //打开总中断
}

/*****
*****
* 函 数 名      : main
* 函数功能      : 主函数
* 输    入      : 无
* 输    出      : 无
*****
*****/
void main()
{
    Int0Init(); // 设置外部中断 0
    while(1);
}

/*****
*****
* 函 数 名      : Int0() interrupt 0
* 函数功能      : 外部中断 0 的中断函数

```

```

* 输    入        : 无
* 输    出        : 无
*****
*****/

void Int0()    interrupt 0        //外部中断 0 的中断函数
{
    delay(1000);    //延时消抖
    if(k3==0)
    {
        led=~led;
    }
}

```

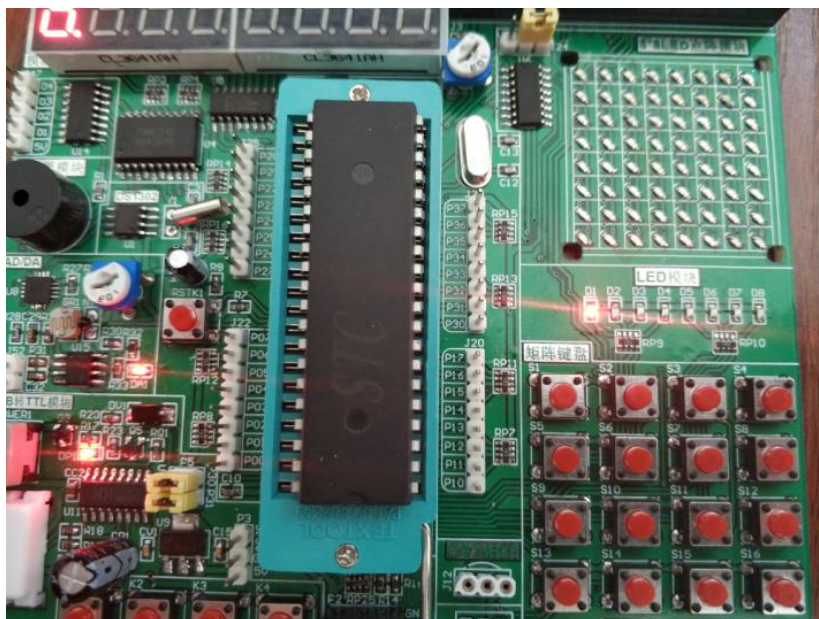
main.c 文件内代码非常少也很简单，首先将 51 单片机的头文件包含进来，然后定义按键 K3 和 LED 的控制脚。主函数功能非常简单，首先调用 Int0Init() 函数配置外部中断 0，该函数配置过程就是按照前面介绍的配置方法所写，即开启总中断、外部中断 0 功能，设置外部中断 0 为下降沿触发。然后进入 while 循环，在循环体内没有执行任何功能程序。

有的朋友就会问，在主函数中怎么没有看到按键对 LED 的控制呢？因为我们在 Int0Init() 函数内就已经把按键管脚配置为外部中断 0 下降沿触发，当有按键按下，即会进入对应中断服务函数执行相应的功能程序，LED 的控制就在中断函数内完成的。

外部中断 1 的使用与外部中断 0 是一样的，大家可以打开“实验 17：外部中断 1”程序查看。这里就不再重复。

16.5 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 驱动串口），把编写好的程序编译后将编译产生的 .hex 文件烧入到芯片内，按照如下图所示的接线方式可以看到当按键 K3 按下后，指示灯 D1 点亮，再次按下则熄灭，如此循环。



(温馨提示：可参考前面实验章节将内容进行整合)

上一章我们介绍了 51 单片机的外部中断，学会了如何配置 51 单片机的外部中断功能。这一章我们来学习下 51 单片机的定时器中断。STC89C5X 含有 3 个定时器：定时器 0、定时器 1、定时器 2（注意：51 系列单片机一定有基本的 2 个定时器（定时器 0 和定时器 1），但不全有 3 个中断，需要查看芯片手册，通常我们使用的都是基本的 2 个定时器：定时器 0/1）。本章以定时器 0 为例进行讲解，让大家学会 51 单片机定时器的使用。本章要实现的功能是：通过定时器 0 中断控制 D1 指示灯间隔 1 秒闪烁。学习本章可以参考前面的实验章节内容。若结合视频学习效果更佳。本章分为如下几部分内容：

166

17.2 定时器配置

17.3 硬件设计

17.4 软件设计

17.5 实验现象

17.1 定时器介绍

在介绍定时器之前我们先科普下几个知识：

1，CPU 时序的有关知识

①振荡周期：为单片机提供定时信号的振荡源的周期（晶振周期或外加振荡周期）。

②状态周期：2 个振荡周期为 1 个状态周期，用 S 表示。振荡周期又称 S 周期或时钟周期。

③机器周期：1 个机器周期含 6 个状态周期，12 个振荡周期。

④指令周期：完成 1 条指令所占用的全部时间，它以机器周期为单位。

例如：外接晶振为 12MHz 时，51 单片机相关周期的具体值为：

振荡周期=1/12us;

状态周期=1/6us;

机器周期=1us;

指令周期=1~4us;

2, 学习定时器前需要明白的几点

①51 单片机有两组定时器/计数器, 因为既可以定时, 又可以计数, 故称之为定时器/计数器。

②定时器/计数器和单片机的 CPU 是相互独立的。定时器/计数器工作的过程是自动完成的, 不需要 CPU 的参与。

③51 单片机中的定时器/计数器是根据机器内部的时钟或者是外部的脉冲信号对寄存器中的数据加 1。

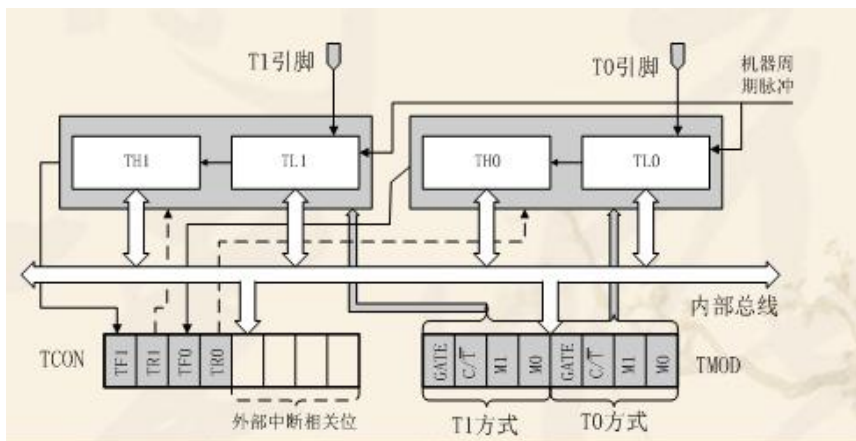
有了定时器/计数器之后, 可以增加单片机的效率, 一些简单的重复加 1 的工作可以交给定时器/计数器处理。CPU 转而处理一些复杂的事情。同时可以实现精确定时作用。

17.1.1 51 单片机定时器原理

STC89C5X 单片机内有两个可编程的定时/计数器 T0、T1 和一个特殊功能定时器 T2。定时/计数器的实质是加 1 计数器 (16 位), 由高 8 位和低 8 位两个寄存器 THx 和 TLx 组成。它随着计数器的输入脉冲进行自加 1, 也就是每来一个脉冲, 计数器就自动加 1, 当加到计数器为全 1 时, 再输入一个脉冲就使计数器回零, 且计数器的溢出使相应的中断标志位置 1, 向 CPU 发出中断请求 (定时/计数器中断允许时)。如果定时/计数器工作于定时模式, 则表示定时时间已到; 如果工作于计数模式, 则表示计数值已满。可见, 由溢出时计数器的值减去计数初值才是加 1 计数器的计数值。

(1) 51 单片机定时/计数器结构

51 单片机定时器/计数器内部结构如下所示:



上图中的 T0 和 T1 引脚对应的是单片机 P3.4 和 P3.5 管脚。51 单片机定时/计数器的工作由两个特殊功能寄存器控制。TMOD 是定时/计数器的工作方式寄存器，确定工作方式和功能；TCON 是控制寄存器，控制 T0、T1 的启动和停止及设置溢出标志。

1, 工作方式寄存器 TMOD

工作方式寄存器 TMOD 用于设置定时/计数器的工作方式，低四位用于 T0，高四位用于 T1。其格式如下：

位	7	6	5	4	3	2	1	0	
字节地址: 89H	GATE	C/T	M1	M0	GATE	C/T	M1	M0	TMOD

GATE 是门控位，GATE=0 时，用于控制定时器的启动是否受外部中断源信号的影响。只要用软件使 TCON 中的 TR0 或 TR1 为 1，就可以启动定时/计数器工作；GATE=1 时，要用软件使 TR0 或 TR1 为 1，同时外部中断引脚 INT0/1 也为高电平时，才能启动定时/计数器工作。即此时定时器的启动条件，加上了 INT0/1 引脚为高电平这一条件。

C/T : 定时/计数模式选择位。C/T =0 为定时模式;C/T =1 为计数模式。

M1M0: 工作方式设置位。定时/计数器有四种工作方式。

定时/计数器工作方式设置表

M1M0	工作方式	说明
00	方式 0	13 位定时/计数器
01	方式 1	16 位定时/计数器
10	方式 2	8 位自动重装定时/计数器
11	方式 3	T0 分成两个独立的 8 位定时/计数器; T1 此方式停止计数

2, 控制寄存器 TCON

TCON 的低 4 位用于控制外部中断, 已在前面介绍。TCON 的高 4 位用于控制定时/计数器的启动和中断申请。其格式如下:

位	7	6	5	4	3	2	1	0	
字节地址: 88H	TF1	TR1	TF0	TR0					TCON

TF1 (TCON. 7): T1 溢出中断请求标志位。T1 计数溢出时由硬件自动置 TF1 为 1。CPU 响应中断后 TF1 由硬件自动清 0。T1 工作时, CPU 可随时查询 TF1 的状态。所以, TF1 可用作查询测试的标志。TF1 也可以用软件置 1 或清 0, 同硬件置 1 或清 0 的效果一样。

TR1 (TCON. 6): T1 运行控制位。TR1 置 1 时, T1 开始工作; TR1 置 0 时, T1 停止工作。TR1 由软件置 1 或清 0。所以, 用软件可控制定时/计数器的启动与停止。

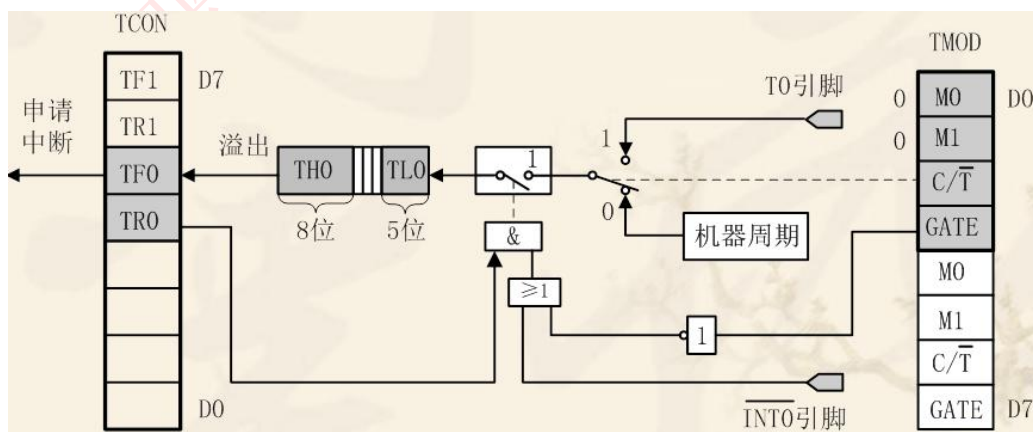
TF0 (TCON. 5): T0 溢出中断请求标志位, 其功能与 TF1 类同。

TR0 (TCON. 4): T0 运行控制位, 其功能与 TR1 类同。

(2) 定时/计数器的工作方式

1, 方式 0

方式 0 为 13 位计数, 由 TL0 的低 5 位 (高 3 位未用) 和 TH0 的 8 位组成。TL0 的低 5 位溢出时向 TH0 进位, TH0 溢出时, 置位 TCON 中的 TF0 标志, 向 CPU 发出中断请求。其结构图如下所示:

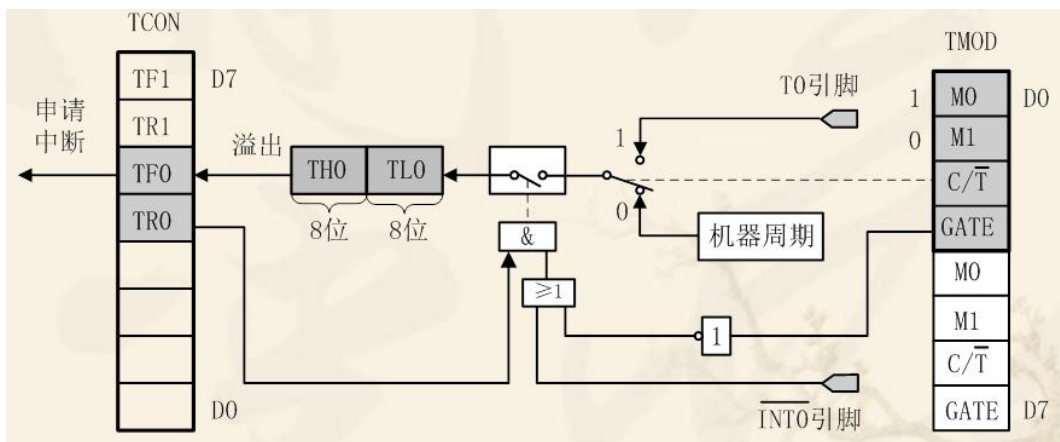


门控位 GATE 具有特殊的作用。当 GATE=0 时, 经反相后使或门输出为 1, 此时仅由 TR0 控制与门的开启, 与门输出 1 时, 控制开关接通, 计数开始; 当 GATE=1

时，由外中断引脚信号控制或门的输出，此时控制与门的开启由外中断引脚信号和 TR0 共同控制。当 TR0=1 时，外中断引脚信号引脚的高电平启动计数，外中断引脚信号引脚的低电平停止计数。这种方式常用来测量外中断引脚上正脉冲的宽度。计数模式时，计数脉冲是 T0 引脚上的外部脉冲。计数初值与计数个数的关系为： $X=2(13)-N$ 。其中 2(13) 表示 2 的 13 次方。

2，方式 1

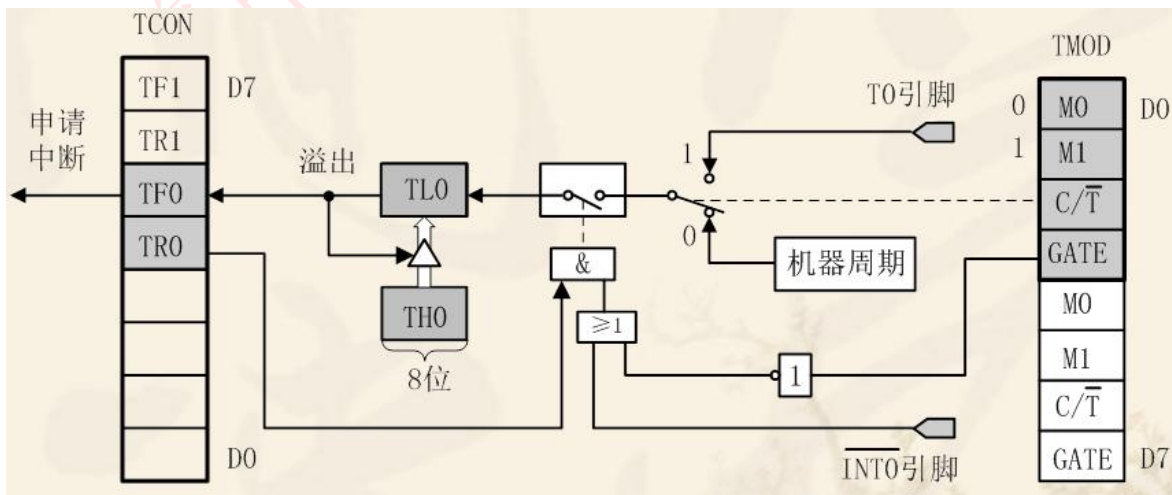
方式 1 的计数位数是 16 位，由 TL0 作为低 8 位，TH0 作为高 8 位，组成了 16 位加 1 计数器。其结构图如下所示：



计数初值与计数个数的关系为： $X=2(16)-N$ 。

3，方式 2

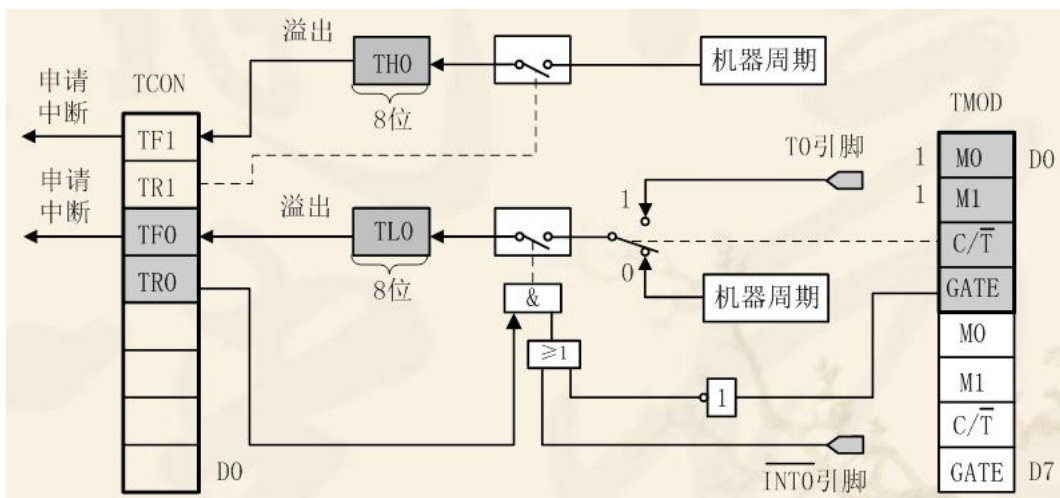
方式 2 为自动重装初值的 8 位计数方式。工作方式 2 特别适合于用作较精确的脉冲信号发生器。其结构图如下所示：



计数初值与计数个数的关系为： $X=2(8)-N$ 。

4，方式 3

方式 3 只适用于定时/计数器 T0， 定时器 T1 处于方式 3 时相当于 TR1=0，停止计数。工作方式 3 将 T0 分成为两个独立的 8 位计数器 TH0 和 TL0。其结构如下所示：



这几种工作方式中应用较多的是方式 1 和方式 2。定时器中通常使用定时器方式 1，串口通信中通常使用方式 2。

17.2 定时器配置

在使用定时器时，应该如何配置使其工作？其步骤如下（各步骤顺序可任意）：

- ①对 TMOD 赋值，以确定 T0 和 T1 的工作方式，如果使用定时器 0 即对 T0 配置，如果使用定时器 1 即对 T1 配置。
- ②根据所要定时的时间计算初值，并将其写入 TH0、TL0 或 TH1、TL1。
- ③如果使用中断，则对 EA 赋值，开放定时器中断。
- ④使 TR0 或 TR1 置位，启动定时/计数器定时或计数。

上述中有一个定时/计数器初值的计算，下面我们来看下如何计算定时/计数器初值。

前面我们介绍过机器周期的概念，它是 CPU 完成一个基本操作所需要的时间。其计算公式是：机器周期=1/单片机的时钟频率。51 单片机内部时钟频率是外部时钟的 12 分频，也就是说当外部晶振的频率输入到单片机里面的时候要进行 12 分频。比如说你用的是 12MHZ 晶振，那么单片机内部的时钟频率就是 12/12MHZ，当你使用 12MHZ 的外部晶振的时候，机器周期=1/1M=1us。如果我们想定时 1ms

的初值是多少呢？ $1\text{ms}/1\mu\text{s}=1000$ 。也就是要计数 1000 个，初值= $65535-1000+1$ （因为实际上计数器计数到 66636（2 的 16 次方）才溢出，所以后面要加 1）= $64536=\text{FC18H}$ ，所以初值即为 $\text{THx}=\text{0XFC}$ ， $\text{TLx}=\text{0X18}$ 。

知道了如何计算定时/计数器初值，那么想定时多长时间都可以计算出，当然由于定时计数器位数有限，我们不可能直接通过初值定时很长时间，如果要实现很长时间的定时，比如定时 1 秒钟。可以通过初值设置定时 1ms，每当定时 1ms 结束后又重新赋初值，并且设定一个全局变量累计定时 1ms 的次数，当累计到 1000 次，表示已经定时 1 秒了。需要其他定时时间类似操作，这样我们就可以使用定时器来实现精确延时来替代之前的 delay 函数。

这里以定时器 0 为例介绍配置定时器工作方式 1、设定 1ms 初值，开启定时器计数功能以及总中断，如下：

```
void Timer0Init()  
{  
    TMOD|=0X01;//选择为定时器 0 模式，工作方式 1，仅用 TR0 打开启动。  
  
    TH0=0XFC; //给定时器赋初值，定时 1ms  
    TL0=0X18;  
    ET0=1;//打开定时器 0 中断允许  
    EA=1;//打开总中断  
    TR0=1;//打开定时器  
}
```

对于定时器 1 的使用方法是一样的，只是将上述的 0 变为 1 即可，具体可参考我们定时器 1 实验例程。

17.3 硬件设计

本实验使用到硬件资源如下：

- (1) LED 模块 (D1)
- (2) 定时器 0

本章硬件电路非常简单，只使用到开发板上 LED 模块的 D1，至于定时器 0

它属于 51 单片机内部资源，只需通过软件配置即可使用。开发板上 LED 模块电路在前面已经介绍，这里就不多说。

如果想单片机控制 LED 闪烁，就必须通过单片机管脚来控制 LED 模块的 D1 指示灯。由于开发板上部分模块电路是集成的，所以直接使用单片机 P2.0 IO 口即可。

17.4 软件设计

本章所要实现的功能是：通过定时器 0 中断控制 D1 指示灯间隔 1 秒闪烁。

我们直接复制上一章创建的工程，在此模板基础上进行程序开发。为了能够与开发攻略教程对应，将复制过来的模板文件夹重新命名为“实验 18：定时器 0 中断”。打开工程直接在 main.c 源文件内进行编程，main.c 内代码如下：

```

/*****
*****
实验现象：下载程序后，D1 小灯循环点亮 1 秒，熄灭 1 秒。使用单片机内部
定时器可以实现准确延时

接线说明：（具体接线图可见开发攻略对应实验的“实验现象”章节）
1，单片机-->LED&交通灯模块
P20-->D1

注意事项：

*****
*****/

#include "reg52.h"          //此文件中定义了单片机的一些特殊功能
寄存器

typedef unsigned int u16;    //对数据类型进行声明定义
typedef unsigned char u8;

sbit led=P2^0;              //定义 P20 口是 led

```

```

/*****
*****
* 函数名      : Timer0Init
* 函数功能    : 定时器 0 初始化
* 输 入      : 无
* 输 出      : 无
*****
*****/
void Timer0Init()
{
    TMOD|=0X01;//选择为定时器 0 模式，工作方式 1，仅用 TR0 打开启动。

    TH0=0XFC; //给定时器赋初值，定时 1ms
    TL0=0X18;
    ET0=1;//打开定时器 0 中断允许
    EA=1;//打开总中断
    TR0=1;//打开定时器
}

/*****
*****
* 函数名      : main
* 函数功能    : 主函数
* 输 入      : 无
* 输 出      : 无
*****
*****/
void main()
{
    Timer0Init(); //定时器 0 初始化
    while(1);
}

/*****
*****
* 函数名      : void Timer0() interrupt 1
* 函数功能    : 定时器 0 中断函数
* 输 入      : 无
* 输 出      : 无
*****
*****/
void Timer0() interrupt 1
```

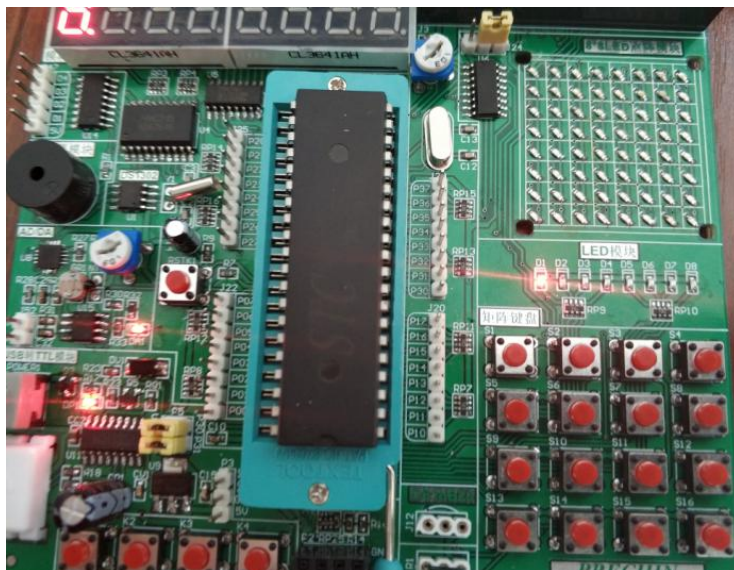
```
{
    static unsigned long i;
    TH0=0XFC; //给定时器赋初值，定时 1ms
    TL0=0X18;
    i++;
    if(i==1000)
    {
        i=0;
        led=~led;
    }
}
```

main.c 文件内代码非常少也很简单，首先将 51 单片机的头文件包含进来，然后定义 LED 的控制脚。主函数功能非常简单，首先调用 Timer0Init() 函数配置定时器 0 并开启中断功能，该函数配置过程就是按照前面介绍的配置方法所写，即选择定时器工作模式 0、工作方式 1、设置定时 1ms 初值、打开定时器计数功能和开启总中断功能。然后进入 while 循环，在循环体内没有执行任何功能程序。当定时时间到达即会进入定时器 0 中断，在中断服务函数内，重新赋初值准备下次计数，并且定义一个静态变量来累计定时 1ms 次数，当变量等于 1000 时，表示定时时间达 1 秒，然后清零变量以及控制 LED 状态翻转。执行完成后退出中断返回主函数，当时间到达又进入中断，如此循环。

对于定时器 1 的使用方法是同样的，只是将上述的 0 变为 1 即可，具体可参考我们定时器 1 实验例程。

17.5 实验现象

使用 USB 线将开发板和电脑连接成功后（电脑能识别开发板上 CH340 驱动串口），把编写好的程序编译后将编译产生的 .hex 文件烧入到芯片内，按照如下图所示的接线方式可以看到指示灯 D1 间隔 1 秒闪烁。



课后作业

- (1) 修改定时器初值，设定 3 秒钟的定时时间让 D1 指示灯闪烁。
 - (2) 使用定时器 1 和数码管设计一个数字时钟。
- (温馨提示：参考本章实验例程，将定时器及数码管程序进行整合)

第 18 章 EEPROM-IIC 实验

这一章我们来学习如何使用 51 单片机的 I/O 口模拟 I2C 时序，并实现与 AT24C02（EEPROM）之间的双向通信。开发板上集成了 1 个 EEPROM 模块，可实现 IIC 通信。本章要实现的功能是：系统运行时，数码管后 4 位显示 0，按 K1 将数据写入到 EEPROM 内保存，按 K2 读取 EEPROM 内保存的数据，按 K3 显示数据加 1，按 K4 显示数据清零，最大能写入的数据是 255。学习本章可以参考前面的实验章节内容。若结合视频学习效果更佳。本章分为如下几部分内容：

18.1 I2C 介绍

18.2 AT24C02 芯片介绍

25.3 硬件设计

18.4 软件设计

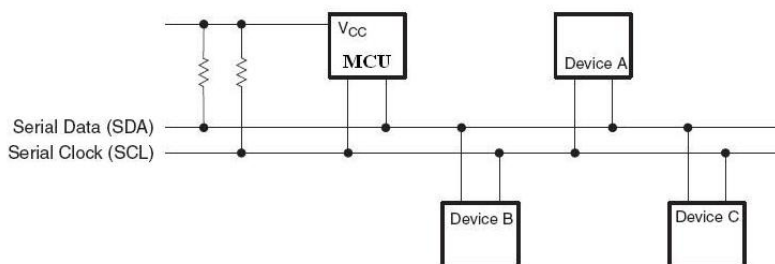
18.5 实验现象

18.1 I2C 介绍

I2C (Inter-Integrated Circuit) 总线是由 PHILIPS 公司开发的两线式串行总线, 用于连接微控制器及其外围设备。是微电子通信控制领域广泛采用的一种总线标准。它是同步通信的一种特殊形式, 具有接口线少, 控制方式简单, 器件封装形式小, 通信速率较高等优点。I2C 总线只有两根双向信号线。一根是数据线 SDA, 另一根是时钟线 SCL。由于其管脚少, 硬件实现简单, 可扩展性强等特点, 因此被广泛的使用在各大集成芯片内。下面我们就从 I2C 的物理层与协议层来了解 I2C。

18.1.1 I2C 物理层

I2C 通信设备常用的连接方式如下图所示:



它的物理层有如下特点:

(1) 它是一个支持多设备的总线。“总线”指多个设备共用的信号线。在一个 I2C 通讯总线中, 可连接多个 I2C 通讯设备, 支持多个通讯主机及多个通讯从机。

(2) 一个 I2C 总线只使用两条总线线路, 一条双向串行数据线(SDA), 一条串行时钟线(SCL)。数据线即用来表示数据, 时钟线用于数据收发同步。

(3) 每个连接到总线的设备都有一个独立的地址, 主机可以利用这个地址进行不同设备之间的访问。

(4) 总线通过上拉电阻接到电源。当 I2C 设备空闲时, 会输出高阻态, 而当所有设备都空闲, 都输出高阻态时, 由上拉电阻把总线拉成高电平。

(5) 多个主机同时使用总线时, 为了防止数据冲突, 会利用仲裁方式决定由哪个设备占用总线。

(6) 具有三种传输模式：标准模式传输速率为 100kbit/s，快速模式为 400kbit/s，高速模式下可达 3.4Mbit/s，但目前大多 I2C 设备尚不支持高速模式。

(7) 连接到相同总线的 IC 数量受到总线的最大电容 400pF 限制。

下面我们来了解下 I2C 总线常用的一些术语：

主机：启动数据传送并产生时钟信号的设备；

从机：被主机寻址的器件；

多主机：同时有多于一个主机尝试控制总线但不破坏传输；

主模式：用 I2CNDAT 支持自动字节计数的模式； 位 I2CRM, I2CSTT, I2CSTP 控制数据的接收和发送；

从模式：发送和接收操作都是由 I2C 模块自动控制的；

仲裁：是一个在有多个主机同时尝试控制总线但只允许其中一个控制总线并使传输不被破坏的过程；

同步：两个或多个器件同步时钟信号的过程；

发送器：发送数据到总线的器件；

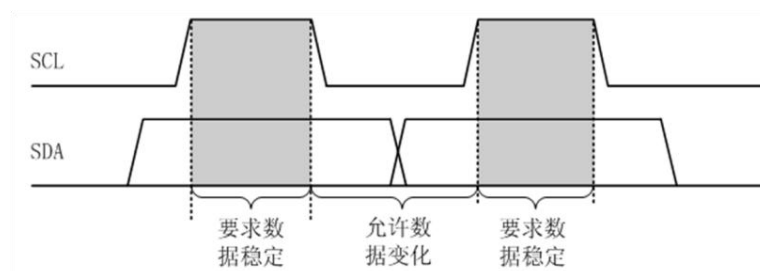
接收器：从总线接收数据的器件。

18.1.2 I2C 协议层

I2C 的协议定义了通信的起始和停止信号、数据有效性、响应、仲裁、时钟同步和地址广播等环节。下面我们就来简单介绍下。

(1) 数据有效性规定

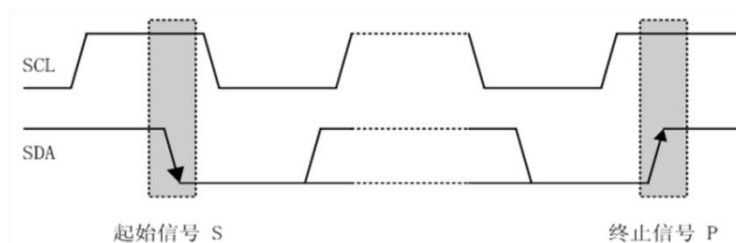
I2C 总线进行数据传送时，时钟信号为高电平期间，数据线上的数据必须保持稳定，只有在时钟线上的信号为低电平期间，数据线上的高电平或低电平状态才允许变化。如下图：



每次数据传输都以字节为单位，每次传输的字节数不受限制。

(2) 起始和停止信号

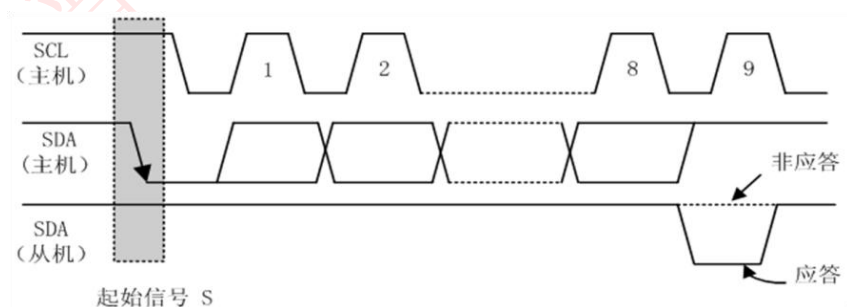
SCL 线为高电平期间，SDA 线由高电平向低电平的变化表示起始信号；SCL 线为高电平期间，SDA 线由低电平向高电平的变化表示终止信号。如下图：



起始和终止信号都是由主机发出的，在起始信号产生后，总线就处于被占用的状态；在终止信号产生后，总线就处于空闲状态。

(3) 应答响应

每当发送器件传输完一个字节的的数据后，后面必须紧跟一个校验位，这个校验位是接收端通过控制 SDA（数据线）来实现的，以提醒发送端数据我这边已经接收完成，数据传送可以继续进行。这个校验位其实就是数据或地址传输过程中的响应。响应包括“应答(ACK)”和“非应答(NACK)”两种信号。作为数据接收端时，当设备(无论主从机)接收到 I2C 传输的一个字节数据或地址后，若希望对方继续发送数据，则需要向对方发送“应答(ACK)”信号即特定的低电平脉冲，发送方会继续发送下一个数据；若接收端希望结束数据传输，则向对方发送“非应答(NACK)”信号即特定的高电平脉冲，发送方接收到该信号后会产生一个停止信号，结束信号传输。应答响应时序图如下：



每一个字节必须保证是 8 位长度。数据传送时，先传送最高位 (MSB)，每一个被传送的字节后面都必须跟随一位应答位 (即一帧共有 9 位)。

由于某种原因从机不对主机寻址信号应答时 (如从机正在进行实时性的处理工作而无法接收总线上的数据)，它必须将数据线置于高电平，而由主机产生一

个终止信号以结束总线的数据传送。

如果从机对主机进行了应答，但在数据传送一段时间后无法继续接收更多的数据时，从机可以通过对无法接收的第一个数据字节的“非应答”通知主机，主机则应发出终止信号以结束数据的继续传送。

当主机接收数据时，它收到最后一个数据字节后，必须向从机发出一个结束传送的信号。这个信号是由对从机的“非应答”来实现的。然后，从机释放 SDA 线，以允许主机产生终止信号。

这些信号中，起始信号是必需的，结束信号和应答信号都可以不要。

(4) 总线的寻址方式

I2C 总线寻址按照从机地址位数可分为两种，一种是 7 位，另一种是 10 位。采用 7 位的寻址字节（寻址字节是起始信号后的第一个字节）的位定义如下：

位:	7 ⁰	6 ⁰	5 ⁰	4 ⁰	3 ⁰	2 ⁰	1 ⁰	0 ⁰
	从机地址 ⁰							R/ $\overline{W}$

D7~D1 位组成从机的地址。D0 位是数据传送方向位，为“0”时表示主机向从机写数据，为“1”时表示主机由从机读数据。

10 位寻址和 7 位寻址兼容，而且可以结合使用。10 位寻址不会影响已有的 7 位寻址，有 7 位和 10 位地址的器件可以连接到相同的 I2C 总线。我们就以 7 位寻址为例进行介绍。

当主机发送了一个地址后，总线上的每个器件都将头 7 位与它自己的地址比较，如果一样，器件会判定它被主机寻址，其他地址不同的器件将被忽略后面的数据信号。至于是从机接收器还是从机发送器，都由 R/W 位决定的。从机的地址由固定部分和可编程部分组成。在一个系统中可能希望接入多个相同的从机，从机地址中可编程部分决定了可接入总线该类器件的最大数目。如一个从机的 7 位寻址位有 4 位是固定位，3 位是可编程位，这时仅能寻址 8 个同样的器件，即可以有 8 个同样的器件接入到该 I2C 总线系统中。

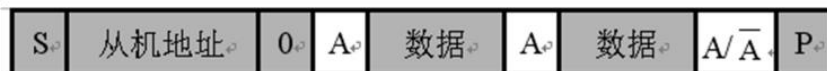
(5) 数据传输

I2C 总线上传送的数据信号是广义的，既包括地址信号，又包括真正的数据信号。在起始信号后必须传送一个从机的地址（7 位），第 8 位是数据的传送

方向位 (R/W)，用“0”表示主机发送(写)数据(W)，“1”表示主机接收数据(R)。每次数据传送总是由主机产生的终止信号结束。但是，若主机希望继续占用总线进行新的数据传送，则可以不产生终止信号，马上再次发出起始信号对另一从机进行寻址。

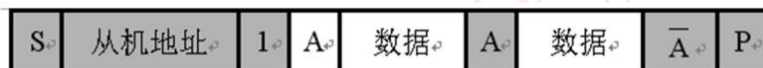
在总线的一次数据传送过程中，可以有以下几种组合方式：

a、主机向从机发送数据，数据传送方向在整个传送过程中不变



注意：有阴影部分表示数据由主机向从机传送，无阴影部分则表示数据由从机向主机传送。A 表示应答，A 非表示非应答(高电平)。S 表示起始信号，P 表示终止信号。

b、主机在第一个字节后，立即从从机读数据



c、在传送过程中，当需要改变传送方向时，起始信号和从机地址都被重复产生一次，但两次读/写方向位正好相反



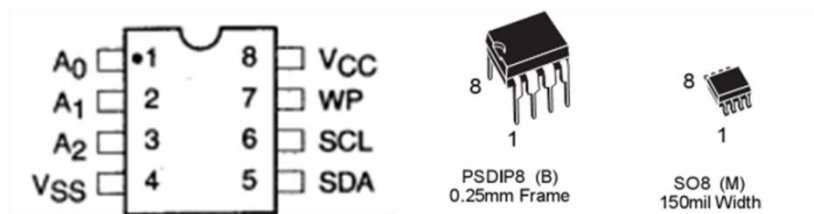
到这里我们就介绍完了 I2C 总线，由于 51 单片机没有硬件 IIC 接口，即使有硬件接口我们通常还是采用软件模拟 I2C。主要原因是硬件 IIC 设计的比较复杂，而且稳定性不怎么好，程序移植比较麻烦，而用软件模拟 IIC，最大的好处就是移植方便，同一个代码兼容所有单片机，任何一个单片机只要有 IO 口(不需要特定 IO)，都可以很快的移植过去。

下面我们再来了解下开发板上的 AT24C02 芯片。

18.2 AT24C02 芯片介绍

AT24C01/02/04/08/16... 是一个 1K/2K/4K/8K/16K 位串行 CMOS，内部含有 128/256/512/1024/2048 个 8 位字节，AT24C01 有一个 8 字节页写缓冲器，AT24C02/04/08/16 有一个 16 字节页写缓冲器。该器件通过 I2C 总线接口进行操作，它有一个专门的写保护功能。我们开发板上使用的是 AT24C02 (EEPROM)

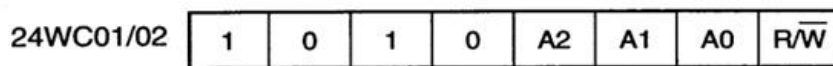
芯片，此芯片具有 I2C 通信接口，芯片内保存的数据在掉电情况下都不丢失，所以通常用于存放一些比较重要的数据等。AT24C02 芯片管脚及外观图如下图所示：



芯片管脚说明如下图所示：

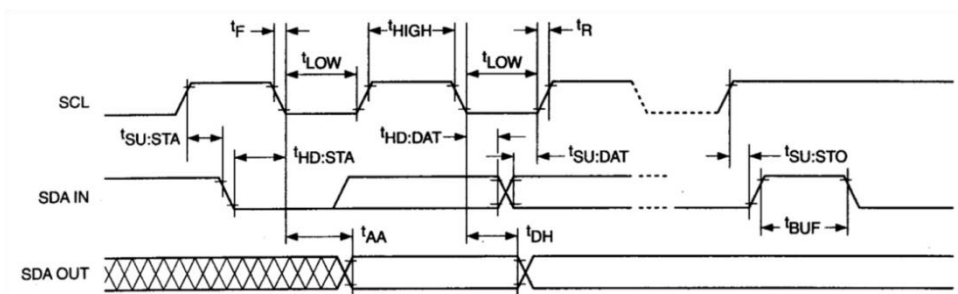
引脚号	引脚名称	功能说明
1	A0	地址输入。A2、A1和A0是器件地址输入引脚。
2	A1	24C02/32/64使用A2、A1和A0输入引脚作为硬件地址，总线上可同时级联8个24C02/32/64器件（详见器件寻址）。
3	A2	24C04使用A2和A1输入引脚作为硬件地址，总线上可同时级联4个24C04器件，A0为空脚，可接地。
		24C08使用A2输入引脚作为硬件地址，总线上可同时级联2个24C08器件，A0和A1为空脚，可接地。
		24C16未使用器件地址引脚，总线上最多只可连接一个16K器件，A2、A1和A0为空脚，可接地。
5	SDA	串行地址和数据输入/输出。SDA是双向串行数据传输引脚，漏极开路，需外接上拉电阻到Vcc（典型值10kΩ）。
6	SCL	串行时钟输入。SCL同步数据传输，上升沿数据写入，下降沿数据读出。
7	WP	写保护。WP引脚提供硬件数据保护。当WP接地时，允许数据正常读写操作；当WP接Vcc时，写保护，只读。
4	GND	地
8	Vcc	正电源

AT24C02 器件地址为 7 位，高 4 位固定为 1010，低 3 位由 A0/A1/A2 信号线的电平决定。因为传输地址或数据是以字节为单位传送的，当传送地址时，器件地址占 7 位，还有最后一位（最低位 R/W）用来选择读写方向，它与地址无关。其格式如下：



我们开发板已经将芯片的 A0/A1/A2 连接到 GND，所以器件地址为 1010000，即 0x50（未计算最低位）。如果要对芯片进行写操作时，R/W 即为 0，写器件地址即为 0xA0；如果要对芯片进行读操作时，R/W 即为 1，此时读器件地址为 0xA1。开发板上我们也将 WP 引脚直接接在 GND 上，此时芯片允许数据正常读写。

I2C 总线时序如下图所示：



读写周期范围

符号	参数	1.8 V, 2.5 V		4.5V~5.5V		单位
		最小	最大	最小	最大	
F_{SCL}	时钟频率		100		400	KHz
T_I	SCL,SDA 输入的噪声抑制时间		200		200	ns
t_{AA}	SCL 变低至 SDA 数据输出及应答信号		3.5		1	μs
t_{BUF}	新的发送开始前总线空闲时间	4.7		1.2		μs
$t_{HD: STA}$	起始信号保持时间	4		0.6		μs
t_{LOW}	时钟低电平周期	4.7		1.2		μs
t_{HIGH}	时钟高电平周期	4		0.6		μs
$t_{SU: STA}$	起始信号建立时间	4.7		0.6		μs
$t_{HD: DAT}$	数据输入保持时间	0		0		ns
$t_{SU: DAT}$	数据输入建立时间	50		50		ns
t_R	SDA 及 SCL 上升时间		1		0.3	μs
t_F	SDA 及 SCL 下降时间		300		300	ns
$t_{SU: STO}$	停止信号建立时间	4		0.6		μs
t_{DH}	数据输出保持时间	100		100		ns

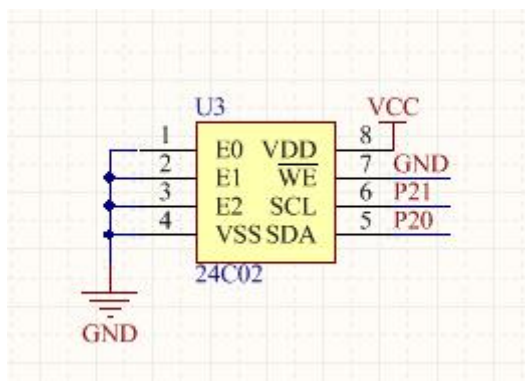
关于 AT24C02(EEPROM)的更多信息,可参考“\6--开发板芯片资料”内 24C02 数据手册来了解。

18.3 硬件设计

本实验使用到硬件资源如下:

- (1) 独立按键 (K1-K4)
- (2) 动态数码管
- (3) EEPROM 模块电路 (AT24C02)

独立按键和动态数码管电路在前面章节都介绍过,这里就不再重复。下面我们来看下开发板上 EEPROM 模块电路,如下图所示:



从上图中可以看出，该电路是独立的，芯片的 SCL 和 SDA 管脚接至 J4 端子上，在介绍 IIC 总线的时候我们说过，为了让 IIC 总线默认为高电平，通常会在 IIC 总线上接上拉电阻，在图中并没有看到 SCL 和 SDA 管脚有上拉电阻，这是因为我们单片机 IO 都外接了 10K 上拉电阻，当单片机 IO 口连接到芯片的 SCL 和 SDA 脚时即相当于它们外接上拉电阻，所以此处可以省去。

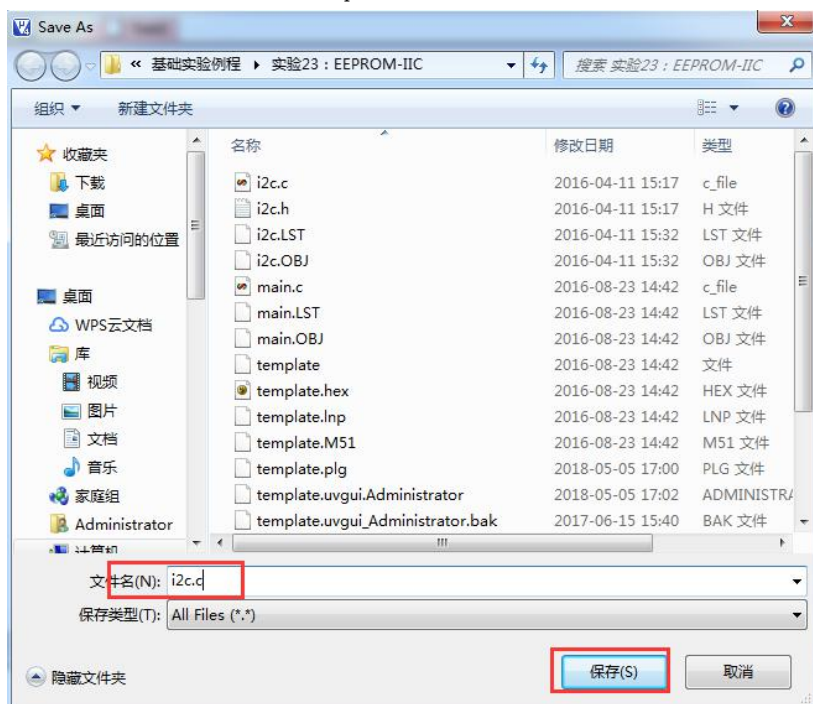
由于该模块电路是集成的，所以 24C02 芯片的 SCL 和 SDA 管脚可以通过使用单片机的 P2⁰~P2¹ 管脚连接即可。

18.4 软件设计

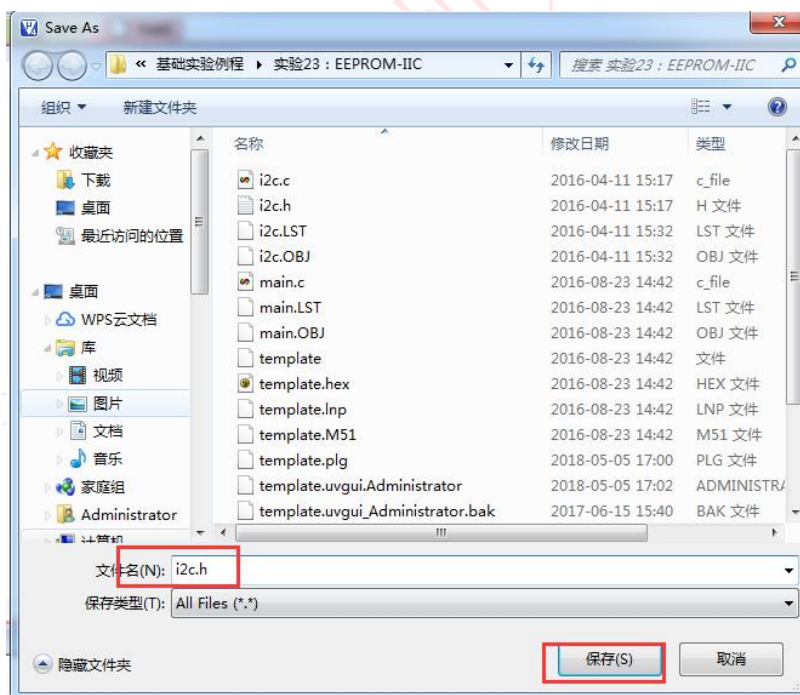
本章所要实现的功能是：系统运行时，数码管后 4 位显示 0，按 K1 将数据写入到 EEPROM 内保存，按 K2 读取 EEPROM 内保存的数据，按 K3 显示数据加 1，按 K4 显示数据清零，最大能写入的数据是 255。

我们直接复制上一章创建的工程，在此模板基础上进行程序开发。为了能够与开发攻略教程对应，将复制过来的模板文件夹重新命名为“实验 23：EEPROM-IIC”。打开工程文件夹，发现里面多了 2 个文件：i2c.c 源文件和 i2c.h 头文件，该文件保存的是 AT24C02 的驱动程序。如何在工程中创建多源文件工程呢？很简单，具体步骤如下：

(1) 打开工程，新建一个源文件，点击保存，命名为 i2c.c，如下所示：



(2) 同样的方法, 新建一个源文件对应的头文件, 点击保存, 命名为 i2c.h, 如下所示:



在头文件内编写如下代码:

```
#ifndef __I2C_H_
#define __I2C_H_

#include <reg52.h>
```

```
#endif
```

这个格式在前面章节我们就介绍过，这里不再重复。

(3) 将新建好的源文件添加到工程中，在工程模板创建章节就已经介绍如何添加文件到工程，这里不多说。添加进来后在 i2c.c 源文件中编写如下代码：

```
#include "i2c.h"

/*****
*****
* 函数名      : Delay10us()
* 函数功能    : 延时 10us
* 输入        : 无
* 输出        : 无
*****
*****/

void Delay10us()
{
    unsigned char a,b;
    for(b=1;b>0;b--)
        for(a=2;a>0;a--);
}

/*****
*****
* 函数名      : I2cStart()
* 函数功能    : 起始信号：在 SCL 时钟信号在高电平期间 SDA 信号产生一个下降沿
* 输入        : 无
* 输出        : 无
* 备注        : 起始之后 SDA 和 SCL 都为 0
*****
*****/

void I2cStart()
{
    SDA=1;
    Delay10us();
    SCL=1;
```

```

    Delay10us(); //建立时间是 SDA 保持时间>4.7us
    SDA=0;
    Delay10us(); //保持时间是>4us
    SCL=0;
    Delay10us();
}

/*****
*****
* 函数名      : I2cStop()
* 函数功能    : 终止信号：在 SCL 时钟信号高电平期间 SDA 信号产生一个上升沿
* 输入        : 无
* 输出        : 无
* 备注        : 结束之后保持 SDA 和 SCL 都为 1；表示总线空闲
*****
*****/

void I2cStop()
{
    SDA=0;
    Delay10us();
    SCL=1;
    Delay10us(); //建立时间大于 4.7us
    SDA=1;
    Delay10us();
}

/*****
*****
* 函数名      : I2cSendByte(unsigned char dat)
* 函数功能    : 通过 I2C 发送一个字节。在 SCL 时钟信号高电平期间，保持发送信号 SDA 保持稳定
* 输入        : num
* 输出        : 0 或 1。发送成功返回 1，发送失败返回 0
* 备注        : 发送完一个字节 SCL=0, SDA=1
*****
*****/

unsigned char I2cSendByte(unsigned char dat)
{
    unsigned char a=0,b=0; //最大 255，一个机器周期为 1us，最大延时 255us。
    for(a=0;a<8;a++) //要发送 8 位，从最高位开始
    {
        SDA=dat>>7;    //起始信号之后 SCL=0，所以可以直接改变 SDA 信

```

号

```

        dat=dat<<1;
        Delay10us();
        SCL=1;
        Delay10us(); //建立时间>4.7us
        SCL=0;
        Delay10us(); //时间大于 4us
    }
    SDA=1;
    Delay10us();
    SCL=1;
    while(SDA) //等待应答，也就是等待从设备把 SDA 拉低
    {
        b++;
        if(b>200) //如果超过 2000us 没有应答发送失败，或者为非应答，
表示接收结束
        {
            SCL=0;
            Delay10us();
            return 0;
        }
    }
    SCL=0;
    Delay10us();
    return 1;
}

/*****
*****
* 函数名      : I2cReadByte()
* 函数功能    : 使用 I2c 读取一个字节
* 输入        : 无
* 输出        : dat
* 备注        : 接收完一个字节 SCL=0, SDA=1.
*****
*****/

unsigned char I2cReadByte()
{
    unsigned char a=0, dat=0;
    SDA=1; //起始和发送一个字节之后 SCL 都是 0
    Delay10us();
    for(a=0;a<8;a++) //接收 8 个字节
    {
        SCL=1;

```

```

        Delay10us();
        dat<<=1;
        dat|=SDA;
        Delay10us();
        SCL=0;
        Delay10us();
    }
    return dat;
}

/*****
*****
* 函数名          : void At24c02Write(unsigned char addr,unsigned
char dat)
* 函数功能        : 往 24c02 的一个地址写入一个数据
* 输入            : 无
* 输出            : 无
*****
*****/

void At24c02Write(unsigned char addr,unsigned char dat)
{
    I2cStart();
    I2cSendByte(0xa0); //发送写器件地址
    I2cSendByte(addr); //发送要写入内存地址
    I2cSendByte(dat); //发送数据
    I2cStop();
}

/*****
*****
* 函数名          : unsigned char At24c02Read(unsigned char addr)
* 函数功能        : 读取 24c02 的一个地址的一个数据
* 输入            : 无
* 输出            : 无
*****
*****/

unsigned char At24c02Read(unsigned char addr)
{
    unsigned char num;
    I2cStart();
    I2cSendByte(0xa0); //发送写器件地址
    I2cSendByte(addr); //发送要读取的地址

```



```

I2cStart();
I2cSendByte(0xa1); //发送读器件地址
num=I2cReadByte(); //读取数据
I2cStop();
return num;
}

```

该源文件内代码即为 AT24C02 驱动程序，其实现的原理是按照前面介绍 I2C 总线时序编写。具体功能可参考程序内注释。源文件内对 SDA 和 SCL 管脚的操作在其头文件内已经定义。

将源文件内的函数名复制到 i2c.h 头文件内进行声明，方便其他源文件调用，比如 main.c 文件。

i2c.h 文件代码如下：

```

#ifndef __I2C_H_
#define __I2C_H_

#include <reg52.h>

sbit SCL=P2^1;
sbit SDA=P2^0;

void I2cStart();
void I2cStop();
unsigned char I2cSendByte(unsigned char dat);
unsigned char I2cReadByte();
void At24c02Write(unsigned char addr,unsigned char dat);
unsigned char At24c02Read(unsigned char addr);

#endif

```

下面来看下 main.c 文件代码，main.c 内代码如下：

```

/*****
*****

```

实验现象：下载程序后数码管后 4 位显示 0，按 K1 保存显示的数据，按 K2 读取上次保存的数据，

按 K3 显示数据加一，按 K4 显示数据清零。最大能写入的数据是 255

接线说明：（具体接线图可见开发攻略对应实验的“实验现象”章节）

注意事项：

```

*****
*****/

#include "reg52.h"           //此文件中定义了单片机的一些特殊功能
寄存器
#include "i2c.h"

typedef unsigned int u16;    //对数据类型进行声明定义
typedef unsigned char u8;

sbit LSA=P2^2;
sbit LSB=P2^3;
sbit LSC=P2^4;

sbit k1=P3^1;
sbit k2=P3^0;
sbit k3=P3^2;
sbit k4=P3^3; //定义按键端口

char num=0;
u8 disp[4];
u8                                     code
smgduan[10]={0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07, 0x7f, 0x6f};

/*****
*****
* 函数名      : delay
* 函数功能    : 延时函数, i=1 时, 大约延时 10us
*****
*****/
void delay(u16 i)
{
    while(i--);
}

/*****
*****
* 函数名      : Keypros()
* 函数功能    : 按键处理函数
* 输入        : 无
* 输出        : 无
*****
*****/

```

```

void Keypros()
{
    if(k1==0)
    {
        delay(1000); //消抖处理
        if(k1==0)
        {
            At24c02Write(1,num); //在地址 1 内写入数据 num
        }
        while(!k1);
    }
    if(k2==0)
    {
        delay(1000); //消抖处理
        if(k2==0)
        {
            num=At24c02Read(1); //读取 EEPROM 地址 1 内的数据保存在
num 中
        }
        while(!k2);
    }
    if(k3==0)
    {
        delay(100); //消抖处理
        if(k3==0)
        {
            num++; //数据加 1
            if(num>255) num=0;
        }
        while(!k3);
    }
    if(k4==0)
    {
        delay(1000); //消抖处理
        if(k4==0)
        {
            num=0; //数据清零
        }
        while(!k4);
    }
}

/*****
*****/

```

```

* 函数名      :datapros()
* 函数功能    :数据处理函数
* 输入        : 无
* 输出        : 无
*****
*****/
void datapros()
{
    disp[0]=smgduan[num/1000]; //千位
    disp[1]=smgduan[num%1000/100]; //百位
    disp[2]=smgduan[num%1000%100/10]; //个位
    disp[3]=smgduan[num%1000%100%10];
}

/*****
*****/
* 函数名      :DigDisplay()
* 函数功能    :数码管显示函数
* 输入        : 无
* 输出        : 无
*****
*****/
void DigDisplay()
{
    u8 i;
    for(i=0;i<4;i++)
    {
        switch(i) //位选，选择点亮的数码管，
        {
            case(0):
                LSA=1;LSB=1;LSC=0; break; //显示第 0 位
            case(1):
                LSA=0;LSB=1;LSC=0; break; //显示第 1 位
            case(2):
                LSA=1;LSB=0;LSC=0; break; //显示第 2 位
            case(3):
                LSA=0;LSB=0;LSC=0; break; //显示第 3 位
        }
        P0=disp[i]; //发送数据
        delay(100); //间隔一段时间扫描
        P0=0x00; //消隐
    }
}

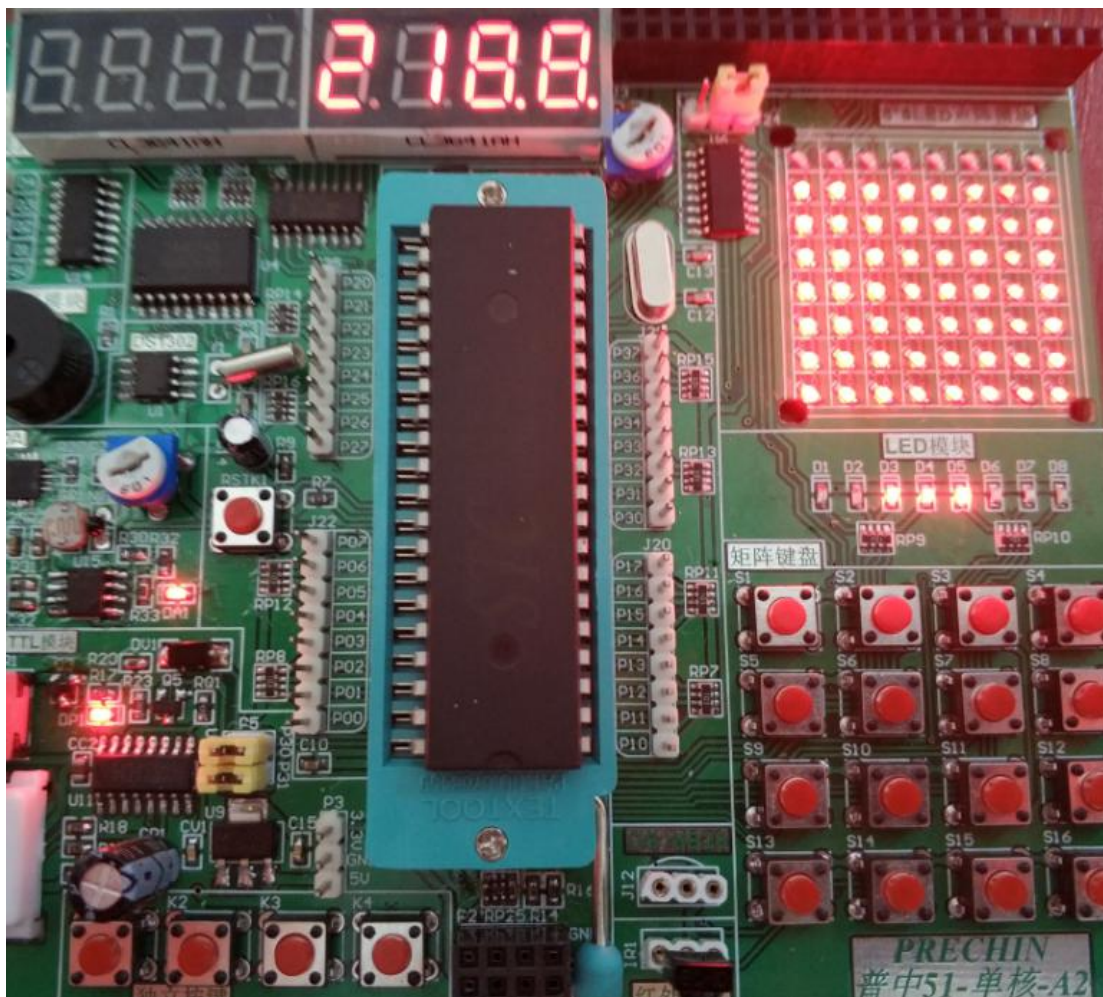
```

```
/******  
*****  
* 函 数 名      : main  
* 函数功能      : 主函数  
* 输    入      : 无  
* 输    出      : 无  
*****  
*****/  
void main()  
{  
    while(1)  
    {  
        Keypros(); //按键处理函数  
        datapros(); //数据处理函数  
        DigDisplay(); //数码管显示函数  
    }  
}
```

main.c 文件内代码非常少也很简单，首先将 51 单片机的头文件和 i2c.h 的头文件包含进来，然后定义动态数码管、74HC138 译码器、独立按键 K1-K4 的控制管脚。主函数功能非常简单，首先调用 Keypros() 按键处理函数对 24C02 芯片进行读写操作。当 K1 按下调用 At24c02Write 函数将变量 num 数据写入到对应的地址内，当 K2 按下调用 At24c02Read 函数进行读取操作，将从对应地址内读取的值存储在变量 num 中，当 K3 按下对变量 num 加 1，最大只能到 255，当 K4 按下对变量 num 清零。然后调用 datapros() 数据函数将 num 值进行拆分为数码管显示的每个段码数据存储在数组 disp 中。最后调用 DigDisplay() 动态数码管显示函数，将处理好的数据显示。

18.5 实验现象

使用 USB 线将开发板和电脑连接成功后，把编写好的程序编译后将编译产生的 .hex 文件烧入到芯片内，按照如下图所示的接线方式可以看到：数码管前 4 位显示 0，按 K1 将数据写入到 EEPROM 内保存，按 K2 读取 EEPROM 内保存的数据，按 K3 显示数据加 1，按 K4 显示数据清零，最大能写入的数据是 255。



课后作业

(1) 通过 AT24C02 芯片对功能程序进行加密。(温馨提示：开始写入一个数据到 AT24C02 中，通过读取判断是否为写入的数据，从而执行程序后面的功能)。

第 19 章 DS18B20 温度传感器实验

在前面章节，我们介绍了如何使用单片机 I/O 口模拟 IIC 总线时序。这一章我们来学习下精度较高的外部 DS18B20 数字温度传感器，由于此传感器是单总线接口，所以需要使用 51 单片机的一个 I/O 口模拟单总线时序与 DS18B20 通信，将检测的环境温度读取出来。开发板上集成了 1 个 DS18B20 温度传感器/DHT11 接口，需插上配置的 DS18B20 温度传感器后才能测试温度。本章要实现的功能是：系统运行时，插上 DS18B20 温度传感器，数码管显示检测的温度值。学习本章可以参考前面的实验章节内容。若结合视频学习效果更佳。本章分为如下几部分内容：

19.1 DS18B20 介绍

19.2 硬件设计

19.3 软件设计

19.4 实验现象

19.1 DS18B20 介绍

DS18B20 是由 DALLAS 半导体公司推出的一种的“一线总线（单总线）”接口的温度传感器。与传统的热敏电阻等测温元件相比，它是一种新型的体积小、适用电压宽、与微处理器接口简单的数字化温度传感器。

DS18B20 温度传感器具有如下特点：

1、适应电压范围更宽，电压范围：3.0~5.5V，在寄生电源方式下可由数据线供电。

2、独特的单线接口方式，DS18B20 在与微处理器连接时仅需要一条口线即可实现微处理器与 DS18B20 的双向通讯。

3、DS18B20 支持多点组网功能，多个 DS18B20 可以并联在唯一的三线上，实现组网多点测温。

4、DS18B20 在使用中不需要任何外围元件，全部传感元件及转换电路集成在形如一只三极管的集成电路内。

5、温范围 $-55^{\circ}\text{C}\sim+125^{\circ}\text{C}$ ，在 $-10\sim+85^{\circ}\text{C}$ 时精度为 $\pm 0.5^{\circ}\text{C}$

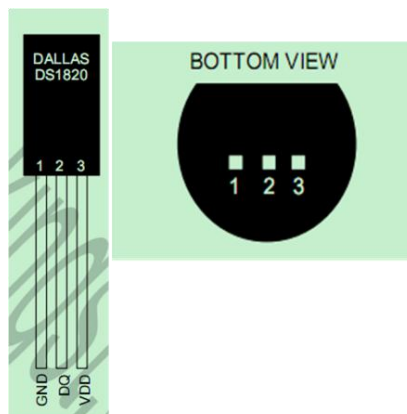
6、可编程的分辨率为 9~12 位，对应的可分辨温度分别为 0.5°C 、 0.25°C 、 0.125°C 和 0.0625°C ，可实现高精度测温。

7、在 9 位分辨率时最多在 93.75ms 内把温度转换为数字，12 位分辨率时最多在 750ms 内把温度值转换为数字，速度更快。

8、测量结果直接输出数字温度信号，以“一根总线”串行传送给 CPU，同时可传送 CRC 校验码，具有极强的抗干扰纠错能力。

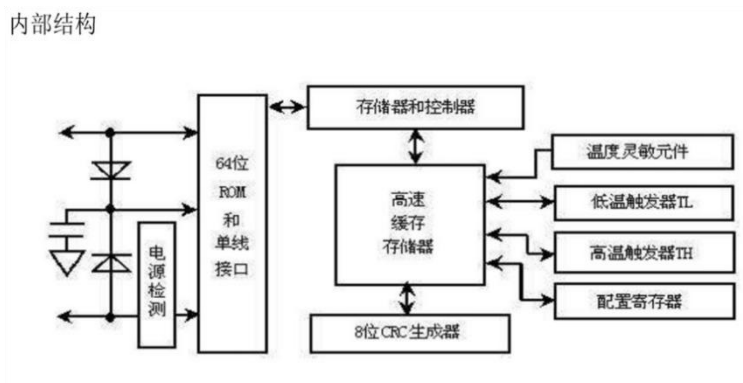
9、负压特性：电源极性接反时，芯片不会因发热而烧毁，但不能正常工作。

DS18B20 外观实物如下图所示：



从 DS18B20 外观图可以看到，当我们正对传感器切面（传感器型号字符那一面）时，传感器的管脚顺序是从左到右排列。管脚 1 为 GND，管脚 2 为数据 DQ，管脚 3 为 VDD。如果把传感器插反，那么电源将短路，传感器就会发烫，很容易损坏，所以一定要注意传感器方向，通常我们在开发板上都会标出传感器的凸起处，所以只需要把传感器凸起的方向对着开发板凸起方向插入即可。

DS18B20 内部结构如下图所示：



ROM 中的 64 位序列号是出厂前被光刻好的，它可以看作是该 DS18B20 的地址序列号。64 位光刻 ROM 的排列是：开始 8 位（28H）是产品类型标号，接着的 48 位是该 DS18B20 自身的序列号，最后 8 位是前面 56 位的循环冗余校验码。光刻 ROM 的作用是使每一个 DS18B20 都各不相同，这样就可以实现一根总线上挂接多个 DS18B20 的目的。

DS18B20 温度传感器的内部存储器包括一个高速的暂存器 RAM 和一个非易失性的可电擦除的 EEPROM，后者存放高温度和低温度触发器 TH、TL 和配置寄存器。

配置寄存器是配置不同的位数来确定温度和数字的转化，配置寄存器结构如下：



低五位一直都是“1”，TM 是测试模式位，用于设置 DS18B20 在工作模式还是在测试模式。在 DS18B20 出厂时该位被设置为 0，用户不需要去改动。R1 和 R0 用来设置 DS18B20 的精度（分辨率），可设置为 9，10，11 或 12 位，对应的分辨率温度是 0.5℃，0.25℃，0.125℃和 0.0625℃。R0 和 R1 配置如下图：

R1	R0	精度	最大转换时间	
0	0	9-bit	93.75 ms	($t_{CONV}/8$)
0	1	10-bit	187.5 ms	($t_{CONV}/4$)
1	0	11-bit	375 ms	($t_{CONV}/2$)
1	1	12-bit	750 ms	(t_{CONV})

在初始状态下默认的精度是 12 位，即 R0=1、R1=1。高速暂存存储器由 9 个字节组成，其分配如下：

寄存器内容	字节地址
温度值低位 (LS Byte)	0
温度值高位 (MS Byte)	1
高温限值 (TH)	2
低温限值 (TL)	3
配置寄存器	4
保留	5
保留	6
保留	7
CRC校验值	8

当温度转换命令 (44H) 发布后，经转换所得的温度值以二字节补码形式存放在高速暂存存储器的第 0 和第 1 个字节。存储的两个字节，高字节的前 5 位是符号位 S，单片机可通过单线接口读到该数据，读取时低位在前，高位在后，数据格式如下：

温度寄存器格式 图 2

	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
LS Byte	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}
	bit 15	bit 14	bit 13	bit 12	bit 11	bit 10	bit 9	bit 8
MS Byte	S	S	S	S	S	2^6	2^5	2^4

如果测得的温度大于 0，这 5 位为 ‘0’，只要将测到的数值乘以 0.0625（默认精度是 12 位）即可得到实际温度；如果温度小于 0，这 5 位为 ‘1’，测到的数值需要取反加 1 再乘以 0.0625 即可得到实际温度。温度与数据对应关系如下：

温度/数据关系 表 2

温度 ℃	数据输出（二进制）	数据输出（十六进制）
+125	0000 0111 1101 0000	07D0h
+85	0000 0101 0101 0000	0550h
+25.0625	0000 0001 1001 0001	0191h
+10.125	0000 0000 1010 0010	00A2h
+0.5	0000 0000 0000 1000	0008h
0	0000 0000 0000 0000	0000h
-0.5	1111 1111 1111 1000	FFF8h
-10.125	1111 1111 0101 1110	FF5Eh
-25.0625	1111 1110 0110 1111	FE6Eh
-55	1111 1100 1001 0000	FC90h

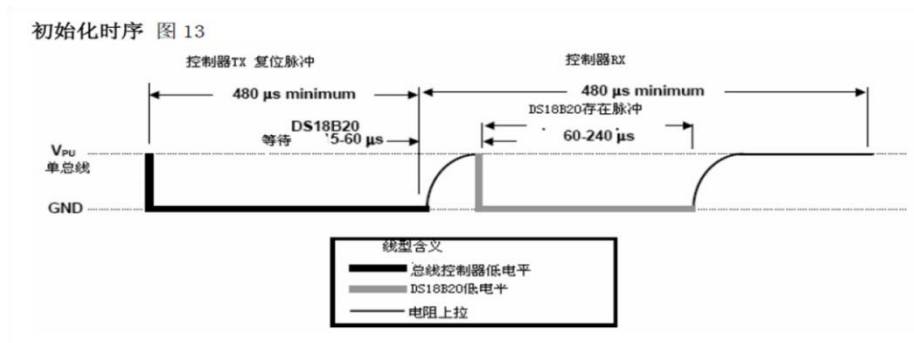
*上电复位时温度寄存器默认值为+85℃

比如我们要计算+85 度，数据输出十六进制是 0X0550，因为高字节的高 5 位为 0，表明检测的温度是正温度，0X0550 对应的十进制为 1360，将这个值乘以 12 位精度 0.0625，所以可以得到+85 度。

知道了怎么计算温度，接下来我们就来看看如何读取温度数据，由于 DS18B20 是单总线器件，所有的单总线器件都要求采用严格的信号时序，以保证数据的完整性。DS18B20 时序包括如下几种：初始化时序、写（0 和 1）时序、读（0 和 1）时序。DS18B20 发送所有的命令和数据都是字节的低位在前。这里我们简单介绍这几个信号的时序：

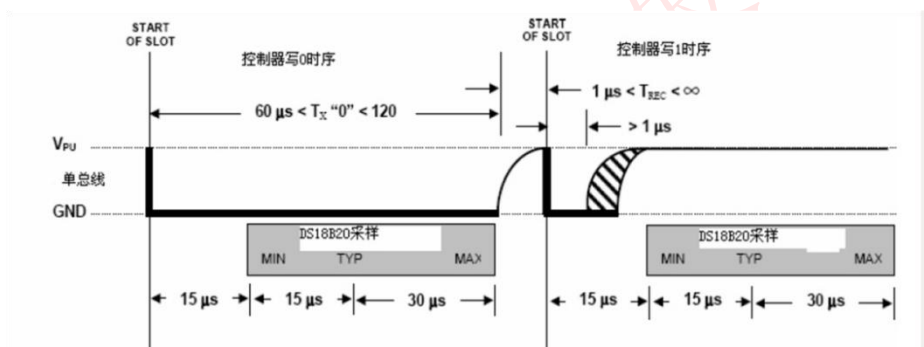
（1） 初始化时序

单总线上的所有通信都是以初始化序列开始。主机输出低电平，保持低电平时间至少 480us（该时间的范围可以从 480 到 960 微妙），以产生复位脉冲。接着主机释放总线，外部的上拉电阻将单总线拉高，延时 15~60 us，并进入接收模式。接着 DS18B20 拉低总线 60~240 us，以产生低电平应答脉冲，若为低电平，还要做延时，其延时的时间从外部上拉电阻将单总线拉高算起最少要 480 微妙。初始化时序图如下：



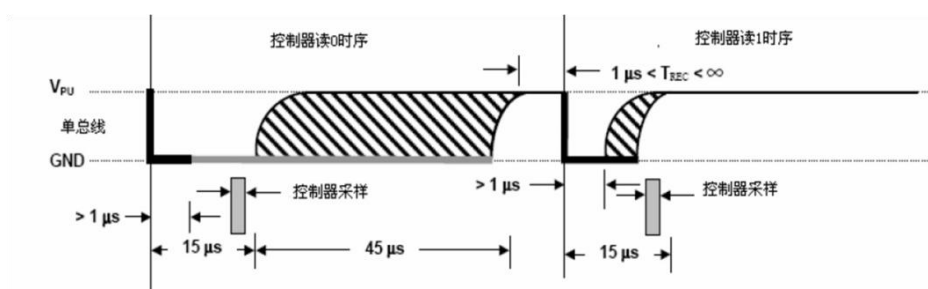
(2) 写时序

写时序包括写 0 时序和写 1 时序。所有写时序至少需要 60us，且在 2 次独立的写时序之间至少需要 1us 的恢复时间，两种写时序均起始于主机拉低总线。写 1 时序：主机输出低电平，延时 2us，然后释放总线，延时 60us。写 0 时序：主机输出低电平，延时 60us，然后释放总线，延时 2us。写时序图如下：



(3) 读时序

单总线器件仅在主机发出读时序时，才向主机传输数据，所以，在主机发出读数据命令后，必须马上产生读时序，以便从机能够传输数据。所有读时序至少需要 60us，且在 2 次独立的读时序之间至少需要 1us 的恢复时间。每个读时序都由主机发起，至少拉低总线 1us。主机在读时序期间必须释放总线，并且在时序起始后的 15us 之内采样总线状态。读时序图如下：



典型的读时序过程为：主机输出低电平延时 2us，然后主机转入输入模式延

时 12us，然后读取单总线当前的电平，然后延时 50us。

在了解了单总线时序之后，我们来看看 DS18B20 的典型温度读取过程，DS18B20 的典型温度读取过程为：复位→发 SKIP ROM 命令（0XCC）→发开始转换命令（0X44）→延时→复位→发送 SKIP ROM 命令（0XCC）→发读存储器命令（0XBE）→连续读出两个字节数据（即温度）→结束。

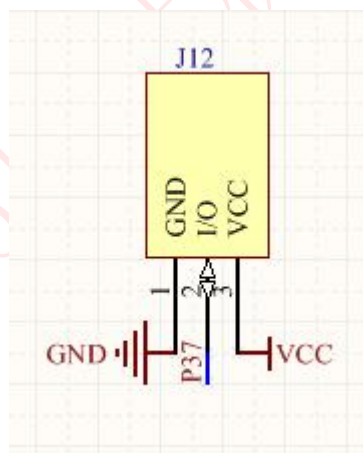
到这里我们就介绍完了 DS18B20，如需更详细的介绍，请大家参考“\6--开发板芯片资料\DS18B20 中文.pdf”。

19.2 硬件设计

本实验使用到硬件资源如下：

- （1）动态数码管
- （2）DS18B20

动态数码管电路在前面章节都介绍过，这里就不再重复。下面我们来看下开发板上 DS18B20 模块电路，如下图所示：



从上图中可以看出，该电路是集成的，传感器接口的单总线管脚接至单片机 P37 I/O 口上，在介绍单总线的时候我们说过，为了让单总线默认为高电平，通常会在单总线上接上拉电阻，在图中并没有看到有上拉电阻，这是因为我们单片机 I/O 都外接了 10K 上拉电阻，当单片机 I/O 口连接到传感器的总线管脚时即相当于它们外接上拉电阻，所以此处可以省去。

由于该模块电路是集成的，所以传感器接口的 DQ 管脚直接与单片机 P37 管脚连接使用即可。

特别注意：一定要注意温度传感器的方向，在接口处我们已经用丝印画了一个凸起，所以只需要将温度传感器对应插入即可。

19.3 软件设计

本章所要实现的功能是：系统运行时，插上 DS18B20 温度传感器，数码管显示检测的温度值。

我们直接复制上一章创建的工程，在此模板基础上进行程序开发。为了能够与开发攻略教程对应，将复制过来的模板文件夹重新命名为“实验 24：DS18B20 温度传感器”。打开工程文件夹，发现里面多了 2 个文件：temp.c 源文件和 temp.h 头文件，该文件保存的是 DS18B20 温度传感器的驱动程序。至于如何创建多文件工程，在上一章已经介绍过，这里就不再重复。

(1) 首先我们看下 temp.c 文件代码，如下：

```
#include "temp.h"
/*****
*****
* 函数名      : Delay1ms
* 函数功能    : 延时函数
* 输入      : 无
* 输出      : 无
*****
*****/

void Delay1ms(uint y)
{
    uint x;
    for( ; y>0; y--)
    {
        for(x=110; x>0; x--);
    }
}
/*****
*****
* 函数名      : Ds18b20Init
* 函数功能    : 初始化
* 输入      : 无
* 输出      : 初始化成功返回 1，失败返回 0
*****
*****/
```

```

uchar Ds18b20Init()
{
    uchar i;
    DSPORT = 0;          //将总线拉低 480us~960us
    i = 70;
    while(i--); //延时 642us
    DSPORT = 1;          //然后拉高总线, 如果 DS18B20 做出反应会将在
15us~60us 后总线拉低
    i = 0;
    while(DSPORT) //等待 DS18B20 拉低总线
    {
        Delay1ms(1);
        i++;
        if(i>5) //等待>5MS
        {
            return 0; //初始化失败
        }
    }
    return 1; //初始化成功
}

/*****
*****
* 函 数 名      : Ds18b20WriteByte
* 函数功能      : 向 18B20 写入一个字节
* 输    入      : 无
* 输    出      : 无
*****
*****/

void Ds18b20WriteByte(uchar dat)
{
    uint i, j;

    for(j=0; j<8; j++)
    {
        DSPORT = 0;          //每写入一位数据之前先把总线拉低 1us
        i++;
        DSPORT = dat & 0x01; //然后写入一个数据, 从最低位开始
        i=6;
        while(i--); //延时 68us, 持续时间最少 60us
        DSPORT = 1;          //然后释放总线, 至少 1us 给总线恢复时间才能接

```

着写入第二个数值

```

        dat >>= 1;
    }
}

/*****
*****
* 函 数 名      : Ds18b20ReadByte
* 函数功能      : 读取一个字节
* 输    入      : 无
* 输    出      : 无
*****
*****/

uchar Ds18b20ReadByte()
{
    uchar byte, bi;
    uint i, j;
    for(j=8; j>0; j--)
    {
        DSPORT = 0; //先将总线拉低 1us
        i++;
        DSPORT = 1; //然后释放总线
        i++;
        i++; //延时 6us 等待数据稳定
        bi = DSPORT; //读取数据, 从最低位开始读取
        /*将 byte 左移一位, 然后与上右移 7 位后的 bi, 注意移动之后移掉
        那位补 0.*/
        byte = (byte >> 1) | (bi << 7);
        i = 4; //读取完之后等待 48us 再接着读取下一个数
        while(i--);
    }
    return byte;
}

/*****
*****
* 函 数 名      : Ds18b20ChangTemp
* 函数功能      : 让 18b20 开始转换温度
* 输    入      : 无
* 输    出      : 无
*****
*****/

void Ds18b20ChangTemp()

```

```

{
    Ds18b20Init();
    Delay1ms(1);
    Ds18b20WriteByte(0xcc);    //跳过 ROM 操作命令
    Ds18b20WriteByte(0x44);    //温度转换命令
    //Delay1ms(100); //等待转换成功，而如果你是一直刷着的话，就不用这个延时了

}
/*****
*****
* 函 数 名      : Ds18b20ReadTempCom
* 函数功能      : 发送读取温度命令
* 输    入      : 无
* 输    出      : 无
*****
*****/

void Ds18b20ReadTempCom()
{
    Ds18b20Init();
    Delay1ms(1);
    Ds18b20WriteByte(0xcc);    //跳过 ROM 操作命令
    Ds18b20WriteByte(0xbe);    //发送读取温度命令
}
/*****
*****
* 函 数 名      : Ds18b20ReadTemp
* 函数功能      : 读取温度
* 输    入      : 无
* 输    出      : 无
*****
*****/

int Ds18b20ReadTemp()
{
    int temp = 0;
    uchar tmh, tml;
    Ds18b20ChangTemp();        //先写入转换命令
    Ds18b20ReadTempCom();      //然后等待转换完后发送读取温度命令
    tml = Ds18b20ReadByte();    //读取温度值共 16 位，先读低字节
    tmh = Ds18b20ReadByte();    //再读高字节
}

```

```
temp = tmh;  
temp <<= 8;  
temp |= tml;  
return temp;  
}
```

各函数功能在代码内有详细的注释，大家可以参考注释理解。在应用中我们主要调用 Ds18b20ReadTemp() 函数来获取检测的温度值，此时获取的仅仅是温度对应的 2 个字节，还需乘以精度 0.0625 才是实际的温度，这个操作在后面 main.c 文件内会有。

程序中我们看到有一个 DSPORT，该变量就是我们在 temp.h 头文件内定义的传感器接口，代码如下：

```
#ifndef __TEMP_H_  
#define __TEMP_H_  
  
#include<reg52.h>  
//---重定义关键词---//  
#ifndef uchar  
#define uchar unsigned char  
#endif  
  
#ifndef uint  
#define uint unsigned int  
#endif  
  
//---定义使用的 I/O 口---//  
sbit DSPORT=P3^7;  
  
//---声明全局函数---//  
void Delay1ms(uint );  
uchar Ds18b20Init();  
void Ds18b20WriteByte(uchar com);  
uchar Ds18b20ReadByte();  
void Ds18b20ChangTemp();  
void Ds18b20ReadTempCom();  
int Ds18b20ReadTemp();  
  
#endif
```

同样，头文件内通常都是用来定义管脚和声明函数或者变量的。下面来看下 main.c 文件代码，main.c 内代码如下：

```
/******
```

*

实验现象：下载程序后，在温度传感器接口处，按照丝印方向插好温度传感器，数码管就会显示

检测的温度值

接线说明：（具体接线图可见开发攻略对应实验的“实验现象”章节）

注意事项：

*/

#include "reg52.h" //此文件中定义了单片机的一些特殊功能寄存器

#include "temp.h"

typedef unsigned int u16; //对数据类型进行声明定义

typedef unsigned char u8;

sbit LSA=P2^2;

sbit LSB=P2^3;

sbit LSC=P2^4;

char num=0;

u8 DisplayData[8];

u8

code

smgduan[10]={0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07, 0x7f, 0x6f};

/******

* 函数名 : delay

* 函数功能 : 延时函数，i=1 时，大约延时 10us

*****/

void delay(u16 i)

{

while(i--);

```

}

/*****
*****
* 函数名      : datapros()
* 函数功能    : 温度读取处理转换函数
* 输入      : temp
* 输出      : 无
*****
*****/

void datapros(int temp)
{
    float tp;
    if(temp< 0)                //当温度值为负数
    {
        DisplayData[0] = 0x40;    // -
        //因为读取的温度是实际温度的补码，所以减 1，再取反求出原码
        temp=temp-1;
        temp=~temp;
        tp=temp;
        temp=tp*0.0625*100+0.5;
        //留两个小数点就*100，+0.5 是四舍五入，因为 C 语言浮点数转换为整型的时候把小数点
        //后面的数自动去掉，不管是否大于 0.5，而+0.5 之后大于 0.5 的就是进 1 了，小于 0.5 的就
        //算加上 0.5，还是在小数点后面。

    }
    else
    {
        DisplayData[0] = 0x00;
        tp=temp; //因为数据处理有小数点所以将温度赋给一个浮点型变量
        //如果温度是正的那么，那么正数的原码就是补码它本身
        temp=tp*0.0625*100+0.5;
        //留两个小数点就*100，+0.5 是四舍五入，因为 C 语言浮点数转换为整型的时候把小数点
        //后面的数自动去掉，不管是否大于 0.5，而+0.5 之后大于 0.5 的就是进 1 了，小于 0.5 的就
        //算加上 0.5，还是在小数点后面。

    }
    DisplayData[1] = smgduan[temp % 10000 / 1000];
    DisplayData[2] = smgduan[temp % 1000 / 100];
}

```

```

DisplayData[3] = smgduan[temp % 100 / 10];
DisplayData[4] = smgduan[temp % 10 / 1];

}

/*****
*****
* 函数名      :DigDisplay()
* 函数功能    :数码管显示函数
* 输入        : 无
* 输出        : 无
*****
*****/
void DigDisplay()
{
    u8 i;
    for(i=0;i<6;i++)
    {
        switch(i) //位选, 选择点亮的数码管,
        {
            case(0):
                LSA=1;LSB=1;LSC=1; break;//显示第 0 位
            case(1):
                LSA=0;LSB=1;LSC=1; break;//显示第 1 位
            case(2):
                LSA=1;LSB=0;LSC=1; break;//显示第 2 位
            case(3):
                LSA=0;LSB=0;LSC=1; break;//显示第 3 位
            case(4):
                LSA=1;LSB=1;LSC=0; break;//显示第 4 位
            case(5):
                LSA=0;LSB=1;LSC=0; break;//显示第 5 位
        }
        P0=DisplayData[i];//发送数据
        delay(100); //间隔一段时间扫描
        P0=0x00;//消隐
    }
}

/*****
*****
* 函数名      : main
* 函数功能    : 主函数
*****
*****/

```

```

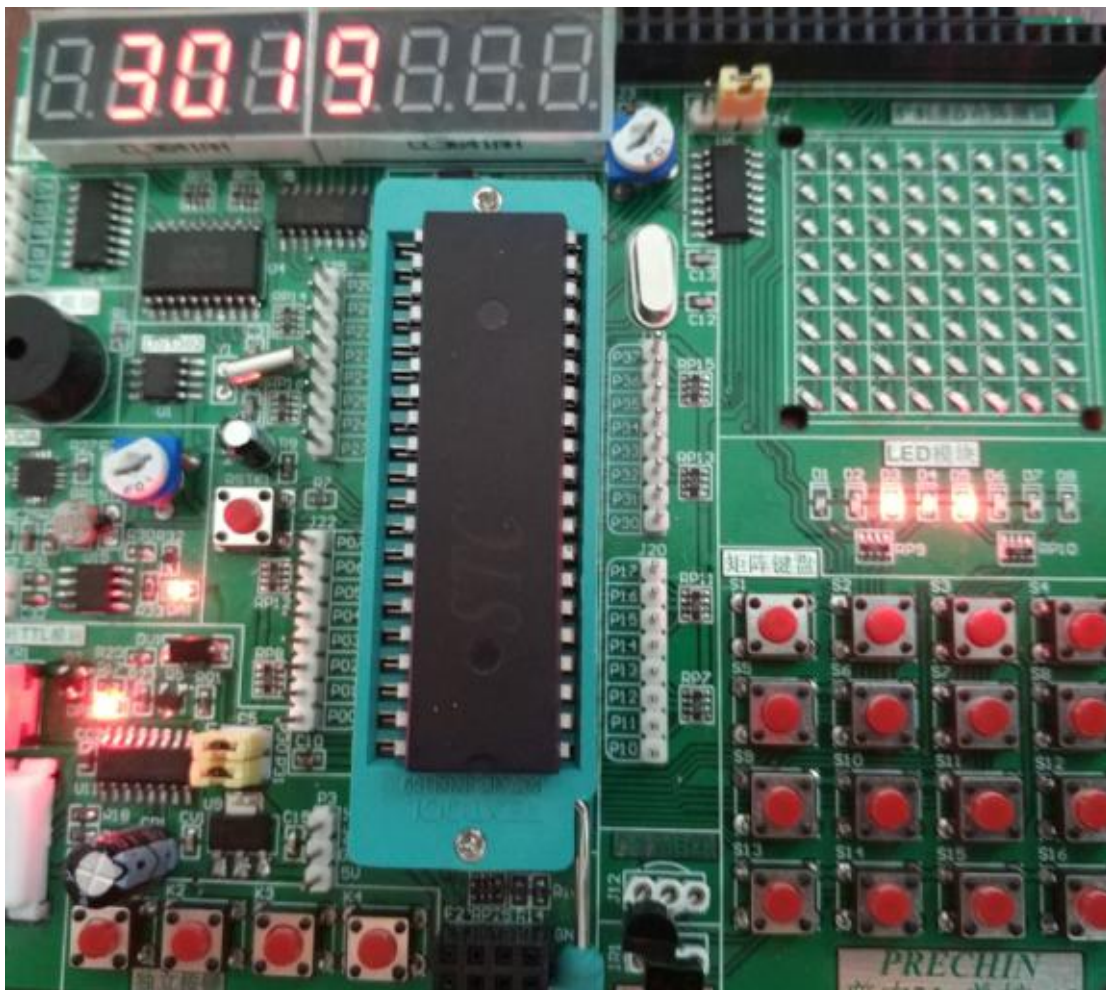
* 输    入    : 无
* 输    出    : 无
*****
*****/
void main()
{
    while(1)
    {
        datapros(Ds18b20ReadTemp());    //数据处理函数
        DigDisplay(); //数码管显示函数
    }
}

```

main.c 文件内代码非常少也很简单，首先将 51 单片机的头文件和 temp.h 的头文件包含进来，然后定义动态数码管、74HC138 译码器的控制管脚。主函数功能非常简单，首先在 while 循环内调用 datapros(Ds18b20ReadTemp()) 温度数据处理函数，将获取的温度数据转换为实际温度值，可以看到 datapros 函数的实际参数就是调用 Ds18b20ReadTemp() 函数来传递温度数据，判断传递进来的温度数据是否为负值进行数据处理和负号显示，最终将温度数据转换为实际温度值，并将温度值放大 100 倍（实际就是保留温度值的后两位小数）后取该温度值的各个位对应的数码管段码数据，将其保存在数组 DisplayData 中。最后调用 DigDisplay() 动态数码管显示函数，将处理好的数据显示。

19.4 实验现象

使用 USB 线将开发板和电脑连接成功后，把编写好的程序编译后将编译产生的 .hex 文件烧入到芯片内，按照如下图所示的接线方式可以看到：插上 DS18B20 温度传感器，数码管显示检测的温度值。



实验说明：DS18B20 默认起始温度是 85 度，之后就会进入到正常温度检测，如果条件允许可以在零下温度下检测看看。

课后作业

(1) 利用 DS18B20 设计一个智能温度控制系统，具有温度上下限值设定，当温度高于上限值电机开启同时蜂鸣器报警，当温度低于下限值时继电器工作加热，同时蜂鸣器报警，当温度处于上下限间正常工作。（温馨提示：把本章实验结合按键控制实验）

第 20 章 DS1302 时钟实验

在前面章节，我们介绍了如何使用单片机 I/O 口模拟 IIC 总线、单总线时序。这一章我们来学习 DS1302 时钟芯片，该芯片是 3 线 SPI 接口，所以需要使用 51 单片机的 3 个 I/O 口模拟 SPI 时序与 DS1302 时钟芯片通信，将时钟日历数据读取出来。开发板上集成了 1 个 DS1302 时钟模块，可使用它设计一个多功能电子时钟。本章要实现的功能是：系统运行时，数码管上显示电子时钟，格式为“XX-XX-XX”。学习本章可以参考前面的实验章节内容。若结合视频学习效果更佳。本章分为如下几部分内容：

20.1 DS1302 时钟芯片介绍

20.2 硬件设计

20.3 软件设计

20.4 实验现象

20.1 DS1302 时钟芯片介绍

20.1.1 DS1302 简介

DS1302 是 DALLAS 公司推出的涓流充电时钟芯片，内含有一个实时时钟/日历和 31 字节静态 RAM，通过简单的串行接口与单片机进行通信。实时时钟/日历电路提供秒、分、时、日、周、月、年的信息，每月的天数和闰年的天数可自动调整。时钟操作可通过 AM/PM 指示决定采用 24 或 12 小时格式。DS1302 与单片机之间能简单地采用同步串行的方式进行通信，仅需用到三根通信线：①RES 复位②I/O 数据线③SCLK 串行时钟。时钟/RAM 的读/写数据以一个字节或多达 31 个字节的字符组方式通信。DS1302 工作时功耗很低保持数据和时钟信息时功率小于 1mW。

DS1302 由 DS1202 改进而来增加了以下的特性：双电源管脚用于主电源和备份电源供应，Vcc1 为可编程涓流充电电源，附加七个字节存储器。它广泛应用于电话、传真、便携式仪器以及电池供电的仪器仪表等产品领域下面。

主要的性能指标：

★ 实时时钟具有能计算 2100 年之前的秒、分、时、日、星期、月、年的能力，还有闰年调整的能力；

★ 31 个 8 位暂存数据存储 RAM；

★ 串行 I/O 口方式使得管脚数量最少；

★ 宽范围工作电压 2.0~5.5V；

★ 工作在 2.0V 时，电流小于 300nA；

★ 读/写时钟或 RAM 数据时有两种传送方式单字节传送和多字节传送字符组方式；

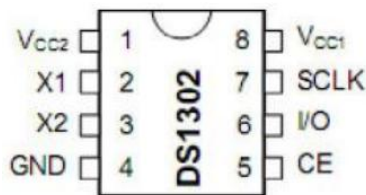
★ 8 脚 DIP 封装或可选的 8 脚 SOIC 封装根据表面装配；

★ 简单 3 线接口；

★ 与 TTL 兼容 Vcc=5V；

★ 可选工业级温度范围-40~+85；

下面我们来看下 DS1302 芯片的管脚及功能。



- 1, VCC2: 主用电源引脚
- 2, X1、X2: DS1302 外部晶振引脚，通常需外接 32.768K 晶振
- 3, GND: 电源地
- 4, CE: 使能引脚，也是复位引脚（新版本功能变）。
- 5, I/O: 串行数据引脚，数据输出或者输入都从这个引脚
- 6, SCLK: 串行时钟引脚
- 7, VCC1: 备用电源

20.1.2 DS1302 使用

操作 DS1302 的大致过程，就是将各种数据写入 DS1302 的寄存器，以设置它当前的时间的格式。然后使 DS1302 开始运作，DS1302 时钟会按照设置情况运转，再用单片机将其寄存器内的数据读出。再用液晶显示，就是我们常说的简易电子钟。所以总的来说 DS1302 的操作分 2 步（显示部分属于液晶显示的内容，不属于 DS1302 本身的内容），但是在讲述操作时序之前，我们要先看看寄存器，DS1302 有一个控制寄存器、12 个日历、时钟寄存器和 31 个 RAM。

（1）控制寄存器

控制寄存器用于存放 DS1302 的控制命令字，DS1302 的 RST 引脚回到高电平后写入的第一个字节就为控制命令。它用于对 DS1302 读写过程进行控制，格式如下：

7	6	5	4	3	2	1	0
1	RAM	A4	A3	A2	A1	A0	RD
	CK						WR

上图是 DS1302 的寄存器样式，我们看到：

- 1、第 7 位永远都是 1；
- 2、第 6 位，1 表示 RAM，寻址内部存储器地址；0 表示 CK，寻址内部寄存器；

3、第 5 到第 1 位，为 RAM 或者寄存器的地址；

4、最低位，高电平表示 RD，即下一步操作将要“读”；低电平表示 W，即下一步操作将要“写”。（与 AT24C02 寄存器类似，这点要理解好）。

寄存器名称	D7	D6	D5	D4	D3	D2	D1	D0
	1	RAM/CK	A4	A3	A2	A1	A0	R/W
秒寄存器	1	0	0	0	0	0	0	0或1
分寄存器	1	0	0	0	0	0	1	0或1
小时寄存器	1	0	0	0	0	1	0	0或1
日寄存器	1	0	0	0	0	1	1	0或1
月寄存器	1	0	0	0	1	0	0	0或1
星期寄存器	1	0	0	0	1	0	1	0或1
年寄存器	1	0	0	0	1	1	0	0或1

寄存器名称	D7	D6	D5	D4	D3	D2	D1	D0
	1	RAM/CK	A4	A3	A2	A1	A0	R/W
写保护寄存器	1	0	0	0	1	1	1	0或1
慢充电寄存器	1	0	0	1	0	0	0	0或1
时钟突发模式	1	0	1	1	1	1	1	0或1
RAM0	1	1	0	0	0	0	0	0或1
...	1	1	...	...	...	...	...	0或1
RAM30	1	1	1	1	1	1	0	0或1
RAM突发模式	1	1	1	1	1	1	1	0或1

比如要读秒寄存器则命令为 10000001，反之写为 1000 0000，要注意其含义。

（2）日历/时钟寄存器

DS1302 共有 12 个寄存器，其中有 7 个与日历、时钟相关，存放的数据为 BCD 码形式。格式如下：

寄存器名称	取值范围	D7	D6	D5	D4	D3	D2	D1	D0
秒寄存器	00~59	CH	秒的十位			秒的个位			
分寄存器	00~59	0	分的十位			分的个位			
小时寄存器	01~12或 00~23	12/2 4	0	A/ P	HR	小时的个位			
日寄存器	01~31	0	0	日的十位			日的个位		

寄存器名称	取值范围	D7	D6	D5	D4	D3	D2	D1	D0
月寄存器	01~12	0	0	0	1或0	月的个位			
星期寄存器	01~07	0	0	0	0	星期几			
年寄存器	01~99	年的十位				年的个位			
写保护寄存器		WP	0	0	0	0	0	0	0
慢充电寄存器		TCS	TCS	TCS	TCS	DS	DS	RS	RS
时钟突发寄存器									

下面对几个寄存器做下说明：

秒寄存器：低四位为 SEC，高的次三位为 10SEC。最高位 CH 为 DS1302 的运行标志，当 CH=0 时，DS1302 内部时钟运行，反之 CH=1 时停止；

小时寄存器：时寄存器。最高位为 12/24 小时的格式选择位，该位为 1 时表示 12 小时格式。当设置为 12 小时显示格式时，第 5 位的高电平表示下午（PM）；而当设置为 24 小时格式时，第 5 位具体的时间数据。

写保护寄存器：当该寄存器最高位 WP 为 1 时，DS1302 只读不写，所以要在往 DS1302 写数据之前确保 WP 为 0。

慢充电寄存器（涓细电流充电）寄存器：我们知道，当 DS1302 掉电时，可以马上调用外部电源保护时间数据。该寄存器就是配置备用电源的充电选项的。其中高四位（4 个 TCS）只有在 1010 的情况下才能使用充电选项；低四位的情况与 DS1302 内部电路有关，具体大家可以查看“\6--开发板芯片资料\DS1302 中文手册.pdf”。

前面我们提到在日历/时钟寄存器中都是以 BCD 码存放数据，那么 BCD 码是什么呢？BCD 码是通过 4 位二进制码来表示 1 位十进制中的 0~9 这 10 个数码。如下所示：

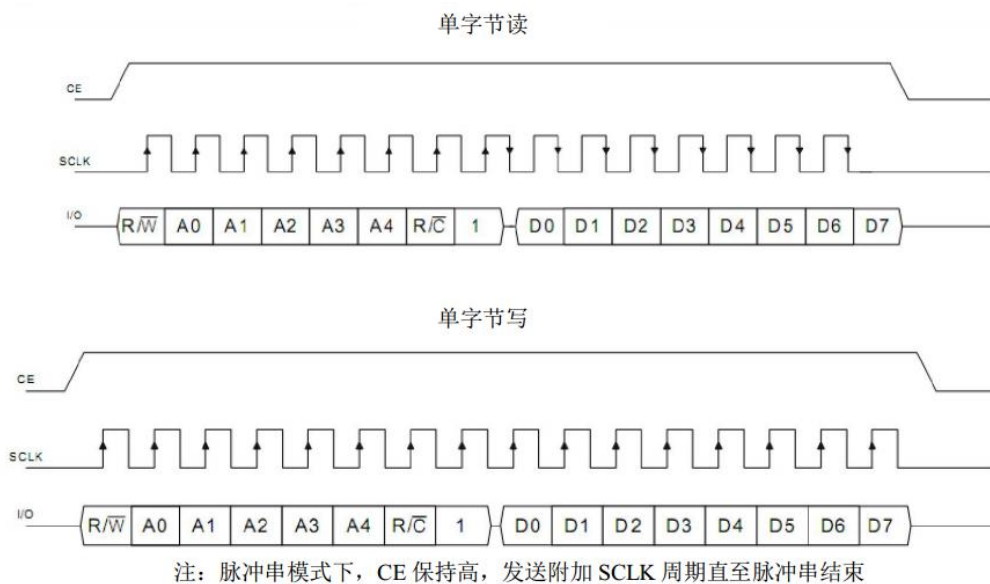
0	0000	5	0101
1	0001	6	0110
2	0010	7	0111
3	0011	8	1000
4	0100	9	1001

所以从 DS1302 中读取出来的时钟数据均为 BCD 码格式，需转换为我们习惯

的 10 进制，转换方法在源程序里，后面我们会介绍。

(3) DS1302 的读写时序

在控制指令字输入后的下一个 SCLK 时钟的上升沿时，数据被写入 DS1302，数据输入从低位（位 0）开始。同样，在紧跟 8 位的控制指令字后的下一个 SCLK 脉冲的下降沿读出 DS1302 的数据，读出数据时从低位 0 位到高位 7。其时序图如下所示：



上图就是 DS1302 的三个时序：复位时序，单字节写时序，单字节读时序；

CE (RST)：复位时序，即在 RST 引脚产生一个正脉冲，在整个读写器件，RST 要保持高电平，一次字节读写完毕之后，要注意把 RST 返回低电平准备下次读写周期；

单字节读时序：注意读之前还是要先对寄存器写命令，从最低位开始写；可以看到，写数据是在 SCLK 的上升沿实现，而读数据在 SCLK 的下降沿实现。所以，在单字节读时序中，写命令的第八个上升沿结束后紧接着的第八个下降沿就将要读寄存器的第一位数据读到数据线上了！这个就是 DS1302 操作中最特别的地方。当然读出来的数据也是最低位开始。

单字节写时序：两个字节的的数据配合 16 个上升沿将数据写入即可。

程序注意事项：

- ★要记得在操作 DS1302 之前关闭写保护；
- ★注意用延时来降低单片机的速度以配合器件时序；

★DS1302 读出来的数据是 BCD 码形式，要转换成我们习惯的 10 进制，转换方法在源程序里；

★读取字节之前，将 I0 设置为输入口，读取完之后，要将其改回输出口；

★在写程序的时候，建议实现开辟数组（内存空间）来集中放置 DS1302 的一系列数据，方便以后扩展键盘输入。

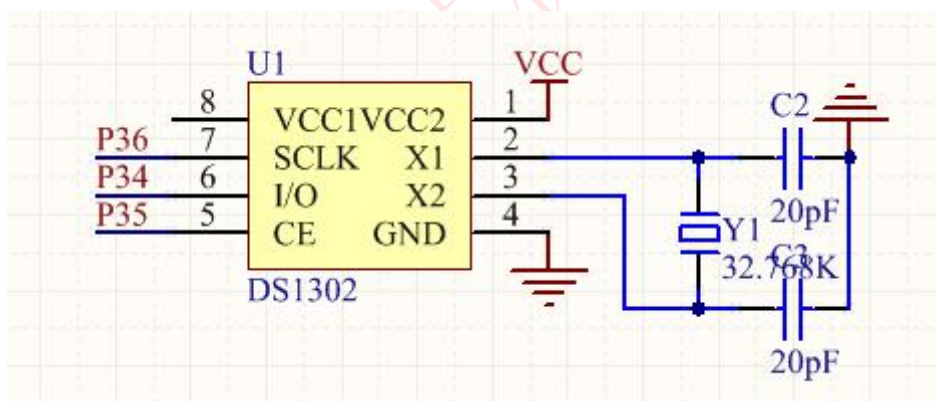
到这里我们就介绍完了 DS1302 时钟芯片，如需更详细的介绍，请大家参考“\6--开发板芯片资料\DS1302 中文手册.pdf”。

20.2 硬件设计

本实验使用到硬件资源如下：

- (1) 动态数码管
- (2) DS1302

动态数码管电路在前面章节都介绍过，这里就不再重复。下面我们来看下开发板上 DS1302 时钟模块电路，如下图所示：



从上图中可以看出，该电路是集成的，DS1302 芯片的控制管脚接至单片机 P3⁴~P3⁶ 上，在芯片的 X1、X2 管脚处外接了一个 32.768KHZ 晶振，为时钟运行提供一个稳定的时钟频率。

由于该模块电路是集成的，所以 DS1302 芯片的控制管脚通过单片机管脚控制即可。

20.3 软件设计

本章所要实现的功能是：系统运行时，数码管上显示电子时钟，格式为“XX-XX-XX”。

我们直接复制上一章创建的工程，在此模板基础上进行程序开发。为了能够与开发攻略教程对应，将复制过来的模板文件夹重新命名为“实验 25：DS1302 时钟”。打开工程文件夹，发现里面多了 2 个文件：ds1302.c 源文件和 ds1302.h 头文件，该文件保存的是 DS1302 时钟的驱动程序。至于如何创建多文件工程，在前面章节已经介绍过，这里就不再重复。

(1) 首先我们看下 ds1302.c 文件代码，如下：

```
#include "ds1302.h"

//---DS1302 写入和读取时分秒的地址命令---//
//---秒分时日月周年 最低位读写位;-----//
uchar code READ_RTC_ADDR[7] = {0x81, 0x83, 0x85, 0x87, 0x89, 0x8b, 0x8d};
uchar code WRITE_RTC_ADDR[7] = {0x80, 0x82, 0x84, 0x86, 0x88, 0x8a, 0x8c};

//---DS1302 时钟初始化 2016 年 5 月 7 日星期六 12 点 00 分 00 秒。---//
//---存储顺序是秒分时日月周年, 存储格式是用 BCD 码---//
uchar TIME[7] = {0, 0, 0x12, 0x07, 0x05, 0x06, 0x16};

/*****
*****
* 函 数 名      : Ds1302Write
* 函数功能      : 向 DS1302 命令 (地址+数据)
* 输    入      : addr, dat
* 输    出      : 无
*****
*****/

void Ds1302Write(uchar addr, uchar dat)
{
    uchar n;
    RST = 0;
    _nop_();

    SCLK = 0; //先将 SCLK 置低电平。
```

```

    _nop_();
    RST = 1; //然后将 RST(CE)置高电平。
    _nop_();

    for (n=0; n<8; n++)//开始传送八位地址命令
    {
        DSIO = addr & 0x01;//数据从低位开始传送
        addr >>= 1;
        SCLK = 1;//数据在上升沿时，DS1302 读取数据
        _nop_();
        SCLK = 0;
        _nop_();
    }
    for (n=0; n<8; n++)//写入 8 位数据
    {
        DSIO = dat & 0x01;
        dat >>= 1;
        SCLK = 1;//数据在上升沿时，DS1302 读取数据
        _nop_();
        SCLK = 0;
        _nop_();
    }

    RST = 0;//传送数据结束
    _nop_();
}

/*****
*****
* 函 数 名      : Ds1302Read
* 函数功能      : 读取一个地址的数据
* 输    入      : addr
* 输    出      : dat
*****
*****/

uchar Ds1302Read(uchar addr)
{
    uchar n, dat, dat1;
    RST = 0;
    _nop_();

    SCLK = 0;//先将 SCLK 置低电平。
    _nop_();

```

```

RST = 1; //然后将 RST (CE) 置高电平。
_nop_();

for(n=0; n<8; n++) //开始传送八位地址命令
{
    DSIO = addr & 0x01; //数据从低位开始传送
    addr >>= 1;
    SCLK = 1; //数据在上升沿时, DS1302 读取数据
    _nop_();
    SCLK = 0; //DS1302 下降沿时, 放置数据
    _nop_();
}
_nop_();
for(n=0; n<8; n++) //读取 8 位数据
{
    dat1 = DSIO; //从最低位开始接收
    dat = (dat>>1) | (dat1<<7);
    SCLK = 1;
    _nop_();
    SCLK = 0; //DS1302 下降沿时, 放置数据
    _nop_();
}

RST = 0;
_nop_(); //以下为 DS1302 复位的稳定时间, 必须的。
SCLK = 1;
_nop_();
DSIO = 0;
_nop_();
DSIO = 1;
_nop_();
return dat;
}

/*****
*****
* 函 数 名      : Ds1302Init
* 函数功能      : 初始化 DS1302.
* 输    入      : 无
* 输    出      : 无
*****
*****/

void Ds1302Init()

```

```

{
    uchar n;
    Ds1302Write(0x8E, 0X00);    //禁止写保护，就是关闭写保护功能
    for (n=0; n<7; n++)//写入 7 个字节的时钟信号：分秒时日月周年
    {
        Ds1302Write(WRITE_RTC_ADDR[n], TIME[n]);
    }
    Ds1302Write(0x8E, 0x80);    //打开写保护功能
}

/*****
*****
* 函 数 名      : Ds1302ReadTime
* 函数功能      : 读取时钟信息
* 输    入      : 无
* 输    出      : 无
*****
*****/

void Ds1302ReadTime()
{
    uchar n;
    for (n=0; n<7; n++)//读取 7 个字节的时钟信号：分秒时日月周年
    {
        TIME[n] = Ds1302Read(READ_RTC_ADDR[n]);
    }
}

```

各函数功能在代码内有详细的注释，大家可以参考注释理解。程序开头处我们看到定义了 3 个数组：READ_RTC_ADDR、WRITE_RTC_ADDR 和 TIME。READ_RTC_ADDR 和 WRITE_RTC_ADDR 数组内存储的是 DS1302 写入和读取时分秒的地址命令，这个可以对照前面介绍的寄存器看。TIME 数组是用来存储我们初始化时间的，每个数据对应含义在代码内已做注释。在应用中我们主要调用 Ds1302Init() 函数和 Ds1302ReadTime() 函数来初始化 DS1302 时钟数据以及读取 DS1302 时钟数据。在初始化时钟数据时，首先要禁止写保护即关闭写保护功能，然后将所需设置的时钟数据写入到对应的时钟寄存器地址内，最后打开写保护功能，以防止意外修改 DS1302 内部寄存器。对于读取时钟数据其实很简单，只要从对应的时钟寄存器地址内读取数据即可，然后将读取的数据存储到一个缓存数组中，方便数据的处理与显示，这个操作在后面 main.c 文件内会有。

```
#ifndef __DS1302_H_
#define __DS1302_H_

//---包含头文件---//
#include<reg52.h>
#include<intrins.h>

//---重定义关键词---//
#ifndef uchar
#define uchar unsigned char
#endif

#ifndef uint
#define uint unsigned int
#endif

//---定义 ds1302 使用的 IO 口---//
sbit DSI0=P3^4;
sbit RST=P3^5;
sbit SCLK=P3^6;

//---定义全局函数---//
void Ds1302Write(uchar addr, uchar dat);
uchar Ds1302Read(uchar addr);
void Ds1302Init();
void Ds1302ReadTime();

//---加入全局变量---//
extern uchar TIME[7];    //加入全局变量

#endif
```

```

/*****
*****/

```

接线说明：（具体接线图可见开发攻略对应实验的“实验现象”章节）
 注意事项：

```

*****
*****

*/

#include "reg52.h"          //此文件中定义了单片机的一些特殊功能
寄存器
#include "ds1302.h"

typedef unsigned int u16;    //对数据类型进行声明定义
typedef unsigned char u8;

sbit LSA=P2^2;
sbit LSB=P2^3;
sbit LSC=P2^4;

char num=0;
u8 DisplayData[8];
u8                                     code
smgduan[10]={0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07, 0x7f, 0x6f};

/*****
*****
* 函 数 名      : delay
* 函数功能      : 延时函数, i=1 时, 大约延时 10us
*****
*****/
void delay(u16 i)
{
    while(i--);
}

/*****
*****
* 函 数 名      : datapros()
* 函数功能      : 时间读取处理转换函数
* 输    入      : 无
* 输    出      : 无
*****
*****/

```



```

void datapros()
{
    Ds1302ReadTime();
    DisplayData[0] = smgduan[TIME[2]/16];           //时
    DisplayData[1] = smgduan[TIME[2]&0x0f];
    DisplayData[2] = 0x40;
    DisplayData[3] = smgduan[TIME[1]/16];           //分
    DisplayData[4] = smgduan[TIME[1]&0x0f];
    DisplayData[5] = 0x40;
    DisplayData[6] = smgduan[TIME[0]/16];           //秒
    DisplayData[7] = smgduan[TIME[0]&0x0f];
}

/*****
*****
* 函数名          :DigDisplay()
* 函数功能        :数码管显示函数
* 输入            : 无
* 输出            : 无
*****
*****/
void DigDisplay()
{
    u8 i;
    for(i=0;i<8;i++)
    {
        switch(i)    //位选，选择点亮的数码管，
        {
            case(0):
                LSA=1;LSB=1;LSC=1; break;//显示第 0 位
            case(1):
                LSA=0;LSB=1;LSC=1; break;//显示第 1 位
            case(2):
                LSA=1;LSB=0;LSC=1; break;//显示第 2 位
            case(3):
                LSA=0;LSB=0;LSC=1; break;//显示第 3 位
            case(4):
                LSA=1;LSB=1;LSC=0; break;//显示第 4 位
            case(5):
                LSA=0;LSB=1;LSC=0; break;//显示第 5 位
            case(6):
                LSA=1;LSB=0;LSC=0; break;//显示第 6 位

```

```

        case(7):
            LSA=0;LSB=0;LSC=0; break;//显示第 7 位
    }
    P0=DisplayData[i];//发送数据
    delay(100); //间隔一段时间扫描
    P0=0x00;//消隐
}
}

/*****
*****
* 函 数 名      : main
* 函数功能      : 主函数
* 输    入      : 无
* 输    出      : 无
*****
*****/
void main()
{
    //Ds1302Init();      //第一次初始化后就可以注释该条语句，这样下次重启就不会再次初始化了
    while(1)
    {
        datapros();      //数据处理函数
        DigDisplay();    //数码管显示函数
    }
}

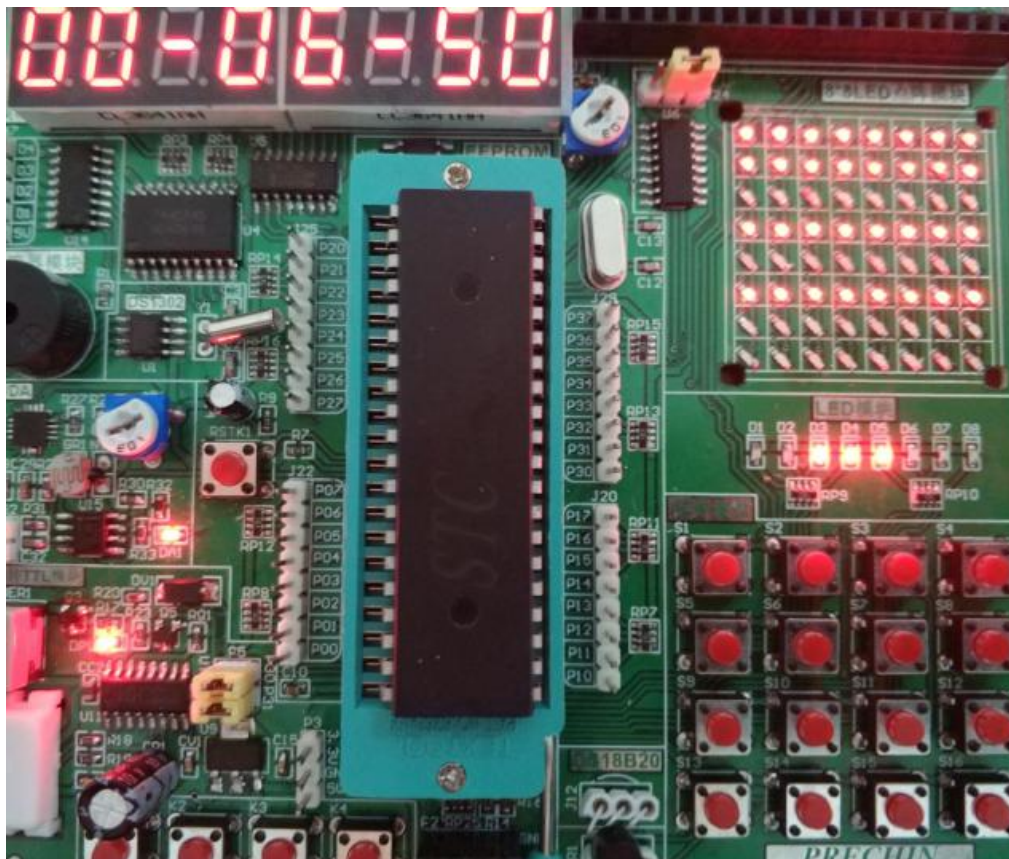
```

main.c 文件内代码非常少也很简单，首先将 51 单片机的头文件和 ds1302.h 的头文件包含进来，然后定义动态数码管、74HC138 译码器的控制管脚。主函数功能非常简单，首先调用 Ds1302Init() 函数初始化 DS1302 时钟数据，然后在 while 循环内调用 datapros() 时钟数据处理函数，该函数内部又调用 Ds1302ReadTime() 函数来读取时钟数据，前面说过读取出来的数据是 BCD 码，需要将其处理转换为我们常用的十进制数，怎么转换？将时钟数据对应的除以 16 即可，然后在取出各位对应的数码管段码数据保存到数组 DisplayData 中。最后调用 DigDisplay() 动态数码管显示函数，将处理好的数据显示。

20.4 实验现象

使用 USB 线将开发板和电脑连接成功后，把编写好的程序编译后将编译产生

的.hex 文件烧入到芯片内，按照如下图所示的接线方式可以看到：数码管上显示电子时钟，格式为：时-分-秒。



课后作业

(1) 利用 AT24C02 掉电不丢失功能，存储数据用来辨别 DS1302 时钟初始化是否已经初始化，如果初始化就不执行 Ds1302Init() 函数。

(温馨提示：写入一个数据到 AT24C02 内，然后读取判断是否写入的数据，如果是就不运行 Ds1302Init() 函数，否则运行)

(2) 设计一个多功能电子时钟：需具备时间可调、闹钟等功能。

(温馨提示：把本章实验与独立按键实验、蜂鸣器实验结合)

第 21 章 红外通信实验

在前面章节，我们介绍的通信，如 SPI、IIC 等通信均为有线，这一章我们来介绍一种无线通信技术——红外遥控通信。我们开发板标配了一个一体化红外接收头和红外遥控器，我们来学习如何使用 51 单片机解码红外遥控器的信号。本章我们使用 51 单片机的外部中断功能来解码红外遥控器的编码信号。本章要实现的功能是：使用外部中断功能将遥控器键值编码数据解码后通过数码管显示。学习本章可以参考前面的实验章节内容。若结合视频学习效果更佳。本章分为如下几部分内容：

21.1 红外遥控介绍

21.2 硬件设计

21.3 软件设计

21.4 实验现象

21.1 红外遥控介绍

21.1.1 红外线简介

人的眼睛能看到的可见光按波长从长到短排列,依次为红、橙、黄、绿、青、蓝、紫。其中红光的波长范围为 $0.62\sim 0.76\mu\text{m}$; 紫光的波长范围为 $0.38\sim 0.46\mu\text{m}$ 。比紫光波长还短的光叫紫外线,比红光波长还长的光叫红外线。红外线遥控就是利用波长为 $0.76\sim 1.5\mu\text{m}$ 之间的近红外线来传送控制信号的。

21.1.2 红外遥控的原理

红外遥控是一种无线、非接触控制技术,具有抗干扰能力强,信息传输可靠,功耗低,成本低,易实现等显著优点,被诸多电子设备特别是家用电器广泛采用,并越来越多的应用到计算机系统中。

由于红外线遥控不具有像无线电遥控那样穿过障碍物去控制被控对象的能力,所以,在设计红外线遥控器时,不必要像无线电遥控器那样,每套(发射器和接收器)要有不同的遥控频率或编码(否则,就会隔墙控制或干扰邻居的家用电器),所以同类产品的红外线遥控器,可以有相同的遥控频率或编码,而不会出现遥控信号“串门”的情况。这对于大批量生产以及在家用电器上普及红外线遥控提供了极大的方便。由于红外线为不可见光,因此对环境影响很小,再由红外光波动波长远小于无线电波的波长,所以红外线遥控不会影响其他家用电器,也不会影响临近的无线电设备。

红外遥控通信系统一般由红外发射装置和红外接收设备两大部分组成。

(1) 红外发射装置

红外发射装置,也就是通常我们说的红外遥控器是由键盘电路、红外编码电路、电源电路和红外发射电路组成。红外发射电路的主要元件为红外发光二极管。它实际上是一只特殊的发光二极管;由于其内部材料不同于普通发光二极管,因而在其两端施加一定电压时,它便发出的是红外线而不是可见光。目前大量的使用的红外发光二极管发出的红外线波长为 940nm 左右,外形与普通发光二极管相同。红外发光二极管有透明的,还有不透明的,在我们的红外遥控器上可以看

到这个红外发光二极管。红外遥控器和红外发光二极管如下图所示：

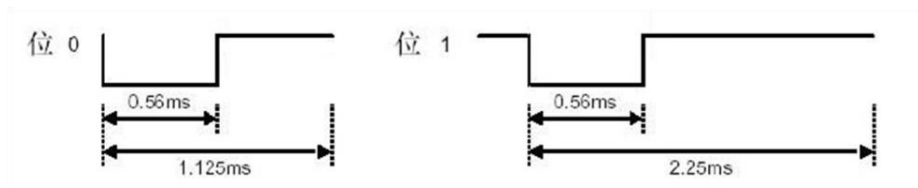


通常红外遥控为了提高抗干扰性能和降低电源消耗，红外遥控器常用载波的方式传送二进制编码，常用的载波频率为 38kHz，这是由发射端所使用的 455kHz 晶振来决定的。在发射端要对晶振进行整数分频，分频系数一般取 12，所以 $455\text{kHz} \div 12 \approx 37.9\text{kHz} \approx 38\text{kHz}$ 。也有一些遥控系统采用 36kHz、40 kHz、56 kHz 等，一般由发射端晶振的振荡频率来决定。所以，通常的红外遥控器是将遥控信号（二进制脉冲码）调制在 38KHz 的载波上，经缓冲放大后送至红外发光二极管，转化为红外信号发射出去的。

二进制脉冲码的形式有多种，其中最为常用的是 NEC Protocol 的 PWM 码（脉冲宽度调制）和 Philips RC-5 Protocol 的 PPM 码（脉冲位置调制码，脉冲串之间的时间间隔来实现信号调制）。如果要开发红外接收设备，一定要知道红外遥控器的编码方式和载波频率，我们才可以选取一体化红外接收头和制定解码方案。我们配套的红外遥控器使用的是 NEC 协议，其特征如下：

- 1、8 位地址和 8 位指令长度；
- 2、地址和命令 2 次传输（确保可靠性）
- 3、PWM 脉冲位置调制，以发射红外载波的占空比代表“0”和“1”；
- 4、载波频率为 38Khz；
- 5、位时间为 1.125ms 或 2.25ms；

NEC 码的位定义：一个脉冲对应 560us 的连续载波，一个逻辑 1 传输需要 2.25ms(560us 脉冲+1680us 低电平)，一个逻辑 0 的传输需要 1.125ms（560us 脉冲+560us 低电平）。而红外接收头在收到脉冲的时候为低电平，在没有脉冲的时候为高电平，这样，我们在接收头端收到的信号为：逻辑 1 应该是 560us 低+1680us 高，逻辑 0 应该是 560us 低+560us 高。所以可以通过计算高电平时间判断接收到的数据是 0 还是 1。NEC 码位定义时序图如下图所示：



NEC 遥控指令的数据格式为：引导码、地址码、地址反码、控制码、控制反码。引导码由一个 9ms 的低电平和一个 4.5ms 的高电平组成，地址码、地址反码、控制码、控制反码均是 8 位数据格式。按照低位在前，高位在后的顺序发送。采用反码是为了增加传输的可靠性（可用于校验）。数据格式如下：



NEC 码还规定了连发码（由 9ms 低电平+2.5ms 高电平+0.56ms 低电平+97.94ms 高电平组成），如果在一帧数据发送完毕之后，红外遥控器按键仍然没有放开，则发射连发码，可以通过统计连发码的次数来标记按键按下的长短或次数。

（2）红外接收设备

红外接收设备是由红外接收电路、红外解码、电源和应用电路组成。红外遥控接收器的主要作用是将遥控发射器发来的红外光信号转换成电信号，再放大、限幅、检波、整形，形成遥控指令脉冲，输出至遥控微处理器。近几年不论是业余制作还是正式产品，大多都采用成品红外接收头。成品红外接收头的封装大致有两种：一种采用铁皮屏蔽；一种是塑料封装。均有三只引脚，即电源正（VDD）、电源负（GND）和数据输出（VOUT）。其外观实物图如下图所示：



正对接收头的凸起处看，从左至右，管脚依次是 1：VOUT，2：GND，3：VDD。

由于红外接收头在没有脉冲的时候为高电平，当收到脉冲的时候为低电平，所以可以通过外部中断的下降沿触发中断，在中断内通过计算高电平时间来判断接收到的数据是 0 还是 1。外部中断的相关知识，大家可以参考前面的“外部

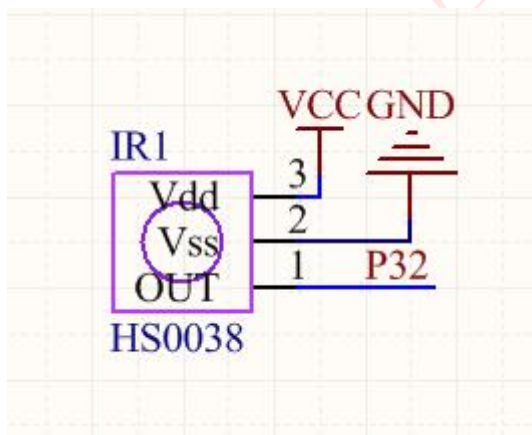
中断实验”，如果想要更加深入的了解红外遥控通信，大家可以百度下，网上这方面的资料可谓是铺天盖地。

21.2 硬件设计

本实验使用到硬件资源如下：

- (1) 动态数码管
- (2) 红外接收头和遥控器

动态数码管电路在前面章节都介绍过，这里就不再重复。红外遥控器和接收头是一体的，内部结构不用去管它。下面我们来看下开发板上红外接收模块电路，如下图所示：



从上图中可以看出，该电路是集成的，红外接收头的输出管脚接至单片机 P32 管脚上，为了保证红外接收头输出管脚默认为高电平，需外接一个上拉电阻，同样的与单片机 IO 口连接就相当于外接了 10K 上拉电阻。

由于该模块电路是集成的，所以红外接收头的输出管脚与单片机 P32 管脚连接使用即可。

21.3 软件设计

本章所要实现的功能是：使用外部中断功能将遥控器键值编码数据解码后通过数码管显示。

我们直接复制上一章创建的工程，在此模板基础上进行程序开发。为了能够与开发攻略教程对应，将复制过来的模板文件夹重新命名为“实验 26：红外通

信”。打开工程直接在 main.c 源文件内进行编程，main.c 内代码如下：

```

/*****
*****

*
实验现象：下载程序后，数码管显示红外遥控键值数据

接线说明：（具体接线图可见开发攻略对应实验的“实验现象”章节）

注意事项：红外遥控器上的电池绝缘隔片要拿掉

*****
*****

*/

#include "reg52.h"           //此文件中定义了单片机的一些特殊功能
寄存器

typedef unsigned int u16;    //对数据类型进行声明定义
typedef unsigned char u8;

sbit LSA=P2^2;
sbit LSB=P2^3;
sbit LSC=P2^4;

sbit IRIN=P3^2;

u8 IrValue[6];
u8 Time;

u8 DisplayData[8];
u8 code smgduan[17]={
0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07,
0x7f, 0x6f, 0x77, 0x7c, 0x39, 0x5e, 0x79, 0x71, 0x76};
//0、1、2、3、4、5、6、7、8、9、A、b、C、d、E、F、H 的显示码

/*****
*****

* 函 数 名          : delay

```

```

* 函数功能      : 延时函数, i=1 时, 大约延时 10us
*****
*****/
void delay(u16 i)
{
    while(i--);
}

/*****
*****
* 函数名      : DigDisplay()
* 函数功能    : 数码管显示函数
* 输入        : 无
* 输出        : 无
*****
*****/
void DigDisplay()
{
    u8 i;
    for(i=0; i<3; i++)
    {
        switch(i) //位选, 选择点亮的数码管,
        {
            case(0):
                LSA=1; LSB=1; LSC=1; break; //显示第 0 位
            case(1):
                LSA=0; LSB=1; LSC=1; break; //显示第 1 位
            case(2):
                LSA=1; LSB=0; LSC=1; break; //显示第 2 位
        }
        P0=DisplayData[i]; //发送数据
        delay(100); //间隔一段时间扫描
        P0=0x00; //消隐
    }
}

/*****
*****
* 函数名      : IrInit()
* 函数功能    : 初始化红外线接收
* 输入        : 无
* 输出        : 无

```

```

*****
*****/

void IrInit()
{
    IT0=1;//下降沿触发
    EX0=1;//打开中断 0 允许
    EA=1; //打开总中断

    IRIN=1;//初始化端口
}

/*****
*****
* 函 数 名      : main
* 函数功能      : 主函数
* 输    入      : 无
* 输    出      : 无
*****
*****/
void main()
{
    IrInit();
    while(1)
    {
        DisplayData[0] = smgduan[IrValue[2]/16];
        DisplayData[1] = smgduan[IrValue[2]%16];
        DisplayData[2] = smgduan[16];
        DigDisplay();
    }
}

/*****
*****
* 函数名      : ReadIr()
* 函数功能      : 读取红外数值的中断函数
* 输入        : 无
* 输出        : 无
*****
*****/

void ReadIr() interrupt 0
{

```

```

u8 j,k;
ul6 err;
Time=0;
delay(700);    //7ms
if(IRIN==0)    //确认是否真的接收到正确的信号
{
    err=1000;    //1000*10us=10ms, 超过说明接收

```

到错误的信号

/*当两个条件都为真是循环, 如果有一个条件为假的时候跳出循环, 免得程序出

错的时

候, 程序死在这里*/

```
while((IRIN==0)&&(err>0))    //等待前面 9ms 的低电平过去
```

```

{
    delay(1);
    err--;
}

```

```
if(IRIN==1)    //如果正确等到 9ms 低电平
```

```

{
    err=500;
    while((IRIN==1)&&(err>0))    //等待 4.5ms 的起始高电

```

平过去

```

{
    delay(1);
    err--;
}
for(k=0;k<4;k++)    //共有 4 组数据
{
    for(j=0;j<8;j++)    //接收一组数据
    {

```

```

        err=60;
        while((IRIN==0)&&(err>0))//等待信号前面的 560us

```

低电平过去

```

    {
        delay(1);
        err--;
    }
}

```

```

err=500;
while((IRIN==1)&&(err>0))    //计算高电平

的时间长度。

    {
        delay(10); //0.1ms
        Time++;
        err--;
        if(Time>30)
        {
            return;
        }
    }
    IrValue[k]>>=1;    //k 表示第几组数据
    if(Time>=8)        //如果高电平出

现大于 565us，那么是 1
    {
        IrValue[k]|=0x80;
    }
    Time=0;           //用完时间要重新赋值

    }
}
}
if(IrValue[2]!=~IrValue[3])
{
    return;
}
}
}
}

```

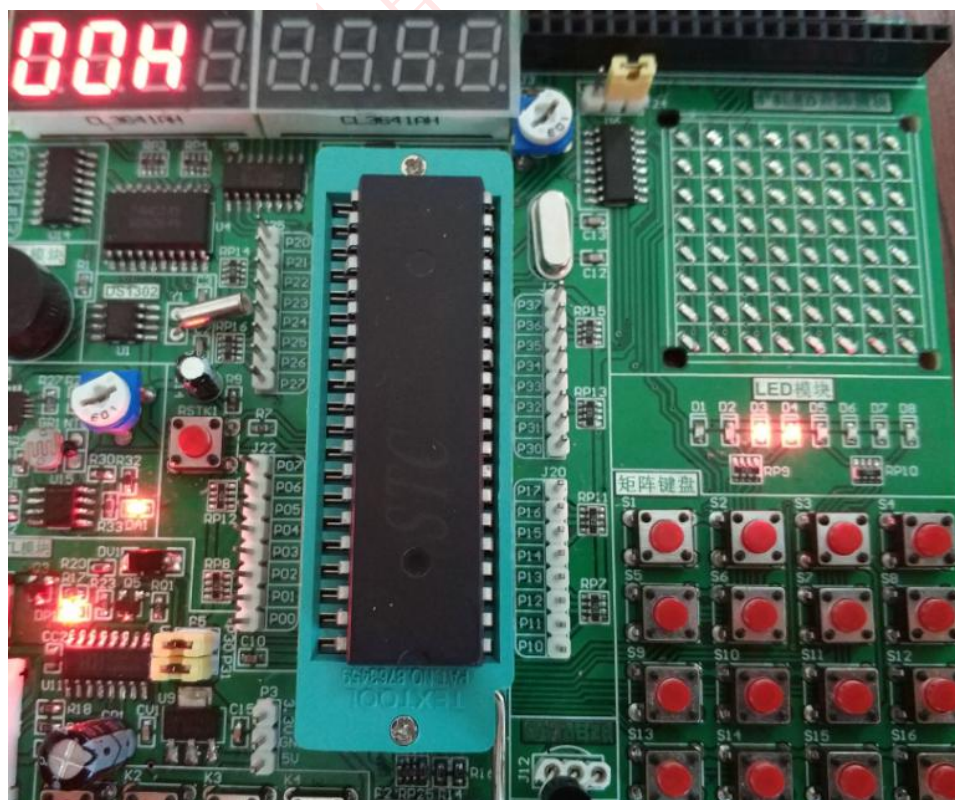
main.c 文件内代码非常少也很简单，首先将 51 单片机的头文件包含进来，然后定义动态数码管、74HC138 译码器、红外接收头的控制管脚。主函数功能非常简单，首先调用 IrInit() 函数配置外部中断 0、下降沿触发等功能，然后在 while 循环内将红外键码控制值取出各位对应的数码管段码数据保存到数组 DisplayData 中。最后调用 DigDisplay() 动态数码管显示函数，将处理好的数据显示。

当红外遥控器按下键后，红外接收头就会发出一个低电平信号触发中断，进入中断函数内首先确认是否接收到的信号（引导码）是真正的信号，如果是引导

码则继续判断输出数据是 1 还是 0，一个标准数据供有 32 位，将其分别保存在 IrValue 数组内，并且根据数据与数据反码的规律判断此数据是否正确，当接收外一组数据后会自动退出中断服务函数，从而继续执行主函数，将接收到的键值码显示在数码管上，程序中我们只将数据控制码显示，至于地址码及其反码和数据控制反码我们不显示。

21.4 实验现象

使用 USB 线将开发板和电脑连接成功后，把编写好的程序编译后将编译产生的 .hex 文件烧入到芯片内，按照如下图所示的接线方式可以看到：按下遥控器上按键，在数码管上显示该键对应的键值码。注意：出厂时的遥控器电池盖内都有一张绝缘片，使用时必须将此绝缘片拿掉。还有键值码不是大家所想的按键上的数字字符码，这个要区分开。



实验说明：我们配的遥控器所有键的地址码与反码都是相同的，不同的是控制码和控制反码，其实知道了控制码就知道了控制反码，所以如果要使用红外遥控控制其他设备，可以通过区分控制码来实现。

如果程序下载了，能显示“00H”，按键时却没有反应，首先检查遥控器是否拔下绝缘片。如果要检查遥控器好坏，可以在光线暗的地方，打开手机摄像机，将遥控器前方的发射灯对准手机摄像头，按下按键，在手机上是否能看见灯闪，来判断遥控器好坏。

课后作业

（1）使用红外遥控器控制开发板上的 LED 指示灯、蜂鸣器和继电器。（温馨提示：对解码后的信号数据进行处理，通过比较来控制）

（2）使用红外遥控器设计一个智能小电扇，实现电机开启、停止。

第 22 章 ADC 模数转换实验

在前面章节，我们介绍的都是对数字信号的操作也就是 0 和 1 操作，本章我们就来讲解如何检测外部模拟信号，因为我们使用的 STC89C5x 单片机内部不含 ADC 接口，所以需要外接一个 ADC 转换芯片将模拟信号转换成数字信号供单片机处理。我们开发板上集成了一个 ADC 模数转换电路，选用的 ADC 芯片是 12 位的 AD 芯片-XPT2046。本章要实现的功能是：通过 ADC 转换电路采集电位器电压值，将采集转换后的 AD 值通过数码管显示。学习本章可以参考前面的实验章节内容。若结合视频学习效果更佳。本章分为如下几部分内容：

22.1 ADC 介绍

22.2 XPT2046 芯片介绍

22.3 硬件设计

22.4 软件设计

22.5 实验现象

22.1 ADC 介绍

我们知道 51 单片机系统内部运算时用的全部是数字量，即 0 和 1，因此对单片机系统而言，我们无法直接操作模拟量，必须将模拟量转换成数字量。所谓数字量，就是用一系列 0 和 1 组成的二进制代码表示某个信号大小的量。用数字量表示同一个模拟量时，数字位数可以多也可以少，位数越多则表示的精度越高，位数越少表示的精度就越低。

22.1.1 ADC 简介

ADC (analog to digital converter) 也称为模数转换器，是指一个将模拟信号转变为数字信号。单片机在采集模拟信号时，通常都需要在前端加上 A/D 芯片。

下面我们看下 ADC 的主要技术指标：

(1) 分辨率

ADC 的分辨率是指对于允许范围内的模拟信号，它能输出离散数字信号值的个数。这些信号值通常用二进制数来存储，因此分辨率经常用比特作为单位，且这些离散值的个数是 2 的幂指数。

例如：12 位 ADC 的分辨率就是 12 位，或者说分辨率为满刻度的 $1/(2^{12})$ 。

一个 10V 满刻度的 12 位 ADC 能分辨输入电压变化最小值是

$$10V \times 1/(2^{12}) = 2.4mV。$$

(2) 转换误差

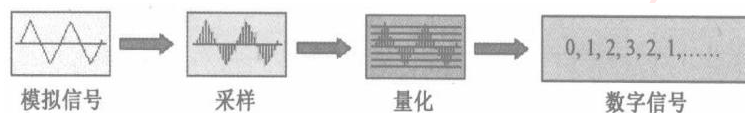
转换误差通常是以输出误差的最大值形式给出。它表示 A/D 转换器实际输出的数字量和理论上的输出数字量之间的差别。常用最低有效位的倍数表示。例如给出相对误差 $\leq \pm \text{LSB}/2$ ，这就表明实际输出的数字量和理论上应得到的输出数字量之间的误差小于最低位的半个字。

(3) 转换速率

ADC 的转换速率是能够重复进行数据转换的速度，即每秒转换的次数。而完成一次 A/D 转换所需的时间（包括稳定时间），则是转换速率的倒数。

22.1.2 ADC 转换原理

在 A/D 转换器中, 因为输入的模拟信号在时间上是连续的, 而输出的数字信号代码是离散的, 所以 A/D 转换器在进行转换时, 必须在一系列选定的瞬间 (时间坐标轴上的一些规定点上) 对输入的模拟信号采样, 然后再把这些采样值转换为数字量。因此, 一般的 A/D 转换过程是通过采样保持、量化和编码这三个步骤完成的, 即首先对输入的模拟电压信号采样, 采样结束后进入保持时间, 在这段时间内将采样的电压量转化为数字量, 并按一定的编码形式给出转换结果, 然后开始下一次采样。下图给出模拟量到数字量转换过程的框图。



模拟量到数字量转换过程框图

1. 采样定理

可以证明, 为了正确无误地用下左图中所示的采样信号 V_s 表示模拟信号 V_i , 必须满足:

$$f_s \geq 2f_{imax}$$

在满足采样定理的条件下, 可以用一个低通滤波器将信号 V_s 还原为 V_i , 这个低通滤波器的电压传输系数 $|A(f)|$ 在低于 $f(imax)$ 的范围内应保持不变, 而在 $f(s)-f(imax)$ 以前应迅速下降为零。如下右图所示。因此, 采样定理规定了 A/D 转换的频率下限。



因此, A/D 转换器工作时的采样频率必须高于所规定的频率。采样频率提高以后, 留给 A/D 转换器每次进行转换的时间也相应缩短了, 这就要求转换电路必须具备更快的工作速度。因此, 不能无限制地提高采样频率, 通常取 $f(s)=(3-5)f(imax)$ 已经能够满足要求。

因为每次把采样电压转换为相应的数字量都需要一定的时间，所以在每次采样以后，必须把采样电压保持一段时间。可见，进行 A/D 转换时所用的输入电压，实际上是每次采样结束时的 V_i 值。

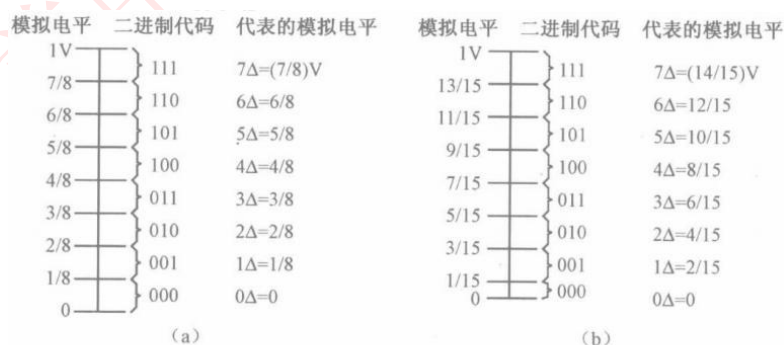
2. 量化和编码

我们知道，数字信号不仅在时间上是离散的，而且数值上的变化也不是连续的。这就是说，任何一个数字量的大小，都是以某个最小数量单位的整倍数来表示的。因此，在用数字量表示采样电压时，也必须把它化成这个最小数量单位的整倍数，这个转化过程就叫做量化。

所规定的最小数量单位叫做量化单位，用 Δ 表示。显然，数字信号最低有效位中的 1 表示的数量大小，就等于 Δ 。把量化的数值用二进制代码表示，称为编码。这个二进制代码就是 A/D 转换的输出信号。

既然模拟电压是连续的，那么它就不一定能被 Δ 整除，因而不可避免地会引入误差，我们把这种误差称为量化误差。在把模拟信号划分为不同的量化等级时，用不同的划分方法可以得到不同的量化误差。

假定需要把 $0 \sim 1V$ 的模拟电压信号转换成三位二进制代码，这时便可以取 $\Delta = (1/8)V$ ，并规定凡数值在 $0 \sim (1/8)V$ 之间的模拟电压都当做 $0X\Delta$ 看待，用二进制的 000 表示；凡数值在 $((1/8) \sim (2/8)V$ 之间的模拟电压都当做 $1X\Delta$ 看待，用二进制的 001 表示，……，如下图所示。不难看出，最大的量化误差可达 Δ ，即 $(1/8)V$ 。



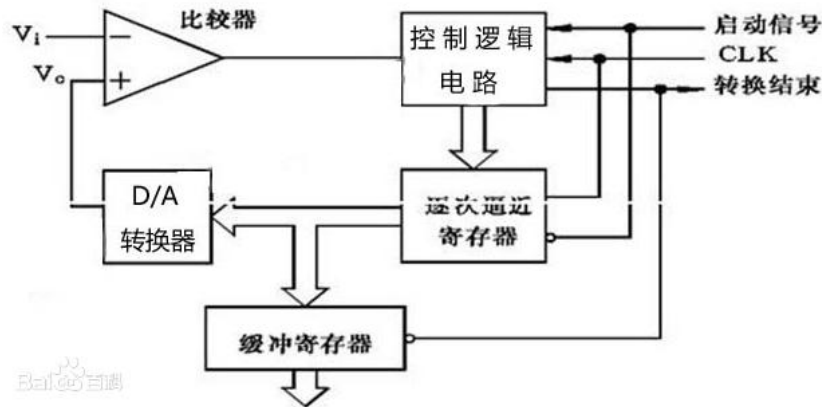
划分量化电平的两种方法

为了减少量化误差，通常采用图(b)所示的划分方法，取量化单位 $\Delta = (2/15)V$ ，并将 000 代码所对应模拟电压规定为 $0 \sim (1/15)V$ ，即 $0 \sim \Delta/2$ 。这时，最大量化误差将减少为 $\Delta/2 = (1/15)V$ 。这个道理不难理解，因为现在把每个二进制代码所代表的模拟电压值规定为它所对应的模拟电压范围的中点，所以最大的量化

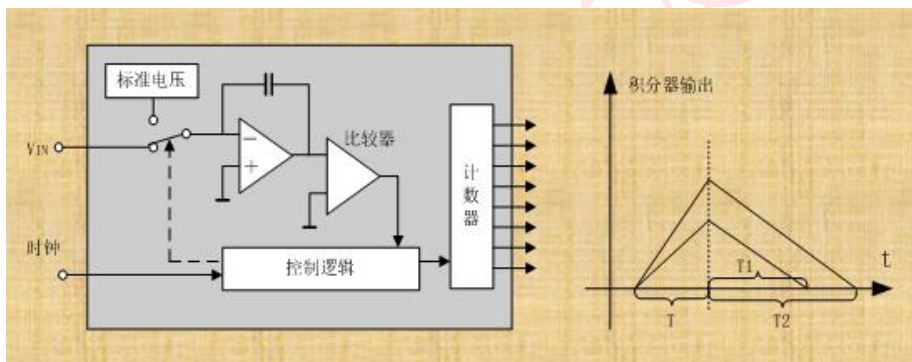
误差自然就缩小为 $\Delta/2$ 了。

ADC 内转换方式主要有以下 2 种：逐次逼近式 ADC 和双积分式 ADC。

(1) 逐次逼近式 ADC



(2) 双积分式 ADC



它们的工作原理大家可以百度，网上资料一大把，讲解非常详细，这里我们就不深入介绍。下面我们来看下开发板上所用的 ADC 转换芯片 XPT2046。

22.2 XPT2046 芯片介绍

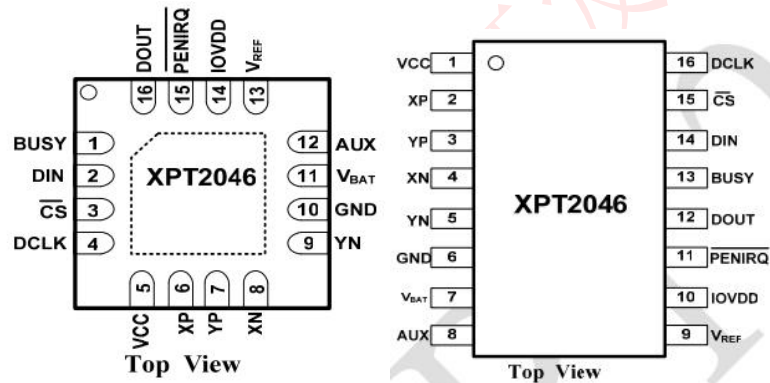
XPT2046 是一款 4 线制电阻式触摸屏控制器，内含 12 位分辨率 125KHz 转换速率逐步逼近型 A/D 转换器。XPT2046 支持从 1.5V 到 5.25V 的低电压 I/O 接口。XPT2046 能通过执行两次 A/D 转换查出被按的屏幕位置，除此之外，还可以测量加在触摸屏上的压力。内部自带 2.5V 参考电压，可以作为辅助输入、温度测量和电池监测之用，电池监测的电压范围可以从 0V 到 6V。XPT2046 片内集成有一个温度传感器。在 2.7V 的典型工作状态下，关闭参考电压，功耗可小于 0.75mW。XPT2046 采用微小的封装形式：TSSOP-16, QFN-16 和 VFBGA-48。

工作温度范围为 $-40^{\circ}\text{C}\sim+85^{\circ}\text{C}$ 。与 ADS7846、TSC2046、AK4182A 完全兼容。

(1) 主要特性

- ①工作电压范围为 1.5V~5.25V
- ②支持 1.5V~5.25V 的数字 I/O 口
- ③内建 2.5V 参考电压源
- ④电源电压测量 (0V~6V)
- ⑤内建结温测量功能
- ⑥触摸压力测量
- ⑦采用 3 线制 SPI 通信接口
- ⑧具有自动省电功能

(2) 芯片管脚说明



QFN引脚号	TSSOP引脚号	VFBGA引脚号	名称	说明
1	13	A5	BUSY	忙时信号线。当CS为高电平时为高阻状态
2	14	A4	DIN	串行数据输入端。当CS为低电平时，数据在DCLK上升沿锁存进来
3	15	A3	CS	片选信号。控制转换时序和使能串行输入输出寄存器，高电平时ADC掉电
4	16	A2	DCLK	外部时钟信号输入
5	1	B1和C1	VCC	电源输入端
6	2	D1	XP	XP位置输入端
7	3	E1	YP	YP位置输入端
8	4	G2	XN	XN位置输入端
9	5	G3	YN	YN位置输入端
10	6	G4和G5	GND	接地
11	7	G6	V _{BAT}	电池监视输入端
12	8	E7	AUX	ADC辅助输入通道
13	9	D7	V _{REF}	参考电压输入/输出
14	10	C7	IOVDD	数字电源输入端
15	11	B7	PENIRQ	笔接触中断引脚
16	12	A6	DOUT	串行数据输出端。数据在DCLK的下降沿移出，当CS高电平时为高阻状态

XPT2046 是一种典型的逐次逼近型模数转换器（SAR ADC），包含了采样/保持、模数转换、串口数据 输出等功能。同时芯片集成有一个 2.5V 的内部参考电压源、温度检测电路，工作时使用外部时钟。XPT2046 可以单电源供电，电源电压范围为 2.7V~5.5V。参考电压值直接决定 ADC 的输入范围，参考电压可以使用内部参考电压，也可以从外部直接输入 1V~VCC 范围内的参考电压（要求外部参考电压源输出阻抗低）。X、Y、Z、VBAT、Temp 和 AUX 模拟信号经过片内的控制寄存器选择后进入 ADC，ADC 可以配置为单端或差分模式。选择 VBAT、Temp 和 AUX 时应该配置为单端模式；作为触摸屏应用时，应该配置为差分模式，这可有效消除由于驱动开关的寄生电阻及外部的干扰带来的测量误差，提高转换精度。单端和差分模式输入配置如下图所示：

表3 单端模式输入配置（ $SER/\overline{DFR}=1$ ）

A2	A1	A0	VBAT	AUXIN	TEMP	YN	XP	YP	Y-位置	X-位置	Z1-位置	Z2-位置	X-驱动	Y-驱动
0	0	0			+IN (TEMP0)								off	off
0	0	1					+IN		测量				Off	On
0	1	0	+IN										Off	Off
0	1	1					+IN				测量		XN, On	YP, On
1	0	0				+IN						测量	XN, On	YP, On
1	0	1						+IN		测量			On	Off
1	1	0		+IN									Off	Off
1	1	1			+IN (TEMP1)								Off	Off

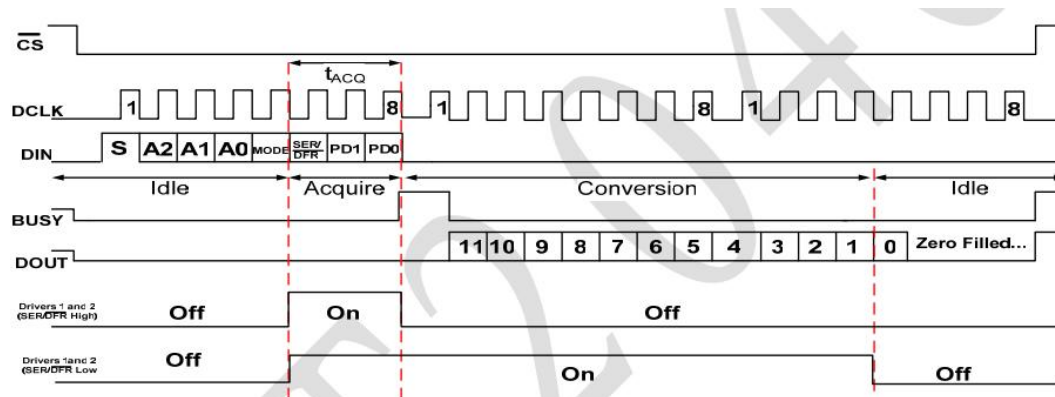
表4 差分模式输入配置（ $SER/\overline{DFR}=0$ ）

A2	A1	A0	+REF	-REF	YN	XP	YP	Y-位置	X-位置	Z1-位置	Z2-位置	驱动
0	0	1	YP	YN		+IN		测量				YP, YN
0	1	1	YP	XN		+IN				测量		YP, XN
1	0	0	YP	XN	+IN						测量	YP, XN
1	0	1	XP	XN			+IN		测量			XP, XN

XPT2046 数据接口是串行接口，其典型工作时序如下图所示，图中展示的信号来自带有基本串行接口的单片机或数据信号处理器。处理器和转换器之间的通信需要 8 个时钟周期，可采用 SPI、SSI 和 Microwire 等同步串行接口。一次完整的转换需要 24 个串行同步时钟（DCLK）来完成。

前 8 个时钟用来通过 DIN 引脚输入控制字节。当转换器获取有关下一次转换的足够信息后，接着根据获得的信息设置输入多路选择器和参考源输入，并进入采样模式，如果需要，将启动触摸面板驱动器。3 个多时钟周期后，控制字节设置完成，转换器进入转换状态。这时，输入采样—保持器进入保持状态，触摸

面板驱动器停止工作（单端工作模式）。接着的 12 个时钟周期将完成真正的模数转换。如果是度量比率转换方式（SER/DFR=0），驱动器在转换过程中将一直工作，第 13 个时钟将输出转换结果的最后一位。剩下的 3 个多时钟周期将用来完成被转换器忽略的最后字节（DOUT 置低）。



在对 XPT2046 进行控制时，控制字节由 DIN 输入的控制字命令格式如下所示：

表5 制字的控制位命令

位 7 (MSB)	位 6	位 5	位 4	位 3	位 2	位 1	位 0 (LSB)
S	A2	A1	A0	MODE	SER/DFR	PD1	PD0

表6 控制字节各位描述

位	名称	功能描述
7	S	开始位。为 1 表示一个新的控制字节到来，为 0 则忽略 PIN 引脚上数据
6-4	A2-A0	通道选择位。参见表 1 和表 2
3	MODE	12 位/8 位转换分辨率选择位。为 1 选择 8 位为转换分辨率，为 0 选择 12 位分辨率
2	SER/DFR	单端输入方式/差分输入方式选择位。为 1 是单端输入方式，为 0 是差分输入方式
1-0	PD1-PD0	低功率模式选择位。若为 11，器件总处于供电状态；若为 00，器件在变换之间处于低功率模式

注意：差分模式仅用于 X 坐标、Y 坐标和触摸压力的测量，其它测量要求采用单端模式。

表7 掉电和内部参考电压选择

PD1	PD0	PENIRQ	功能说明
0	0	使能	在两次 A/D 转换之间掉电，下次转换一开始，芯片立即进入完全上电状态，而无需额外的延时。在这种模式下，YN 开关一直处于 ON 状态
0	1	禁止	参考电压关闭，ADC 打开
1	0	使能	参考电压打开，ADC 关闭
1	1	禁止	芯片处于上电状态，参考电压和 ADC 总是打开

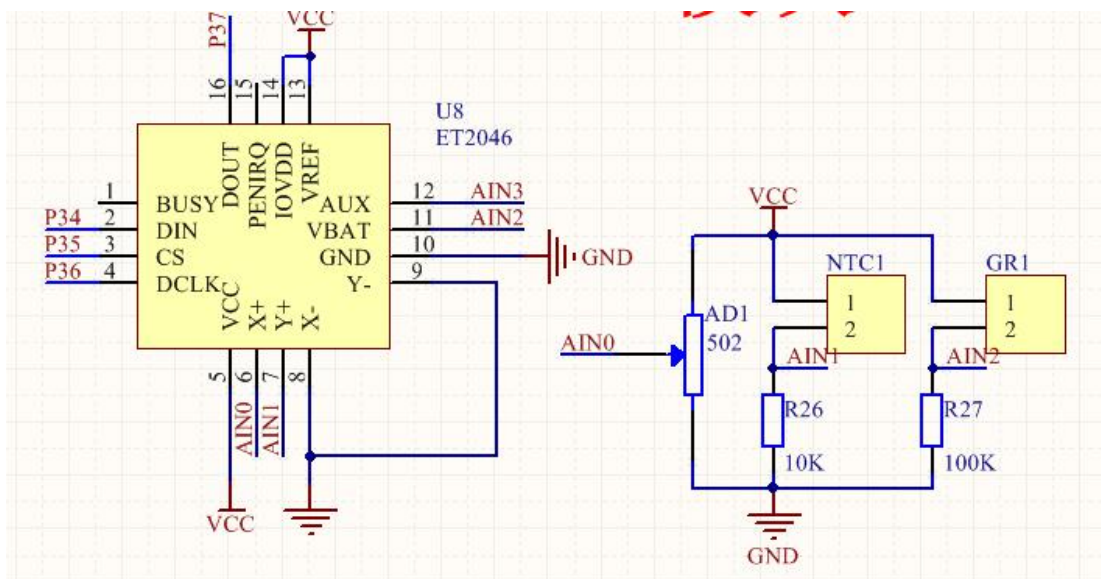
至此，我们就对 ADC 的基本原理及 XPT2046 芯片介绍完了，大家如果想要更详细的了解其工作原理，可以查看光盘资料“\6--开发板芯片资料\xpt2046 中文.pdf”或者百度。

22.3 硬件设计

本实验使用到硬件资源如下：

- (1) 动态数码管
- (2) ADC 模块

动态数码管电路在前面章节都介绍过，这里就不再重复。下面我们来看下开发板上 ADC 模块电路，如下图所示：



从上图可以看出，该电路是集成的，XPT2046 芯片的控制管脚接至单片机 P34~P36 管脚上，XPT2046 芯片的 ADC 输入转换通道分别接入了 AD1 电位器、NTC1 热敏传感器、GR1 光敏传感器，还有一个外接通道 AIN3 接在 DAC (PWM) 模块的 J52 端子上供外部模拟信号检测。

由于该模块电路是集成的，所以 XPT2046 芯片的控制管脚与单片机管脚连接使用即可。

22.4 软件设计

本章所要实现的功能是：通过 ADC 转换电路采集电位器电压值，将采集转换后的 AD 值通过数码管显示。

我们直接复制上一章创建的工程，在此模板基础上进行程序开发。为了能够与开发攻略教程对应，将复制过来的模板文件夹重新命名为“实验 27：AD 模数转换”。打开工程文件夹，发现里面有 4 个工程文件夹，本章实验以电位器 AD

值工程进行讲解,其他3个工程只是在本工程基础上修改了XPT2046的采集通道,其他都一样。打开“电位器AD值”工程,里面多了2个文件:XPT2046.c源文件和XPT2046.h头文件,该文件保存的是XPT2046的驱动程序。至于如何创建多文件工程,在前面章节已经介绍过,这里就不再重复。

(1) 首先我们看下XPT2046.c文件代码,如下:

```
#include "XPT2046.h"

/*****
*****
*函数名: SPI_Write
*输 入: dat: 写入数据
*输 出: 无
*功 能: 使用 SPI 写入数据
*****
*****/

void SPI_Write(uchar dat)
{
    uchar i;
    CLK = 0;
    for(i=0; i<8; i++)
    {
        DIN = dat >> 7; //放置最高位
        dat <<= 1;
        CLK = 0;        //上升沿放置数据

        CLK = 1;

    }
}

/*****
*****
*函数名: SPI_Read
*输 入: 无
*输 出: dat: 读取 到的数据
*功 能: 使用 SPI 读取数据
*****
*****/

uint SPI_Read(void)
{
    uint i, dat=0;
```

```

    CLK = 0;
    for(i=0; i<12; i++)      //接收 12 位数据
    {
        dat <<= 1;

        CLK = 1;
        CLK = 0;

        dat |= DOUT;
    }
    return dat;
}

/*****
*****
*函数名: Read_AD_Data
*输 入: cmd: 读取的 X 或者 Y
*输 出: endValue: 最终信号处理后返回的值
*功 能: 读取触摸数据
*****
*****/
uint Read_AD_Data(uchar cmd)
{
    uchar i;
    uint AD_Value;
    CLK = 0;
    CS = 0;
    SPI_Write(cmd);
    for(i=6; i>0; i--); //延时等待转换结果
    CLK = 1;           //发送一个时钟周期, 清除 BUSY
    _nop_();
    _nop_();
    CLK = 0;
    _nop_();
    _nop_();
    AD_Value=SPI_Read();
    CS = 1;
    return AD_Value;
}

```

各函数功能在代码内有详细的注释, 大家可以参考注释理解。因为 XPT2046 芯片也是 SPI 接口, 所以其通信方式与前面学习的 DS1302 或者 74HC595 类似。在应用中我们主要调用 Read_AD_Data(uchar cmd) 函数来获取对应通道的 AD 数

据。该函数其实很简单，按照前面介绍的时序，先将对应通道的控制字命令写入到 XPT2046 内，然后延时一段时间读取检测 AD 值，因为 XPT2046 是 12 位的，所以读取出来的数据是 12 位，其范围是 0-4095。这个操作在后面 main.c 文件内会有。

程序中我们看到有 DIN、CLK、CS 和 DOUT 的操作，该变量就是我们在 XPT2046.h 头文件内定义的控制管脚，代码如下：

```
#ifndef __XPT2046_H_
#define __XPT2046_H_

//---包含头文件---//
#include<reg52.h>
#include<intrins.h>

//---重定义关键词---//
#ifndef uchar
#define uchar unsigned char
#endif

#ifndef uint
#define uint unsigned int
#endif

#ifndef ulong
#define ulong unsigned long
#endif

//---定义使用的 I/O 口---//
sbit DOUT = P3^7; //输出
sbit CLK = P3^6; //时钟
sbit DIN = P3^4; //输入
sbit CS = P3^5; //片选

uint Read_AD_Data(uchar cmd);
uint SPI_Read(void);
void SPI_Write(uchar dat);

#endif
```

同样，头文件内通常都是用来定义管脚和声明函数或者变量的。下面来看下 main.c 文件代码，main.c 内代码如下：

```
/******
```

*

实验现象: 下载程序后数码管前 4 位显示电位器检测的 AD 值, 范围是 0-4095, 一般达不到最大, 这个受
供电电压的影响

接线说明: (具体接线图可见开发攻略对应实验的“实验现象”章节)

注意事项:

*/

#include "reg52.h" //此文件中定义了单片机的一些特殊功能
寄存器

#include "XPT2046.h"

typedef unsigned int u16; //对数据类型进行声明定义

typedef unsigned char u8;

sbit LSA=P2^2;

sbit LSB=P2^3;

sbit LSC=P2^4;

u8 disp[4];

u8

code

smgduan[10]={0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07, 0x7f, 0x6f};

/*****

* 函数名 : delay

* 函数功能 : 延时函数, i=1 时, 大约延时 10us

*****/

void delay(u16 i)

{

while(i--);

}


```

/*****
*****
* 函数名      :datapros()
* 函数功能    :数据处理函数
* 输入        : 无
* 输出        : 无
*****
*****/
void datapros()
{
    u16 temp;
    static u8 i;
    if(i==50)
    {
        i=0;
        temp = Read_AD_Data(0x94);    //   AIN0 电位器
    }
    i++;
    disp[0]=smgduan[temp/1000]; //千位
    disp[1]=smgduan[temp%1000/100]; //百位
    disp[2]=smgduan[temp%1000%100/10]; //个位
    disp[3]=smgduan[temp%1000%100%10];
}

/*****
*****
* 函数名      :DigDisplay()
* 函数功能    :数码管显示函数
* 输入        : 无
* 输出        : 无
*****
*****/
void DigDisplay()
{
    u8 i;
    for(i=0;i<4;i++)
    {
        switch(i)    //位选，选择点亮的数码管，
        {
            case(0):
                LSA=1;LSB=1;LSC=1; break; //显示第 0 位
            case(1):
                LSA=0;LSB=1;LSC=1; break; //显示第 1 位

```

```
        case(2):
            LSA=1;LSB=0;LSC=1; break;//显示第 2 位
        case(3):
            LSA=0;LSB=0;LSC=1; break;//显示第 3 位
    }
    P0=disp[i];//发送数据
    delay(100); //间隔一段时间扫描
    P0=0x00;//消隐
}

}

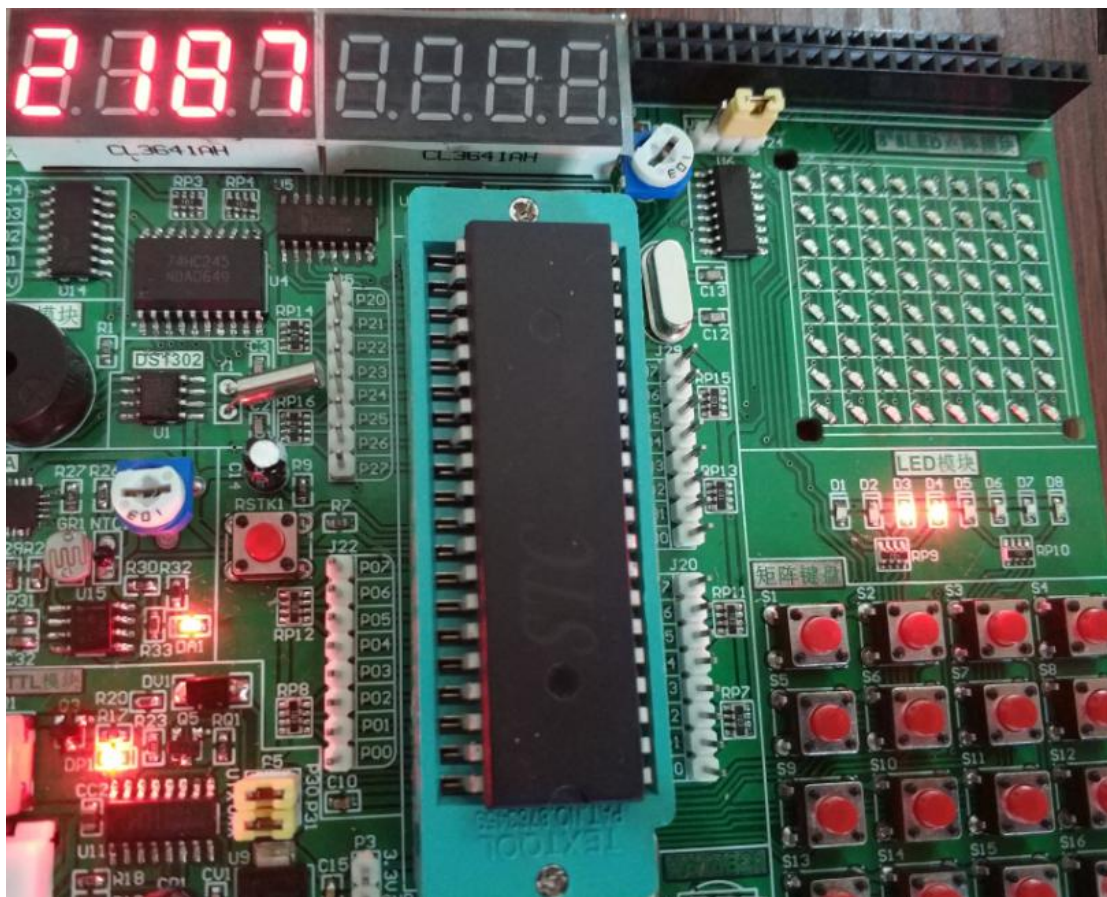
/*****
*****
* 函 数 名      : main
* 函数功能      : 主函数
* 输    入      : 无
* 输    出      : 无
*****
*****/
void main()
{
    while(1)
    {
        datapros();    //数据处理函数
        DigDisplay(); //数码管显示函数
    }
}
```

main.c 文件内代码非常少也很简单,首先将 51 单片机的头文件和 XPT2046.h 的头文件包含进来,然后定义动态数码管、74HC138 译码器的控制管脚。主函数功能非常简单,在 while 循环内调用 datapros()AD 数据处理函数,该函数内部又调用 Read_AD_Data(0x94)函数来读取电位器对应通道(AIN0)的 AD 值,0x94 即为 AIN0 通道的控制命令,至于 AIN1、AIN2、AIN3 通道值可根据前面介绍的控制命令寄存器计算(AIN1 对应的命令值是 0xA4,AIN2 对应的命令值是 0xD4,AIN3 对应的命令值是 0xE4),然后在取出各位对应的数码管段码数据保存到数组 disp 中。最后调用 DigDisplay()动态数码管显示函数,将处理好的数据显示。

22.5 实验现象

使用 USB 线将开发板和电脑连接成功后(使用短接片将 USB 转串口模块上 J39

端子的 P31T 与 URXD 短接, J44 端子的 P30R 与 UTXD 短接, 电脑能识别开发板上 CH340 驱动串口), 把编写好的程序编译后将编译产生的 .hex 文件烧入到芯片内, 按照如下图所示的接线方式可以看到: 数码管上显示采集到电位器的 AD 值, 范围是 0-4095, 注意: 由于供电电压的不同, 所以 ADC 参考电压也不同, 对应计算的 AD 值就会有误差, 可能不能达到最大值 4095。



课后作业

(1) 使用 ADC 模块采集电位器的电压值。

(温馨提示: 本章实验程序即为采集电位器的 AD 值, 要想转换为电压值只需根据公式: $V = (5 * AD) / 4096$ 计算, 在程序中要注意设置 V 变量为浮点, 然后强制转换 AD 为浮点数, 否则计算出来的电压值就会把小数部分滤除掉)

第 23 章 DAC 数模转换实验

在前面章节，我们介绍了如何使用 51 单片机来采集外部模拟信号，这一章我们来介绍下如何使用 51 单片机输出模拟信号，要让 51 单片机输出模拟信号，同样需要相应的转换器，这种转换器我们称之为 DAC 数模转换器，还可以使用在实际应用较高的 PWM 技术来模拟 DAC 输出。我们开发板上集成了一个 DAC (PWM) 模块电路，通过单片机 IO 口生成 PWM 波来模拟 DAC 输出。本章要实现的功能是：系统运行时，DAC (PWM) 模块上的指示灯 DA1 呈呼吸灯效果，由暗变亮再由亮变暗。学习本章可以参考前面的实验章节内容。若结合视频学习效果更佳。本章分为如下几部分内容：

- 23.1 DAC 介绍
- 23.2 PWM 介绍
- 23.3 硬件设计
- 23.4 软件设计
- 23.5 实验现象

23.1 DAC 介绍

23.1.1 DAC 简介

DAC (Digital to analog converter) 即数字模拟转换器, 它可以将数字信号转换为模拟信号。它的功能与 ADC 相反。在常见的数字信号系统中, 大部分传感器信号被转化成电压信号, 而 ADC 把电压模拟信号转换成易于计算机存储、处理的数字编码, 由计算机处理完成后, 再由 DAC 输出电压模拟信号, 该电压模拟信号常常用来驱动某些执行器件, 使人类易于感知。如音频信号的采集及还原就是这样一个过程。

DAC 的主要技术指标如下:

(1) 分辨率

DAC 的分辨率是输入数字量的最低有效位 (LSB) 发生变化时, 所对应的输出模拟量 (电压或电流) 的变化量。它反映了输出模拟量的最小变化值。分辨率与输入数字量的位数有确定的关系, 可以表示成 $FS / (2^n)$ 。FS 表示满量程输入值, n 为二进制位数。对于 5V 的满量程, 采用 8 位的 DAC 时, 分辨率为 $5V/256 = 19.5mV$; 当采用 12 位的 DAC 时, 分辨率则为 $5V/4096 = 1.22mV$ 。显然, 位数越多分辨率就越高。

(2) 线性度

线性度 (也称非线性误差) 是实际转换特性曲线与理想直线特性之间的最大偏差。常以相对于满量程的百分数表示。如 $\pm 1\%$ 是指实际输出值与理论值之差在满刻度的 $\pm 1\%$ 以内。

(3) 绝对精度和相对精度

绝对精度 (简称精度) 是指在整个刻度范围内, 任一输入数码所对应的模拟量实际输出值与理论值之间的最大误差。绝对精度是由 DAC 的增益误差 (当输入数码为全 1 时, 实际输出值与理想输出值之差)、零点误差 (数码输入为全 0 时,

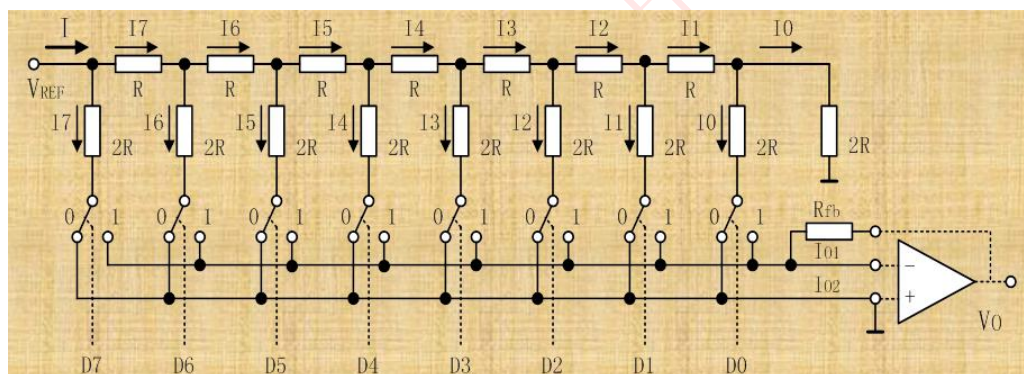
DAC 的非零输出值)、非线性误差和噪声等引起的。绝对精度(即最大误差)应小于 1 个 LSB。相对精度与绝对精度表示同一含义,用最大误差相对于满刻度的百分比表示。

(4) 建立时间

建立时间是指输入的数字量发生满刻度变化时,输出模拟信号达到满刻度值的 $\pm 1/2\text{LSB}$ 所需的时间。是描述 D/A 转换速率的一个动态指标。根据建立时间的长短,可以将 DAC 分成超高速($<1\mu\text{S}$)、高速($10\sim 1\mu\text{S}$)、中速($100\sim 10\mu\text{S}$)、低速($\geq 100\mu\text{S}$)几档。

23.1.2 DAC 工作原理

了解了 DAC 基本概念及特性后,再来看下其工作原理,下面以 T 型电阻网络 DAC 来介绍。其内部结构图如下所示:



$$\text{DAC 输出电压计算公式: } V_0 = V_{\text{ref}} * z / 256$$

公式中的 z 表示单片机给的数字量, v_{ref} 为参考电压,通常我们是接在系统电源上,即 5V,数值 256 表示 DAC 精度为 8 位。

DAC 主要由数字寄存器、模拟电子开关、位权网络、求和运算放大器和基准电压源(或恒流源)组成。用存于数字寄存器的数字量的各位数码,分别控制对应位的模拟电子开关,使数码为 1 的位在位权网络上产生与其位权成正比的电流值,再由运算放大器对各电流值求和,并转换成电压值。

上述的模拟电子开关都分别接着一个分压的器件,比如说电阻。模拟开关的个数取决于 DAC 的精度。那么 N 个电子开关就把基准电压分为 N 份(并不是平均分哦),而这些开关根据输入的二进制每一位数据对应开启或者关闭,把分压的器件上的电压引入输出电路中。

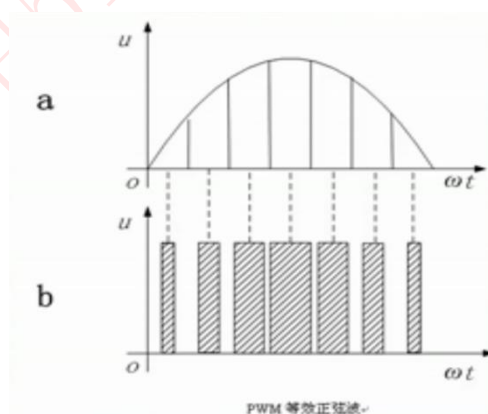
23.2 PWM 介绍

出于成本考虑，在实际开发应用中，使用最多的还是通过 PWM 来模拟 DAC 输出。下面就先来了解下 PWM 相关概念。

PWM 是 Pulse Width Modulation 的缩写，中文意思就是脉冲宽度调制，简称脉宽调制。它是利用微处理器的数字输出来对模拟电路进行控制的一种非常有效的技术，其控制简单、灵活和动态响应好等优点而成为电力电子技术最广泛应用的控制方式，其应用领域包括测量，通信，功率控制与变换，电动机控制、伺服控制、调光、开关电源，甚至某些音频放大器，因此学习 PWM 具有十分重要的现实意义。

其实我们也可以这样理解，PWM 是一种对模拟信号电平进行数字编码的方法。通过高分辨率计数器的使用，方波的占空比被调制用来对一个具体模拟信号的电平进行编码。PWM 信号仍然是数字的，因为在给定的任何时刻，满幅值的直流供电要么完全有(ON)，要么完全无(OFF)。电压或电流源是以一种通(ON)或断(OFF)的重复脉冲序列被加到模拟负载上去的。通的时候即是直流供电被加到负载上的时候，断的时候即是供电被断开的时候。只要带宽足够，任何模拟值都可以使用 PWM 进行编码。

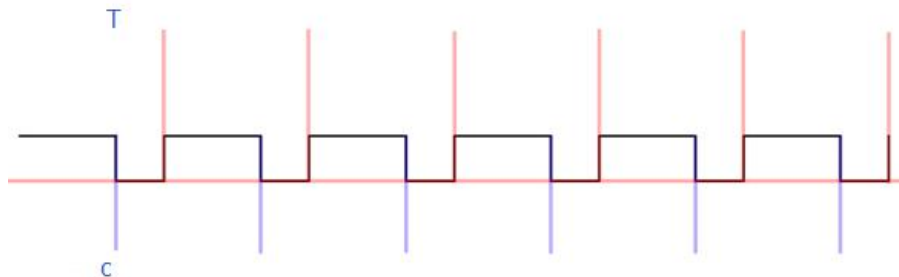
PWM 对应模拟信号的等效图，如下图所示：



从图中可以看到，上图 a 是一个正弦波即模拟信号，b 是一个数字脉冲波形即数字信号。我们知道在计算机系统中只能识别是 1 和 0，对于 51 单片机芯片，要么输出高电平（5V），要么输出低电平（0），假如要输出 1.5V 的电压，那么就必须通过相应的处理，比如本章所要讲解的 PWM 输出，其实从上图也可以看到，只要保证数字信号脉宽足够就可以使用 PWM 进行编码，从而输出

1. 5V 的电压。

PWM 的输出其实就是对外输出脉宽可调（即占空比调节）的方波信号，信号频率是由 T 的值决定，占空比由 C 的值决定。其示意图如图所示：



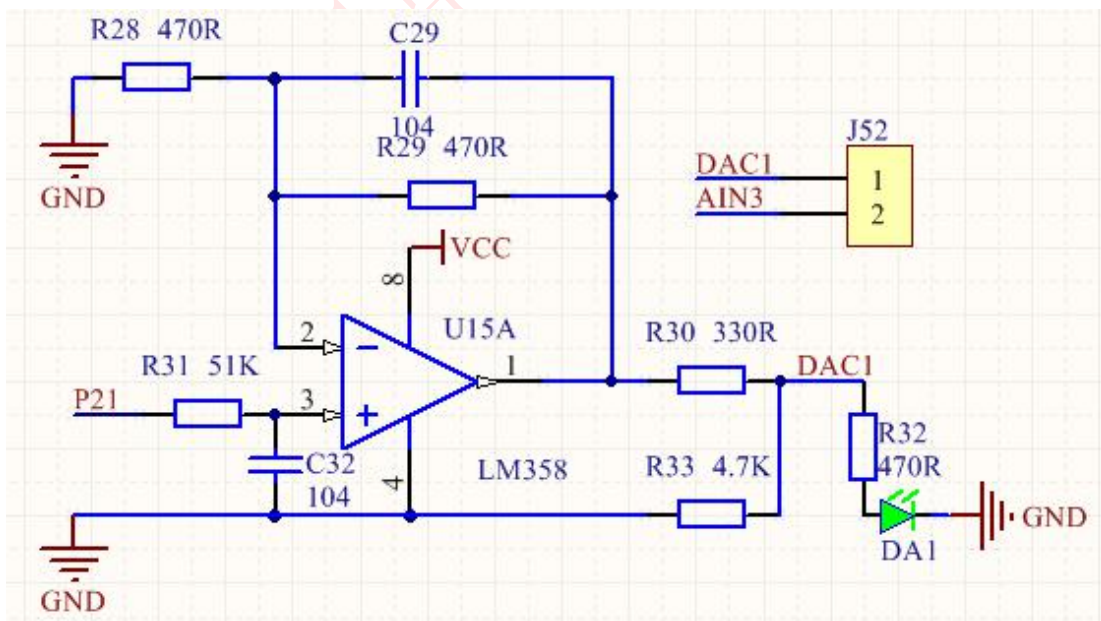
从上图中可以看到，PWM 输出频率是不变的，改变的是 C 的值，此值的改变将导致 PWM 输出信号占空比的改变。占空比其实就是一个周期内高电平时间与周期的比值。而频率的话可以使用 51 单片机的定时器确定。

23.3 硬件设计

本实验使用到硬件资源如下：

（1）DAC (PWM) 模块

下面我们来看下开发板上 DAC (PWM) 模块电路，如下图所示：



从上图中可以看出，该电路是集成的，PWM 输出控制管脚接至单片机 P21 管脚上，DAC1 为 PWM 输出信号，将其连接一个 LED，这样可以通过指示灯的状态直观的反映出 PWM 输出电压值变化。LM358 芯片与这些电容电阻构成了一个跟随电

路，即输入是多少，输出即为多大电压，输出电压范围是 0-5V。输出信号由 J52 端子的 DAC1 引出，在其端子上还有一个 AIN3 脚，它是上一章介绍 ADC 时的外部模拟信号输入通道。如果使用短接片将 DAC1 和 AIN3 短接，这样就可以使用 XPT2046 芯片采集检测 PWM 输出信号。

由于该模块电路是独立的，所以 PWM 控制管脚可以通过使用任意单片机管脚连接使用即可。

如果需要检测 PWM 输出的电压大小，可以使用短接片将 J52 端子的 DAC 和 AIN3 短接，这样就可以使用 XPT2046 来检测模拟信号。

23.4 软件设计

本章所要实现的功能是：系统运行时，DAC (PWM) 模块上的指示灯 DA1 呈呼吸灯效果，由暗变亮再由亮变暗。。

我们直接复制上一章创建的工程，在此模板基础上进行程序开发。为了能够与开发攻略教程对应，将复制过来的模板文件夹重新命名为“实验 28：DA 数模转换”。打开工程直接在 main.c 源文件内进行编程，main.c 内代码如下：

```
/*
*****
*****

*
实验现象：下载程序后 AD/DAC 模块上的 DA1 指示灯呈呼吸灯效果，由暗变亮再由亮变暗

接线说明：（具体接线图可见开发攻略对应实验的“实验现象”章节）

注意事项：

*****
*****

*/

#include "reg52.h"          //此文件中定义了单片机的一些特殊功能寄存器
```

```
typedef unsigned int u16;      //对数据类型进行声明定义
typedef unsigned char u8;

//--定义使用的 I/O 口--//
sbit PWM=P2^1;
bit DIR;

//--定义一个全局变量--//
u16 count,value,timer1;

/*****
*****
* 函 数 名      : Timer1Init
* 函数功能      : 定时器 1 初始化
* 输    入      : 无
* 输    出      : 无
*****
*****/
void Timer1Init()
{
    TMOD|=0x10;//选择为定时器 1 模式，工作方式 1，仅用 TR1 打开启动。

    TH1 = 0xFF;
    TL1 = 0xff;    //1us

    ET1=1;//打开定时器 1 中断允许
    EA=1;//打开总中断
    TR1=1;//打开定时器
}

/*****
*****
* 函 数 名      : main
* 函数功能      : 主函数
* 输    入      : 无
* 输    出      : 无
*****
*****/
void main()
{
    Timer1Init(); //定时器 1 初始化
    while(1)
```

```
{
    if(count>100)
    {
        count=0;
        if(DIR==1)                //DIR 控制增
加或减小
        {
            value++;
        }
        if(DIR==0)
        {
            value--;
        }
    }

    if(value==1000)
    {
        DIR=0;
    }
    if(value==0)
    {
        DIR=1;
    }

    if(timer1>1000) //PWM 周期为 1000*1us
    {
        timer1=0;
    }
    if(timer1 <value)
    {
        PWM=1;
    }
    else
    {
        PWM=0;
    }
}

/*****
*****
* 函 数 名      : Timer1
```

```

* 函数功能      : 定时器 1 的中断函数
* 输    入      : 无
* 输    出      : 无
*****
*****/

void Timer1(void) interrupt 3    //3 为定时器 1 的中断号 1 定时器 0
的中断号 0 外部中断 1 2

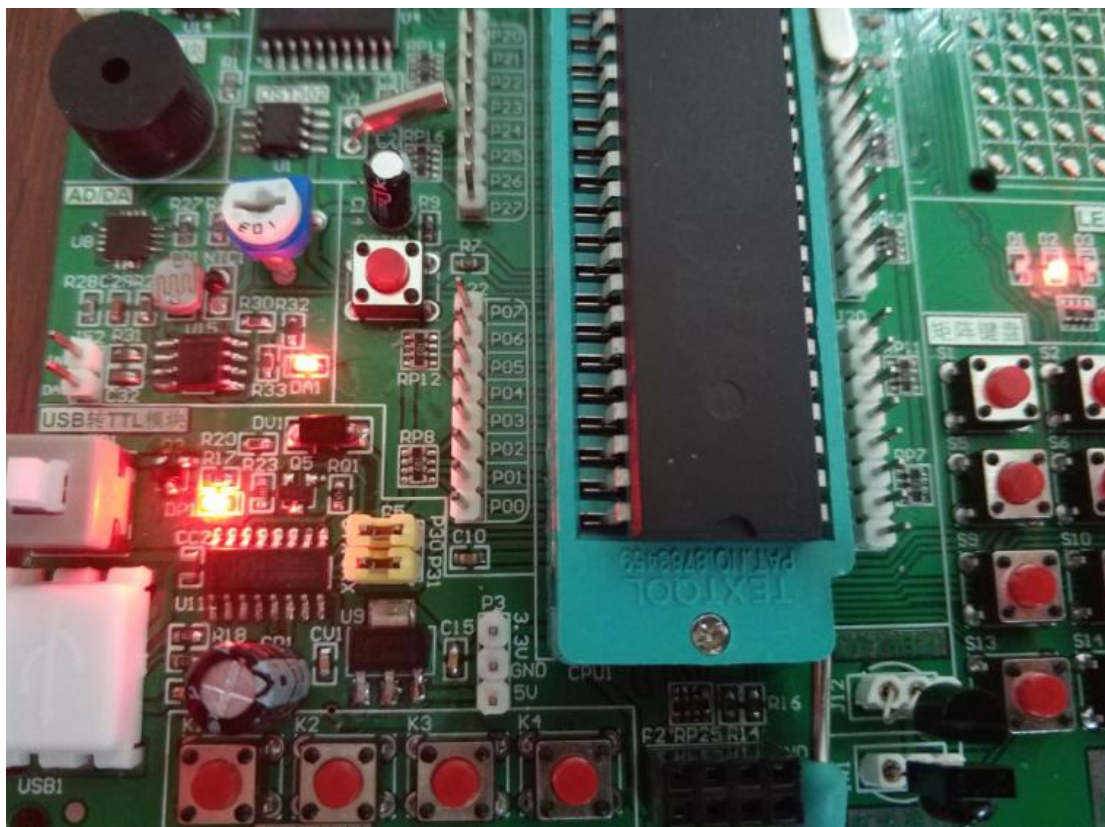
    外部中断 2 4 串口中断
    {
        TH1 = 0xFF;
        TL1 = 0xFF;    //1us
        timer1++;
        count++;
    }

```

main.c 文件内代码非常少也很简单，首先将 51 单片机的头文件包含进来，然后定义 PWM 的控制管脚。主函数功能非常简单，首先调用 Timer1Init() 函数配置定时器 1 定时中断功能，定时时间设置为 1us，然后在 while 循环内通过相应变量的值控制调节占空比大小的方向和 PWM 高低电平的输出。当定时时间达到，进入中断服务函数，在中断服务函数内重新给定时器赋初值，并且将变量 timer1 和 count 加 1。程序内有详细的注释，大家结合注释来理解。

23.5 实验现象

使用 USB 线将开发板和电脑连接成功后(使用短接片将 USB 转串口模块上 J39 端子的 P31T 与 URXD 短接，J44 端子的 P30R 与 UTXD 短接，电脑能识别开发板上 CH340 驱动串口)，把编写好的程序编译后将编译产生的 .hex 文件烧入到芯片内，按照如下图所示的接线方式可以看到：系统运行时，DAC(PWM) 模块上的指示灯 DA1 呈呼吸灯效果，由暗变亮再由亮变暗。



课后作业

(1) 通过 ADC 模块来检测 PWM 输出值，并使用数码管来显示其输出电压大小。（温馨提示：将 ADC 实验与 PWM 实验进行整合，注意要将 J52 端子的 DAC 和 AIN3 脚短接）

第 24 章 LCD1602 液晶显示实验

在前面章节，我们已经学习过几种显示装置，例如静态数码管、动态数码管、16*16 双色点阵、串口通信。使用它们可以很直观方便地显示一些字符数据，但是它们也有各种局限性，比如显示字符数据信息太少，硬件设计复杂、代码编写难度大等。这一章我们就来介绍一种非常简单且常用的显示装置——LCD1602 液晶显示器，使用它可以显示更多的字符数字信息。我们开发板上集成了一个 LCD1602 液晶显示器接口，将配置的 LCD1602 液晶对应插入即可。本章要实现的功能是：系统运行时，在 LCD1602 液晶上显示字符信息。学习本章可以参考前面的实验章节内容。若结合视频学习效果更佳。本章分为如下几部分内容：

24.1 LCD1602 介绍

24.2 硬件设计

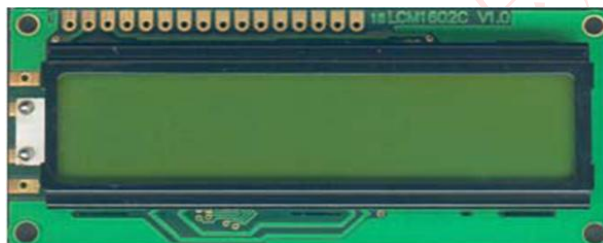
24.3 软件设计

24.4 实验现象

24.1 LCD1602 介绍

24.1.1 LCD1602 简介

1602 液晶也叫 1602 字符型液晶，它能显示 2 行字符信息，每行又能显示 16 个字符。它是一种专门用来显示字母、数字、符号的点阵型液晶模块。它是由若干个 5x7 或者 5x10 的点阵字符位组成，每个点阵字符位都可以用显示一个字符，每位之间有一个点距的间隔，每行之间也有间隔，起到了字符间距和行间距的作用，正因为如此，所以它不能很好的显示图片。其实物图如下所示：



大家手上拿到的 LCD1602 外观可能和上图不一样，这是由于不同厂家设计所致，使用方法是一样的。在上图中可以看到有 16 个管脚孔，从左至右管脚编号顺序是 1-16，其功能定义如下所示：

编号	符号	引脚说明	编号	符号	引脚说明
1	VSS	电源地	9	D2	Data I/O
2	VDD	电源正极	10	D3	Data I/O
3	VL	液晶显示偏压信号	11	D4	Data I/O
4	RS	数据/命令选择端 (H/L)	12	D5	Data I/O
5	R/W	读/写选择端 (H/L)	13	D6	Data I/O
6	E	使能信号	14	D7	Data I/O
7	D0	Data I/O	15	BLA	背光源正极
8	D1	Data I/O	16	BLK	背光源负极

下面对几个管脚做下说明：

3 脚：VL，液晶显示偏压信号，用于调整 LCD1602 的显示对比度，一般会外接电位器用以调整偏压信号，注意此脚电压为 0 时可以得到最强的对比度。

4 脚：RS，数据/命令选择端，当此脚为高电平时，可以对 1602 进行数据字节的传输操作，而为电平时，则是进行命令字节的传输操作。命令字节，即是用来对 LCD1602 的一些工作方式作设置的字节；数据字节，即使用以在 1602 上显示的字节。值得一提的是，LCD1602 的数据是 8 位的。

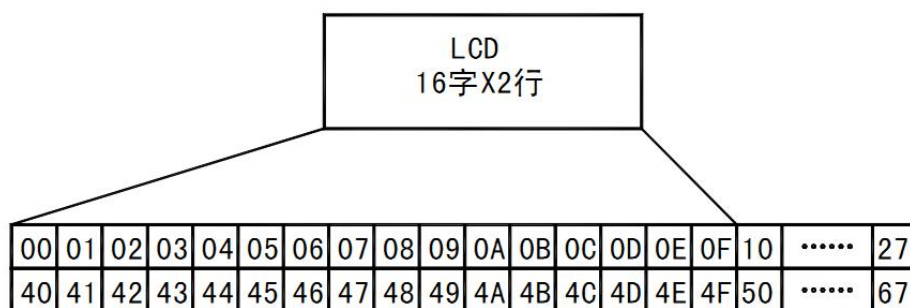
5 脚：R/W，读写选择端。当此脚为高电平可对 LCD1602 进行读数据操作，反之进行写数据操作。

6 脚：E，使能信号，其实是 LCD1602 的数据控制时钟信号，利用该信号的上升沿实现对 LCD1602 的数据传输。

7~14 脚：8 位并行数据口，而 51 单片机一组 I/O 也是 8 位，使得对 LCD1602 的数据读写大为方便。

在 LCD1602 内部含有 80 个字节的 DDRAM，它是用来寄存显示字符的。其地址和屏幕的对应关系如下表：

	显示位置	1	2	3	4	5	6	7		40
DDRAM 地 址	第一行	00H	01H	02H	03H	04H	05H	06H		27H
	第二行	40H	41H	42H	43H	44H	45H	46H		67H



从上图可知，不是所有的地址都可以直接用来显示字符数据，只有第一行中的 00~0F，第二行中的 40~4F 才能显示，其他地址只能用于存储。要显示字符时，要先输入显示字符地址，也就是告诉模块在哪里显示字符，例如第二行第一个字符的地址是 40H，那么是否直接写入 40H 就可以将光标定位在第二行第一个字符的位置呢？这样不行，因为写入显示地址时要求最高位 D7 恒定为高电平 1，所以实际写入的数据应该是 01000000B(40H) + 10000000B(80H) = 11000000B(C0H)。在 1602 中我们就用前 16 个就行了。第二行也一样用前 16 个地址。

24.1.2 LCD1602 常用指令

在使用 LCD1602 时，我们需要掌握一些常用的指令，这些指令对于 LCD1602 初始化是必须的。

(1) 清屏指令

指令功能	指令编码										执行时间 /ms
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
清屏	0	0	0	0	0	0	0	0	0	1	1.64

功能：

- <1> 清除液晶显示器，即将 DDRAM 的内容全部填入“空白”的 ASCII 码 20H；
- <2> 光标归位，即将光标撤回液晶显示屏的左上方；
- <3> 将地址计数器(AC)的值设为 0。

(2) 模式设置指令

指令功能	指令编码										执行时间 /us
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
进入模式设置	0	0	0	0	0	0	0	1	I/D	S	40

功能：

设定每次写入 1 位数据后光标的移位方向，并且设定每次写入的一个字符是否移动。

I/D: 0=写入新数据后光标左移 1=写入新数据后光标右移

S: 0=写入新数据后显示屏不移动 1=写入新数据后显示屏整体右移 1 个字符

(3) 显示开关控制指令

指令功能	指令编码										执行时间 /us
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
显示开关控制	0	0	0	0	0	0	1	D	C	B	40

功能：

控制显示器开/关、光标显示/关闭以及光标是否闪烁。

D: 0=显示功能关 1=显示功能开

C: 0=无光标 1=有光标

B: 0=光标闪烁 1=光标不闪烁

(4) 功能设定指令

指令功能	指令编码										执行时间/ μ s
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
功能设定	0	0	0	0	1	DL	N	F	X	X	40

功能:

设定数据总线位数、显示的行数及字型。

DL: 0=数据总线为 4 位 1=数据总线为 8 位

N: 0=显示 1 行 1=显示 2 行

F: 0=5 $\times$ 7 点阵/每字符 1=5 $\times$ 10 点阵/每字符

24.1.3 LCD1602 使用

要使用 LCD1602，首先需要对其初始化，即通过写入一些特定的指令实现。然后选择要在 LCD1602 的哪个位置显示并将所要显示的数据发送到 LCD 的 DDRAM。使用 LCD1602 通常都是用于写数据进去，很少使用读功能。LCD1602 操作步骤如下所示：

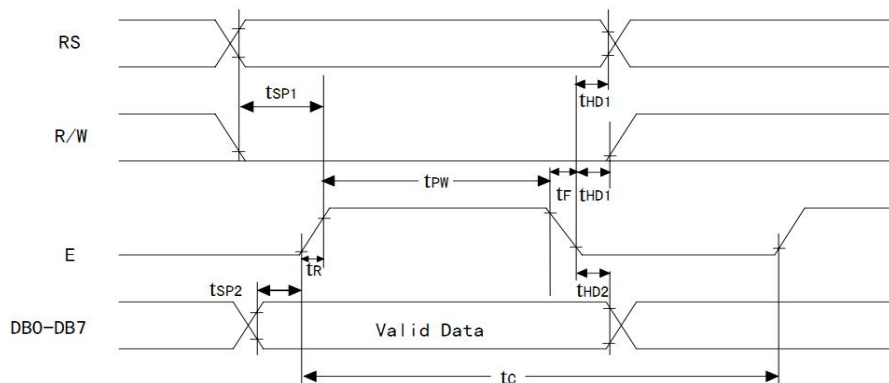
- (1) 初始化
- (2) 写命令 (RS=L)，设置显示坐标
- (3) 写数据 (RS=H)

在此，不需要读出它的数据的状态或者数据本身。所以只需要看两个写时序：

① 当要写指令字，设置 LCD1602 的工作方式时：需要把 RS 置为低电平，RW 置为低电平，然后将数据送到数据口 D0~D7，最后 E 引脚一个高脉冲将数据写入。

② 当要写入数据字，在 1602 上实现显示时：需要把 RS 置为高电平，RW 置为低电平，然后将数据送到数据口 D0~D7，最后 E 引脚一个高脉冲将数据写入。

写指令和写数据，差别仅仅在于 RS 的电平不一样而已。以下是 LCD1602 的时序图：



时序参数	符号	极限值			单位
		最小值	典型值	最大值	
E 信号周期	t_C	400	—	—	ns
E 脉冲宽度	t_{PW}	150	—	—	ns
E 上升沿/下降沿时间	t_R, t_F	—	—	25	ns
地址建立时间	t_{SP1}	30	—	—	ns
地址保持时间	t_{HD1}	10	—	—	ns
数据建立时间(读操作)	t_D	—	—	100	ns
数据保持时间(读操作)	t_{HD2}	20	—	—	ns
数据建立时间(写操作)	t_{SP2}	40	—	—	ns
数据保持时间(写操作)	t_{HD2}	10	—	—	ns

从上图可以看到，以上给的时间参数全部是 ns 级别的，而 51 单片机的机器周期是 1 μ s，指令周期是 2-4 个机器周期，所以即便在程序里不加延时程序，也可以很好的配合 LCD1602 的时序要求了。

当要写命令字节的时候，时间由左往右，RS 变为低电平，R/W 变为低电平，注意看是 RS 的状态先变化完成。然后这时，DB0~DB7 上数据进入有效阶段，接着 E 引脚有一个整脉冲的跳变，接着要维持时间最小值为 $t_{pw}=400\text{ns}$ 的 E 脉冲宽度。然后 E 引脚负跳变，RS 电平变化，R/W 电平变化。这样便是一个完整的 LCD1602 写命令的时序。

注意：这里介绍的是 8 位 LCD1602，现在某些公司为简化引脚数，使用 4 位 LCD1602。使用 4 位 LCD1602 时，应该多看手册，找到不同点，对原有程序加以修改。我们例程也做了 4 位和 8 位 LCD1602 的兼容，具体后面会介绍。

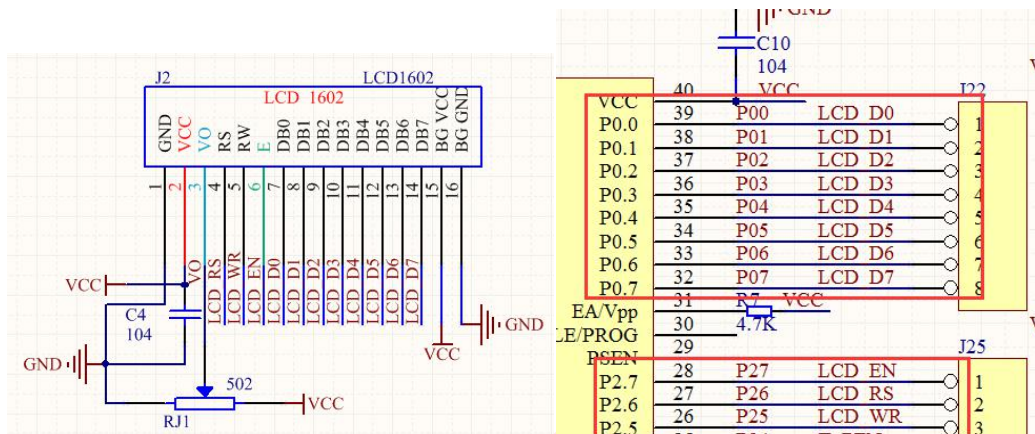
至此，我们就把 LCD1602 介绍完了，大家如果想要更详细的了解它，可以查看光盘资料“\6—开发板芯片资料\LCD1602 液晶完整中文资料.pdf”或者百度。

24.2 硬件设计

本实验使用到硬件资源如下：

(1) LCD1602 液晶

开发板上集成了一个 LCD1602 液晶接口，下面我们来看下开发板上 LCD1602 液晶接口电路，如下图所示：



从上图中可以看出，该电路是并不是独立的，LCD1602 的 8 位数据口 DB0-DB7 与单片机的 P00-P07 管脚连接，LCD1602 的 RS、RW、E 脚与单片机的 P26、P25、P27 管脚连接。RJ1 是一个电位器，用来调节 LCD1602 对比度即显示亮度。

注意：这里原理图是使用的 8 位 LCD1602 接口设计，如果使用 4 位 LCD1602 则需要使用转接板。在出厂配套 LCD1602 时，有的是 8 位的，有的是 4 位的。出厂的 4 位 LCD1602 已带有转接板，仅仅将 8 位 LCD1602 引脚的高四位用于 4 位 LCD1602。也就是说在传输数据的时候需要将 8 位的数据截成两段，先发送高四位，在发送低四位。其它引脚操作方法不变。

做本章实验时只需要将配置的 LCD1602 液晶插到开发板的 LCD1602 接口处，无需额外接线，如下所示（具体可参考实验现象接线图）：



24.3 软件设计

本章所要实现的功能是：系统运行时，在 LCD1602 液晶上显示字符信息。

我们直接复制上一章创建的工程，在此模板基础上进行程序开发。为了能够与开发攻略教程对应，将复制过来的模板文件夹重新命名为“实验 29：LCD1602 液晶”。打开工程文件夹，发现里面多了 2 个文件：lcd.c 源文件和 lcd.h 头文件，该文件保存的是 LCD1602 的驱动程序。至于如何创建多文件工程，在前面章节已经介绍过，这里就不再重复。

(1) 首先我们看下 lcd.c 文件代码，如下：

```
#include "lcd.h"

/*****
*****
* 函 数 名      : Lcd1602_Delay1ms
* 函数功能      : 延时函数，延时 1ms
* 输    入      : c
* 输    出      : 无
* 说    名      : 该函数是在 12MHZ 晶振下，12 分频单片机的延时。
*****
*****/

void Lcd1602_Delay1ms(uint c)  //误差 0us
{
    uchar a,b;
    for (; c>0; c--)
    {
        for (b=199;b>0;b--)
        {
            for(a=1;a>0;a--);
        }
    }
}

/*****
*****
* 函 数 名      : LcdWriteCom
* 函数功能      : 向 LCD 写入一个字节的命令
* 输    入      : com
* 输    出      : 无
*****
*****/

#ifndef LCD1602_4PINS //当没有定义这个 LCD1602_4PINS 时
void LcdWriteCom(uchar com)  //写入命令
```



```

{
    LCD1602_E = 0;    //使能
    LCD1602_RS = 0;   //选择发送命令
    LCD1602_RW = 0;   //选择写入

    LCD1602_DATAPINS = com;    //放入命令
    Lcd1602_Delay1ms(1);    //等待数据稳定

    LCD1602_E = 1;           //写入时序
    Lcd1602_Delay1ms(5);    //保持时间
    LCD1602_E = 0;
}
#else
void LcdWriteCom(uchar com)    //写入命令
{
    LCD1602_E = 0;    //使能清零
    LCD1602_RS = 0;   //选择写入命令
    LCD1602_RW = 0;   //选择写入

    LCD1602_DATAPINS = com; //由于 4 位的接线是接到 P0 口的高四位，
    所以传送高四位不用改
    Lcd1602_Delay1ms(1);

    LCD1602_E = 1;    //写入时序
    Lcd1602_Delay1ms(5);
    LCD1602_E = 0;

    LCD1602_DATAPINS = com << 4; //发送低四位
    Lcd1602_Delay1ms(1);

    LCD1602_E = 1;    //写入时序
    Lcd1602_Delay1ms(5);
    LCD1602_E = 0;
}
#endif
/*****
*****
* 函 数 名      : LcdWriteData
* 函数功能      : 向 LCD 写入一个字节的数据
* 输    入      : dat
* 输    出      : 无
*****
*****/
#endif    LCD1602_4PINS

```

```

void LcdWriteData(uchar dat)          //写入数据
{
    LCD1602_E = 0;    //使能清零
    LCD1602_RS = 1;   //选择输入数据
    LCD1602_RW = 0;   //选择写入

    LCD1602_DATAPINS = dat; //写入数据
    Lcd1602_Delay1ms(1);

    LCD1602_E = 1;    //写入时序
    Lcd1602_Delay1ms(5); //保持时间
    LCD1602_E = 0;
}
#else
void LcdWriteData(uchar dat)          //写入数据
{
    LCD1602_E = 0;    //使能清零
    LCD1602_RS = 1;   //选择写入数据
    LCD1602_RW = 0;   //选择写入

    LCD1602_DATAPINS = dat; //由于 4 位的接线是接到 P0 口的高四位，
    所以传送高四位不用改
    Lcd1602_Delay1ms(1);

    LCD1602_E = 1;    //写入时序
    Lcd1602_Delay1ms(5);
    LCD1602_E = 0;

    LCD1602_DATAPINS = dat << 4; //写入低四位
    Lcd1602_Delay1ms(1);

    LCD1602_E = 1;    //写入时序
    Lcd1602_Delay1ms(5);
    LCD1602_E = 0;
}
#endif
/*****
*****
* 函 数 名      : LcdInit()
* 函数功能      : 初始化 LCD 屏
* 输    入      : 无
* 输    出      : 无
*****
*****/

```

```

#ifndef LCD1602_4PINS
void LcdInit() //LCD 初始化子程序
{
    LcdWriteCom(0x38); //开显示
    LcdWriteCom(0x0c); //开显示不显示光标
    LcdWriteCom(0x06); //写一个指针加 1
    LcdWriteCom(0x01); //清屏
    LcdWriteCom(0x80); //设置数据指针起点
}
#else
void LcdInit() //LCD 初始化子程序
{
    LcdWriteCom(0x32); //将 8 位总线转为 4 位总线
    LcdWriteCom(0x28); //在四位线下的初始化
    LcdWriteCom(0x0c); //开显示不显示光标
    LcdWriteCom(0x06); //写一个指针加 1
    LcdWriteCom(0x01); //清屏
    LcdWriteCom(0x80); //设置数据指针起点
}
#endif

```

各函数功能在代码内有详细的注释，大家可以参考注释理解。因为 LCD1602 使用的是 6800 总线通信（什么是 6800 总线，大家可以百度），其通信方式在前面介绍 LCD1602 时已经讲解。在程序中我们可以看到如下程序框架：

```

#ifndef LCD1602_4PINS
... 程序段 1...
#else
... 程序段 2...
#endif

```

这个是条件选择编译结构，意思是：如果没有定义 LCD1602_4PINS 则执行程序段 1，否则执行程序段 2。

程序中为了兼容 8 位 LCD1602 和 4 位 LCD1602，所以使用条件选择编译来确定是执行 8 位的 LCD1602 驱动代码还是 4 位的 LCD1602 驱动代码。我们例程中默认没有定义 LCD1602_4PINS 宏，即 LCD1602 是 8 位数据口，如果你手上的 LCD1602 是 4 位数据口（在 LCD1602 屏上可以看到加了一块转接板）则需要定义 LCD1602_4PINS 宏。这个可以在 lcd.h 头文件中看到，如下所示：

当使用的是 4 位数据传输的时候定义，

使用 8 位取消这个定义

```

/*****

```

```
//#define LCD1602_4PINS
```

在应用中我们主要调用 LcdInit()、LcdWriteCom() 和 LcdWriteData() 函数来操作 LCD1602 显示。LcdInit() 函数实现了 LCD1602 初始化操作，即写入一些特定的指令到 LCD1602 内，这些指令功能可在前面介绍的指令查看。在初始化函数中，默认将显示位置设置为第一行的第一个地址。LcdWriteCom() 和 LcdWriteData() 函数是 LCD1602 的写命令和写数据操作，其代码是按照时序图实现。对 LCD1602 的显示操作在后面 main.c 文件内还会有。

程序中我们看到有 LCD1602_4PINS、LCD1602_DATAPINS、LCD1602_E、LCD1602_RS 和 LCD1602_RW 的操作，该变量就是我们在 lcd.h 头文件内定义的，代码如下：

```
#ifndef __LCD_H_
#define __LCD_H_
/*****
当使用的是 4 位数据传输的时候定义，
使用 8 位取消这个定义
*****/
//#define LCD1602_4PINS

/*****
包含头文件
*****/
#include<reg52.h>

//---重定义关键词---//
#ifndef uchar
#define uchar unsigned char
#endif

#ifndef uint
#define uint unsigned int
#endif

/*****
PIN 口定义
*****/
#define LCD1602_DATAPINS P0
sbit LCD1602_E=P2^7;
sbit LCD1602_RW=P2^5;
sbit LCD1602_RS=P2^6;
```

```

/*****
函数声明
*****/
/*在 51 单片机 12MHZ 时钟下的延时函数*/
void Lcd1602_Delay1ms(uint c);    //误差 0us
/*LCD1602 写入 8 位命令子函数*/
void LcdWriteCom(uchar com);
/*LCD1602 写入 8 位数据子函数*/
void LcdWriteData(uchar dat)    ;
/*LCD1602 初始化子程序*/
void LcdInit();

#endif

```

同样，头文件内通常都是用来定义管脚和声明函数或者变量的。下面来看下 main.c 文件代码，main.c 内代码如下：

```

/*****
*****

*
实验现象：下载程序后插上 LCD1602 液晶在开发板上，即可显示

接线说明：（具体接线图可见开发攻略对应实验的“实验现象”章节）


注意事项：根据自己使用的 LCD1602 是否带有转接板，如果带有转接板的即为 4 位，需在 LCD.H 头文件中
将宏#define LCD1602_4PINS 打开，我们这里使用的 LCD1602 是 8 位，所以默认将该宏注释

*****
*****

*/

#include "reg52.h"           //此文件中定义了单片机的一些特殊功能寄存器
#include "lcd.h"

typedef unsigned int u16;    //对数据类型进行声明定义
typedef unsigned char u8;

u8 Disp[]=" Pechin Science ";

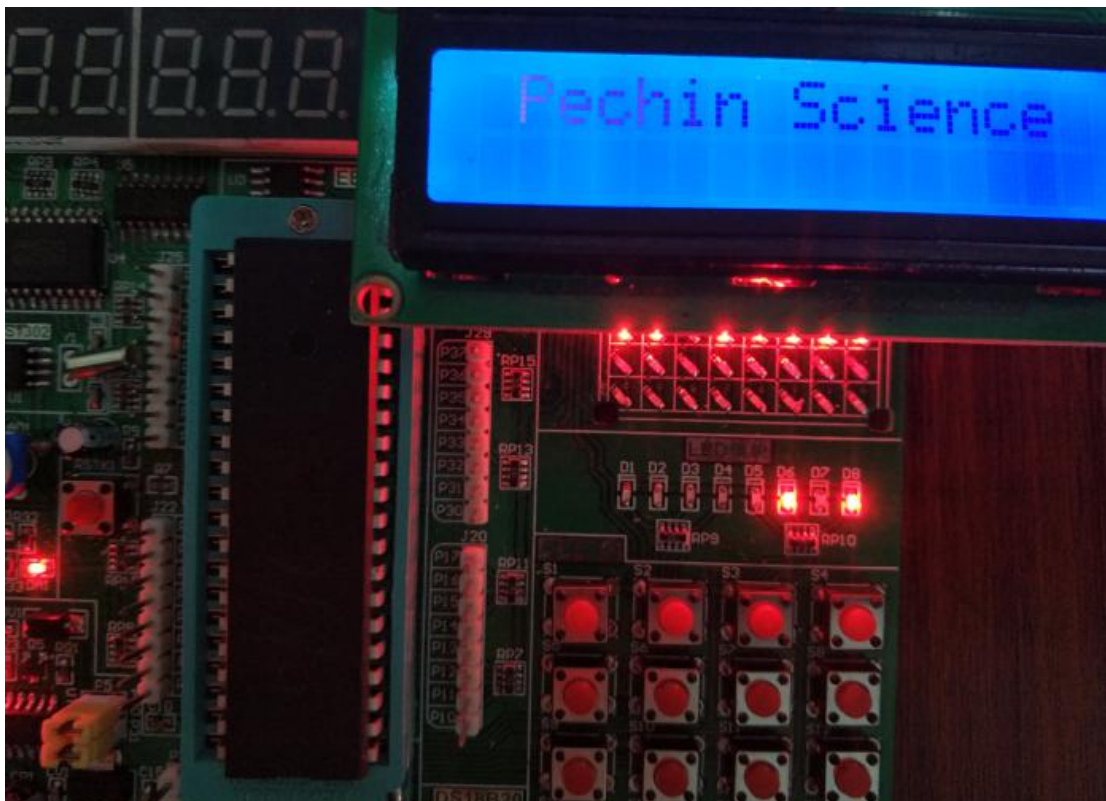
```

```
/******  
*****  
* 函 数 名      : main  
* 函数功能      : 主函数  
* 输    入      : 无  
* 输    出      : 无  
*****  
*****/  
void main(void)  
{  
    u8 i;  
    LcdInit();  
    for(i=0;i<16;i++)  
    {  
        LcdWriteData(Disp[i]);  
    }  
    while(1);  
}
```

main.c 文件内代码非常少也很简单，首先将 51 单片机的头文件和 lcd.h 的头文件包含进来，然后定义相关变量。主函数功能非常简单，首先调用 LcdInit() 函数完成 LCD1602 初始化操作，在初始化函数中，默认将显示位置设置为第一行的第一个地址。然后在 for 循环内发送 Disp 数组内的 16 个字符到 LCD1602 上显示，最后在 while 循环内不执行任何功能代码。

24.4 实验现象

使用 USB 线将开发板和电脑连接成功后，把编写好的程序编译后将编译产生的 .hex 文件烧入到芯片内，按照如下图所示的接线方式可以看到：LCD1602 第一行显示“ Pechin Science ”。



课后作业

(1) 使用 LCD1602 显示时钟。

(温馨提示: LCD1602 只能以 ASCII 字符显示, 所以在显示时钟数值时, 需要将时钟数据转换为字符, 比如数值 1 转换为 ASCII 字符可这样操作: $1+0X30$, 或者 $1+' '$)

第 25 章 LCD12864 液晶显示实验

上一章我们学习了如何在 LCD1602 显示字符数据，我们知道 LCD1602 是字符型显示器，它不能显示汉字图形等。对于需要显示汉字或图形的项目中，LCD1602 不能实现，因此本章就来介绍一个可以实现字符、汉字、图形等显示的液晶屏——LCD12864。LCD12864 分为带字库和不带字库两种，开发板配置的是不带字库 LCD12864。我们开发板上集成了一个 LCD12864 液晶显示器接口，它可兼容带字库和不带字库两种屏，将配置的 LCD12864 液晶对应插入即可。本章要实现的功能是：系统运行时，LCD12864 上显示汉字字符信息。学习本章可以参考前面的实验章节内容。若结合视频学习效果更佳。本章分为如下几部分内容：

25.1 LCD12864 介绍

25.2 硬件设计

25.3 软件设计

25.4 实验现象

25.1 LCD12864 介绍

25.1.1 LCD12864 简介

LCD12864 液晶屏结构上与 LCD1602 一样，只是在行列数与现实像素上区别很大。LCD12864，以下简称 12864，注意区分 LCD1602 和 LCD12864。12864 是 64 行 128 列，当然也有可能会设计成 64 列 128 行，这里的行列不像 1602 那样，1602 是按照八行四列标准英文字符格式，以一行十六个字符，两列字符命名，而 12864 是以 128 列像素，64 行像素，也就是有 128*64 个像素点组成。就好比是 128 列 64 行的点阵。需要一行一列的去显示像素点。

常用的 12864 分为带字库和不带字库两种。以下分别介绍：

1、带字库 LCD12864



上图左侧为带字库 12864 显示字符和汉字，右侧为图形模式显示。注意：这里图形模式虽然显示了字符和汉字，但是并不是使用字库里的，显示的方法也是多样化的。

对于带字库 LCD12864，最常见的标志就是在屏幕背后，会有存放字库的芯片。如下图所示：



2、不带字库 LCD12864



这种不带字库的 LCD12864 背面没有芯片，一般为转接板，将 12864 显示屏转接到开发板。这种显示屏的操作就像带字库的 12864 操作图片形式一样。所有的显示都需要取模，取模方法和点阵取模一样。我们开发板所配置的是不带字库的 LCD12864, 从外形上看其体积比带字库 LCD12864 要小很多，因此我们称之为 MiniLCD12864。

25.1.2 MiniLCD12864 介绍

开发板上配置的 MiniLCD12864 是不带字库的，要想显示汉字或其他字符需通过取字模方式来实现。要让其显示首先还得初始化，这个和 LCD1602 原理类似，即通过对其内部一些特殊寄存器设置实现特定功能。我们配置的 MiniLCD12864 内部驱动芯片是 ST7565P，其数据手册以及转换板原理图在实验例程“\5--实验程序\基础实验例程\实验 30: LCD12864 液晶”内可以查看。

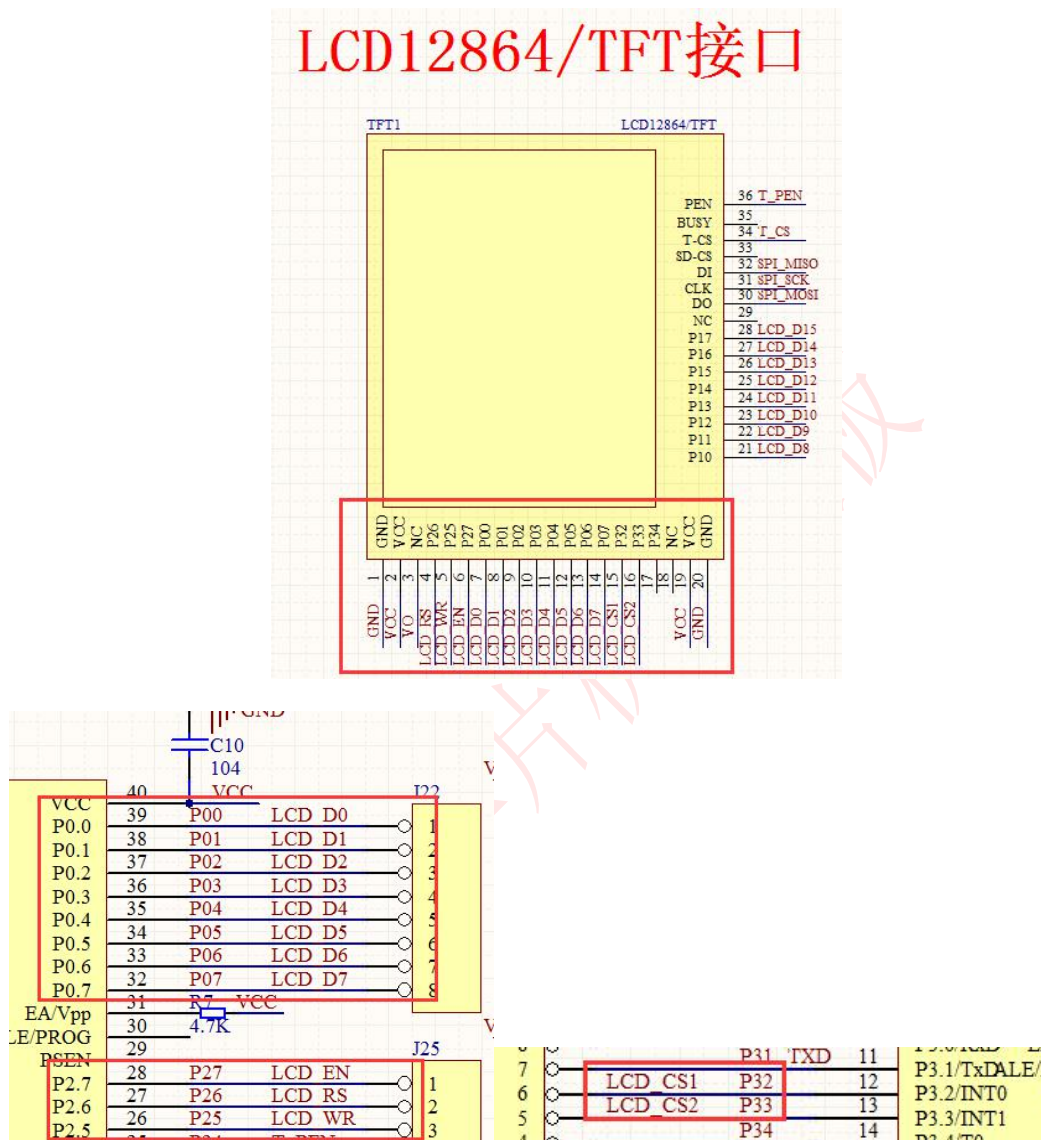
ST7565P 数据手册是液晶屏内核手册，里面有驱动方法和指令和程序有关的数据。通常液晶屏的数据手册都是英文的，但是其内容并不复杂，我们要查看的通常都是某几个寄存器内容，没有必要把手册整篇都看完。而且寄存器的介绍都是一些较容易看懂的单词，如果不明白可以借助翻译软件理解。在这里要降调一下，既然打算进入嵌入式行业，一定要习惯看英文资料，以后进入高级单片机或者 ARM 的学习会遇到很多英文资料的查阅。关于 ST7565P 的介绍大家可以查阅手册，这里就不列举了。

25.2 硬件设计

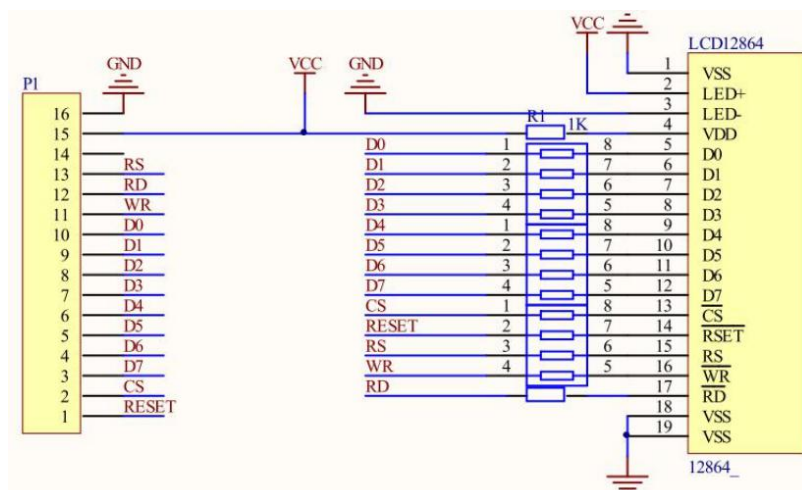
本实验使用到硬件资源如下：

(1) LCD12864 液晶

开发板上集成了一个 LCD12864 液晶接口，下面我们来看下开发板上 LCD12864 液晶接口电路，如下图所示：



LCD12864 液晶屏转接板原理图如下所示：



上图中的 LCD12864 是液晶屏 LCD12864 的排线接口，焊接在转接板的顶层，而图中的电阻 R1 及排阻都焊接在转接板的底层，P1 插针接口是 LCD12864 液晶屏的引出控制口，直接与开发板上的接口连接。

25.3 软件设计

本章所要实现的功能是：系统运行时，LCD12864 上显示汉字字符信息。

我们直接复制上一章创建的工程，在此模板基础上进行程序开发。为了能够与开发攻略教程对应，将复制过来的模板文件夹重新命名为“实验 30：LCD12864 液晶”。打开工程文件夹，发现里面有 3 个工程实验文件夹，分别是：刷屏、显示文字、显示图片。刷屏实验是最基础的，对于初学者可以先看下这个实验了解 MiniLCD12864 是如何初始化的，然后显示文字实验，该实验是在刷屏基础上实现了文字显示函数。最后显示图片实验是在显示文字基础上实现了图片显示函数。这里我们以显示文字实验为例介绍 MiniLCD12864 的驱动方法。

打开该工程，里面多了 3 个文件：st7565.c 源文件、st7565.h 头文件和 charcode.h 头文件。前两个文件保存的是 MiniLCD12864 的驱动程序，最后一个头文件保存的是显示汉字及其他字符的字模数据，这些数据是通过取模软件实现，与 LED 点阵一样。至于如何创建多文件工程，在前面章节已经介绍过，这里就不再重复。下面我们就来分析下 MiniLCD12864 的驱动。

25.3.1 显示文字

(1) 设置 I/O

在程序里设置要使用的 I/O，例程里使用如下定义：

```
//--定时使用的 I/O 口--//  
#define DATA_PORT P0  
sbit LCD12864_CS = P3^2;  
sbit LCD12864_RSET = P3^3;  
sbit LCD12864_RS = P2^6;  
sbit LCD12864_RW = P2^7;  
sbit LCD12864_RD = P2^5;
```

这样定义的好处是，以后修改 I/O 时更方便，移植性更好。可以看到数据口是 P0 口连接，其它五个是控制 I/O。

(2) 驱动函数声明

首先必须知道需要用到什么函数，定义函数如下：

```
//--定义全局函数--//  
void LcdSt7565_WriteCmd(cmd);  
void LcdSt7565_WriteData(dat);  
void Lcd12864_Init();  
void Lcd12864_ClearScreen(void);  
uchar Lcd12864_Write16CnCHAR(uchar x, uchar y, uchar *cn);
```

这些函数分别用来写指令，写数据，初始化，清屏和输出字符。

(3) 函数定义

1、写指令函数；

```
void LcdSt7565_WriteCmd(cmd)  
{  
    LCD12864_CS = 0; //chip select, 打开片选  
    LCD12864_RD = 1; //disable read, 读失能  
    LCD12864_RS = 0; //select command, 选择命令  
    LCD12864_RW = 0; //select write, 选择写模式  
    _nop_();  
    _nop_();  
    DATA_PORT = cmd; //put command, 放置命令  
    _nop_();  
    _nop_();  
    LCD12864_RW = 1; //command writing, 写入命令  
}
```

2、写数据函数；

```
void LcdSt7565_WriteData(dat)
```



```
{
    LCD12864_CS = 0; //chip select, 打开片选
    LCD12864_RD = 1; //disable read, 读失能
    LCD12864_RS = 1; //select data, 选择数据
    LCD12864_RW = 0; //select write, 选择写模式
    _nop_();
    _nop_();
    DATA_PORT = dat; //put data, 放置数据
    _nop_();
    _nop_();
    LCD12864_RW = 1; //data writing, 写数据
}
```

写指令和数据使用的是同一个时序图，在下图中 A0 是数据指令控制线，在程序里使用的是 RS 宏定义；/CS1 是片选线在程序中是 CS 宏定义，CS2 是复位线，是 RSET 宏定义；图中读写控制线是 WR/RD，程序中是 RW 宏定义；图中的 D[0:7]数据总线，在程序中是 DATA_PORT 宏定义。

System Bus Read/Write Characteristics 1 (For the 8080 Series MPU)

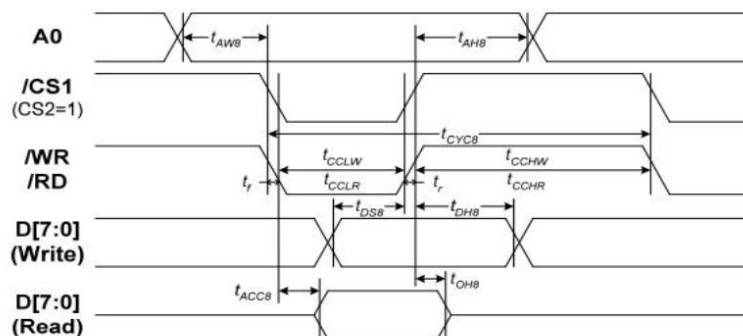


Figure 37

Table 24

(VDD = 3.3V, Ta = -30 to 85°C)

Item	Signal	Symbol	Condition	Rating		Units
				Min.	Max.	
Address hold time	A0	t _{AHB}		0	—	
Address setup time		t _{AWB}		0	—	
System cycle time		t _{CYC8}		240	—	

3、初始化函数；

```

/*****
*****
* 函数名      : LCD12864_Init
* 函数功能    : 初始化 12864
* 输入        : 无
* 输出        : 无
* 说明        : LCD12864 的命令指令可以查看例程文件夹下的
《ST7565p 数据手册》
*              * 的第 51 页的位置。
*****

```



```

*****/

void Lcd12864_Init()
{
    uchar i;
    LCD12864_RSET = 0;
    for (i=0; i<100; i++);
    LCD12864_CS = 0;
    LCD12864_RSET = 1;

    //-----Star Initial Sequence-----//
    //-----程序初始化设置，具体命令可以看文件夹下---//

    //---软件初始化---//
    LcdSt7565_WriteCmd(0xE2); //reset
    for (i=0; i<100; i++); //延时一下

    //---表格第 8 个命令，0xA0 段（左右）方向选择正常方向（0xA1 为
    反方向）---//
    LcdSt7565_WriteCmd(0xA1); //ADC select segment direction

    //---表格第 15 个命令，0xC8 普通(上下)方向选择选择反向，0xC0
    为正常方向---//
    LcdSt7565_WriteCmd(0xC8); //Common direction

    //---表格第 9 个命令，0xA6 为设置字体为黑色，背景为白色---//
    //---0xA7 为设置字体为白色，背景为黑色---//
    LcdSt7565_WriteCmd(0xA6); //reverse display

    //---表格第 10 个命令，0xA4 像素正常显示，0xA5 像素全开---//
    LcdSt7565_WriteCmd(0xA4); //normal display

    //---表格第 11 个命令，0xA3 偏压为 1/7, 0xA2 偏压为 1/9---//
    LcdSt7565_WriteCmd(0xA2); //bias set 1/9

    //---表格第 19 个命令，这个是个双字节的命令，0xF800 选择增压为
    4X;---//
    //---0xF801, 选择增压为 5X，其实效果差不多---//
    LcdSt7565_WriteCmd(0xF8); //Boost ratio set
    LcdSt7565_WriteCmd(0x01); //x4

    //---表格第 18 个命令，这个是个双字节命令，高字节为 0X81，低字
    节可以---//
    //---选择从 0x00 到 0X3F。用来设置背景光对比度。---//

```

```

LcdSt7565_WriteCmd(0x81); //V0 a set
LcdSt7565_WriteCmd(0x23);

//--表格第 17 个命令，选择调节电阻率--//
LcdSt7565_WriteCmd(0x25); //Ra/Rb set

//--表格第 16 个命令，电源设置。--//
LcdSt7565_WriteCmd(0x2F);
for (i=0; i<100; i++);

//--表格第 2 个命令，设置显示开始位置--//
LcdSt7565_WriteCmd(0x40); //start line

//--表格第 1 个命令，开启显示--//
LcdSt7565_WriteCmd(0xAF); // display on
for (i=0; i<100; i++);

}

```

初始化函数是对时序线的初始化和初始指令的写入。这些指令的意义都在程序有标注，详细的解释可以查看“ST7565p 数据手册.pdf”。

4、清屏函数：

```

void Lcd12864_ClearScreen(void)
{
    uchar i, j;
    for(i=0; i<8; i++)
    {
        //--表格第 3 个命令，设置 Y 的坐标--//
        //--Y 轴有 64 个，一个坐标 8 位，也就是有 8 个坐标--//
        //所以一般我们使用的也就是从 0xB0 到 0xB7,就够了--//
        LcdSt7565_WriteCmd(0xB0+i);
        //--表格第 4 个命令，设置 X 坐标--//
        //--当你的段初始化为 0xA1 时，X 坐标从 0x10,0x04 到
        0x18,0x04,一共 128 位--//
        //--当你的段初始化为 0xA0 时，X 坐标从 0x10,0x00 到
        0x18,0x00,一共 128 位--//
        //--在写入数据之后 X 坐标的坐标是会自动加 1 的，我们初
        始化使用 0xA0 所以--//
        //--我们的 X 坐标从 0x10,0x00 开始---//
        LcdSt7565_WriteCmd(0x10);
        LcdSt7565_WriteCmd(0x04);
        //--X 轴有 128 位，就一共刷 128 次，X 坐标会自动加 1，
        所以我们不用再设置坐标--//
        for(j=0; j<128; j++)
    }
}

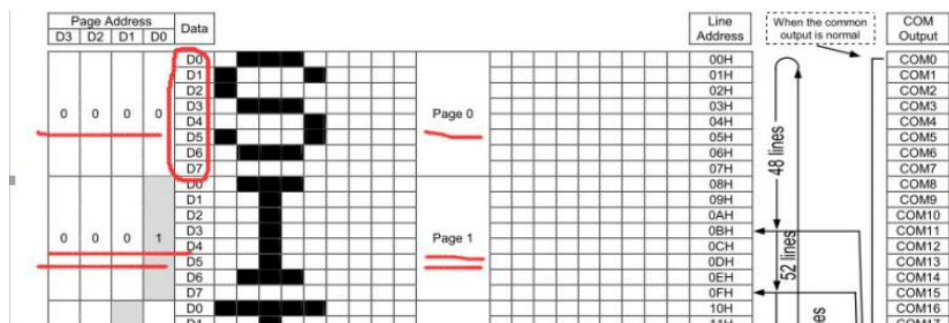
```

```

{
    LcdSt7565_WriteData(0x00); //如果设置背景为白色时， 清屏
    选择 0xFF
}
}
}

```

在清屏函数中，指令 `0xB0+i`，是“页”地址设置，从下图中可以看出页地址使用低四位配置。一共有 8 页，每页 8 行，共 64 行。从上图中还可以看出，数据是纵向的，这里可以认为，在取模的时候需要纵向取模。可以知道设置了页地址以后，加上数据的每一位，就能控制到每一行，也就等同于是设置了行地址。



从上图得到的是行，还需要设置列，清屏函数中使用 `0x10, 0x04` 写入初始列。

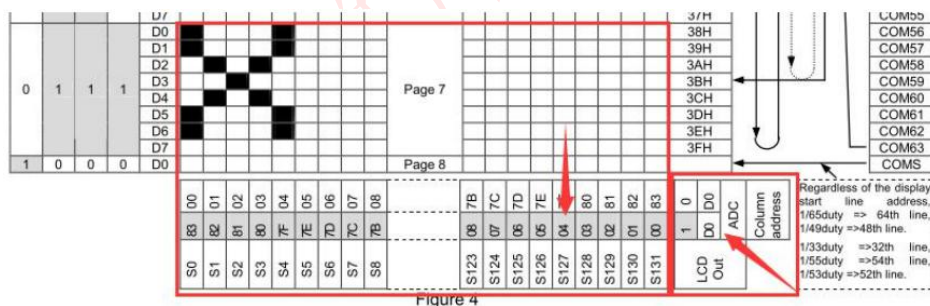


图 32.1

Column Address Set

This command specifies the column address of the display data RAM shown in Figure 4. The column address is split into two sections (the higher 4 bits and the lower 4 bits) when it is set (fundamentally, set continuously). Each time the display data RAM is accessed, the column address automatically increments (+1), making it possible for the MPU to continuously read from/write to the display data. The column address increment is topped at 83H. This does not change the page address continuously. See the function explanation in "The Column Address Circuit," for details.

	E R/W			D7 D6 D5				D4 D3 D2 D1 D0				A7 A6 A5 A4 A3 A2 A1 A0				Column address
	A0	/RD	/WR													
High bits →	0	1	0	0	0	0		1	A7	A6	A5	A4	0	0	0	0
Low bits →								0	A3	A2	A1	A0	0	0	0	1
													0	0	0	0
													0	0	0	1
													0	0	0	0
													0	0	0	1
													1	0	0	0
													0	0	1	1

图 32.2

在图 32.1 的右侧有设置列的增长方向，在初始化函数中设置的 0xA1，反方向，表示设置列地址应该从右至左开始。图中箭头指向 04，这里是表示程序中设置的指令 0x10, 0x04，意思是从这一列向左到 0x83，也就是倒数第 S127 列到第 S0 列，共 128 列。在图 32.2 中可以看到设置列的指令使用方法，是先把列数据的高四位放在 0x1x 的低位中发送，紧接着把列数据的低四位放到 0x0x 的低位中发送。在 12864 内部会自动组合成右侧的 A7-A0 的列地址，一共可操作 132 列。但显示只能有 128 列，故需要设置从第几列开始显示。程序中设置的是右侧第四列开始，就去掉了四列。这四列是显示不出来的。

函数最后是输出数据 0x00，一共输出 128 次，可以看到这里的列是在自动增长，这个增长方向是有之前的初始化函数设置的。然后就是循环 8 次，每次修改页地址到下一页，列地址从新开始。这样一次循环就完成一次清屏。

5、写字符函数；

```

uchar Lcd12864_Write16CnCHAR(uchar x, uchar y, uchar *cn)
{
    uchar j, x1, x2, wordNum;

    //--Y 的坐标只能从 0 到 7，大于则直接返回--//
    if(y > 7)
    {
        return 0;
    }

    //--X 的坐标只能从 0 到 128，大于则直接返回--//
    if(x > 128)
    {
        return 0;
    }
    y += 0xB0;    //求取 Y 坐标的值
    //--设置 Y 坐标--//
    LcdSt7565_WriteCmd(y);
    while ( *cn != '\0') //在 C 语言中字符串结束以 '\0' 结尾
    {

        //--设置 Y 坐标--//
        LcdSt7565_WriteCmd(y);

        x1 = (x >> 4) & 0x0F;    //由于 X 坐标要两句命令，分高低 4 位，
        所以这里先取出高 4 位
    }
}

```

```

x2 = x & 0x0F;          //去低四位
//--设置 X 坐标--//
LcdSt7565_WriteCmd(0x10 + x1);    //高 4 位
LcdSt7565_WriteCmd(0x04 + x2);    //低 4 位

for (wordNum=0; wordNum<50; wordNum++)
{
    //--查询要写的字在字库中的位置--//
    if ((CN16CHAR[wordNum].Index[0] == *cn)
        &&(CN16CHAR[wordNum].Index[1] == *(cn+1)))
    {
        for (j=0; j<32; j++) //写一个字
        {
            if (j == 16)    //由于 16X16 用到两个 Y 坐标,当大于
等于 16 时,切换坐标
            {
                //--设置 Y 坐标--//
                LcdSt7565_WriteCmd(y + 1);

                //--设置 X 坐标--//
                LcdSt7565_WriteCmd(0x10 + x1);    //高 4 位
                LcdSt7565_WriteCmd(0x04 + x2);    //低 4 位
            }
            LcdSt7565_WriteData(CN16CHAR[wordNum].Msk[j]);
        }
        x += 16;
    } //if 查到字结束
} //for 查字结束
cn += 2;
} //while 结束
return 1;
}

```

该函数的目的是在一个大小 16*16 的像素窗口显示一个字,有可能是中文,也可以是字符(这里字符要做成 16*16 的)。这个函数有三个参数,一个是 X 坐标,即页地址,一个是 Y 坐标,即列地址,最后一个是一个指针,是字的索引。函数一开始判断页地址和列地址时候超出范围,超出范围就退出该函数,否则向下执行。

首先求取这个字要从哪一页。

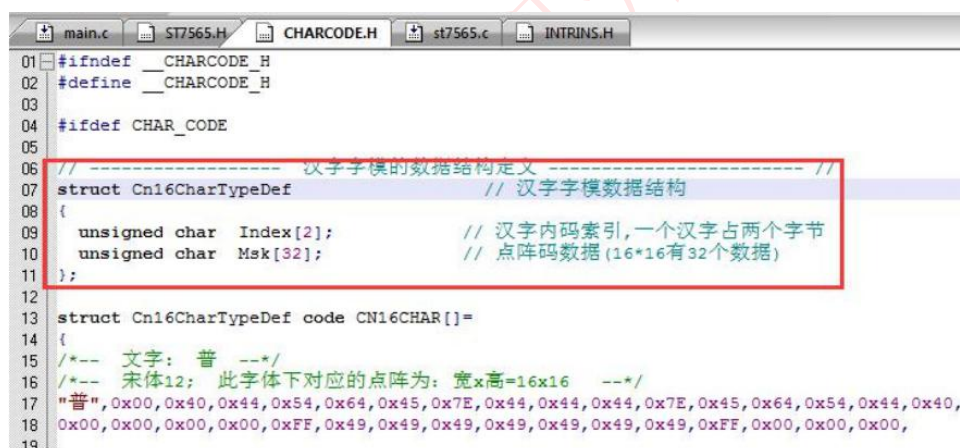
然后是进入写数据循环, 这里判断*cn!=' /0',表示该函数可以连续写入字(字串的结尾一般是' /0')。在函数结尾有 cn += 2;这里字以两个地址增

长，也就是说如果不是汉字是字符，就应该用字符加空格做索引。

然后在此写入页地址，计算从哪一列开始写，在写完一个字以后，重新写入页地址，且列地址以加 16 列变化，表示下一个连续的字写在这个字的后面。

然后有一个 for(wordNum=0;wordNum<50;wordNum++)，这个循环的目的是从字库文件里找字，这个 wordNum 限制在 50 以内，表示字库里的字不能超过 50 个，如果要超过，就需要修改这里。循环 50 次来查找这个字。如果没找到就不显示，继续查找下一个字。

如何判断是否有这个字。在字库文件里有一个汉字字模结构体，该结构体前两个位字的索引，对应的是这个汉字的字符串，在查找时对比汉字的两个字节是否相等。如果都相等，表示这个汉字在字库里。如果是使用的字符，那么第一个对比的是字符第二个是空格，如果两个相等，表示字库里有这个字符。在结构体第二个元素是一个 32 字节的字符串，是存放字模的，当找到相应字的索引时，就可以调用该字的字模来显示这个字。

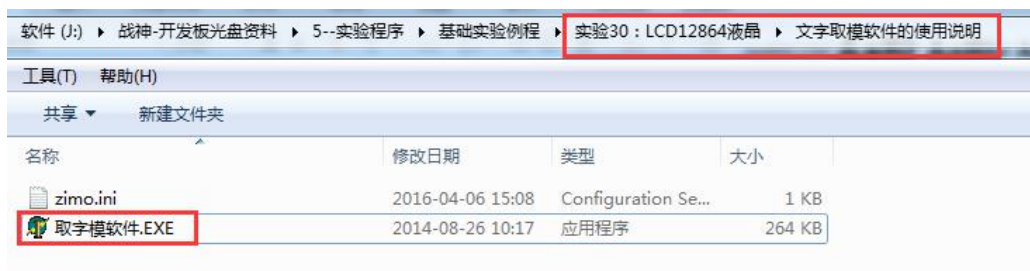


如果查到字库有这个字，就会进入循环写数据中。一个字像素点是 16*16 个，一个字节站 8 个，一共 32 个字节数据，也就是要写 32 次。在写的过程中是先写上一页 16 个数据，当数据超过 16 个时，跳转到下一页开始写 16 个数据，两页上下合并为 16*16 个点，32 个数据。写完一个字，就进入到查找下一个字，直到字串结束写字循环。

(4) 取模软件使用

取模软件的使用方法在前面学习 LED 点阵实验的时候就已经介绍过，这里我们再次介绍下。

1, 打开取字模软件



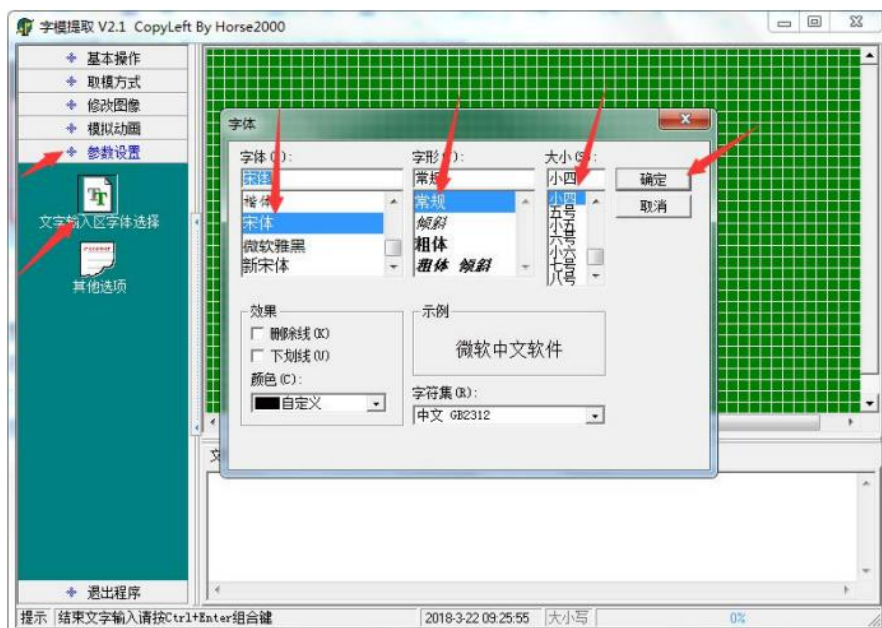
打开界面如下：



在上图中，左侧是各种设置区，这里可以设置取模方向、取模方式等等必要参数。右上侧是显示预览区，会显示这个字在显示屏上需要点亮的点和区域大小。右下侧是数据输入和输出区，在“文字输入区”输入需要取的字或者字符，按 Ctrl+Enter 键，能够把文字的预览显示在预览区。当点击生成方式的时候能够在点阵生成区显示字模。这个操作接下来会详细介绍。

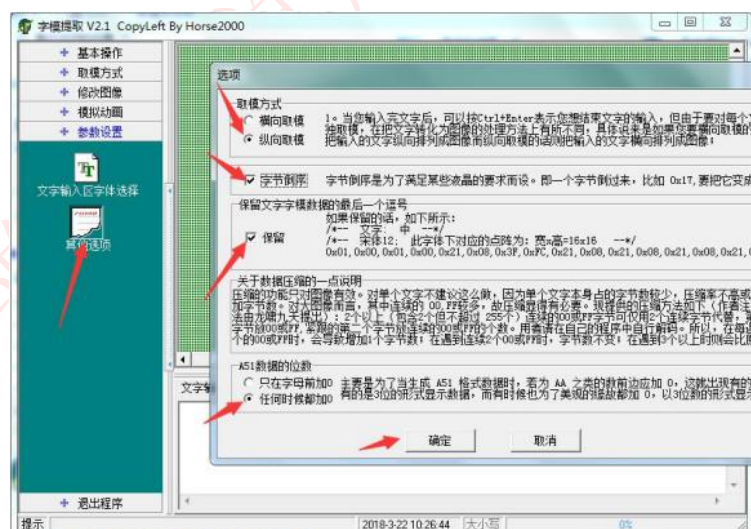
1) 设置字体

首先在设置区选“参数设置”->文字输入区字体选择，这里不用选择“基本操作”->新建图像，在输入数字以后按 Ctrl+Enter 就能产生新的图像。打开字体选择以后弹出如下窗口。

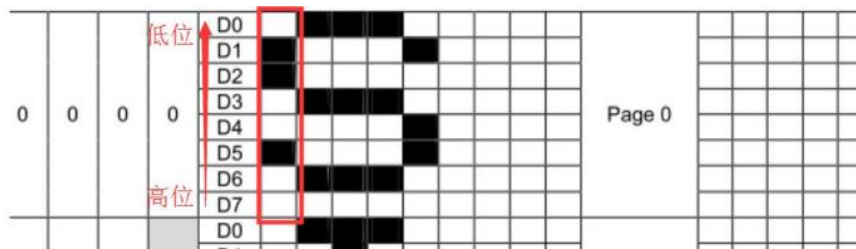


在窗口里设置字体如图所示，在这种字体下生成的字模大小为 16*16。如果字体不对，生成的大小就不一定是 16*16 的，后面会介绍如何修改修改字体生成其它大小的字模。注意字模的大小和输出字的函数有关。这里设置完毕后，点击“确定”。

2) 设置“其它选项”



在“参数设置”->其它选项，设置“纵向取模”，原因是 12864 中数据是列方向向下排列。如下图所示，纵向为一个数据，这里确定为纵向取模。



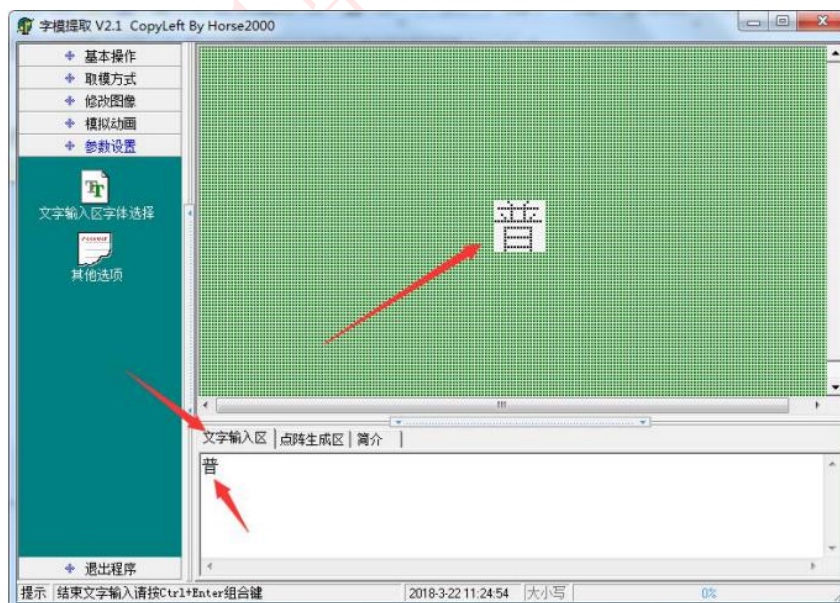
“字节倒序”的确定是看数据高低位，如果不倒序，则从上到下是由高到低，从手册上可以看出，一个字节中数据是由下到上是由高位到低位，故取模的时候要设置选中倒序。当然有时候在程序中找到数据上下颠倒的寄存器设置，也可以不设置倒序，这里需要开发人员自己去思考。

保留最后的逗号，这个可选，也可以不选，需要看你在程序里如何存字库，一般连续存放，最好选上，如果没选，就在程序中输入一个也行。如果只有一个字，就可以不选择。

“A51 的位数”这里可以不用管，我们程序是 C51 格式的，不需要生成 A51 格式。A51 格式适用于汇编编程，是否需要加 0，得看编译方式和显示器，这里不做讲解。

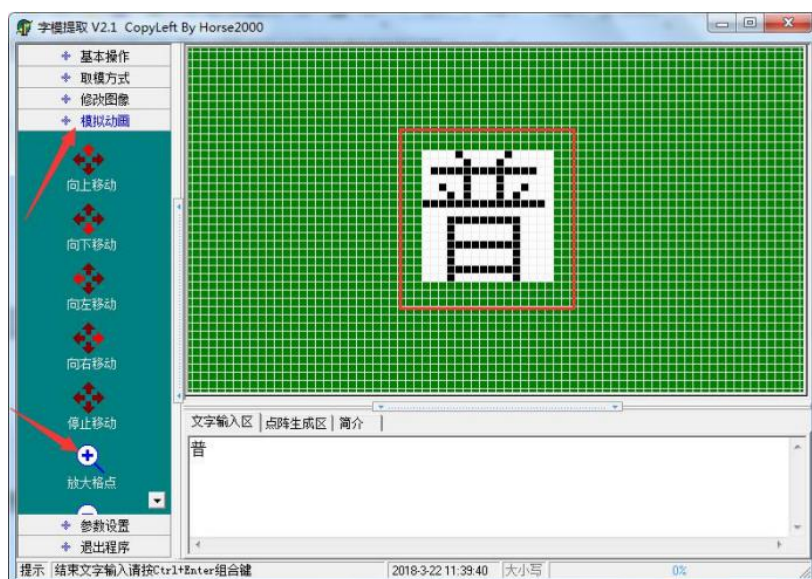
设置完毕后，点击“确定”完成设置。

3) 输入文字



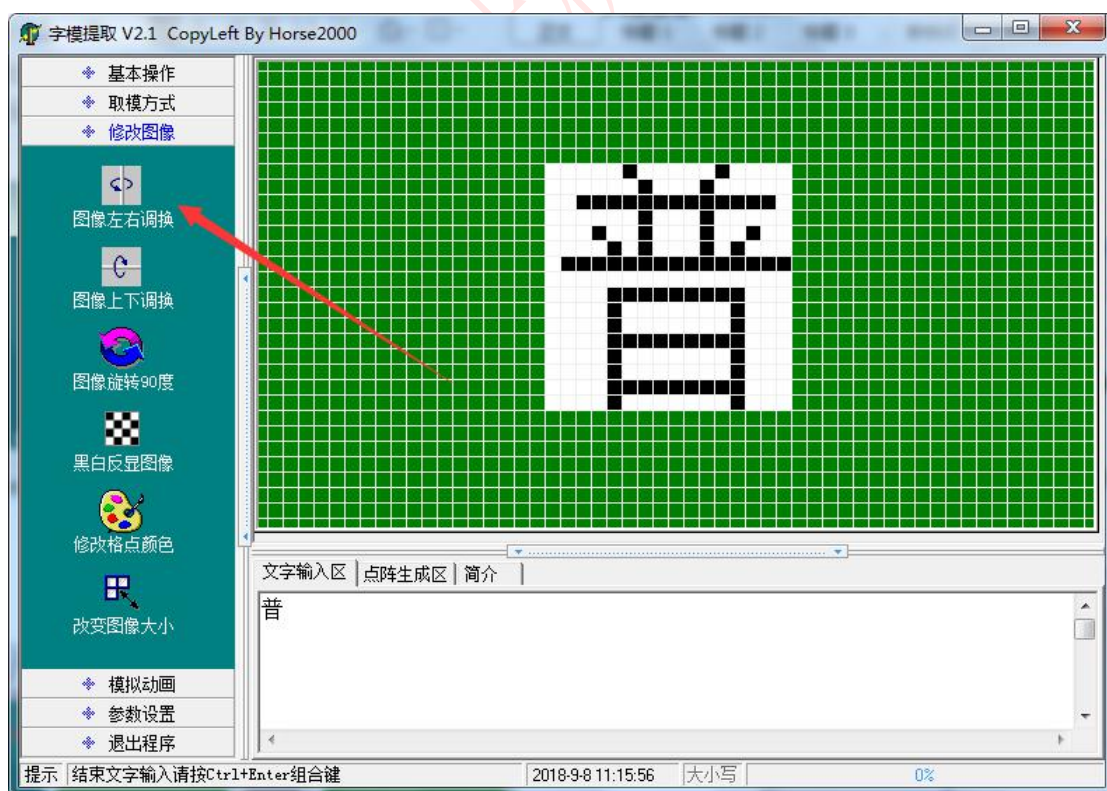
在文字输入区输入汉字，如上图所示，输入需要的汉字，然后按 Ctrl+Enter 键，在预览区会显示这个字的样式，如果不喜欢，可以进行自主修改，建议在学会使用以后再进行。这时候显示的字很小，需要放大。在“模拟动画”设置项里

有“放大格点”，点击可放大预览区。

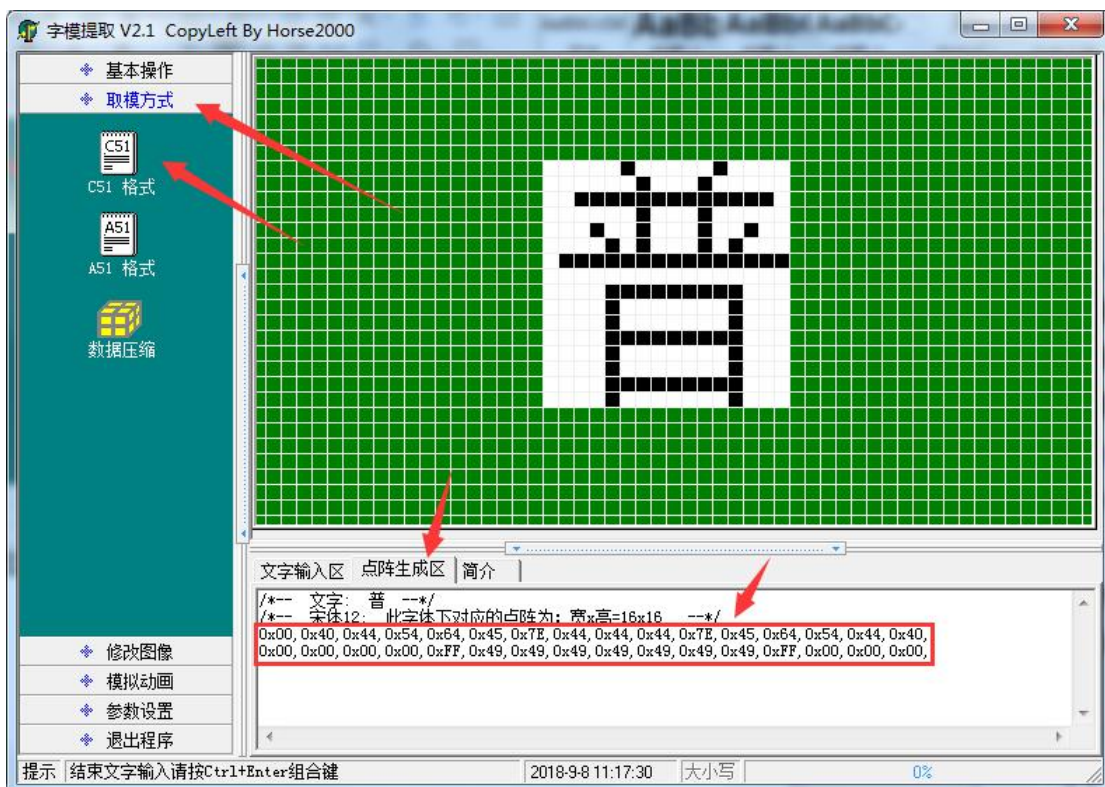


4) 左右调换字模

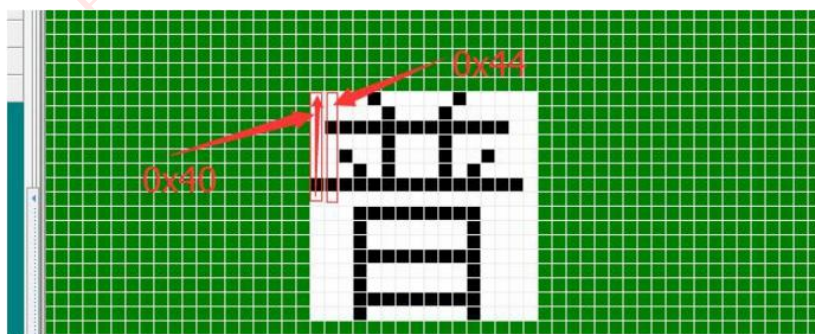
在“修改图像”区选择图像左右调换，文字就会相应的左右调换，如果不左右调换则显示在屏幕上的文字就会左右反过来显示，这个是根据我们程序来设置的。



5) 生成字模

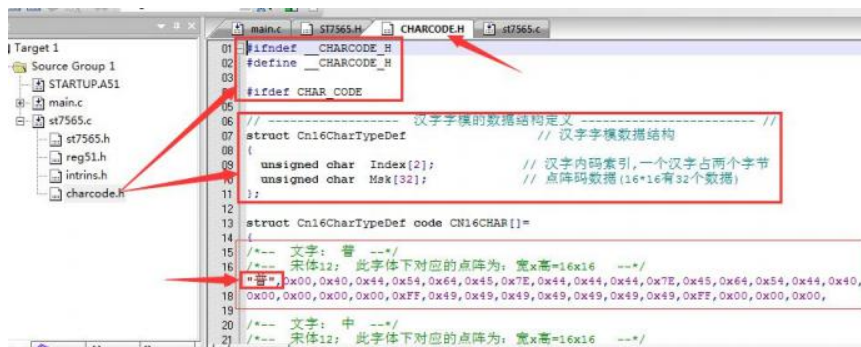


在“设置区”->取模方式里点击“C51 格式”，就会在“输出区”->点阵生成区里有输出对应字的字模。输出区里有这个字和这个字的像素大小，这里是 16*16 的。还有两行是字模，字模一共 32 个字节。第一个字节是 0x40，从下图可以看出第一个字节的八个点从下到上依次是 0100 0000，也就是 0x40，第二个自己从上到下是 0100 0100，也就是 0x44。由此可以看出取模方向是纵向的是从左至右一个字节一个字节的生成的，因此在程序里设计的是页设置以后，写 16 列，在换页写十六列，和这里是对应的。如果两个不对应，就得修改成对应的。



这时候就做出了一个字的字模。现在就是做一个字模文件，一般是做成头文件直接使用。

5) 制作字模文件



这里可以按照例程写头文件，做一个宏定义`#ifdef CHAR_CODE`，这里用宏定义是为了在使用的时候，如果不使用文字，可以不定义这个，就不会被编译进来，是程序易于裁剪和编辑。

从上图可以看到在结构体数组中复制了之前生成的字模，然后在前面添加了索引。这里索引是用双引号加输入的字，做成一个字符串，占用两个字节，在结构体中是放到索引中的。从而只做了一个简单的字库。

如果需要多个，可以重复上述步骤。一般一个数组中使用同一个大小的字做字库。可以看看例程里。

注意：例程中查询字数最多 50 个，字库里字数可能不止 50 个，可以设计更多，如果有足够大的空间，可存放更多个字，这里查询的数量就应该变大，或者以找到改字为准。这个需要用户自己去思考。

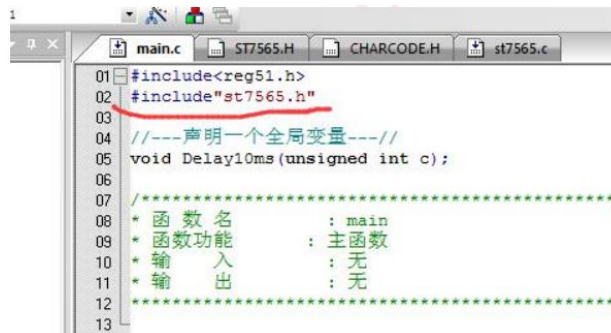
(5) 12864 怎么显示字

如何显示字，需要在函数中对输出字的函数进行调用。

1) 添加头文件支持



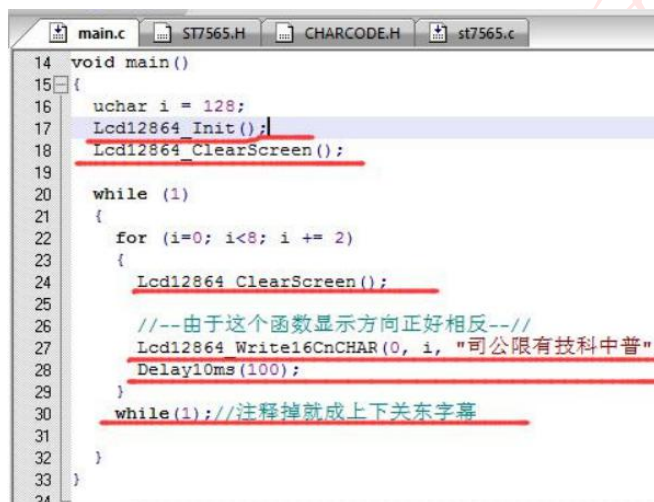
这里添加宏定义，即可包含字库头文件。也会将 ST7565.c 中有关于输出字的函数进行编译。



```
01 #include<reg51.h>
02 #include"st7565.h"
03
04 //---声明一个全局变量---//
05 void Delay10ms(unsigned int c);
06
07 /*
08 * 函数名      : main
09 * 函数功能    : 主函数
10 * 输入        : 无
11 * 输出        : 无
12 */
13
```

在主函数源文件这里添加 ST7565.h 的头文件。这样在主函数里就可以调用显示函数了。

2) 调用函数进行显示



```
14 void main()
15 {
16     uchar i = 128;
17     Lcd12864_Init();
18     Lcd12864_ClearScreen();
19
20     while (1)
21     {
22         for (i=0; i<8; i += 2)
23         {
24             Lcd12864_ClearScreen();
25
26             //---由于这个函数显示方向正好相反---//
27             Lcd12864_Write16CnCHAR(0, i, "司公限有技科中普");
28             Delay10ms(100);
29         }
30         //while(1); //注释掉就成上下关东字幕
31     }
32 }
33
34
```

这里首先调用初始化函数，对 LCD12864 内部寄存器进行初始化配置。然后调用清屏函数，这里清屏是用的是函数里的设置颜色。然后进入了循环调用，这里只有四次循环。用 $i < 8$ ，是为了表示有八页，每次翻两页。然后是重新清屏，这里是为了显示下一行数据，如果没有清屏，下一行显示的时候，上一行的数据是会一直显示的。接着是输出要显示的数据，调用了输出字的函数，函数参数首先是起始列，然后是页，这里页地址按照两页在变化，然后是数据，历程中的字库仅有需要使用的几个字，如果用户只做了自己的字库，就可以显示其他字。否则只有例程中的字。然后是延时，四次循环，结束后停留在最后一行显示。

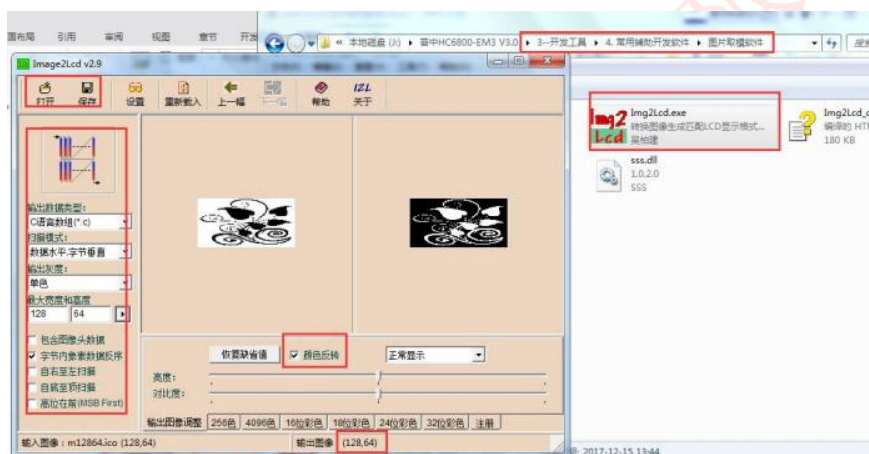
25.3.2 显示图片

(1) 如何将图像取模

Mini 12864 是灰度显示图像，也就是单色的，可以使用图像取模软件，取模方法大同小异，需要用户自己去实践。这里使用光盘中提供的图像取模软件，位置如下图，如果光盘资料不一样，应该会在其他位置，需要用户仔细搜索整个光盘资料。首先我们需要准备一张单色的图像，这里图像大小不应该超过 128*64, 这里是 128 列，64 行的像素，如果不对，就需要进行裁剪，或用 PS 等软件进行像素调整。这里直接使用 128*64 的图像，不介绍如何调整图像，用户可以自己去查阅资料。

1) 打开图像

打开软件以后，电机菜单栏的“打开”，选择修改好的黑白图片。设置如图所示：



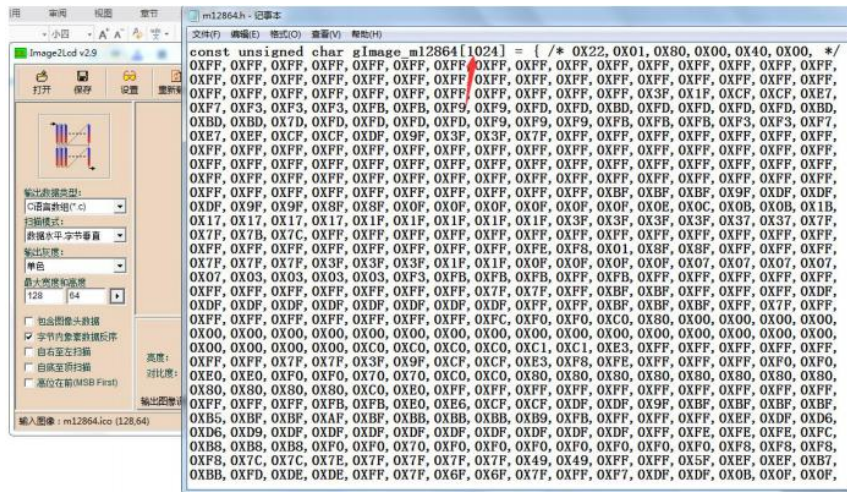
在图上数据是 C 语言类型。扫描模式是按照程序对应来，程序中设计成写完第一页以后，写第二页，故数据取模应该字节垂直，数据水平方向。输出颜色是单色的。大小设置成 12864，这里的大小修改以后，要看右下角的输出是否是 12864，如果不是，就需重新使用画图软件修改大小然后重新打开，直到图像是自己需要的大小。然后设置字节反向，是因为在页内，字节数据由下至上是高到低，这里普通情况是由上至下是由高到低，就需要反序。颜色反转选择是由于在程序里初始化 LCD12864 的时候，输出颜色有反向，对应这里就需要反转。

2) 生成图像的模

设置好以后，点击保存，设置输出 xxx.h 文件，然后会弹出图像的模，如下图。

弹出的模里，可以看到生成了一个大小是 1024 的数组，实际上就是 128*8，由于 64 行分成了 8 字节，所以是 1024 个数据。这里数据中前几个是图像头

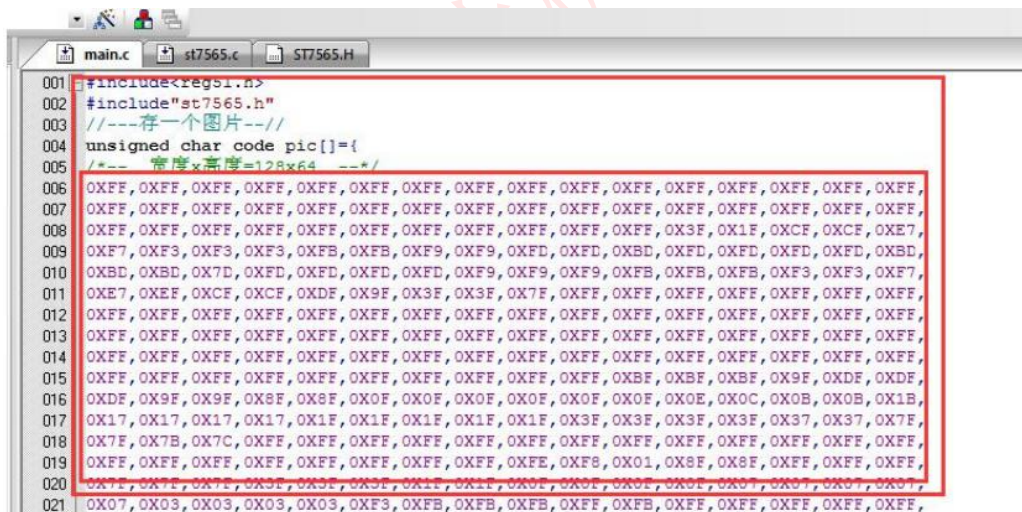
的数据，这个一般不用，复制其余的 1024 个数据到程序中替换历程中的 1024 个数据，就能显示当前这个图像。



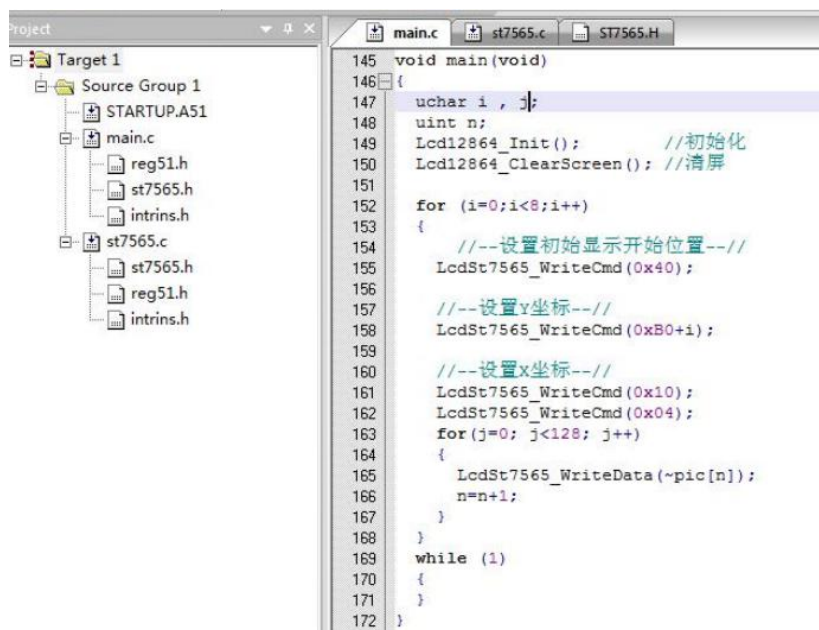
(2) 12864 如何显示图像

1) 添加图像的模到程序中

把图像的模复制到这里覆盖程序里原来的数据，就可以显示当前的图像。这里可以看到包含有 12864 的驱动源文件和头文件，可以对 12864 进行操作。



2) 输出显示代码



主函数中调用了 12864 初始化，清屏。

然后是一个八次循环，这是表示有八页，然后设置显示开始位置，这个其实可以没有，然后设置页地址，每循环一次，页地址会随着循环往下换页，然后设置起始列地址，从每一页的第一列开始写，这里设置起始列以后，直接输出 128 个数据就可以写满一页，这是因为列地址在写的时候会自动增长。这里使用的 128 次循环输出，数组里每一个数据都进行了取反输出，这也表示了为什么在图像取模的时候需要颜色反转。如果这个不取反，取模就不需要反转颜色，用户可以自己实践。程序里的 n 是用来表示当前输出的数组里第几个数，没输出一次加 1，就表示输出下一个数。

这样就完成了一次图像的输出生。然后执行死循环，防止程序跑飞。显示效果如下，就不再是例程里的图像，而是自己定义的。

至此我们就介绍完，具体代码大家可以打开工程查看。

25.4 实验现象

使用 USB 线将开发板和电脑连接成功后（使用短接片将 VCC 与 OE 短接），把编写好的程序编译后将编译产生的 .hex 文件烧入到芯片内，按照如下图所示的接线方式可以看到：LCD12864 逐行显示“普中科技有限公司”。



课后作业

(1) 在 LCD12864 上显示时钟。

(温馨提示：将本章实验与 DS1302 时钟程序组合)